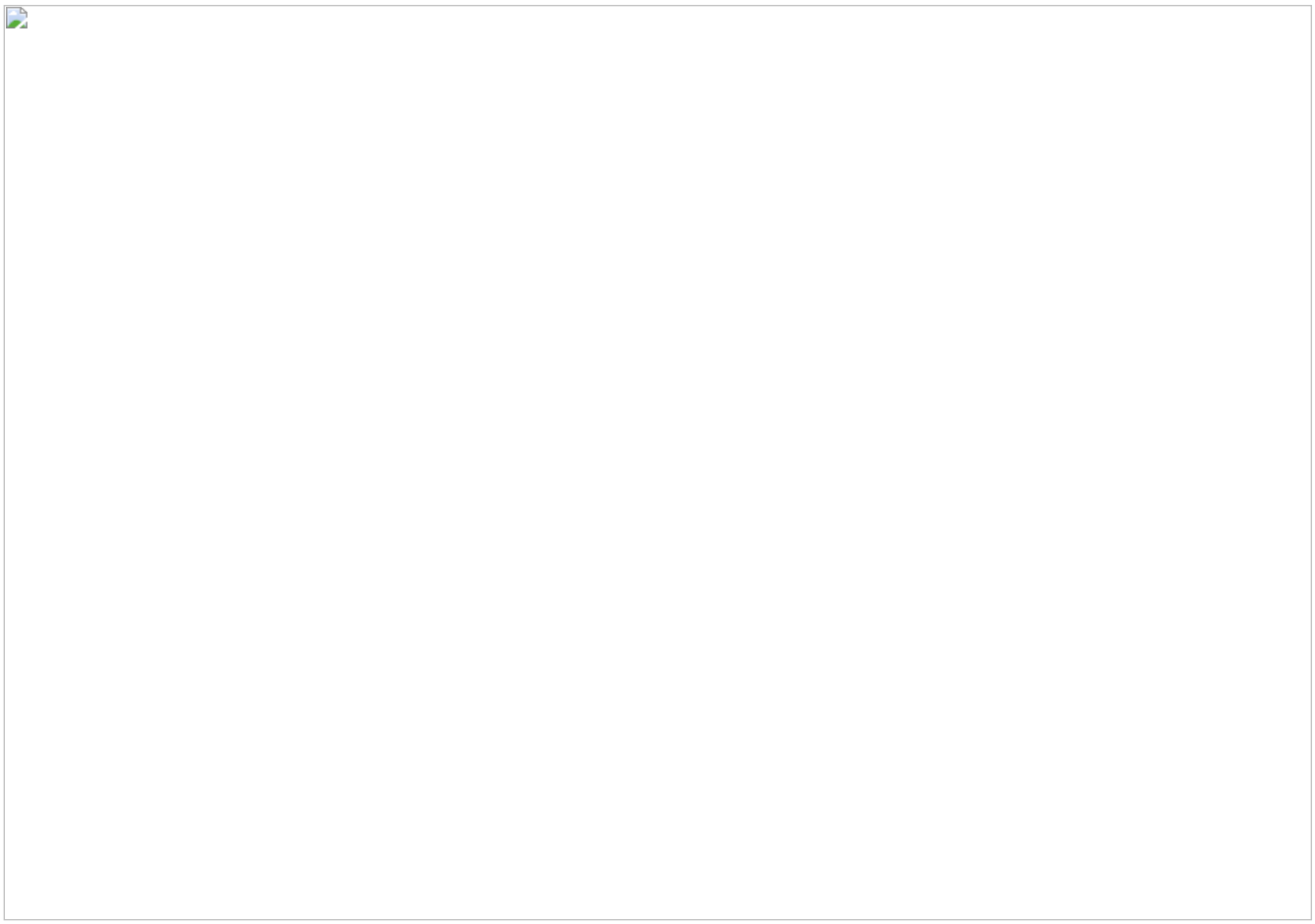
Category Theory illustrated



Praise

> "The range of applications for category theory is immense, and visually conveying meaning through illustration is an indispensable skill for organizational and technical work. Unfortunately, the foundations of category theory, despite much of their utility and simplicity being on par with Venn Diagrams, are locked behind resources that assume far too much academic background.
>
> Should category theory be considered for this academic purpose or any work wherein clear thinking and explanations are valued, beginner-appropriate resources are essential. There is no book on category theory that makes its abstractions so tangible as "Category Theory Illustrated" does. I recommend it for programmers, managers, organizers, designers, or anyone else who values the structure and clarity of information, processes, and relationships."

Evan Burchard, Author of "The Web Game Developer's Cookbook" and "Refactoring JavaScript"

> "The clarity, consistency and elegance of diagrams in 'Category Theory Illustrated' has helped us demystify and explain in simple terms a topic often feared."

Gonzalo Casas, Software developer and lecturer at ETH Zurich

About

In memory of

Francis William Lawvere

1937 - 2023

"Try as you may,

you just can't get away,

from mathematics"

Tom Lehrer

The story behind this book

I was interested in math as a kid, but was always messing up calculations, so I decided it was not my thing and started pursuing other interests, like writing and visual art.

A little later I got into programming and I found that this was similar to the part of mathematics that I enjoyed. I started using functional programming in an effort to explore the similarity and to improve myself as a developer. I discovered category theory a little later.

Some 5 years ago I found myself jobless for a few months and decided to publish some of the diagrams that I drew as part of the notes I kept when was reading "Category Theory for Scientists" by David Spivak. The effort resulted in a rough version of the first two chapters of this book, which I published online.

A few years after that some people found my notes and encouraged me write more. They were so nice that I forgot my imposter syndrome and got to work on the next several chapters.

On math

Ever since Newton's Principia, the discipline of mathematics is viewed in the somewhat demeaning position of "science and engineering's workhorse" — only "useful" as a means for helping scientists and engineers to make technological and scientific advancements, i.e., it is viewed as just a tool for solving "practical" problems.

Because of this, mathematicians are in a weird and, I'd say, unique position of always having to defend what they do with respect to its value for *other disciplines*. I again stress that this is something that would be considered absurd when it comes to any other discipline.

People don't expect any return on investment from *physical theories*, e.g., no one bashes a physical theory for having no utilitarian value.

And bashing philosophical theories for being impractical would be even more absurd — imagine bashing Wittgenstein, for example:

> "All too well, but what can you do with the picture theory of language?" "Well, I am told it does have its applications in programming language theory..."

Or someone being sceptical to David Hume's scepticism:

> "That's all fine and dandy, but your theory leaves us at square one in terms of our knowledge. What the hell are we expected to do from there?"

Although many people don't necessarily subscribe to this view of mathematics as a workhorse, we can see it encoded inside the structure of most mathematics textbooks — each chapter starts with an explanation of a concept, followed by some examples, and then ends with a list of problems that this concept solves.

There is nothing wrong with this approach, but mathematics is so much more than a tool for solving problems. It was the basis of a religious cult in ancient Greece (the Pythagoreans), it was seen by philosophers as means to understanding the laws which govern the universe. It was, and still is, a language which can allow for people with different cultural backgrounds to understand each other. And it is also art and a means of entertainment. It is a mode of thinking, Or we can even say it is thinking itself. Some people say that "writing is thinking", but I would argue that writing, when refined enough, and free from any kind of bias in on the side of the author, automatically becomes *mathematical writing* — you can almost convert the words into formulas and diagrams.

Category theory embodies all these aspects of mathematics, so I think it's very good grounds to writing a book where all of them shine — a book that isn't based on solving of problems, but exploring concepts and seeking connections between them. A book that is, overall, pretty.

Who is this book for

So, who is this book for? Some people would phrase the question as "Who *should* read this book", but if you ask it this way, then the answer is "nobody".

Indeed, if you think in terms of "should", mathematics (or at least the type of mathematics that is reviewed here) won't help you much, although it is falsely advertised as a solution to many problems (whereas it is, in fact, (as we established) something much more).

Let's take an example — many people claim that Einstein's theories of relativity are essential for GPS-es to work properly. Due to relativistic effects, the clocks on GPS satellites tick faster than identical clocks on the ground.

They seem to think that if the theory didn't exist, the engineers that developed the GPSes would have faced this phenomenon in the following way:

> Engineer 1: Whoa, the clocks on the satellites are off by X nanoseconds!
>
> Engineer 2: But that's impossible! Our mathematical model predicts that they should be correct.
>
> Engineer 1: OK, so what do we do now?
>
> Engineer 2: I guess we need to drop this project until we have a viable mathematical model that describes time in the universe.

Although I am not an expert in special relativity, I suspect that the way this conversation would have developed would be closer to the following:

> Engineer 1: Whoa, the clocks on the satellites are off by X nanoseconds!
>
> Engineer 2: This is normal. There are many unknowns.
>
> Engineer 1: OK, so what do we do now?
>
> Engineer 2: Just adjust it by X and see if it works. Oh, and tell that to some physicist. They might find it interesting.

In other words, we can solve problems without any advanced math, or with no math at all, as evidenced by the fact that the Egyptians were able to build the pyramids without even knowing Euclidean geometry. And with that I am not claiming that math is so insignificant, that it is not even good enough to serve as a tool for building stuff. Quite the contrary, I think that math is much more than just a simple tool. So going through any math textbook (and of course especially this one) would help you in ways that are much more vital than finding solutions to "complex" problems.

Some people say that we don't use maths in our daily life. But, if true, that is only because other people have solved all hard problems for us and the solutions are encoded on the tools that we use, however not knowing math means that you will be forever a consumer, bound to use those existing tools and solutions and thinking patterns, not being able to do anything on your own.

And so "Who is this book for" is not to be read as who should, but who *can* read it. Then, the answer is "everyone".

About the language

Explaining mathematics involves a tradeoff between understandable/approachable and being rigorous/correct. Between the the first-grade teacher who says that if you have one apple and you get another one you'd have two, and presenting a 200-page wall of formulas that prove the same statement, as Russell and Whitehead did in "Principia Mathematica".

Here, I try to stay in the middle of this spectrum ("the middle way", as the Buddist teachings call it). This is simply my niche, this is simply because there *aren't so many texts* that are there (a math student often has to make by themselves the leap between talking about apples and oranges and talking about formal statements).

I think that this is because being in the middle is *hard*. You have to build bridges in both directions, to take care of both the less and more advanced, to have both the pictures and formulas.

And I did put of effort to have both: although I am sloppy/handwavy, I value correctness over everything else. Every statement has been checked extensively by me and other people who helped me with this project and all statements that can possibly leave the reader with the wrong impression have been edited.

About category theory

Like we said, the fundaments of mathematics are the fundaments of thought. Category theory allows us to formalize those fundaments that we use in our daily (intellectual) lives.

The way we think and talk is based on intuition that develops naturally and is a very easy way to get our point across. However, intuition also makes it easy to be misunderstood — what we say usually can be interpreted in many ways, some of which are wrong. Misunderstanding of these kinds are the reason why biases appear. Moreover, certain people (called "sophists" in ancient Greece) would introduce biases on purpose in order to twist the discourse in the direction that suits them.

It's in such situations, that people often resort to *formulas* and *diagrams* to refine their thoughts. Diagrams (even more than formulas) are ubiquitous in science and mathematics.

Category theory formalizes the concept of diagrams and their components — arrows and objects — to create a language for presenting all kinds of ideas. In this sense, category theory is a way to unify knowledge, both mathematical and scientific, and to unite various modes of thinking with common terms.

As a consequence of that, category theory and diagrams are also a very understandable way to communicate a formal concept clearly, something I hope to demonstrate in the following pages.

Summary

In this book we will visit various such modes of knowledge and along the way, see all kinds of mathematical objects, viewed through the lens of categories.

We start with *set theory* in chapter 1, which is the original way to formalize different mathematical concepts.

Chapter 2 we will make a (hopefully) gentle transition from sets to *categories* while showing how the two compare and (finally) introducing the definition of category theory.

In the next two chapters, 3 and 4, we jump into two different branches of mathematics and introduce their main means of abstraction, *groups and orders*, observing how they connect to the core category-theoretic concepts that we introduced earlier.

Chapter 5 also follows the main formula of the previous two chapters, getting to the heart of the matter of why category theory is a universal language, by

showing its connection with the ancient discipline of *logic*. As in chapters 3 and 4, we start with a crash course in logic itself.

The connection between all these different disciplines is examined in chapter 6, using one of the most interesting category-theoretical concepts — the concept of a functor.

In chapter 7 we review another more interesting and more advanced categorical concept, the concept of a *natural transformation*.

Acknowledgments

Thanks to my wife Dimitrina, for all her support.

My daughter Daria, my "anti-author" who stayed seated on my knees when I was writing the second and third chapters and mercilessly deleted many sentences, most of them bad.

Thanks to my high-school arts teacher, Mrs Georgieva who told me that I have some talent, but I have to work.

Thanks to Prathyush Pramod who encouraged me to finish the book and is also helping me out with it.

And also to everyone else who submitted feedback and helped me fix some of the numerous errors that I made — knowing myself, I know that there are more.

Sets

Ready, *set*, begin... (you don't know how hard I tried to resist to making that pun). We begin our inquiry with the theory of sets. Set theory and category theory share many similarities. We can view category theory as a *generalization* of set theory. That is, it's meant to describe the same thing as set theory (everything?), but to do it in a more abstract manner, one that is more versatile and (hopefully) simpler.

Also, sets are an *example of a category* (the *proto-example*, we might say), and it is useful to have examples.

What is an Abstract Theory

> Instead of asking what can be defined and deduced from what is assumed to begin with, we ask instead what more general ideas and principles can be found, in terms of which what was our starting-point can be defined or deduced. Bertrand Russell, from Introduction to Mathematical Philosophy

Most scientific and mathematical theories have a specific *domain*, which they are tied to, and in which they are valid. They are created with this domain in mind and are not intended to be used outside of it. For example, Darwin's theory of evolution is created in order to explain how different *biological species* came to evolve using natural selection, quantum mechanics is a description of how particles behave at a specific scale, etc.

Even mathematical theories, although they are not inherently *bound* to a specific domain (like the scientific theories) are at least strongly related to some domain, as for example differential equations are created to model how events change over time.

Set theory and category theory are different, they are not created to provide a rigorous explanation of how a particular phenomenon works, instead they provide a more general framework for explaining all kinds of phenomena. They

work less like tools and more like languages for defining tools. Such theories are called *abstract* theories.

The borders of the two are sometimes blurry. All theories *use abstraction*, otherwise they would be pretty useless: without abstraction Darwin would have to speak about specific animal species or even individual animals. The difference is that abstract theories have *core concepts* that don't refer to anything in particular, and are instead left for people to generalize on. All theories are applicable outside of their domains, but set theory and category theory do not have a domain to begin with.

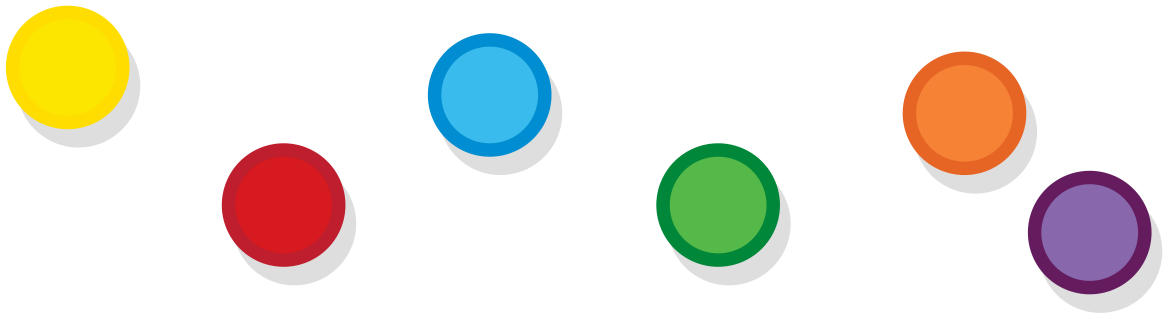
Concrete theories, like the theory of evolution, are composed of concrete concepts. For example, the concept of a *population*, also called a *gene-pool*, refers to a group of individuals that can interbreed. Abstract theories, like set theory, are composed of abstract concepts, like the concept of a set. The concept of a set by itself does not refer to anything. However, we cannot say that it is an empty concept, as there are countless things that can be represented by sets, for example, gene pools can be (very aptly) represented by sets of individual animals. Animal species can also be represented by sets — a set of all populations that can theoretically interbreed.

You've already seen how abstract theories may be useful. Because they are so simple, they can be used as building blocks to many concrete theories. Because they are common, they can be used to unify and compare different concrete theories, by putting these theories in common grounds (this is very characteristic of category theory, as we will see later). Moreover, good (abstract) theories can serve as *mental models* for developing our thoughts.

Sets

> "A set is a gathering together into a whole of definite, distinct objects of our perception or of our thought—which are called elements of the set." - Georg Cantor

Perhaps unsurprisingly, everything in set theory is defined in terms of sets. A set is a collection of things where the "things" can be anything you want (like individuals, populations, genes, etc.) Consider, for example, these balls.



Let's construct a set, call it G (as gray) that contains *all* of them as elements. There can only be one such set, because a sets have no structure.

> **DEFINITION:**
>
> A set is a collection of items, that contains no structure, other than which items belong to it.

So, there is no order, no position, no ball goes before or after another, there are no members which are "special" with respect to their membership of the set. Two sets that contain the same elements are just pictures of the same set.

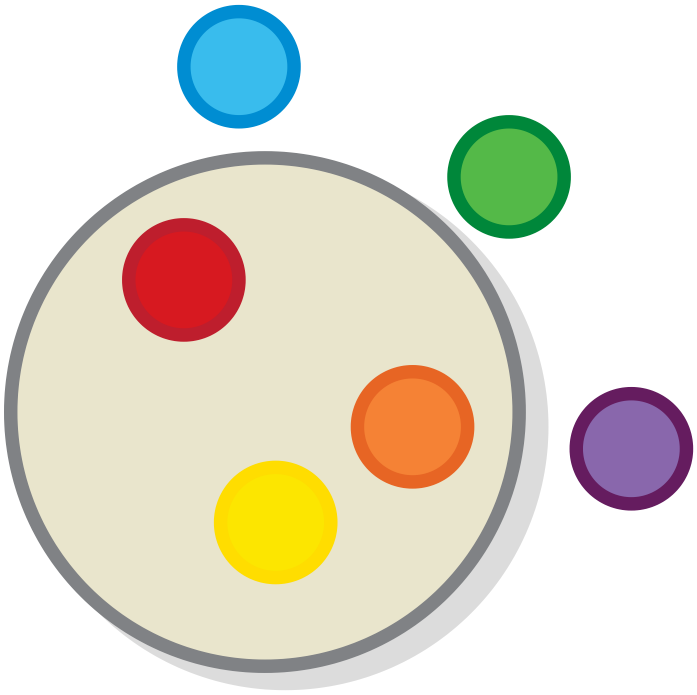


This example may look overly-simple, but in fact, it's just as valid as any other.

The key insight that makes the concept useful is the fact that it enables you to reason about several things as if they were one.

Subsets

Let's construct one more set. The set of *all balls that are warm in color*. Let's call it Y (because in the diagram, it's colored in **y**ellow).



Notice that Y contains only elements that are also present in G. When two sets have this relation, we may say that Y is a *subset* of G.

> **DEFINITION:**
>
> Y is a subset of G when (or $Y \subseteq G$) if every element of the set of Y is also an element in the set G.

A subset resides completely inside its superset when the two are drawn together.

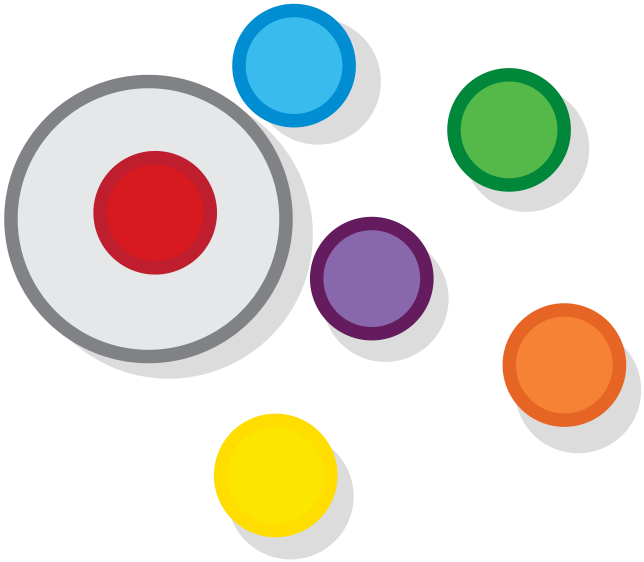


Singleton Sets

The set of all *red balls* contains just one ball. We said above that sets summarize *several* elements into one. Still, sets that contain just one element are perfectly valid &mdash, there are things that are *one of a kind*.

> **DEFINITION:**
>
> A singleton set is a set that contains one element.

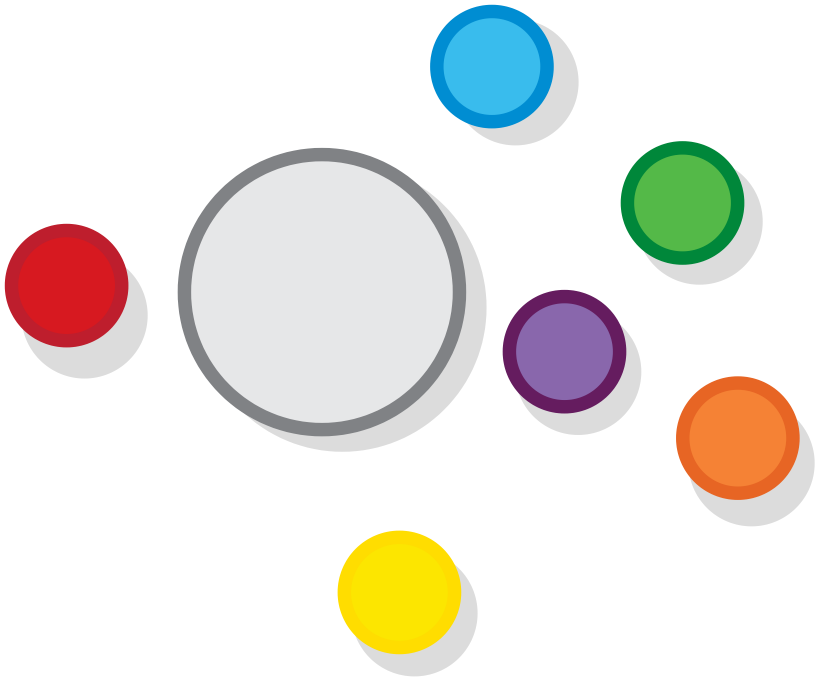
The set of kings/queens that a given kingdom has is a singleton set.



What's the point of singleton sets? Well, it is part of the language of set theory, e.g., if we have a function which expects a set of given items, but if there is only one item that meets the criteria, we can just create a singleton set with that item.

The Empty set

Of course if one is a valid answer, zero can be also. If we want a set of all *black balls* B or all the *white balls*, W, the answer to all these questions is the same — the empty set.



Because a set is defined only by the items it contains, the empty set is *unique* — there is no difference between the set that contains zero *balls* and the set that contains zero *numbers*, for instance.

> **DEFINITION:**
>
> The empty set is a set that contains no elements.

Formally, the empty set is marked with the symbol $\varnothing$ (so $B = W = \varnothing$).

The empty set has some special properties, for example, it is a subset of every other set. Mathematically speaking, $\forall A \rightarrow \varnothing \subseteq A$ ($\forall$ means "for all")

Functions

> "By function I mean the unity of the act of arranging various representations under one common representation." — Immanuel Kant, from "The Critique of Pure Reason"

Now is the time to admit something: this chapter isn't actually about sets. It is about functions and we only started this way because there is no way to

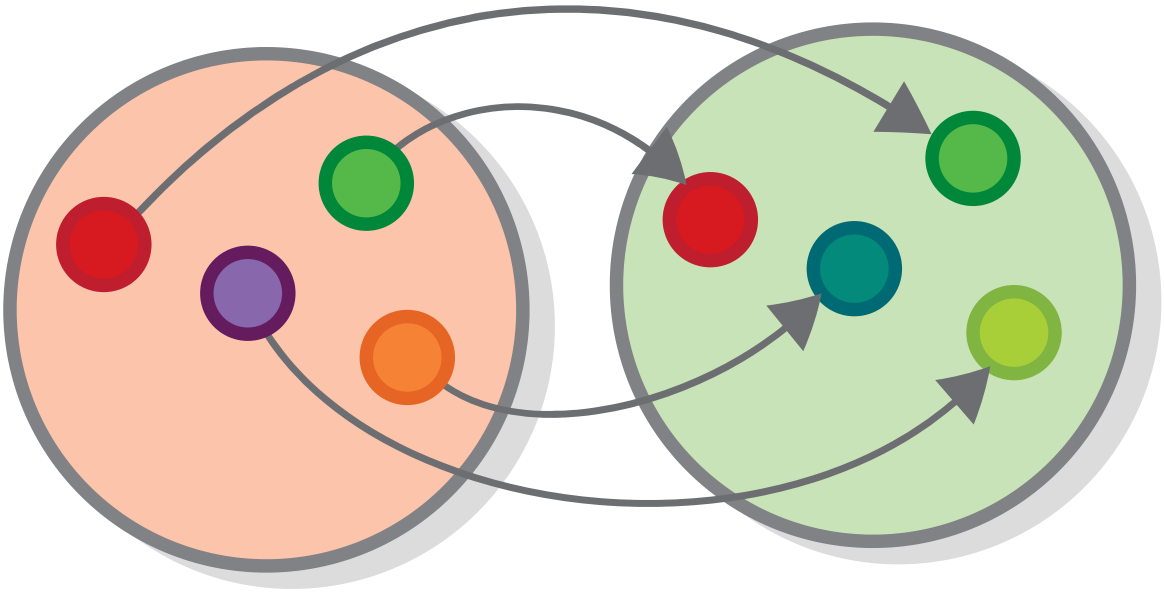
explain functions without explaining sets (although if you read on, you'll find that there actually is one).

> **DEFINITION:**
>
> A function is many-to-one a relationship between two sets: one that matches each element of one set, called the *source set* of the function, with exactly one element from another set, called the *target set* of the function.

These two sets are also called the *domain* and *codomain* of the function, or its *input* and *output*. In programming, they go by the name of *argument type* and *return type*. In logic, they correspond to the *premise* and *conclusion* (we will get there). We might also say, depending on the situation, that a given function *goes* from this set to that other one, *connects* this set to the other, or that it *converts* a value from this set to a value from the other one. These different terms demonstrate the multifaceted nature of the concept of function.

Different types of functions

Here is a function f, which converts each ball from the set R to the ball with the opposite color in another set G (in mathematics a function's name is often accompanied by the names of its source and target sets, like this: $f : R \to G$)



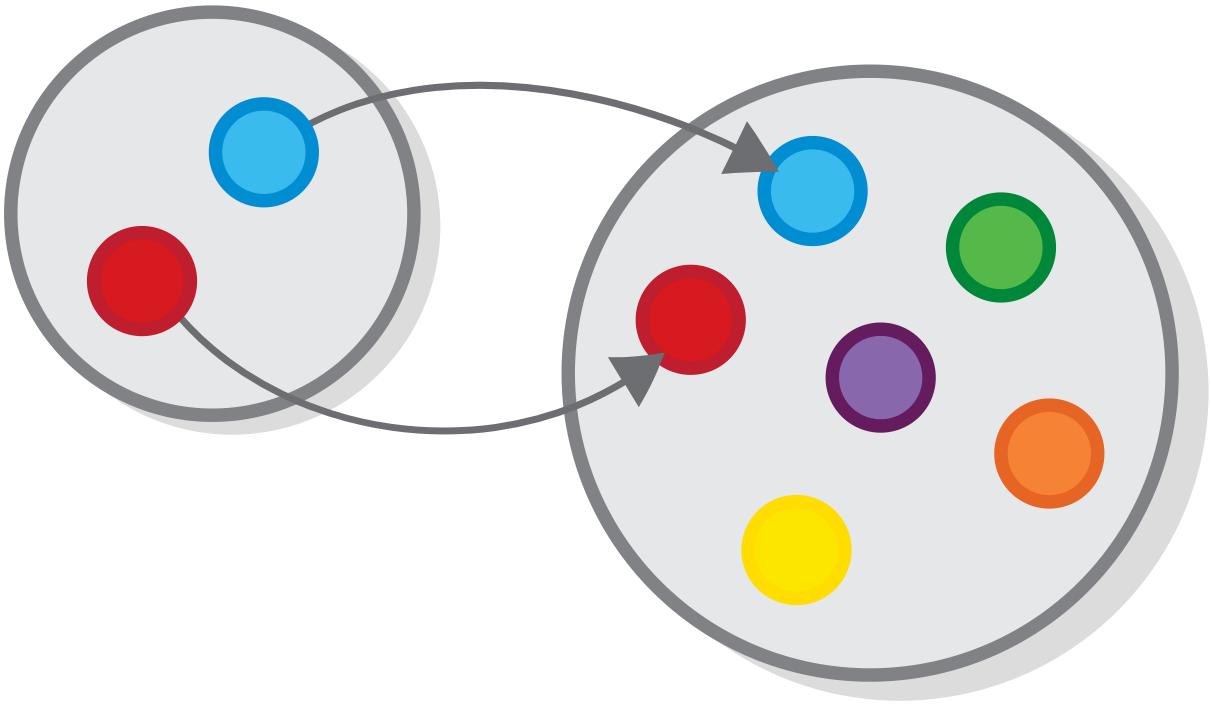
This is probably one of the simplest type of function that exists — it encodes a *one-to-one relationship* between the sets. That is to say, *one* element from the source is connected to exactly *one* element from the target (and the other way around).

But functions usually express relationships of the type *many-to-one*, where *many* elements from the source might be connected to *one* element from the target (but not the other way around). Below is one such function.



Such functions might represent operations such as *categorizing* a given collection of objects by some criteria, or partitioning them, based on some property that they might have.

A function can also express relationships in which some elements from the target set do not play a part.



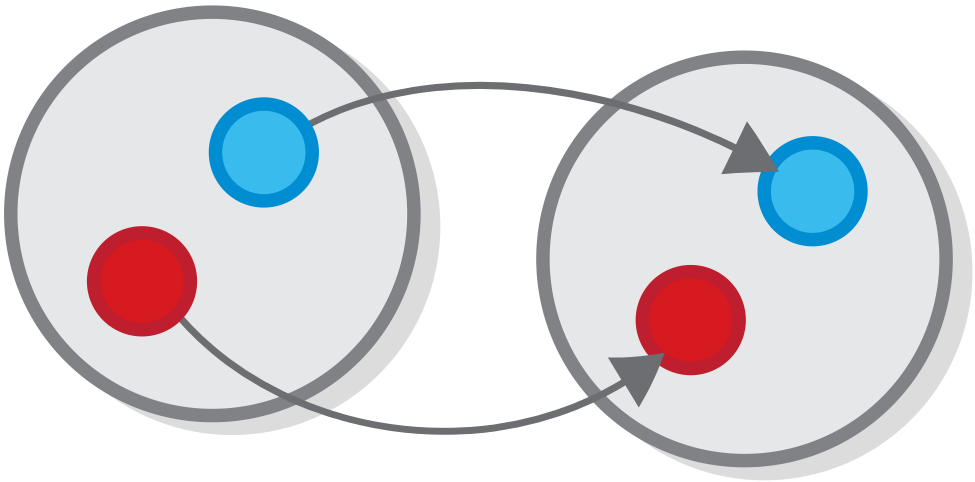
An example might be the relationship between some kind of pattern or structure and the emergence of this pattern in some more complicated context.

We saw how versatile functions are, but there is one thing that you cannot have in a function. You cannot have a source element that is not mapped to anything, or that is mapped to more than one target element — that would constitute a *many-to-many* relationship and as we said functions express many-to-one relationships. There is a reason for that "design decision", and we will arrive at it shortly.

The Identity Function

For every set G, no matter what it represents, we can define the function that does nothing:

> **DEFINITION:**
>
> The identity function for a set G, $ID_G : G \to G$ is a function which maps every element of G to itself.

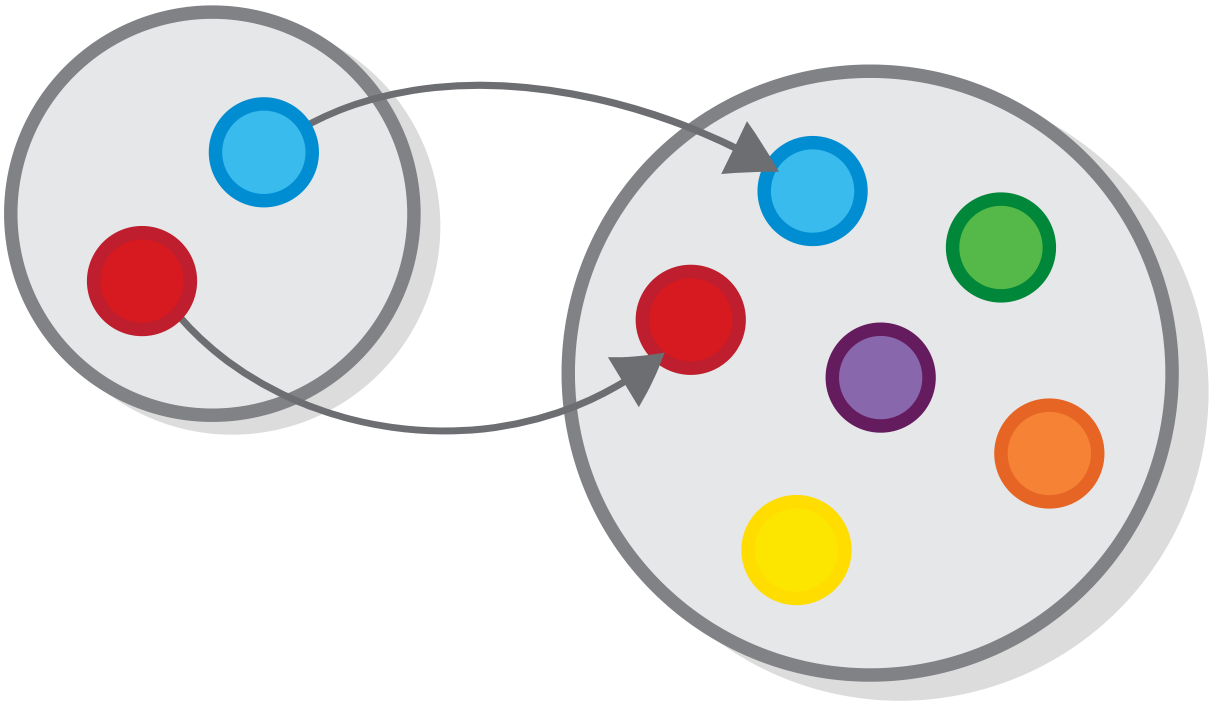


You can think of ID_G as a function which represents the set G in the realm of functions.

Functions and Subsets

Another interesting collection of functions (there is one for each subset).

> **THEOREM:**
>
> For each set and subset, we can define a function (called the *image* of the subset) that maps each element of the subset to itself.

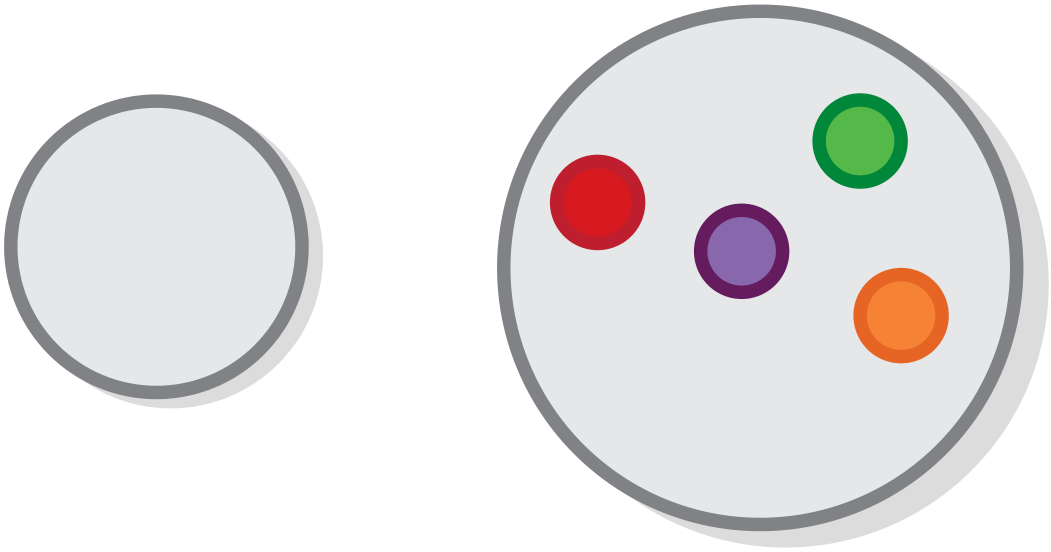


Every set is a subset of itself, in which case this function is the same as the identity.

Functions and the Empty Set

Although it doesn't look like it...

> **THEOREM:**
>
> There is a unique function from the empty set to any other set.



Task 3: Is this really valid? Why? Check the definition.

If you still aren't convinced, check this out:

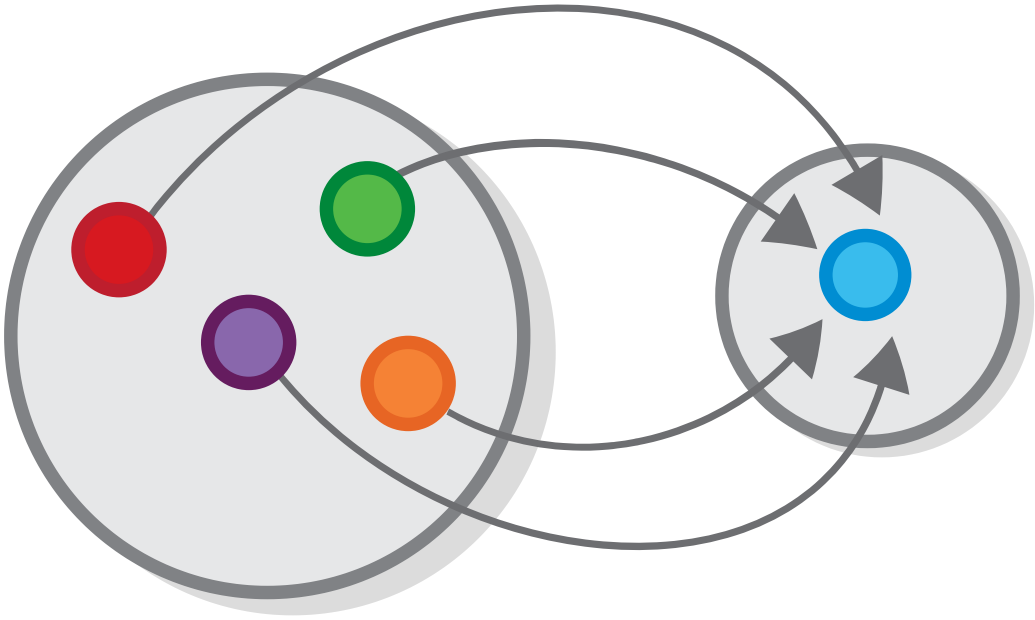
1. There is a function between a subset and a its superset
2. The empty set is a subset of any other set.

So, evidently, this function has to exist!

Task 4: What about the other way around. Are there functions with the empty set as a target as opposed to its source?

Functions and Singleton Sets

And another function that we meet often is this one.



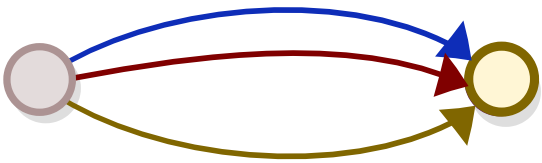
> **THEOREM:**
>
> There is a unique function from any set to any singleton set.

Task 5: Is this really the only way to connect *any* set to a singleton set in a valid way?

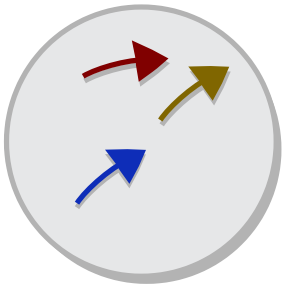
Task 6: Again, what about the other way around?

Hom sets

Given two sets A and B with a bunch of functions between them (here we don't draw the set elements and draw each function as a single arrow)...



...we can construct the *function set* (usually called *hom set*) of A to B, containing those functions as elements.



If there are no functions, this set is empty.

> **DEFINITION:**
>
> For objects A and B the homomorphism set of A to B, (denoted $A \Rightarrow B$) is the set containing one element for each function that goes from A to B.

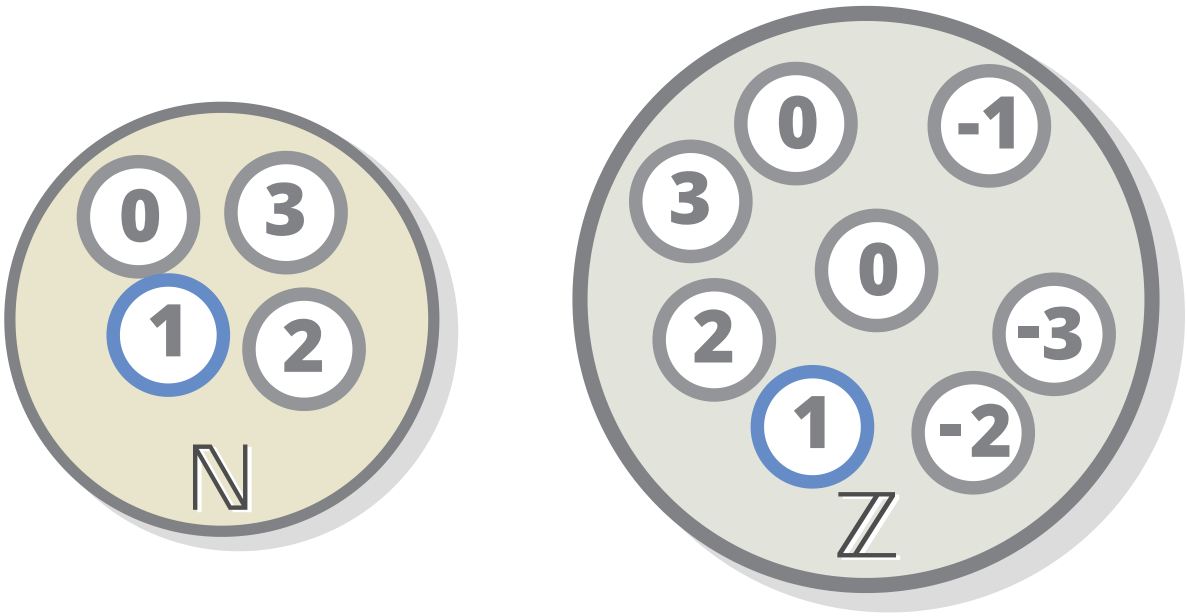
Note that this is only in one direction, the functions between B and A belong in a different hom set.

Numbers

All numerical operations can be expressed as functions acting on the sets of (different types of) numbers.

Number sets

Because not all functions work on all numbers, we separate the set of numbers to several sets, some of which are subsets to one another, such the set of whole numbers $\mathbb{Z} := \ldots -3 -2, -1, 0, 1, 2, 3 \ldots$, the set of positive whole numbers, (also called “natural” numbers), $\mathbb{N} := 0, 1, 2, 3 \ldots$.

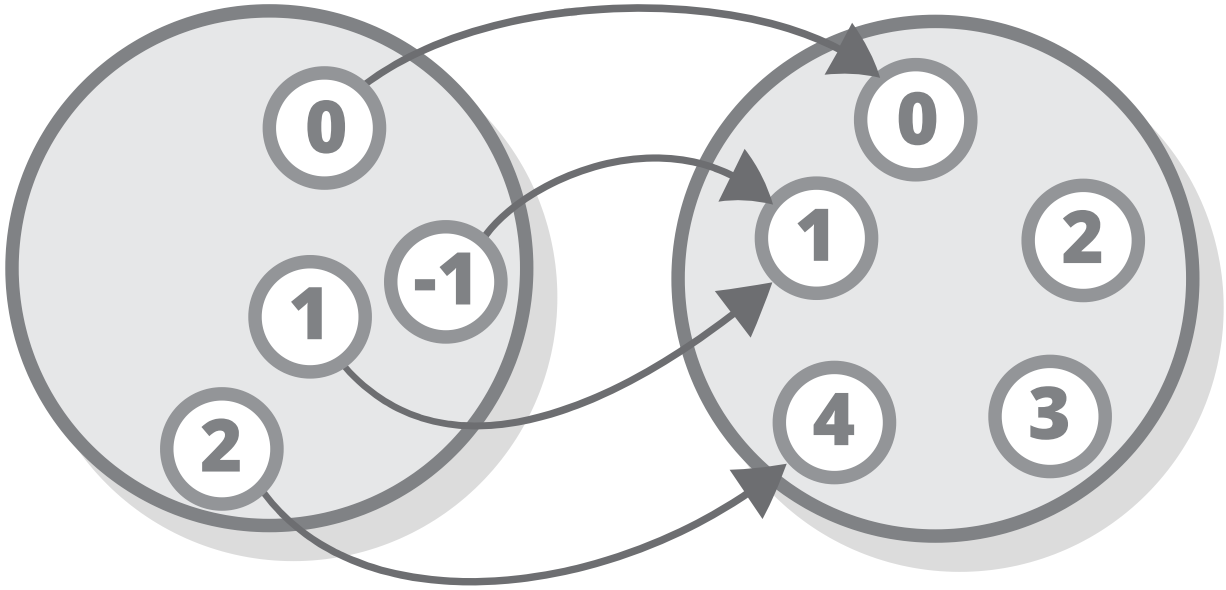


(Because both sets are infinite, we cannot draw them in their entirety, however we can draw a part of them).

Number functions

> Every generalization of number has first presented itself as needed for some simple problem: negative numbers were needed in order that subtraction might be always possible, since otherwise a − b would be meaningless if a were less than b; fractions were needed in order that division might be always possible; and complex numbers are needed in order that extraction of roots and solution of equations may be always possible. — Bertrand Russell, from Introduction to Mathematical Philosophy

Each numerical operation is a function between two number sets. For example, squaring a number is a function from the set of real numbers to the set of real non-negative numbers.



I will use the occasion to reiterate some of the more important characteristics of functions:

- All numbers in the target have (or should have) two arrows pointing at them (one for the positive square root and one for the negative one), and that is OK.
- Zero from the source set is connected to itself in the target set — that is permitted.
- Some numbers aren't the square of any other number — that is also permitted.

Overall everything is permitted, as long as you can always provide exactly one result (also known as *The result™*) per value. For numerical operations, this is always true, simply because math is designed this way.

Note that most mathematical operations, such as addition, multiplication, etc. require two numbers in order to produce a result. This does not mean that they are not functions, it just means they're a little fancier. Depending on what we need, we may present those operations as functions from the sets of *tuples* of numbers to the set of numbers, or we may say that they take a number and return a function. More on that later.

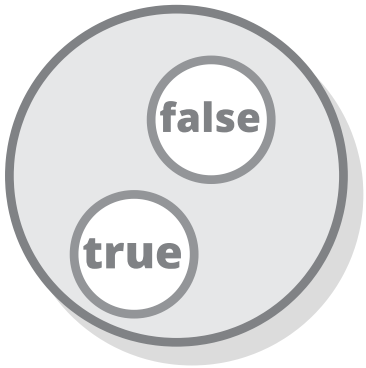
Sets and Functions in Programming

Sets are used extensively in programming, especially in their incarnation as *types* (also called *classes*). All sets of numbers that we discussed earlier also exist in most languages as types.

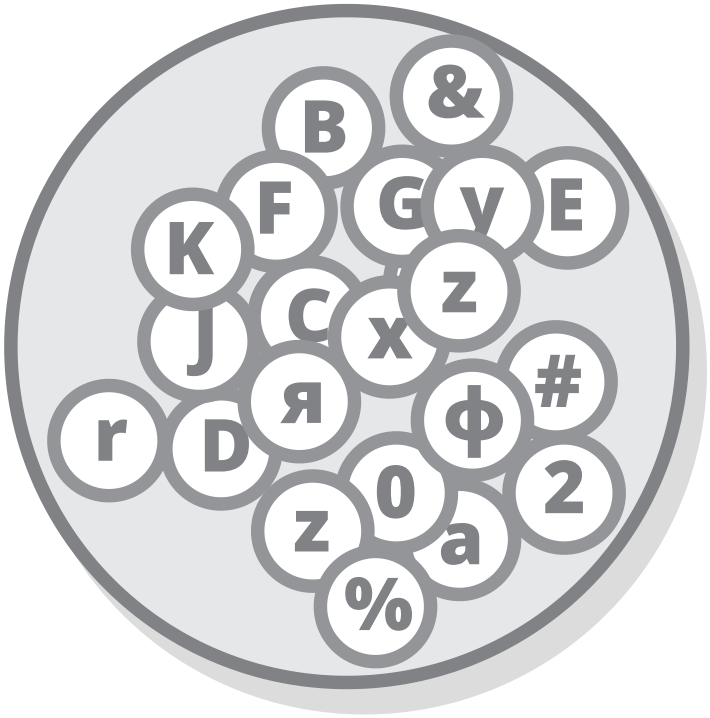
Sets and types

Sets are not the same thing as types, but types can be seen as sets (or they *have* sets we can say).

For example, we can view the `Boolean` type as a set containing two elements — `true` and `false`.



Another very basic set that is used in programming is the set of keyboard characters, or `Char`.



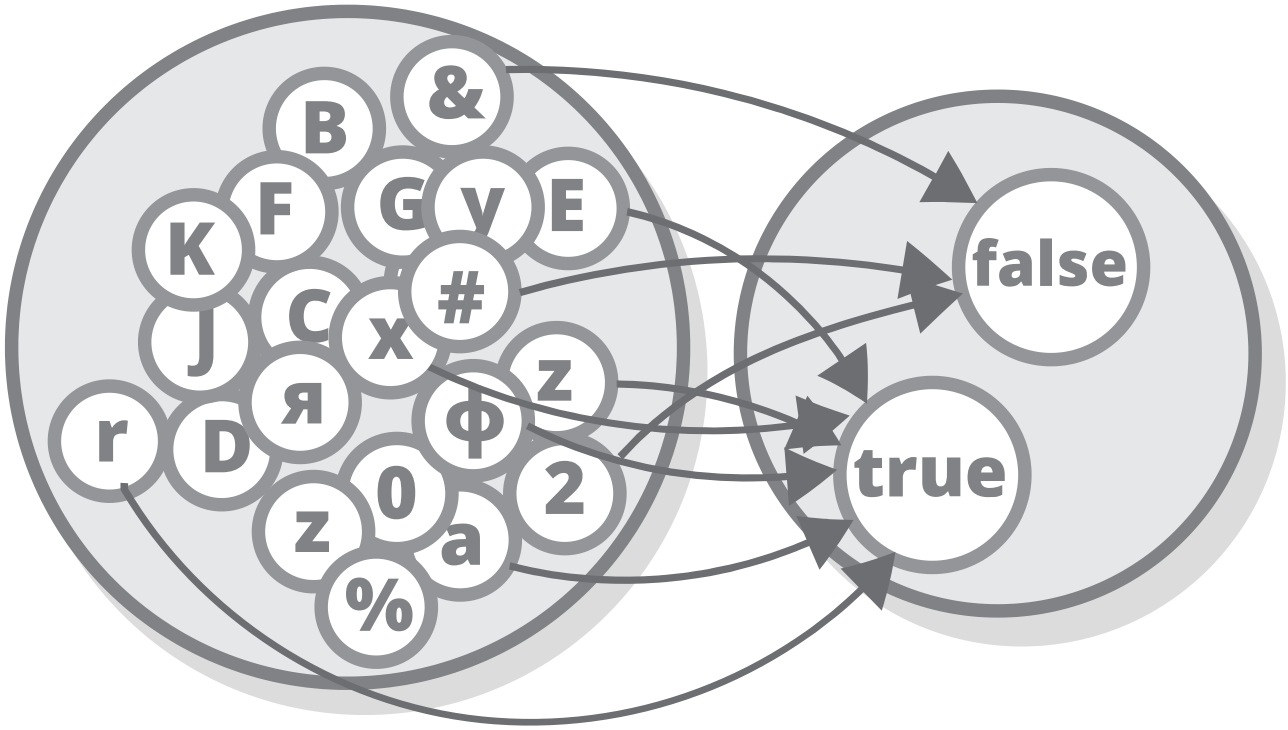
Most of the sets that we use in programming are *composite* sets e.g. make a list of `Char`s and you have a string. We will see how this happens later.

Task 7: What is the type equivalent of subsets in programming?

Functions and methods/subroutines

Functions in programming (also called methods, subroutines, etc.) kinda resemble mathematical functions — they take an element that belongs to a given set and return exactly one element which belongs to another set.

For example, here is a method that takes an argument of type `Char` and returns a `Boolean`, indicating whether the character is a letter.



However functions in most programming languages can also be quite different from mathematical functions — they can perform various operations that have nothing to do with returning a value. These operations are sometimes called *side effects*.

Why are functions in programming different? Well, figuring a way to encode *effectful* functions in a way that is mathematically sound isn't trivial and at the time when most programming paradigms that are in use today were created, people had bigger problems than the their functions not being mathematically sound (e.g. actually being able to run any program at all).

Purely-functional programming languages

Many people feel that mathematical functions are too limiting and hard to use. And they might have a point, but mathematical functions have one big advantage over non-mathematical ones — their type signature tells you almost everything about what the function does (this is probably the reason why most functional languages are strongly-typed). This is why there are some languages that only permit mathematical functions, and for which this equality holds. They are called *purely-functional* programming languages. An example of a such language is Haskell, which we will meet later.

Such languages don't support functions that perform operations like rendering stuff on screen, doing I/O, etc. (in this context, such operations are called "side effects".

There, such operations are *outsourced* to the language's runtime. Instead of writing functions that directly perform a side effect, for example `console.log('Hello')`, we write functions that return a type that represents that side effect (for example, in Haskell side effects are handled by the `IO` type) and the runtime then executes those functions for us.

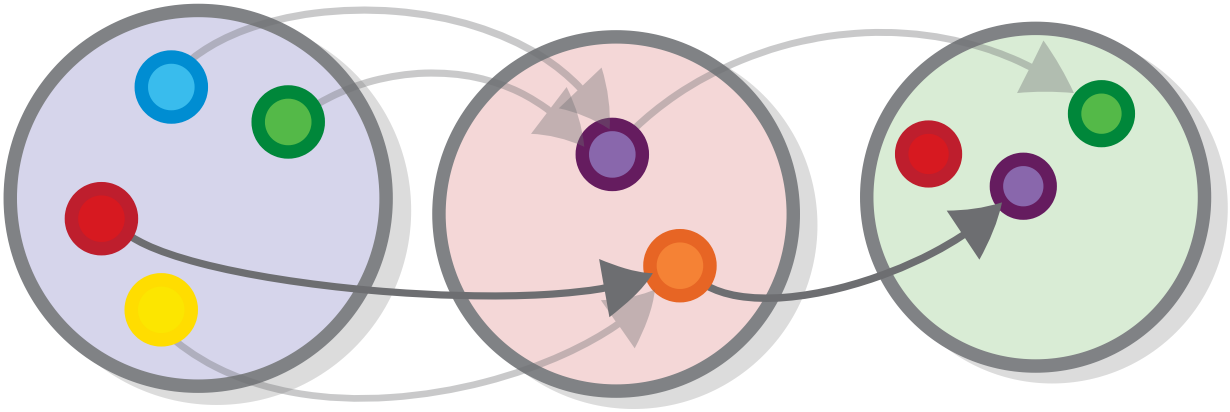
We then compose all those functions into a program (by breaking them down to a thing called *continuation passing style*).

Functional Composition

Now, we were just about to reach the heart of the matter regarding the topic of functions. And that is functional composition. Assume that we have two functions, and the target of the first one is the same set as the source of the second one



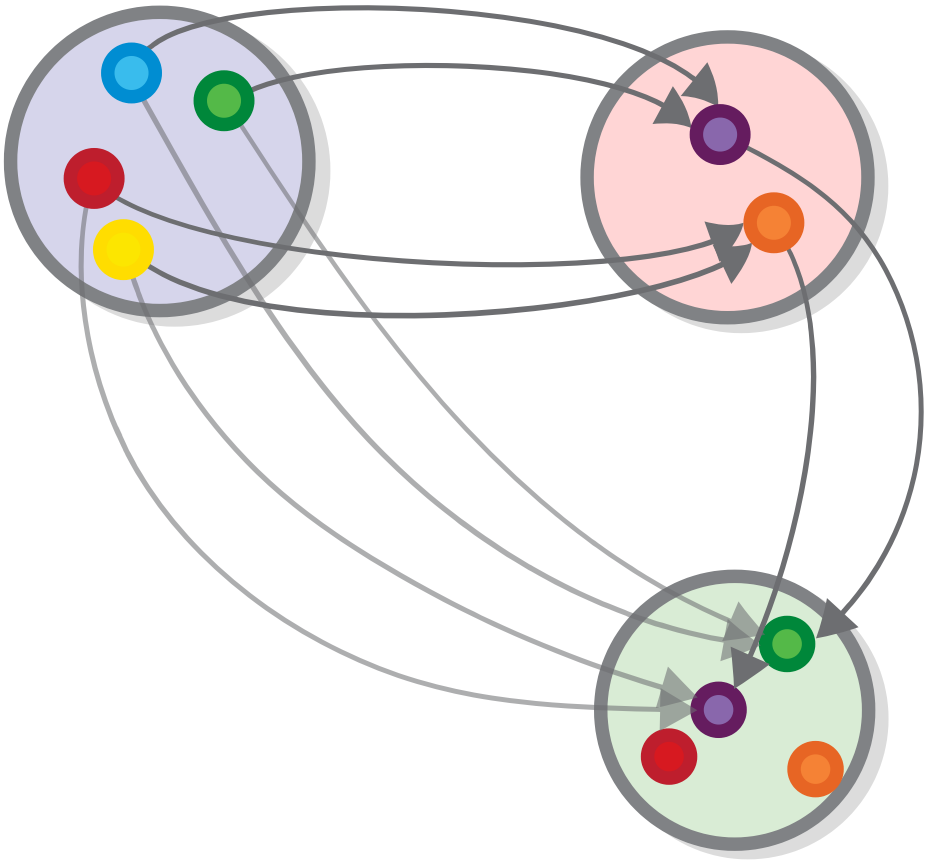
If we apply the first function g to some element from set Y, we will get an element of the set P. Then, if we apply the second function f to *that* element, we will get an element from type G.



We can define a function that is the equivalent to performing the operation described above:

> **THEOREM:**
>
> For any three sets Y, P and G and two functions $g : Y \to P$ and $f : P \to G$ we can define a function $f \circ g$, such that, if you follow the $f \circ g$ arrow for any element of set Y you will get to the same element of the set G as the one you will get if you follow the g arrow and then the f arrow. We call $f \circ g$ the *composition* of g and f.

(notice that in $f \circ g$ the first function is on the right, so it's similar to $f(g(a))$).



Composition is the essence of all things categorical. The key insight is that the sum of two parts is no more complex than the parts themselves (and therefore can be summed (composed) again). This insight is captured by the property called associativity, which we will look into later.

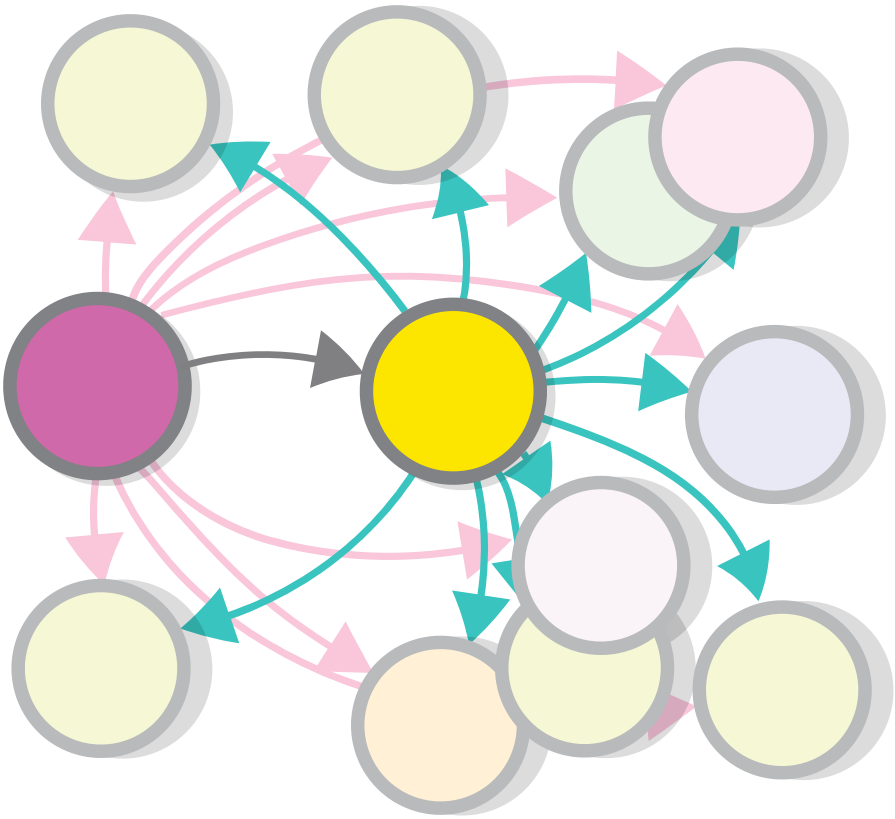
Task 8: Think about which qualities of a function make composition possible, e.g., does it work with other types of relationships, like many-to-many and one-to-many.

Composition in everyday life

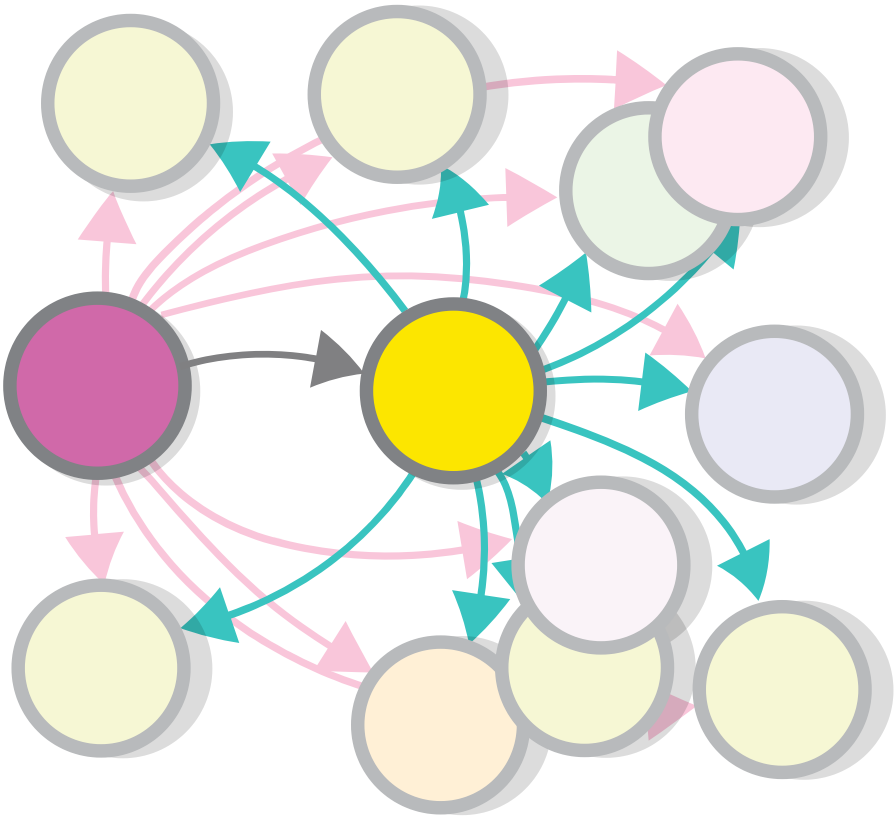
To understand how powerful composition is, consider the following: one set being connected to another means that each function from the second set can be transferred to a corresponding function from the first one.

If we have a function $g : P \to Y$ from set P to set Y, then for every function f from the set Y to any other set, there is a corresponding function $f \circ g$ from the set P to the same set. In other words, every time you define a new

function from Y to some other set, you gain one function from P to that same set for free.



For example, if we take the relationship between a person and his mother as a function called “mother” with the set of all people in the world as source, and the same set as target, then composing this function with itself would yield the function “grandmother”, composing it with the function “sister” would yield the function “aunt”.



And if we keep composing these two functions (as well as their male counterparts “father” and “brother”, we would obtain all possible ancestral relationships.

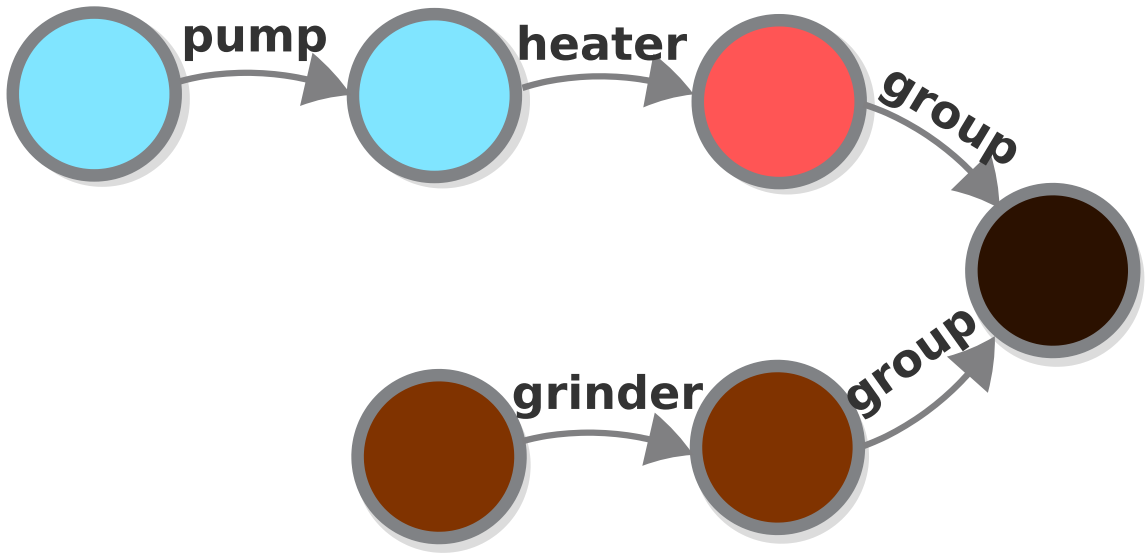
This example highlights an important ability that is enabled to functional composition — the ability to break-down composite relationships to their basic building blocks.

Composition in engineering

Besides being useful for *analyzing* relationships that already exist, the principle of composition enables you to *build* objects that exhibit such relationships (AKA engineering).

The main way in which modern engineering differs from ancient craftsmanship is the concept of a *part/module/component* — a product that performs a given function that is not made to be used directly, but is instead optimized to be combined with other such products in order to form a “end-user” product.

For example, an *espresso machine* is just a combination of the components, such as , *pump*, *heater*, *grinder group* etc, when composed in an appropriate way.

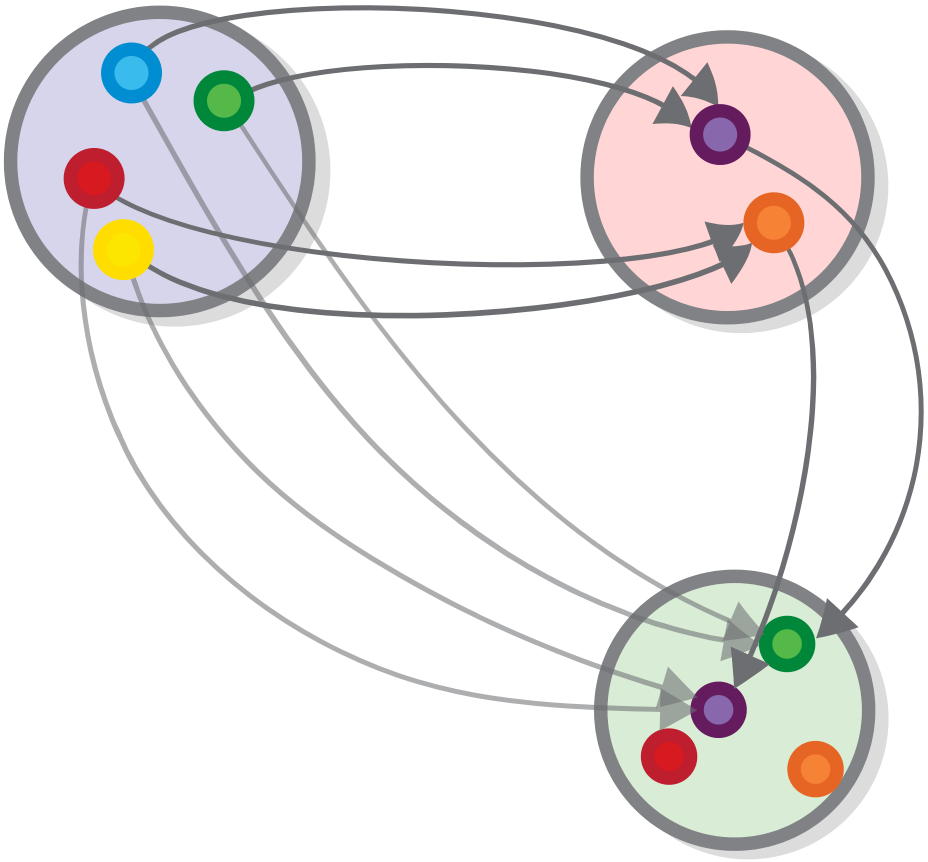


Task 9: Think about what would be those functions' sources and targets.

By the way, diagrams that are "zoomed out" that show functions without showing set elements are called *external diagrams*, as opposed to the ones that we saw before, which are *internal*.

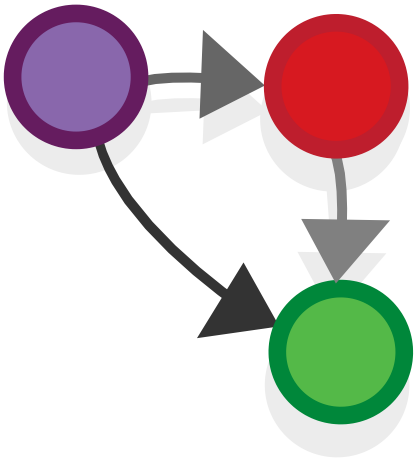
Composition and external diagrams

Let's look at the diagram that demonstrates functional composition in which we showed that successive application of the two composed functions ($f \circ g$) and the new function (h) are equivalent.



We showed this equivalence by drawing an *internal* diagram, and explicitly drawing the elements of the functions' sources and targets in such a way that the two paths are equivalent.

Alternatively, we can just *say* that the arrow paths are all equivalent (all arrows starting from a given set element ultimately lead to the same corresponding element from the resulting set) and draw the equivalence as an external diagram.

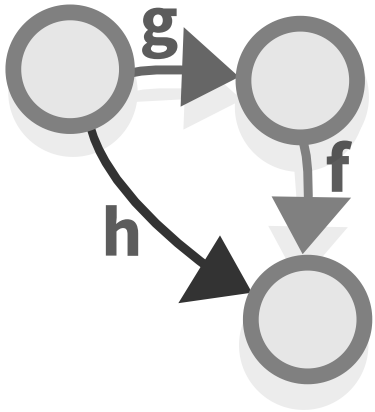


Or alternatively, if you want to express it as a formula.



The external diagram is a more appropriate representation of the concept of composition, as it is more general. In fact, it is *so* general that it can actually serve as a *definition of functional composition*.

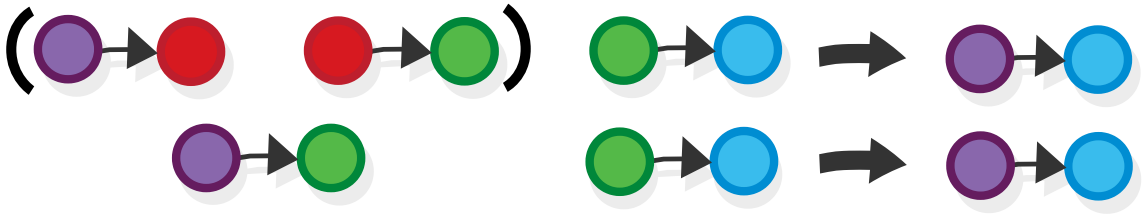
> **DEFINITION:**
>
> The composition of two functions f and g is a third function h defined in such a way that all the paths in this diagram are equivalent.



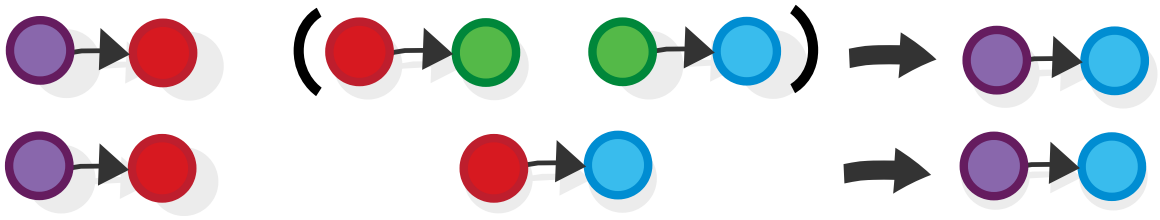
If you continue reading this book, you will hear more about diagrams in which all paths are equivalent (they are called *commuting diagrams*, by the way).

Associativity

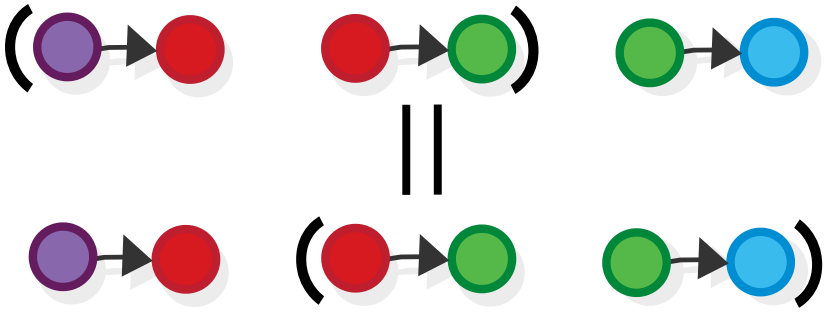
If we want compose more than two functions we might wonder if the order in which we compose the functions matters for the final outcome i.e. whether combining two functions and then combining the result with a third function...



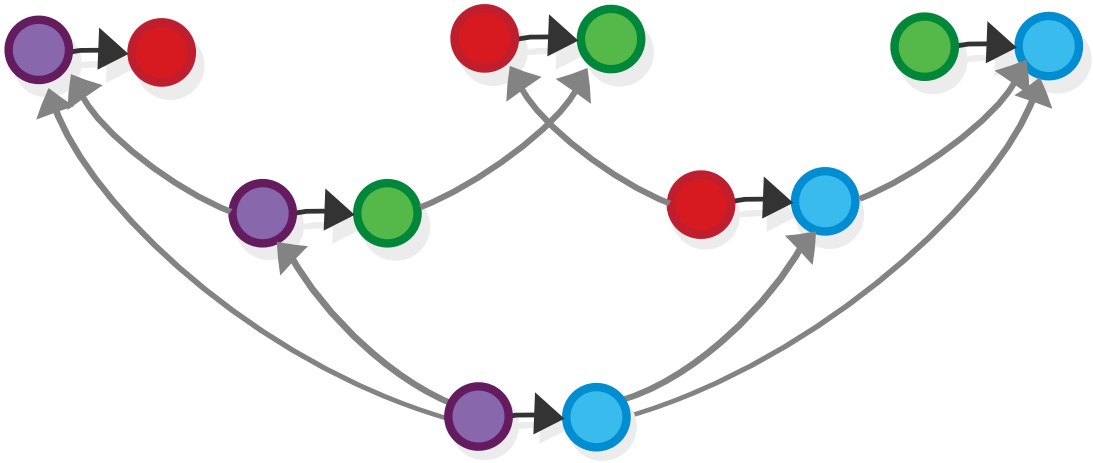
...would yield the same result as composing the second and the third functions, before adding the first one.



The answer is that order of composition doesn't matter — as long as we compose the same functions, the result would always be the same.



i.e. there are many ways to get the same function.



This property of functions is called *associativity*.

> **THEOREM:**
>
> Functional composition is associative i.e., for any functions f g and c with the appropriate type signature $(f \circ g) \circ c$ is the same as $f \circ (g \circ c)$

Task 10: Draw the above diagrams as internal diagrams: define three functions that compose with one another (you can use the two functions that we defined earlier, you only would have to make a third one) compose them in the two ways shown above and check if the result is the same.

Category theory – hints for the definition

At this point you might be worried that I had forgotten that I am supposed to talk about category theory and I am just presenting a bunch of irrelevant concepts. I may indeed do that sometimes, but not right now — the fact that *functional composition* can be presented without even mentioning category theory doesn't stop it from being one of category theory's *most important concepts*.

In fact, we can say (although this is not an official definition) that...

> **DEFINITION:**
>
> Category theory is the study of things that are *function-like* (that compose in an associative way).

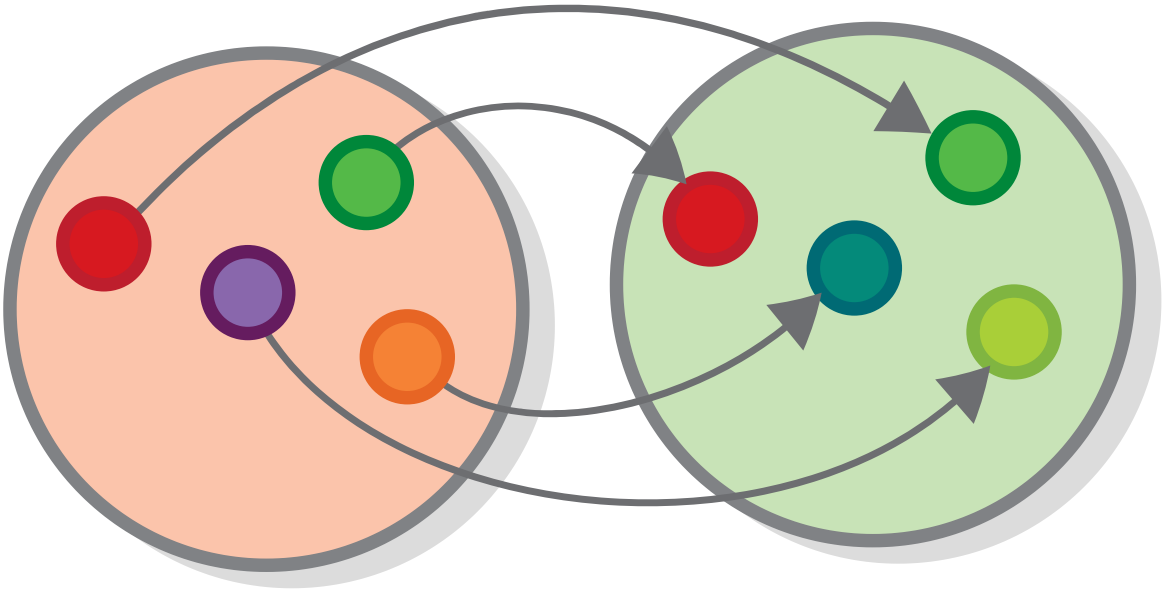
Those things are not necessarily functions, but have a source and a target like functions, they compose with one another like functions (associatively) and they can be represented by external diagrams.

And there is another way of defining category theory, without defining category theory: it is what you get if you replace the concept of equality with the concept of *isomorphism*.

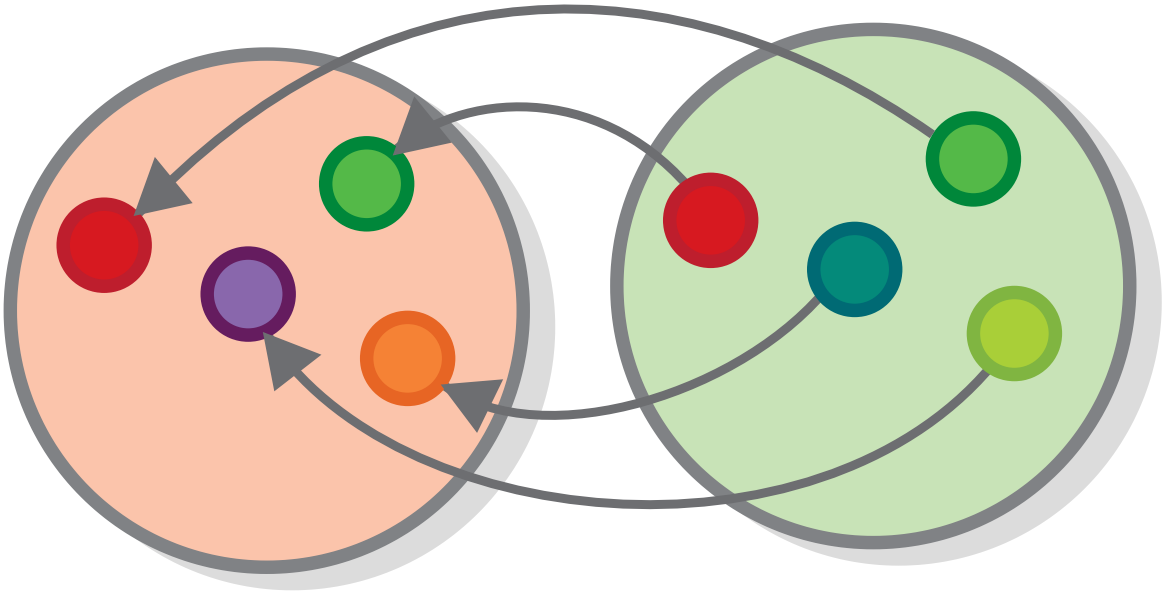
We haven't talked about isomorphisms yet, but this is what we will be doing for the rest of this chapter.

Isomorphism

To explain what isomorphism is, we go back to the examples of the types of relationships that functions can represent, and to the first and most elementary of them all — the *one-to-one* type of relationship. We know that all functions have exactly one element from the source set, pointing to one element from the target set. But for one-to-one functions *the reverse is also true* — exactly one element from the target set points to one element from the source.



If we have a one-to-one-function that connects sets that are of the same size (as is the case here), then this function has the following property: all elements from the target set have exactly one arrow pointing at them. In this case, the function is *invertible*. That is, if you flip the arrows of the function and its source and target, you get another valid function.



Invertible functions are called *isomorphisms*. When there exists an invertible function between two sets, we say that the sets are *isomorphic*. For example, because we have an invertible function that converts the temperature measured in *Celsius* to temperature measured in *Fahrenheit*, and vise versa, we can say that temperatures measured in Celsius and Fahrenheit are isomorphic.

Isomorphism means "same form" in Greek (although actually their form is the only thing which is different between two isomorphic sets).

Now it's time to define isomorphisms formally. One way to do that is the following:

> **DEFINITION:**
>
> (Internal) Two sets A and B are isomorphic (or $A \cong B$) if there exist a one-to-one relationship between their elements.

This definition does not tell us anything about the most important quality of isomorphisms — invertability. So, we will present a different one:

> **DEFINITION:**
>
> (External) Two sets A and B are isomorphic (or $A \cong B$) if there exist functions $f : A \to B$ and its reverse $g : B \to A$, such that $f \circ g = ID_A$ and $g \circ f = ID_B$.

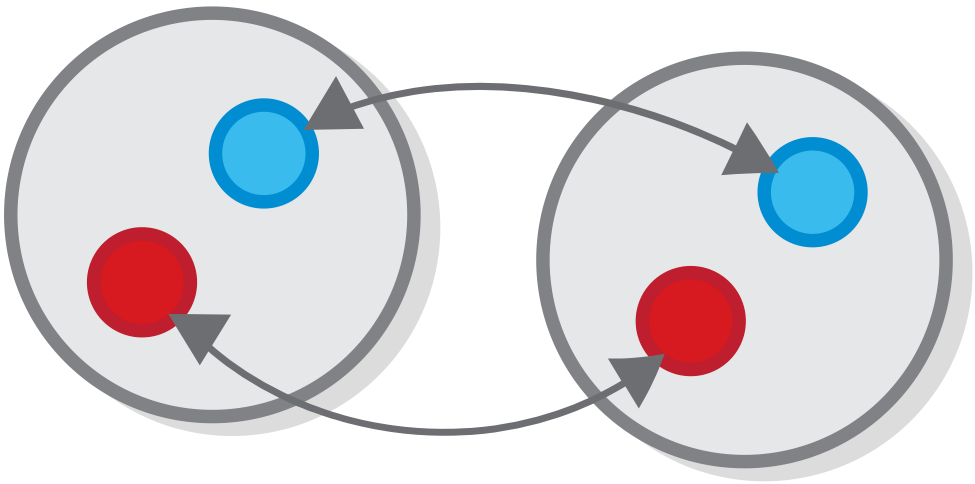
Notice how the identity function comes in handy. In fact, notice that the concept of isomorphism is defined only by using the identity function and functional composition, this is our first completely external, completely *categorical* definition (if you don't count composition itself).

Isomorphism and identity

If you look closely you would see that the identity function is invertible too (its reverse is itself),

> **THEOREM:**
>
> The identity function is an isomorphism.

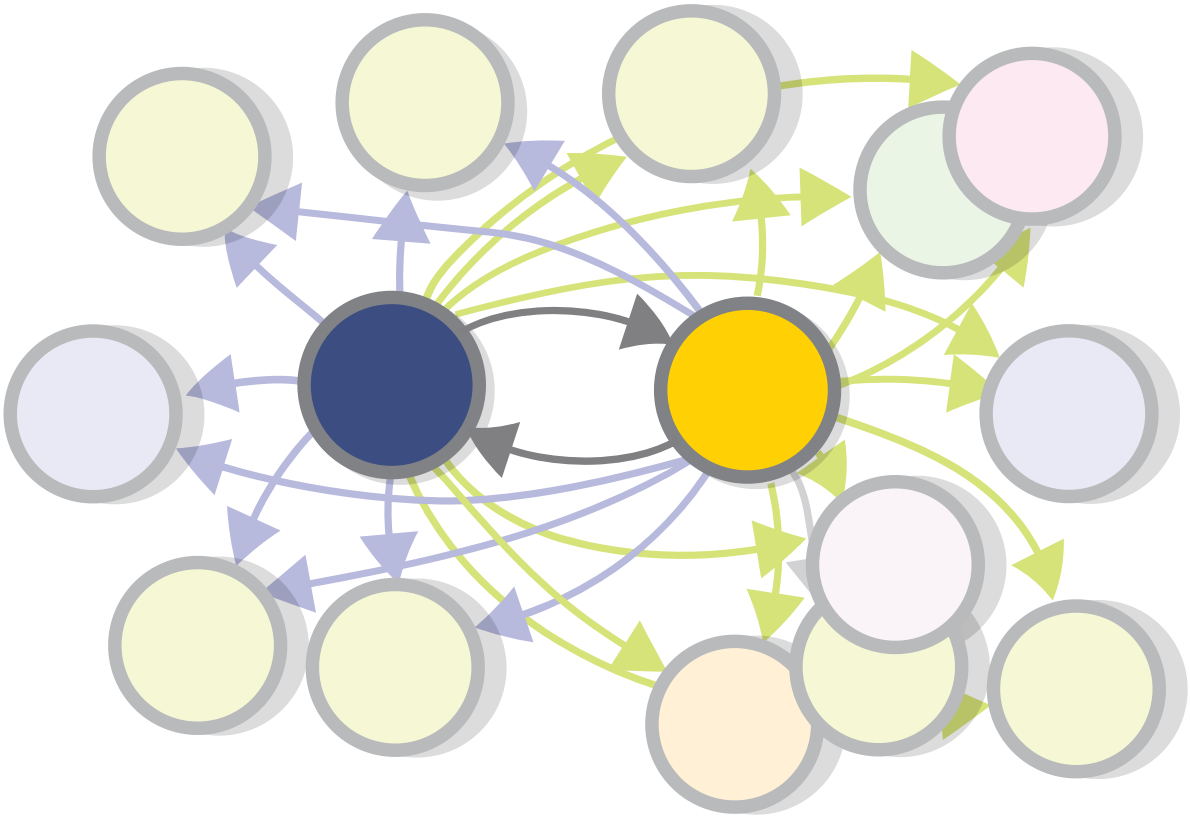
So each set is isomorphic to itself in that way.



So, the concept of an isomorphism contains the concept of equality — all equal things are also isomorphic.

Isomorphism and composition

An interesting fact about isomorphisms is that if we have functions that convert a member of set A to a member of set B, and the other way around, then, because of functional composition, we know that any function from A has a corresponding function from B and the other way around.



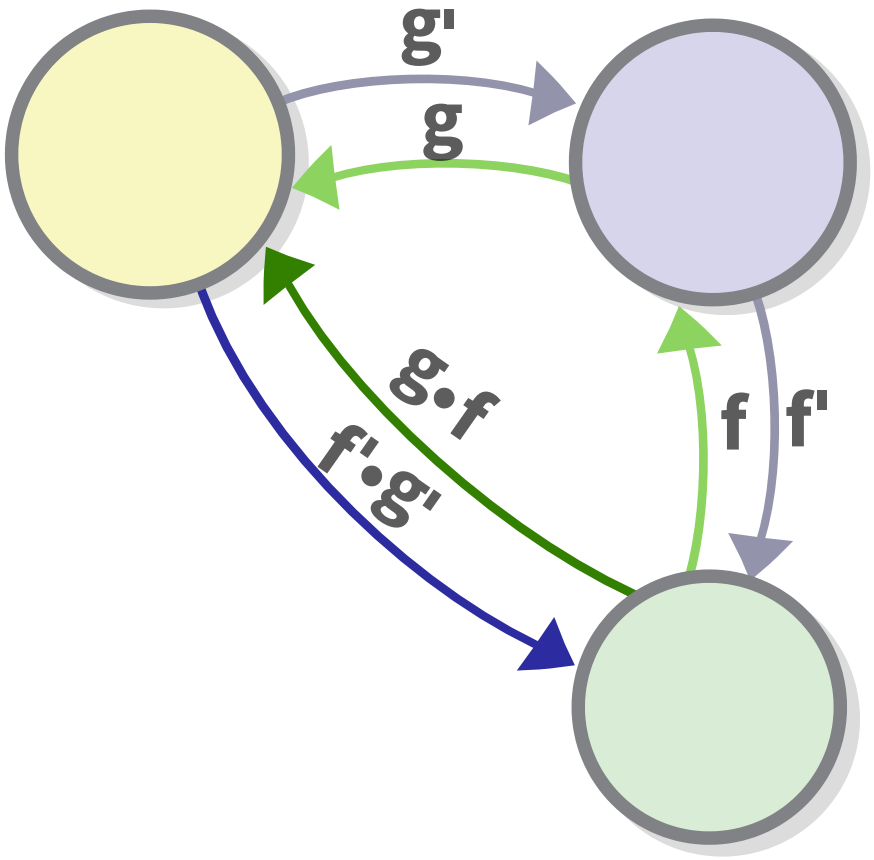
For example, if you have a function “is the partner of” that goes from the set of all married people to the same set, then that function is invertible. That is not to say that you are the same person as your significant other, but rather that every statement about you, or every relation you have to some other person or object is also a relation between them and this person/object, and vice versa.

Composing isomorphisms

Another interesting fact about isomorphisms is that if we have two isomorphisms that have a set in common, then we can obtain a third isomorphism between the other two sets that would be the result of their (the isomorphisms) composition.

> **THEOREM:**
>
> Two sets that are both isomorphic to a third one are isomorphic to one another.

Composing two isomorphisms into another isomorphism is possible by composing the two pairs of functions that make up the isomorphism in the two directions.



Informally, we can see that the two morphisms are indeed reverse to each other and hence form an isomorphism. If we want to prove that fact formally, we will do something like the following:

If two functions are isomorphic, then their composition is equal to an identity function, proving that functions $g \circ f$ and $f' \circ g'$, are isomorphic is equivalent to proving that their composition is equal to identity.

$$g \circ f \circ f' \circ g' = id$$

But we know already that f and f' are isomorphic and hence $f \circ f' = id$, so the above formula is equivalent to (you can reference the diagram to see what that means):

$$g \circ id \circ g' = id$$

And we know that anything composed with id is equal to itself, so it is equivalent to:

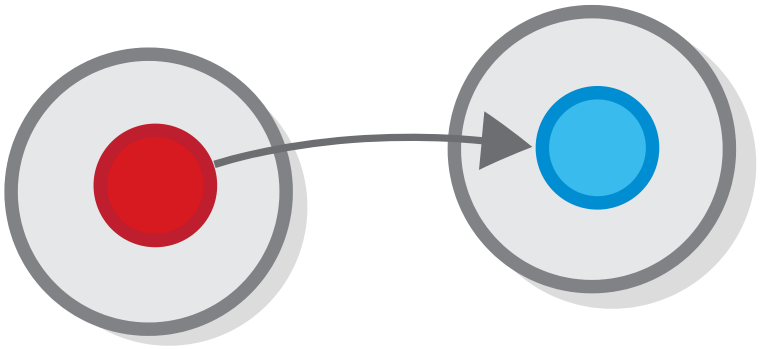
$$g \circ g' = id$$

which is true, because g and g' are isomorphic and isomorphic functions composed are equal to identity.

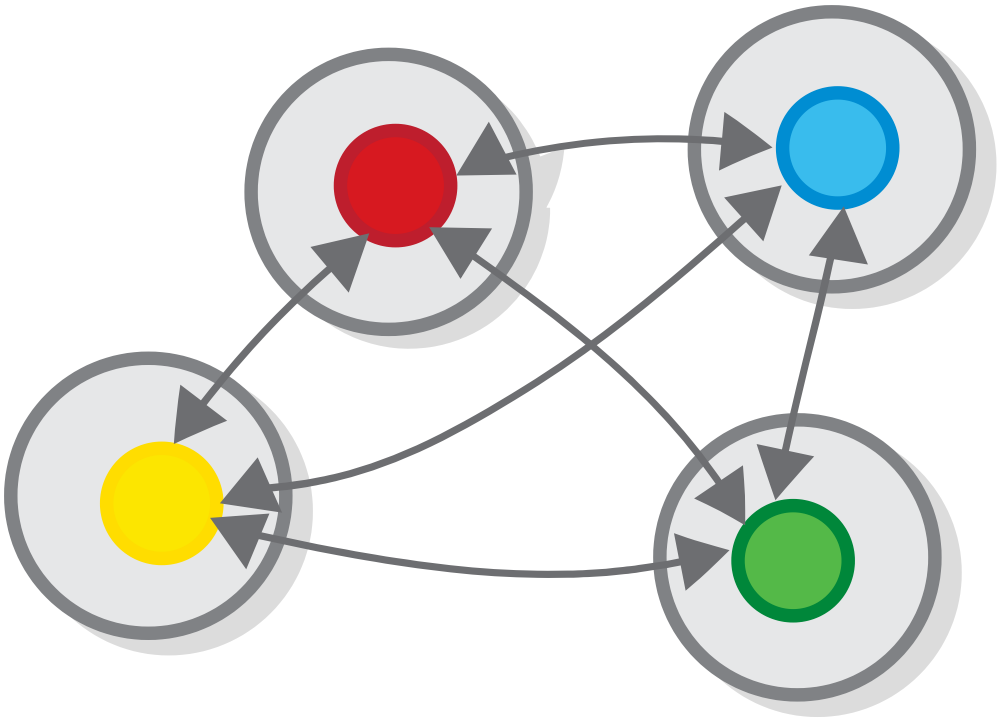
By the way, there is another way to obtain the isomorphism — by composing the two morphisms one way in order to get the third function and then taking its reverse. But to do this, we have to prove that the function we get from composing two isomorphisms is also an isomorphism.

Isomorphisms Between Singleton Sets

Between any two singleton sets, we may define the only possible function.



The function is invertible, which means that all singleton sets are isomorphic to one another, and furthermore (which is important) they are isomorphic *in one unique way*.



Isomorphisms and equivalence

We said that isomorphic sets aren't necessarily the same set (although the reverse is true). However, it is hard to get away from the notion that being isomorphic means that they are *equal* or *equivalent* in some respect. For example, all people who are connected by the *isomorphic* mother/child relationship share some of the same genes.

And in computer science, if we have functions that convert an object of type A to an object of type B and the other way around (as for example the functions between a data structure and its id), we also can pretty much regard A and B as two formats of the same thing, as having one means that we can easily obtain the other.

Equivalence relations

What does it mean for two things to be equivalent? The question sounds quite philosophical, but there is actually is a formal way to answer it, i.e., there is a mathematical concept that captures the concept of equality in a rather elegant way — the concept of an *equivalence relation*.

So what is an equivalence relation? We already know what a relation is — it is a connection between two sets (an example of which is function). But when is a relation an equivalence relation? Well, according the definition, it's when it follows three laws, which correspond to three intuitive ideas about equality.

> **DEFINITION:**
>
> An equivalence relation between sets is a relation that obeys the laws of reflexivity, transitivity, and symmetry.

Let's review them.

Reflexivity

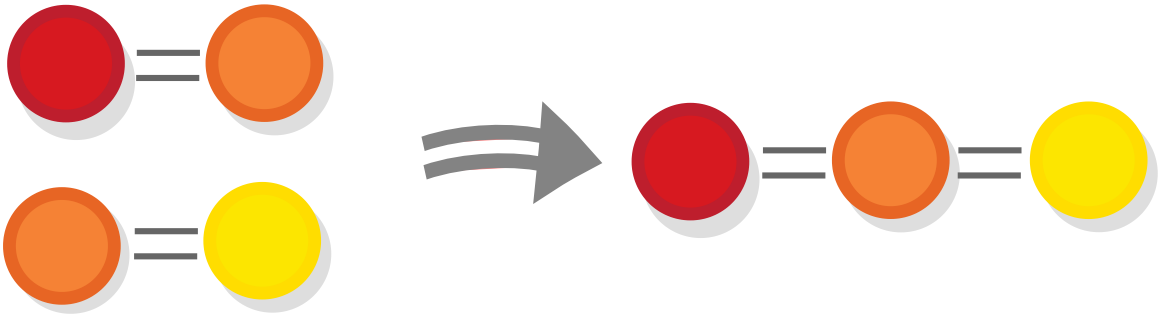
The first idea that defines equivalence, is that *everything is equivalent with itself*.



This simple principle translates to the equally simple law of *reflexivity*: for all sets A, $A = A$.

Transitivity

According to the Christian theology of the Holy Trinity, the Jesus' Father is God, Jesus is God, and the Holy Spirit is also God, however, the Father is not the same person as Jesus (neither is Jesus the Holy Spirit). If this seems weird to you, that's because it breaks the second law of equivalence relations, transitivity. Transitivity is the idea that things that are both equal to a third thing must also equal between themselves.

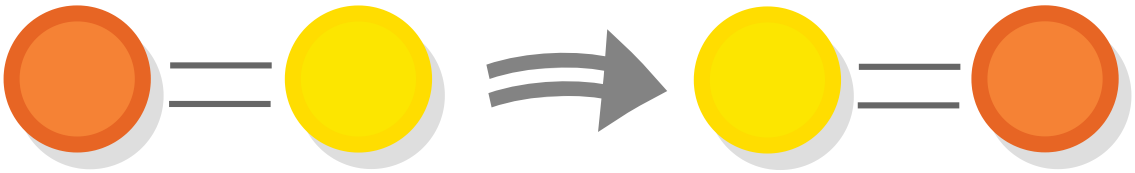


Mathematically, for all sets A B and C, if $A = B$ and $B = C$ then $A = C$.

Note that we don't need to define what happens in similar situations that involve more than three sets, as they can be settled by just multiple application of this same law.

Symmetry

If one thing is equal to another, the reverse is also true, i.e, the other thing is also equal to the first one. This idea is called *symmetry*. Symmetry is probably the most characteristic property of the equivalence relation, which is not true for almost any other relation.



In mathematical terms: if $A = B$ then $B = A$.

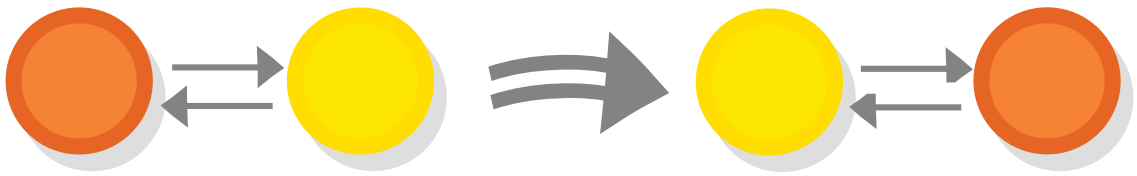
Isomorphisms as equivalence relations

You probably suspect that...

> **THEOREM:**
>
> Isomorphisms are equivalence relations

Isomorphisms *are* indeed equivalence relations. And "incidentally", we already have all the information needed to prove it (in the same way in which James Bond seems to always incidentally have exactly the gadgets that are needed to complete his mission).

We said that the most characteristic property of the equivalence relation is its *symmetry*. And this property is satisfied by isomorphisms, due to the isomorphisms' most characteristic property, namely the fact that they are *invertible*.



Task 11: One law down, two to go: Go through the previous section and verify that isomorphisms also satisfy the other equivalence relation laws.

What I am trying to say with all this is that it *makes sense* to treat any isomorphism as equality. For this reason, the practice of using isomorphisms to define an equivalence relation is very prominent in category theory where isomorphisms are denoted with $\cong$, which is almost the same as the way equality is denoted $=$ (note that the sign is also similar to two parallel arrows connecting one set to the other).

From Sets to Categories

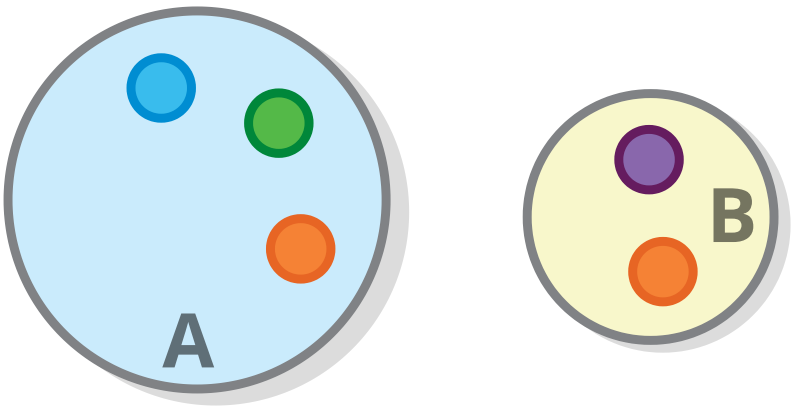
In this chapter, we will see some more set-theoretic constructs, but we will also introduce their category-theoretic counterparts in an effort to gently introduce the concept of a category itself.

When we are finished with that, we will try (and almost succeed) to define categories from scratch, without actually relying on set theory.

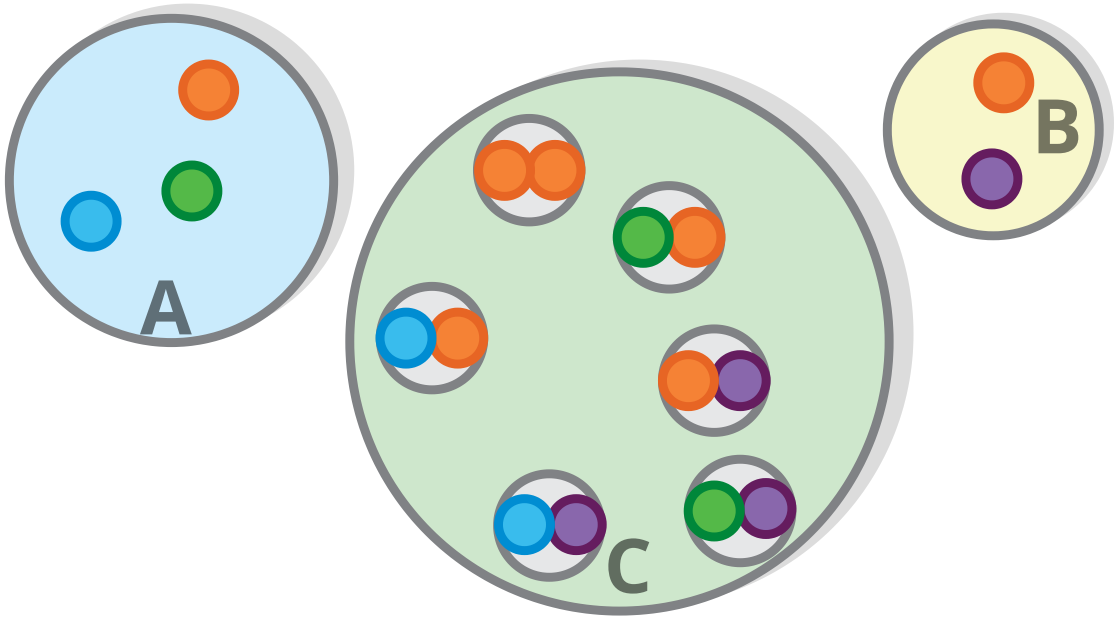
Products

In the previous chapter, we needed a way to construct a set whose elements are *composite* of the elements of some other sets e.g. when we discussed mathematical functions, we couldn't define $+$ and $-$ because we could only formulate functions that take one argument. Similarly, when we introduced the primitive types in programming languages, like `Char` and `Number`, we mentioned that most of the types that we actually use are *composite* types. So how do we construct those?

So, consider a set A (containing a's) and a set B (containing B's)



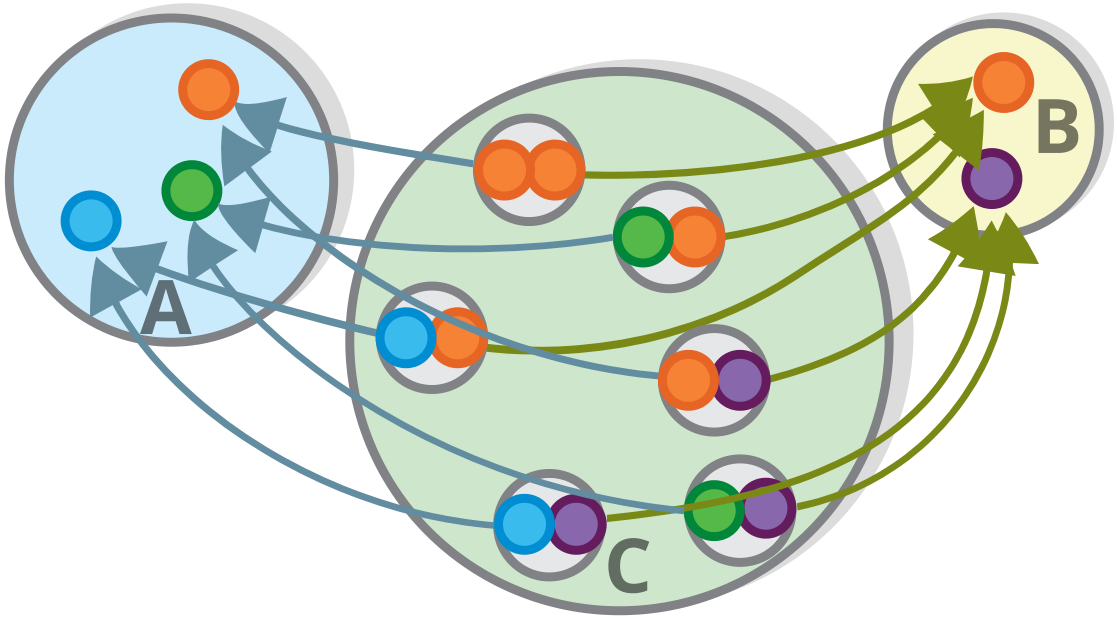
We introduce a new set that combines those two sets into one, their product set.



> **DEFINITION:**
>
> The *Cartesian product* (or *tuple*) of sets A and B (denoted $A \times B$) is the set of *ordered pairs* that contain one element of the set A and one element of the set B. Or formally speaking: $A \times B = (a, b)$ where $a \in A, b \in B$ ($\in$ means "is an element of").

Task 1: Why is this called a product? Hint: How many elements does it have?

Naturally, the product comes equipped with two functions, one for each property, which allow you to take a pair and extracts the value of the property,

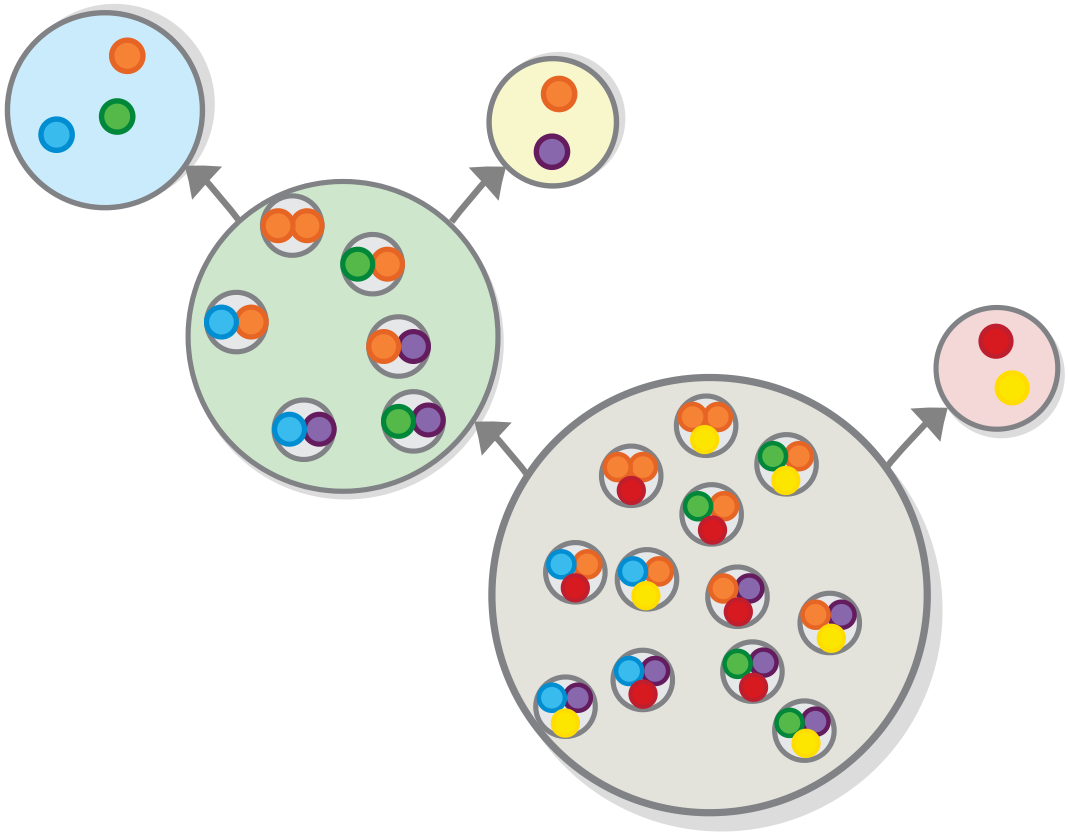


> **THEOREM:** For each set C, that is a product of A and B, there are two functions $C \to A$ and $C \to B$, called the product's *projections* that retrieve back its (the product's) constituent values).

(in programming terms, we would dub these the "getters")

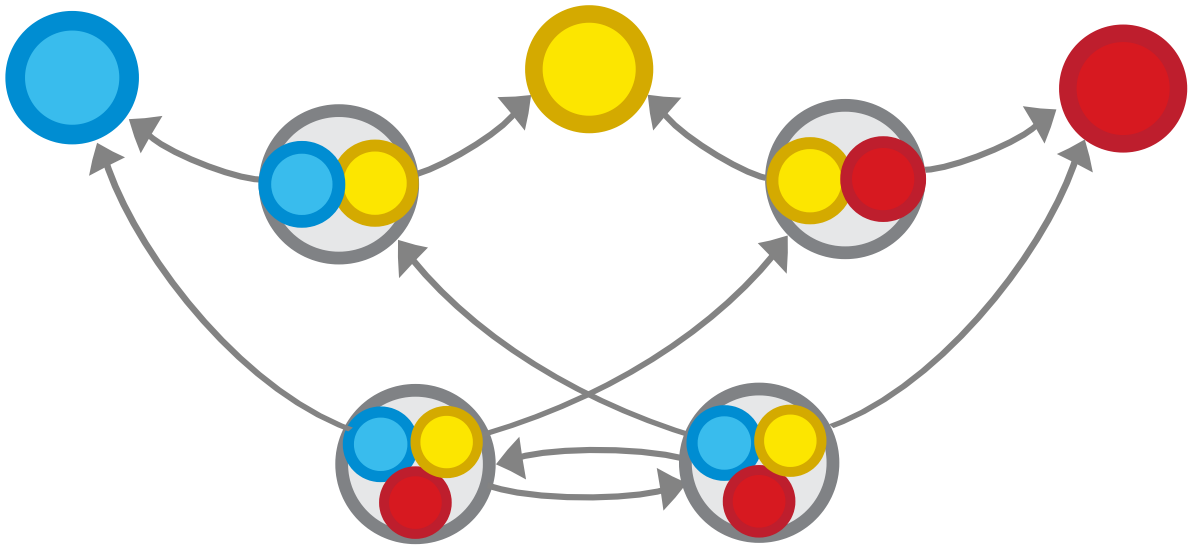
Triple product

There are occasions where we want to combine not two, but three sets into a product (e.g. $A \times B \times C$). But we don't need to define the concept of triple product separately: we can achieve it by combining the first and second one into a product and then combining their product with the third set, (so it will be $(A \times B) \times C$.



There is another way to make a triple product of three sets – combining the second and the third one and then combining the result with the first one (so $A \times (B \times C)$, but it doesn't actually matter which one you use – if we view isomorphic sets as equal, the end results would be the same i.e.

> **THEOREM:**
>
> The two ways of combining three sets into a triple product are isomorphic, $(A \times B) \times C \cong A \times (B \times C)$.



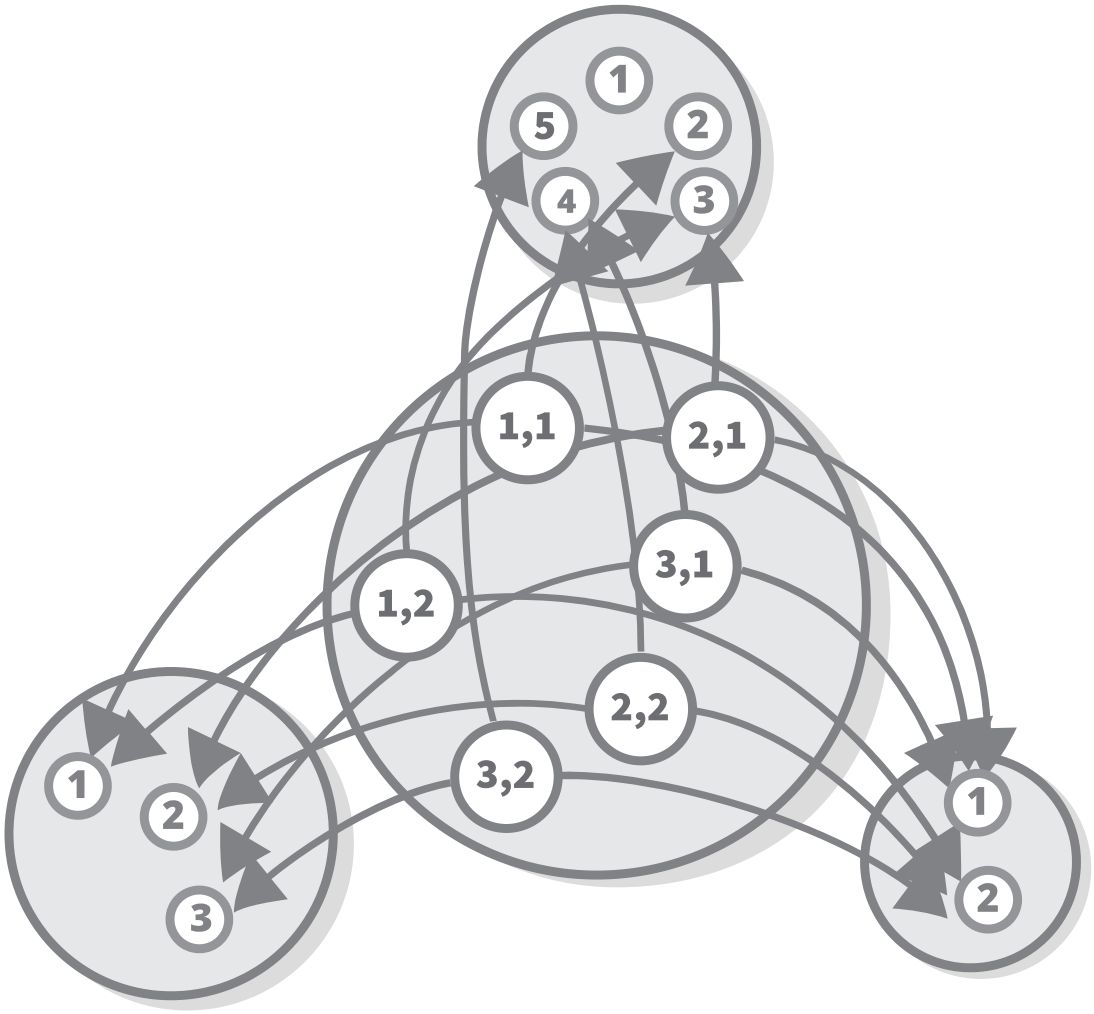
You might recognize this diagram from the section on functional composition. It means that the cartesian product operation is (like functional composition), *associative*.

Products as Objects

In the previous chapter, we established the correspondence of various concepts in programming languages and set theory — sets resemble types, and functions resemble methods/subroutines. This picture is made complete with products, that are like stripped-down *classes* (also called *records* or *structs*) — the sets that form the product correspond to the class's *properties* (also called *members*) and the functions for accessing them are like what programmers call *getter methods* e.g. the famous example of object-oriented programming of a `Person` class with `name` and `age` fields is nothing more than a product of the set of strings, and the sets of numbers. And, as we showed, objects with more than two values can be expressed as compositions of nested products.

Using Products to define Numeric Operations

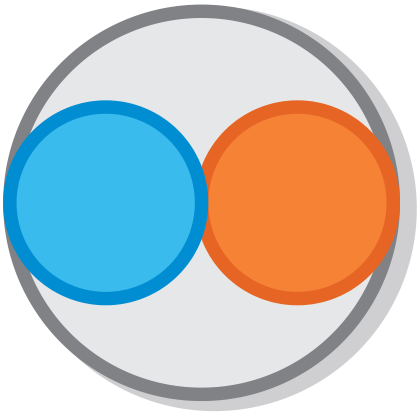
Products can also be used for expressing functions that take more than one argument (and this is indeed how multi-param functions are implemented in some languages, like the ones from the ML family). For example, "plus" is a function from the set of products of two numbers to the set of numbers, so, $+ : \mathbb{Z} \times \mathbb{Z} \to \mathbb{Z}$.



By the way, such functions (ones that take two objects of one type and return a third object of the same type) are called *operations*.

Defining products in terms of sets–Internal definition

A product is, as we said, a set of *ordered* pairs (formally speaking $A \times B \neq B \times A$). So, to define a product we must define the concept of an ordered pair. So how can we do that?



Note that an ordered pair of elements is not just a set containing the two elements (that would be an *unordered pair*)

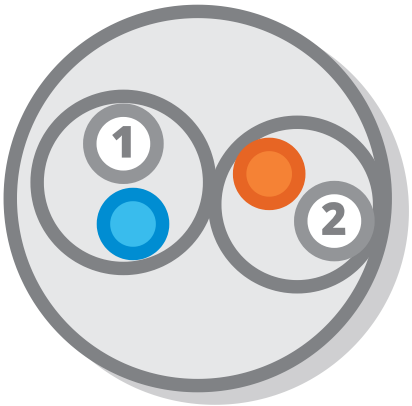


but it also contains information about which of those objects comes first and which one goes second in the pair—some mathematical operations (such as addition) don't care about order, others (such as subtraction) do. And in programming, we have the ability to assign names to each property of an object, which accomplishes the same purpose—allows us to access a specific property of the object, not just any random property.

So, if an ordered pair isn't a set, does that mean that we have to define it as a "primitive" type like we defined sets if we want to use them? That's possible, but there is another approach. We can define a construct that is *isomorphic* to the ordered pair, using only sets. And mathematicians have come up with multiple ingenious ways to do that. Here is the first one, which was suggested by Norbert Wiener in 1914. Note the smart use of the fact that the empty set is unique.



The next one was suggested by Felix Hausdorff in the same year. In order to use that one, we just have to define 1, and 2 first.



Suggested in 1921 by Kazimierz Kuratowski, this one uses just the component of the pair.



All of these definitions work by *zooming in* into the individual elements of the product. We may think of this as a *low-level* approach to the definition, one which which focuses on the product's *internal* structure. But, more interesting, at least for category theory, is the *high-level* approach — instead of zooming in

we *zoom out*, we stay completely oblivious to the contents of our sets and focus only on the functions that are associated with the product.

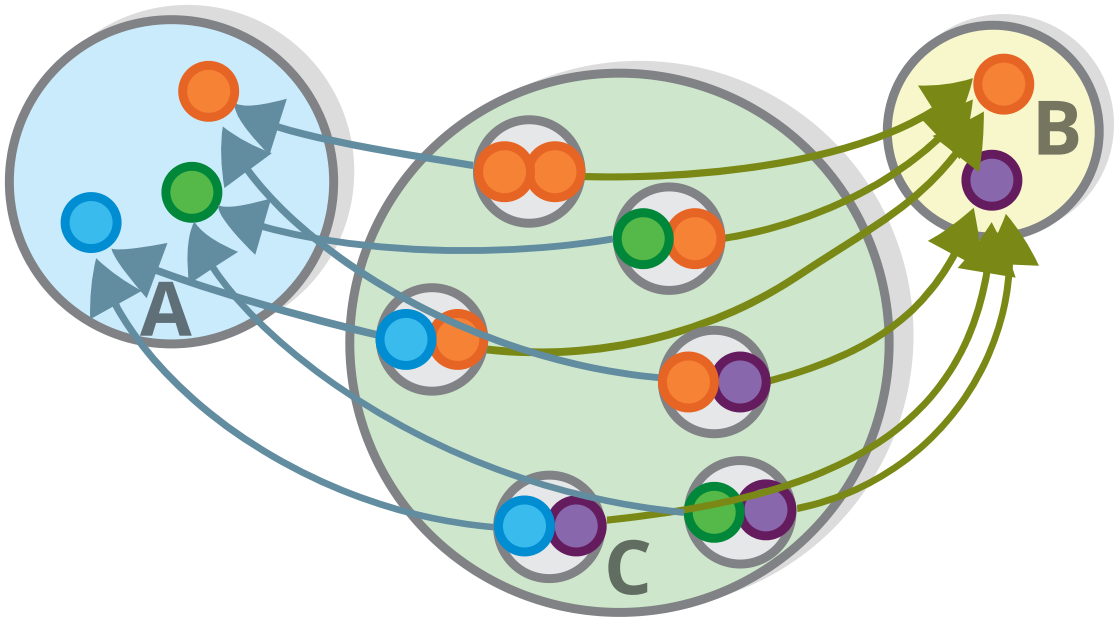
Defining products in terms of functions- external definition

Now, we will look into a category-theoretic definition of the product set. We call this definition *external* because it is based not on the internal structure that the object has, but on it's external behavior (which is defined by the functions that come from and go to it). And because it is strongly related to external diagrams.

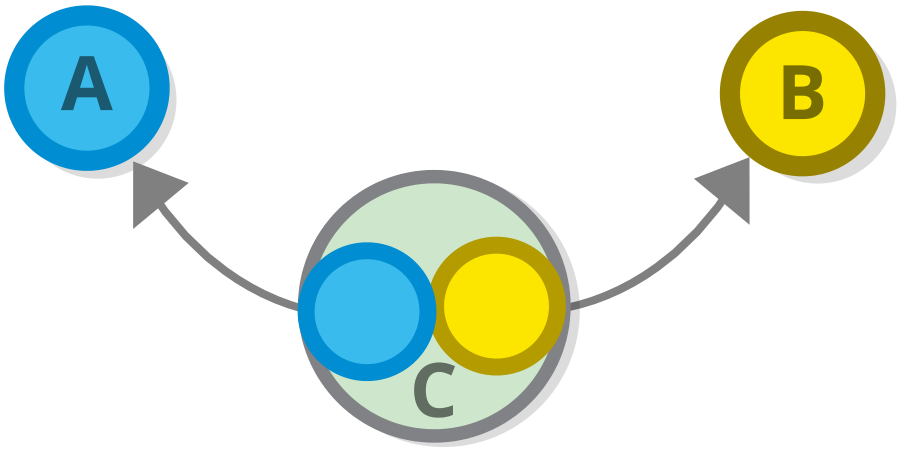
Such definitions are driven by a conceptual model of the object we want to define. For example, we can agree that a product of A and B is some sort of combination that contains A and B (and nothing more).

Now, based on that conceptual model, we must, given two sets, devise a way to pinpoint the set that is their product, by looking at the functions that come from/to them.

So, we said that a product of A and B contains an A and B. So, what are the functions that can fulfils these criteria? Of course that would be the projections, the functions for retrieving back the two elements: $A \times B \to A$ and $A \times B \to B$.



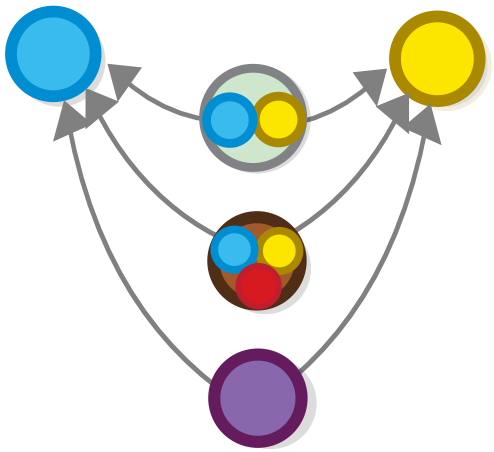
Now if we switch to the (semi) external view, this diagram already provides some definition of what a product is:



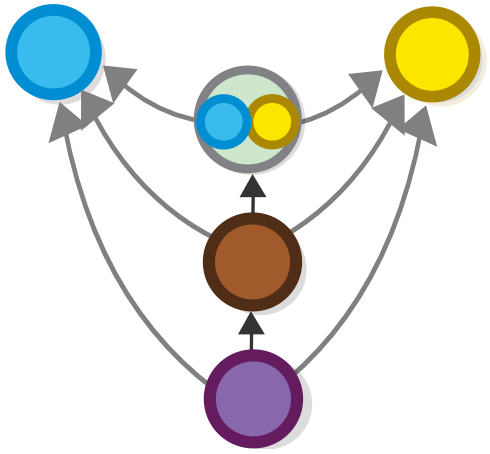
> **DEFINITION:**
>
> The product of A and B, denoted $A \times B$, is a set such that:
>
> 1. There exist two "projection" functions $A \times B \to A$ and $A \times B \to B$...

In other words, if we have a set C for which there are functions $C \to A$ and $A \times B \to B$, then C can potentially be equal to $A \times B$.

However, this definition is not complete, as the product A and B, is not the *only* set for which such functions can be defined. For example, a set of triples, that we already examined (or the triple product) $A \times B \times X$ for any element X also qualifies. Any other set that would happen to have some functions to A and B, and would, by this definition, be "impostor product".



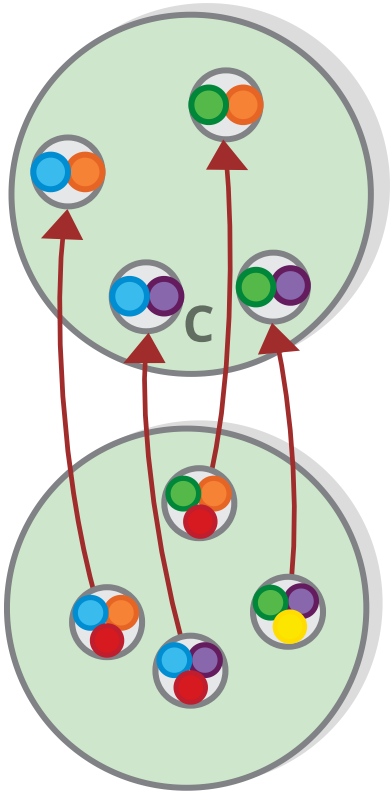
To expose those impostors, we go back to our conceptual definition. Remember that we said $A \times B$ contains an element of A, an element of B *and nothing more*. This tells us that each of these impostors I can be *converted* to $A \times B$, i.e. that there is an arrow $I \to A \times B$. Why? As we said, all such sets would be *more complex* than the product. And you can always have a function that converts a more complex structure to a simpler one by just throwing information away.



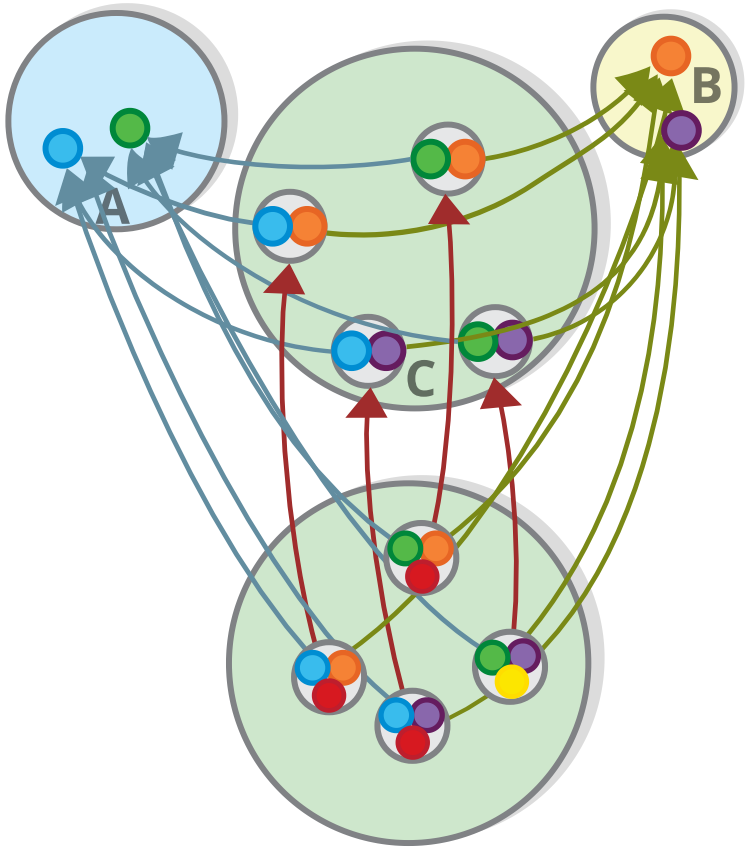
We can know that this arrow would exist for every product because any element of the impostor set I, containing an element of A, an element of B *and something more*, there exist an element of the set $A \times B$ that contains the same element of A and the same element of B (and nothing more). So, we can define a function $I \to A \times B$, that throws away that extra information.

And even more interestingly, the projection functions (for retrieving the elements) $I \to A$ and $I \to B$, because of which I is an impostor, can be defined in terms of this function $I \to A \times B$.

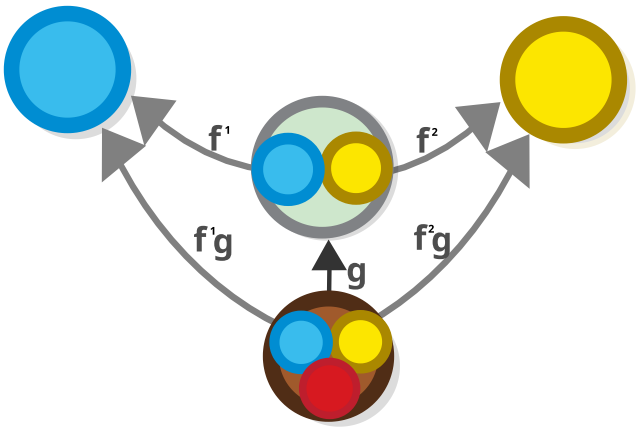
As an example, take the set of triples, $A \times B \times X$ for any X. The canonical function that converts it to a product $A \times B \times X \to A \times B$ is the function that just removes the third element X.



And we can see that the projections can be easily defined using this function.



That is, if we dub this function $g : A \times B \times X \to A \times B$ and let f^1 and f^2 be the projections of the product ($f^1 : A \times B \to A$ and $f^2 : A \times B \to B$), then, the arrow that connects the triple $A \times B \times X$ to A and B are just the compositions $f^1 \circ g$ and $f^2 \circ g$. It is almost as if $A \times B \times X$ *is only connected to* A *and* B *because of this function*.



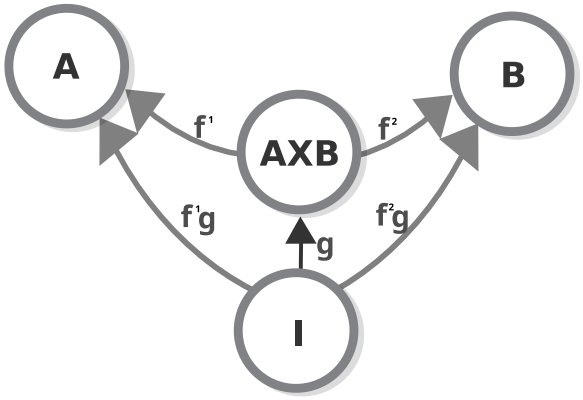
More formally, we can define the product in the following way.

> **DEFINITION:**
>
> The product of A and B, denoted $A \times B$, is a set such that:
>
> 1. There exist two "projection" functions $f^1 : A \times B \to A$ and $f^2 : A \times B \to B$.
> 2. For any impostor product I, that also has such projection functions ($I \to A$ and $I \to B$), there must also exist a unique function (called universal morphism) with the type signature $g : I \to A \times B$, that converts the impostor product to the real product, such that the projections of the impostor would be just the composition of g with the projections of the product i.e. $f^1 \circ g : I \to A$ and $f^2 \circ g : I \to B$.

We prove that a given set is a product by giving a formula for the function g, such that it fits our criteria. Given functions $g^1 : I \to A$ and $g^2 : I \to B$, the function g would be just the function that makes up a pair of the results of those two functions, so if i is an element of I, then $g = (i) \to (g^1(i), g^2(i))$.

So, the function g exist for every object I.

> **DEFINITION:**

> The product of A and B, denoted $A \times B$, defined in such a way that all the paths in this diagram are equivalent, for all objects I, that are connected to $A \times B$.
>
> 
>

You would see a lot of similar definitions and diagrams in this book. In category theory, we often (always) define properties that a given object might possess, by defining a structure such that all similar objects can be converted to it. This is what we call a *universal property*, but it is too early to go into more detail, (after all we haven't even yet said what category theory is).

Isomorphism and equality

If we remember the three definitions of products in terms of sets and set structure, that we saw earlier and try to determine which of them is the "real" product, which is defined by the universal property, we will see that they *all* are the real product. This is because they are all isomorphic to one another. This is OK: when we represent things using universal properties, isomorphism is treated as equality.

> **THEOREM:**
>
> If a set C satisfies a given universal property (such as being the product of A and B), then any set isomorphic to C' would also satisfy it.

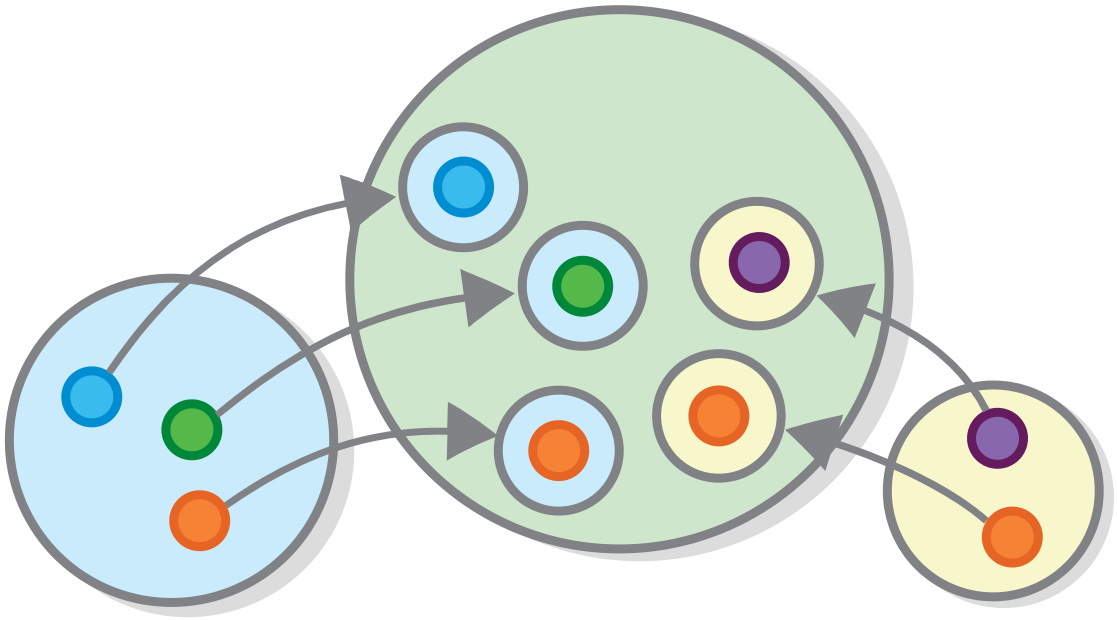
(This is so, because you can easily construct the universal morphism from C' from the universal morphism of C).

We say that the product of two sets is "unique up to *an isomorphism*. This is a shorthand for "there are actually more than one of it, but they are all isomorphic to each other, so we don't care".

This is the same viewpoint that we often adopt in programming, especially when we work on the higher level: although there might be many different implementations list or a pair, or many different formats in which a given data can be stored, as long as we have a way to convert one to the other (and vice versa they are all the same to us.

Sums

We will now study a construct that is pretty similar to the product but at the same time is very different. Similar because, like the product, it is a relation between two sets which allows you to unite them into one, without erasing their structure. But different as it encodes a very different type of relation — a product encodes an *and* relation between two sets, while the sum encodes an *or* relation.



> **DEFINITION:**
>
> The sum of two sets A and B, denoted $A + B$ is a set that contains all elements from A combined with all elements from B.

Defining sums in terms of sets–internal definition

As with the product, representing sums in terms of sets is not so straightforward e.g. when a given object is an element of both sets, then it appears in the sum twice which is not permitted, because a set cannot contain the same element twice.

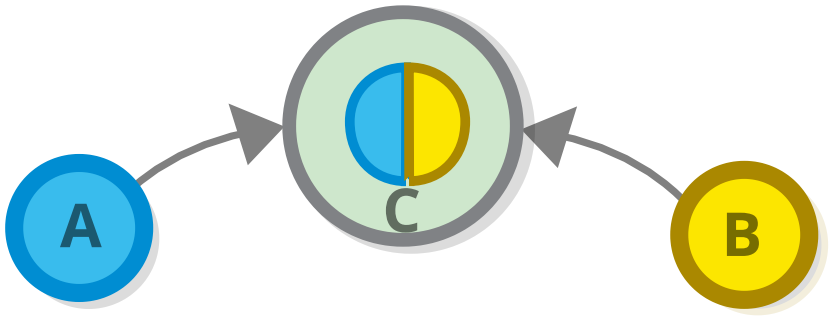
And, as with the product, there is a low-level way to express a sum using sets alone. Incidentally, we can use pairs.



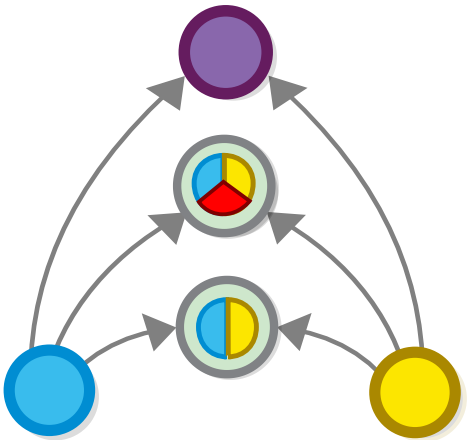
Defining sums in terms of functions–external definition

As you might already suspect, the interesting part is expressing the sum of two sets using functions. To do that, we have to go back to the conceptual part of the definition. We said that sums express an *or* relation between two things.

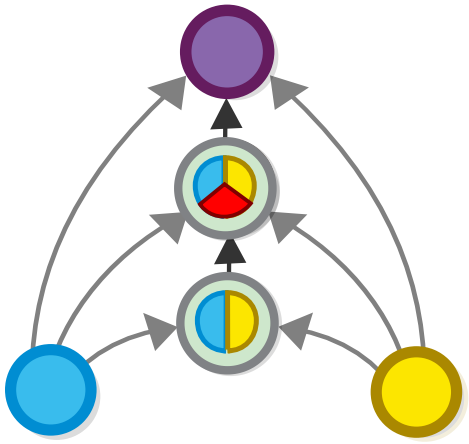
A property of every *or* relation is that if something is an A that something is also an $A \vee B$ (The $\vee$ symbol means *or* by the way). For example, if my hair is *brown*, then my hair is also *either blond or brown*. This is what *or* means, right? This property can be expressed as a function, two functions actually – one for each set that takes part in the sum relation (for example, if parents are either mothers or fathers, then there surely exist functions $mothers \to parents$ and $fathers \to parents$).



As you might have already noticed, this definition is pretty similar to the definition of the product from the previous section — the difference being reversed arrows. And the similarities don't end here. As with products, we have sets that can be thought of as *impostor* sums — ones for which these functions exist, but which also contain additional information.



All these sets express relationships which are more vague than the simple sum, and therefore given such a set, there would exist a unique function that would distinguish it from the true sum. The only difference is that, unlike the functions that define products, this time this function goes *from the sum* to the impostor.

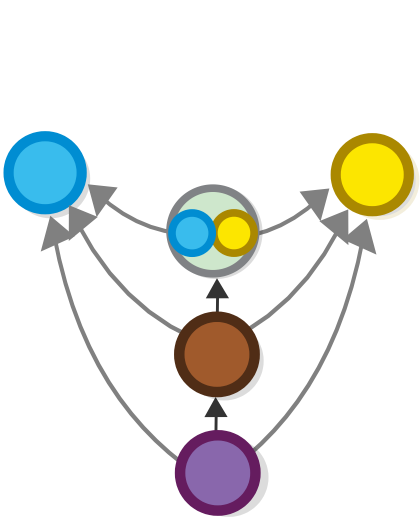


Here is the definition:

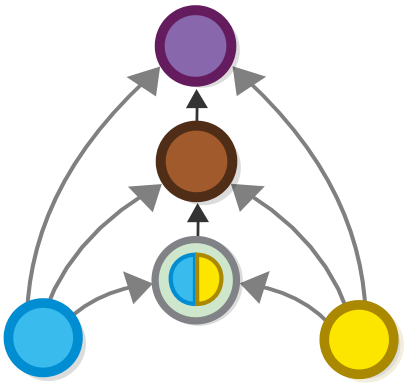
> **DEFINITION:**
>
> The sum of A and B, denoted $A + B$, is a set, such that:
>
> 1. There exists two "projection" functions $A \to A + B$ and $B \to A + B$.
> 2. For any impostor sum I, that also has such projection functions ($A \to I$ and $B \to I$), there must also exist a unique function with the type signature $g : A + B \to I$, that converts the real sum to the impostor sum, such that the projections of the impostor sum be just the composition of g with the projections.

Interlude: Categorical Duality

The concepts of product and sum might already look similar in a way when we view them through their internal diagrams. The *external* view makes this similarity precise — these two diagrams are one and the same diagram, only their arrows are flipped — many-to-one relationships become one-to-many and the other way around.



PRODUCT



COPRODUCT

The universal properties that define the two constructs are the same as well – if we have a sum $A + B$, for each impostor sum, such as $A + B + X$, there exists a trivial function $A + B \to A + B + R$.

And, if you remember, with products the arrows go the other way around – the equivalent example for a product would be the function $A \times B \times R \to A \times B$

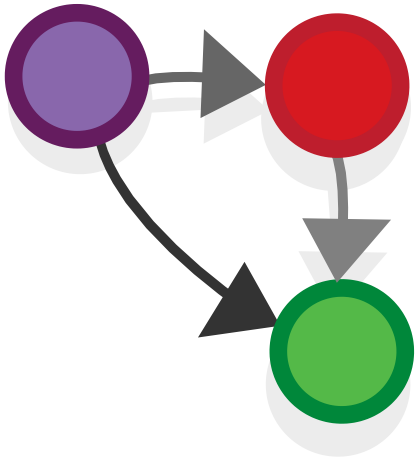
This fact uncovers a deep connection between the concepts of the *product* and *sum*, which is not otherwise apparent – they are each other's opposites. *Product* is the opposite of *sum* and *sum* is the opposite of *product*.

In category theory, concepts that have such a relationship are said to be *dual* to each other. So, the concepts of *product* and *sum* are dual. This is why sums are known in a category-theoretic setting as *converse product*, or *coproduct* for short. This naming convention is used for all dual constructs in category theory.

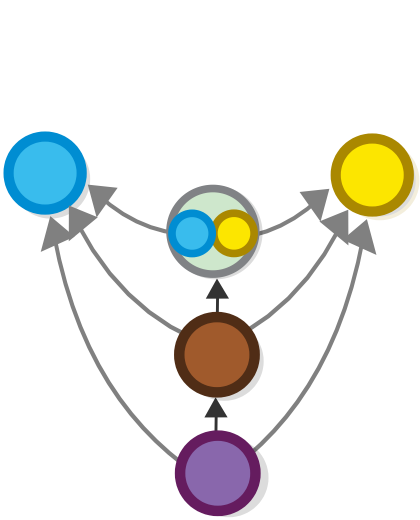
Defining the rest of set theory ~~externally~~ categorically

So far in the book, we saw some amazing ways of defining set-theoretic constructs without looking at the set elements and by only using external diagrams.

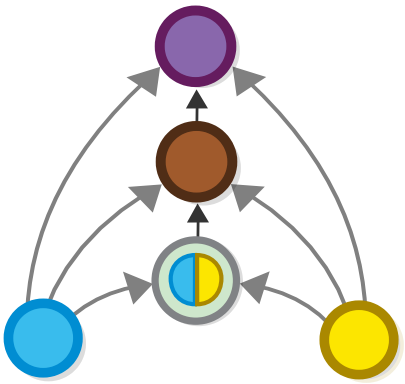
In the first chapter, we defined functions and functional composition with this diagram.



And now, we also defined products and sums.



PRODUCT



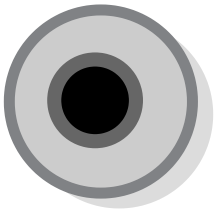
COPRODUCT

What's even more amazing, is that we can define *all of set-theory*, based just on the concept of functions, as discovered by the category theory pioneer Francis William Lawvere.

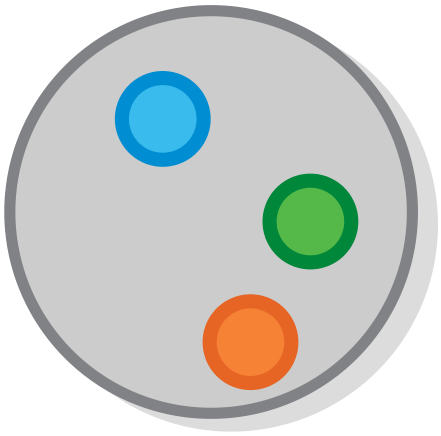
Defining set elements externally

Traditionally, everything in set theory is defined in terms of two things: *sets* and *elements*, so, if we want to define it using *sets* and *functions*, we must define the concept of a *set element* in terms of functions.

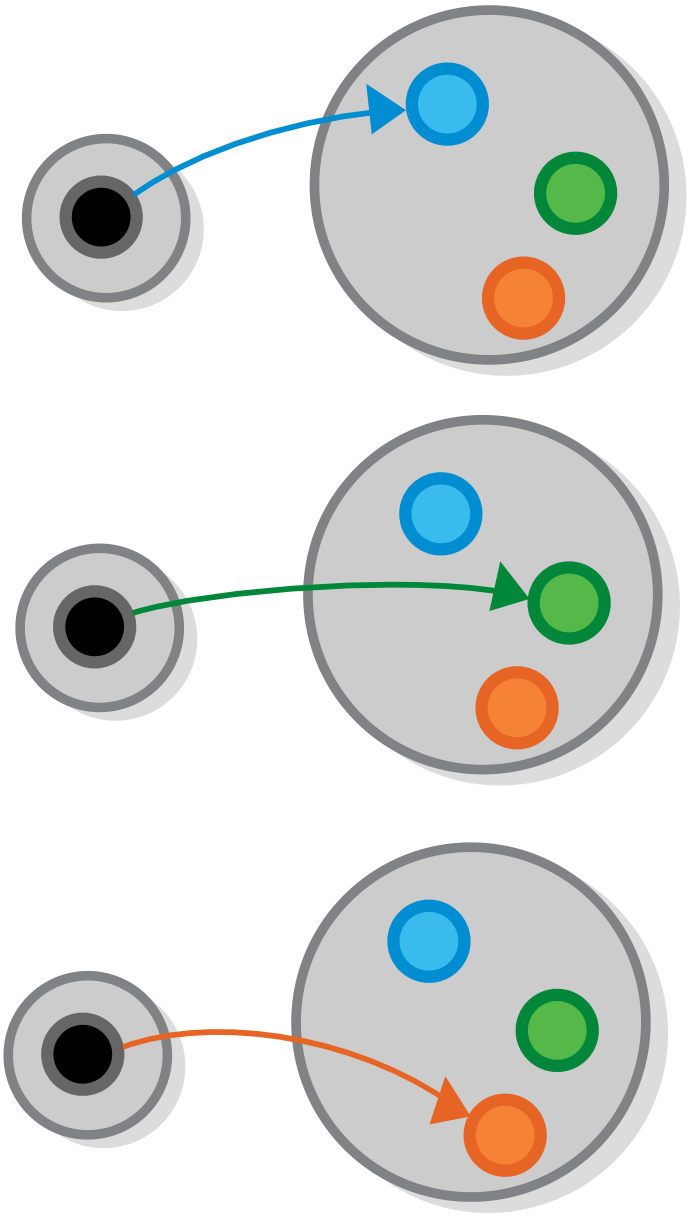
To do so, we will use the singleton set.



OK, let's start by taking a random set which we want to describe.



And let's examine the functions from the singleton set, to that random set.

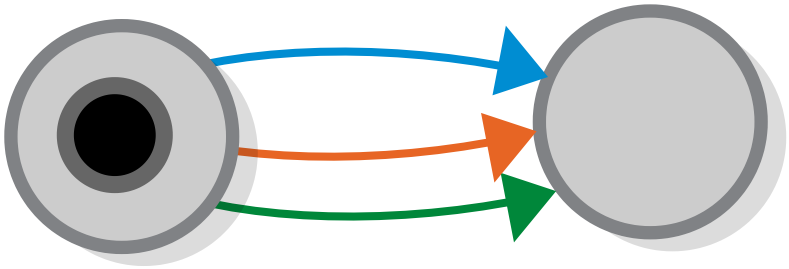


It's easy to see that there would be exactly one function for each element of the set. So we may say that:

> **THEOREM:**
>
> Each element of a set X is isomorphic to a function $1 \to X$ (where 1 means the singleton set).

So, we can say that what we call "elements" of a set are the functions from the singleton set to it.

So, our example set would look like this.



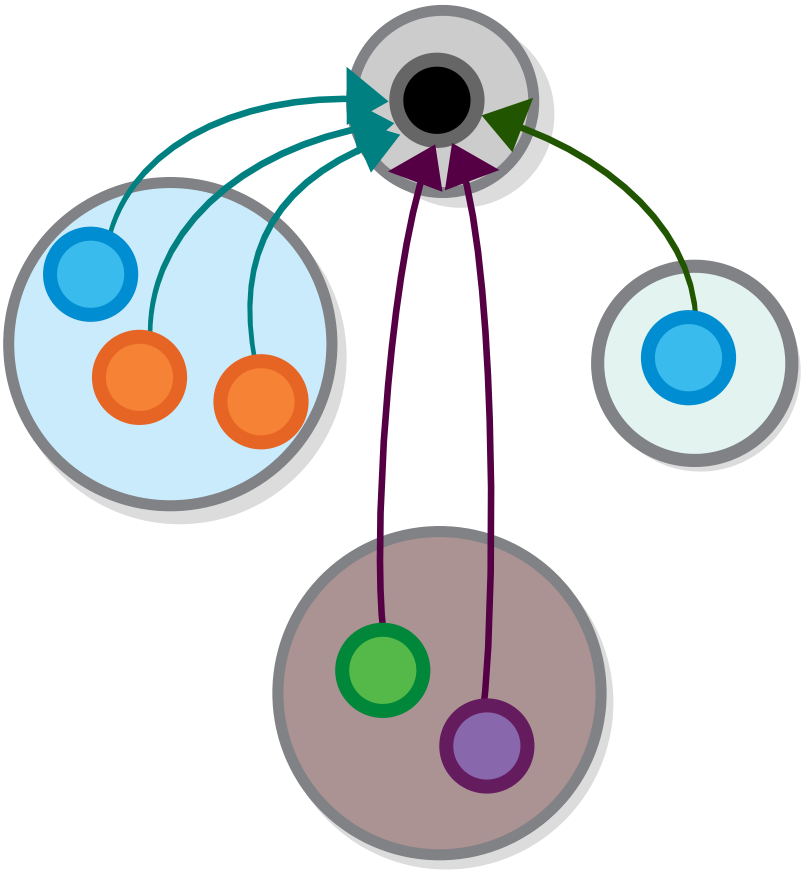
However, our diagram is not yet fully external, as it depends on the idea of the singleton set, i.e. the set with one *element*. Furthermore, this makes the whole definition circular, as we cannot define the concept of a one-element set, without the concept of element.

Defining the singleton set externally

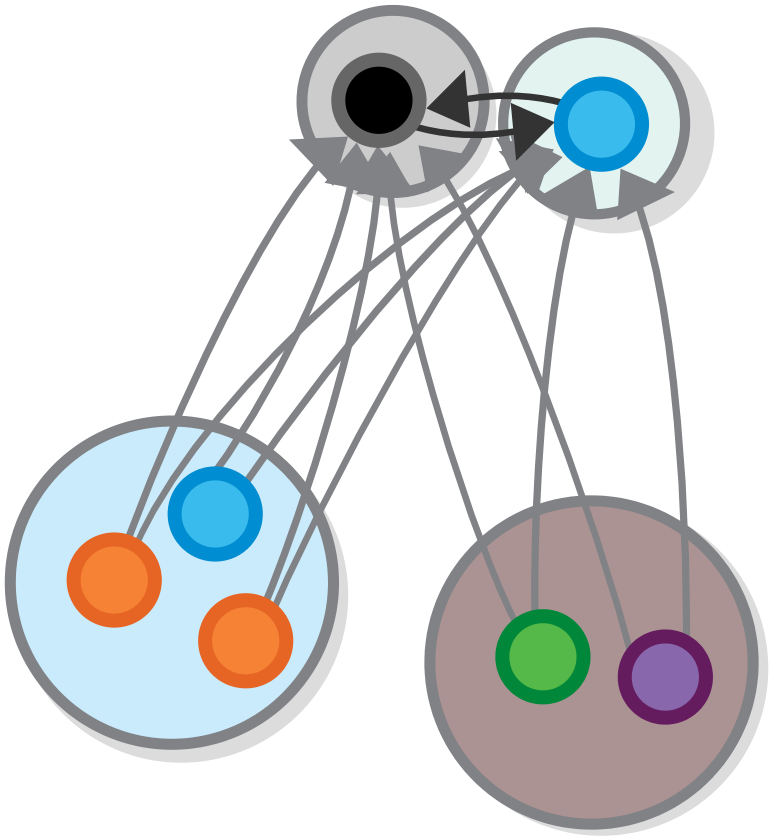
We define the singleton set externally in the same way as we did define products and sums - by using a unique property that the singleton set has. In particular, in the last chapter we learned the following:

> **THEOREM:**
>
> There is a unique function from any set to any singleton set.

If 1 is the singleton set, then we have exactly one function $X \to 1$ for all objects X i.e. $\forall X \exists! (X \to 1)$ (where $\exists!$ means "Exists unique").

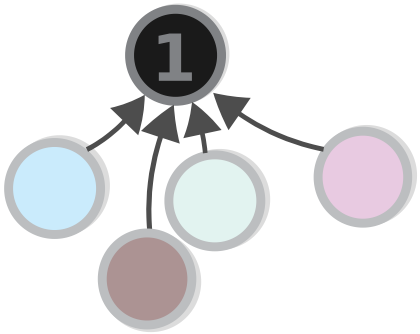


It turns out that this property defines the singleton set uniquely i.e. there is no other set that has it, other than the sets that are isomorphic to the singleton set. This is simply because, if there are two sets that have it, those two sets would also have unique functions between *themselves* so they would be isomorphic to one another. More formally, if we have two sets X and Y such that $\exists! X \to 1 \wedge \exists! Y \to 1$ and they both hold this property ("exactly one function from any other set to this set") then we also have $X \cong Y$.



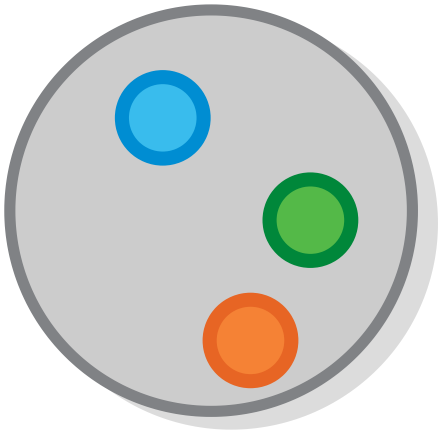
And because there is no other set, other than the singleton set that has this property, we can use it as a definition of the singleton set:

> **DEFINITION:**
>
> The singleton set 1 is one such that there exist a unique functions from any other set to it i.e. we have $\forall X \exists! X \to 1$, then 1 is the singleton set.

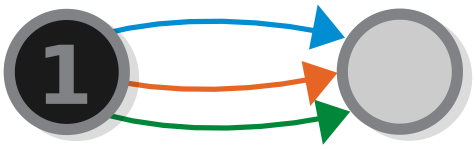


With this, we acquire a fully external definition (up to an isomorphism) of the singleton set, and thus a definition of a set element — the elements of a given

set...



...are just the functions from the singleton set to that set.



Note that from this property it follows that the singleton set has exactly one element, which confirms that our definition is correct.



Task 2: Why exactly does it follow (check the definition)?

Defining the empty set externally

The empty set is, of course, the set that has no elements, but how would we say this without referring to elements?

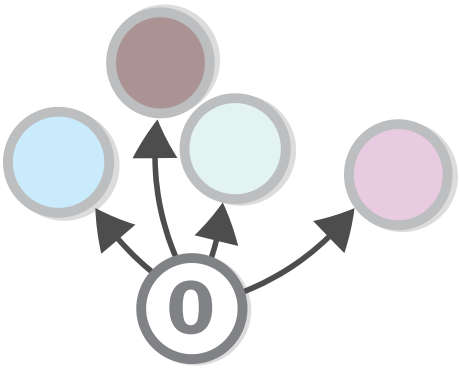
In the previous chapter, we noted an interesting property of the empty set:

> **THEOREM:**
>
> There is a unique function from the empty set to any other set.

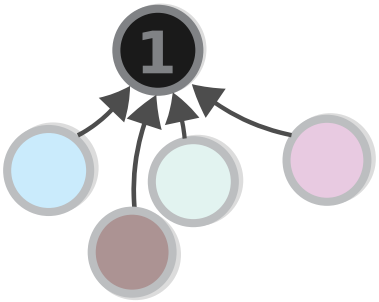
And, again, since the empty set is the only set that has this property, we can reverse the above statement and use it as a definition:

> **DEFINITION:**
>
> The empty set is a set such that there exists a function from it to any other set.

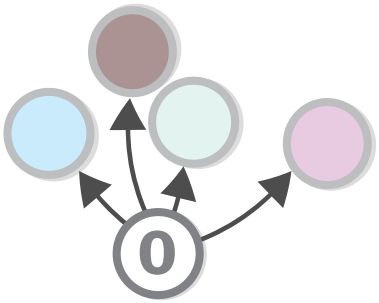
Task 3: why is the function from the empty set unique?



Observant readers will notice the similarities between the diagrams depicting the initial and terminal object (yes the two concepts are, of course, dual of each other).

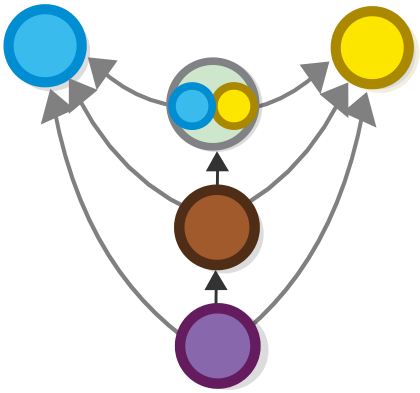


TERMINAL OBJECT

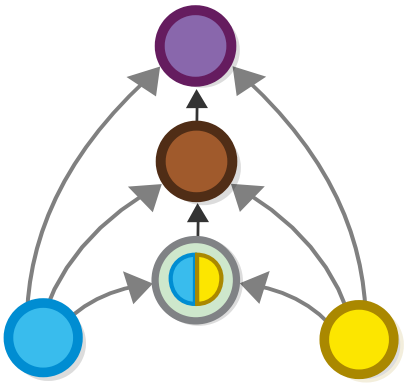


INITIAL OBJECT

Some *even more* observant readers (folks, keep it down please, you are *too observant*) may also notice the similarities between the product/coproduct diagrams and the initial/terminal object diagrams.



PRODUCT

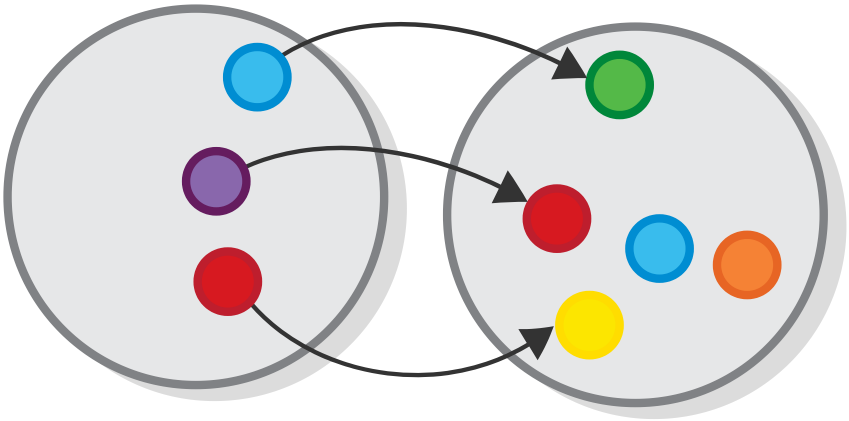


COPRODUCT

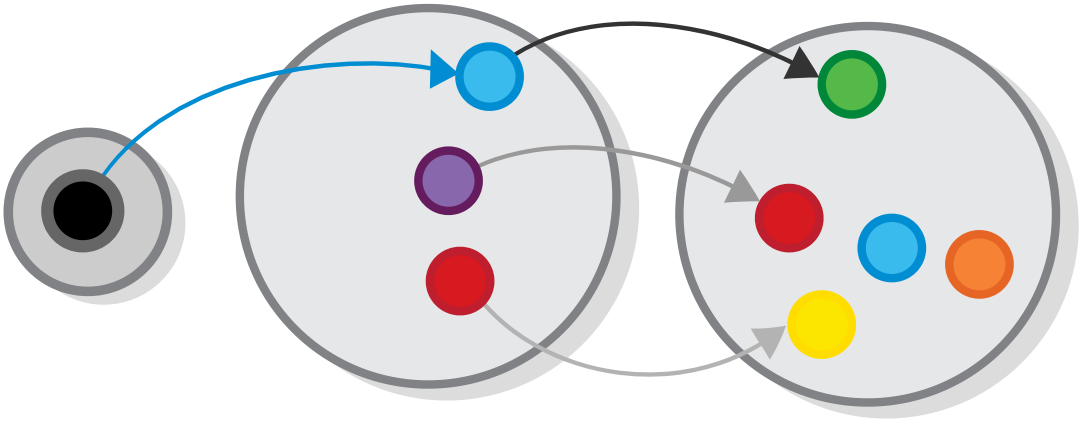
The similarity of the diagrams, is due to a similar general approach of defining things — in both cases we find the property that makes a given concept useful and then define the concept so it has this property*.

Functional application

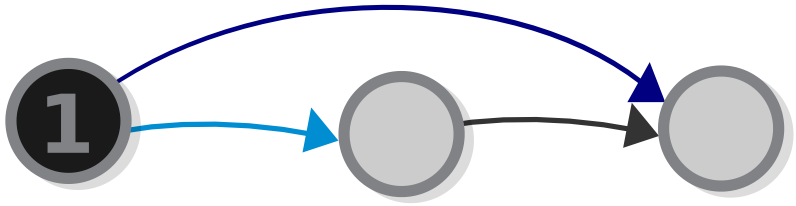
After seeing the functional definition of set elements, we might be inclined to ask the following: If elements are represented by functions, then how do you *apply* a given function to an element of a set, (and retrieve an element of another set)?



The answer is surprisingly simple — *selecting* an element from a set is the same as constructing a function from the singleton set to that element.



And then *applying* a function to an element is the same as *composing* the element function, with the function we want to apply.



The result is the function that represents the element returned by the applied function.

> **THEOREM:**
>
> Let g be an element of set X, let the function $g : 1 \to X$ represent that element, and let $f : X \to Y$ be any function from X to some other set. Then, the composition of the two functions $f \circ g : 1 \to Y$ is exactly the function that would represent the element which is the result of calling the function f with the value of g as an argument $f(g)$.

Conclusion

This was a taste of Lawvere's Elementary Theory of the Category of Sets (ETCS) which constitutes a rigorous definition of set theory (equivalent to ZFC set theory) using only the concept of a function.

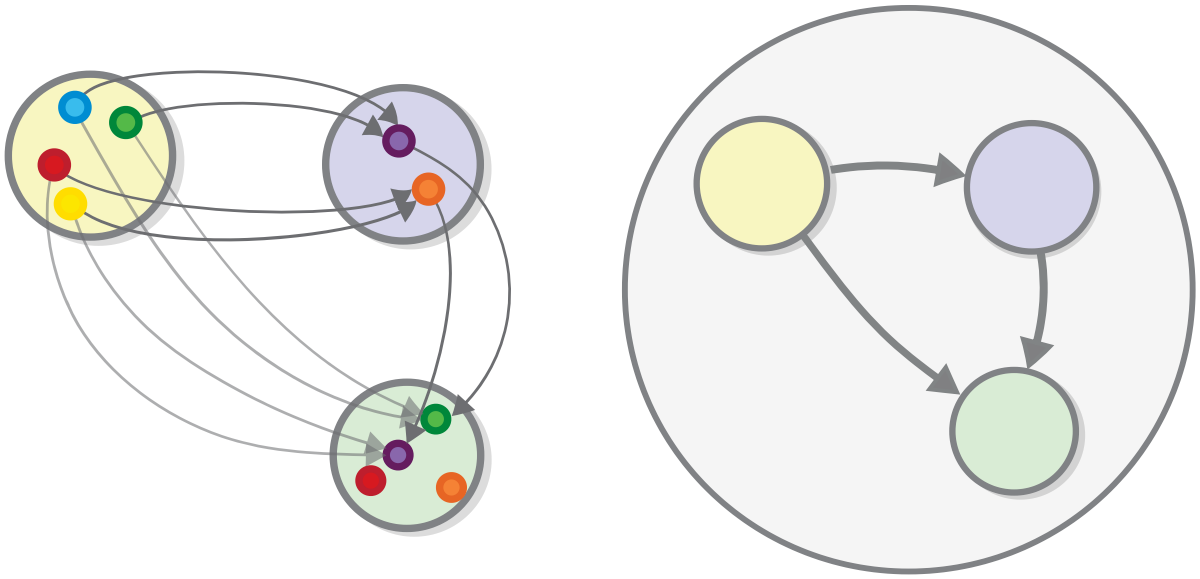
We can cover this theory in it's entirety, listing all axioms that are needed, but for now it is probably more important to understand why do we want it in the first place?

The short answer: because it is more general than the traditional definition, this new definition also applies to objects that are not exactly sets but are *like* sets in some respects.

You may say that they apply to entirely different *categories of objects* (nudge, nudge).

Categories briefly

Maybe it is about time to see what a category is. Here is a short definition: A category consists of objects (an example of which are sets) and morphisms that go from one object to another (which behave as functions) and that are composable. We can say a lot more about categories, and even present a formal definition, but for now, it is sufficient for you to remember that sets are one example of a category and that categorical objects are like sets, except that we don't *see* their elements i.e. category-theoretic notions are captured by the external diagrams, while strictly set-theoretic notions can be captured by internal ones.



When we are within the realm of sets, we can view each set as a collection of individual elements. In category theory, we don't have such a notion. However, taking this notion away allows us to define concepts such as the sum and product sets in a whole different and more general way. Plus we always have a way to "go back" to set theory, using the tricks from the last section.

Category Theory	Set theory
Category	**N/A**
Objects and Morphisms	Sets and functions

Category Theory	Set theory
N/A	Element

By switching to external diagrams, we lose sight of the particular (the elements of our sets), but we gain the ability to zoom out and see the whole universe where we have been previously trapped.

Sets VS Categories

One remark before we continue: in the last section, we may have made it seem like category theory and set theory are somehow competing with each other. Perhaps that notion would be somewhat correct if category and set theory were meant to describe *concrete* phenomena, in the way that the theory of relativity and the theory of quantum mechanics are both supposed to explain the physical world. Concrete theories are conceived mainly as *descriptions* of the world, and as such it makes sense for them to be connected in some sort of hierarchy.

In contrast, abstract theories, like category theory and set theory, are more like *languages* for expressing such descriptions — they still can be connected, and *are* connected in more than one way, but there is no inherent hierarchical relationship between the two and therefore arguing over which of the two is more basic, or more general, is just a chicken-and-egg problem, as you will see in the next chapter.

Categories again

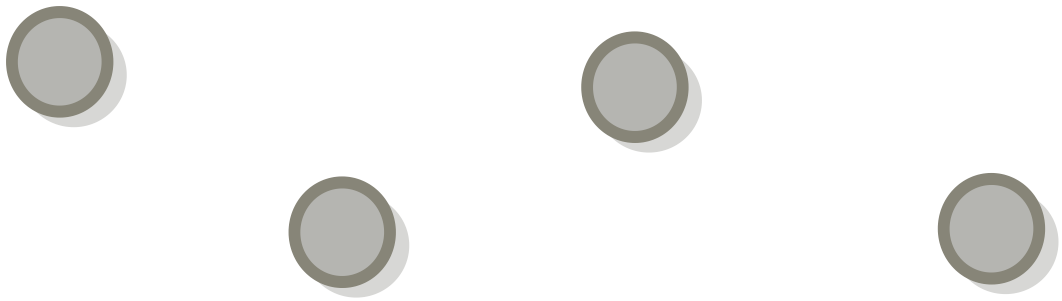
> "...deal with all elements of a set by ignoring them and working with the set's definition." — Dijkstra (from "On the cruelty of really teaching computing science")

All category theory books, including this one, start by talking about set theory. Looking back, I really don't know why this is the case — books that focus on a given subject usually don't start off by introducing an *entirely different subject*, (before even starting to talk about the main one). Perhaps the set-first approach *is* the best way to introduce people to categories. Or perhaps using sets to introduce categories is one of those things that people do just because everyone else does it. But, one thing is for certain — we don't

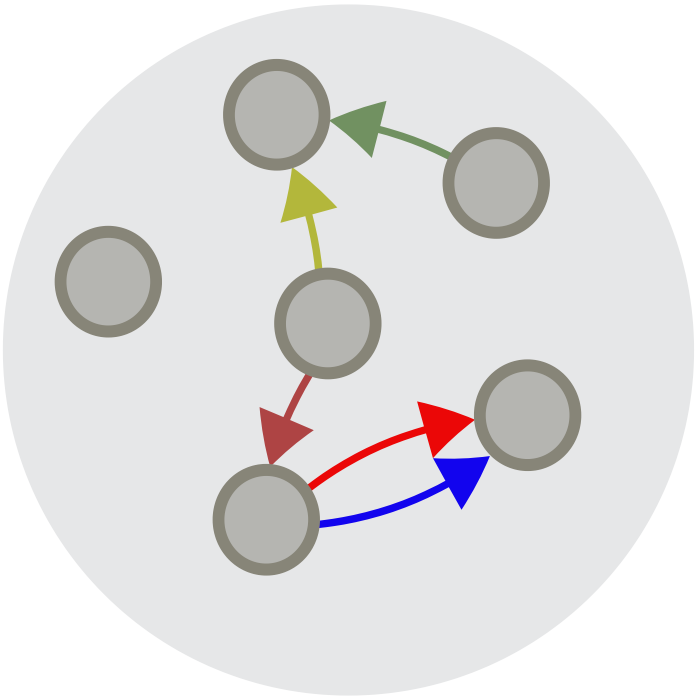
need to study sets in order to understand categories. So now I would like to start over and talk about categories as a foundational concept. So let's pretend like this is a new book, I wonder if I can dedicate this to a different person, like Tom Lehrer, who passed away in 2025 while the first edition still wasn't finished). But anyways.

Objects and morphisms

A category is a collection of objects (things) where the "things" can be anything you want. Consider, for example, these ~~colorful~~ gray balls:



A category consists of a collection of objects as well as some arrows connecting objects to one another. We call the arrows *morphisms*. They have a source object and target object (for now you can think of them as functions).

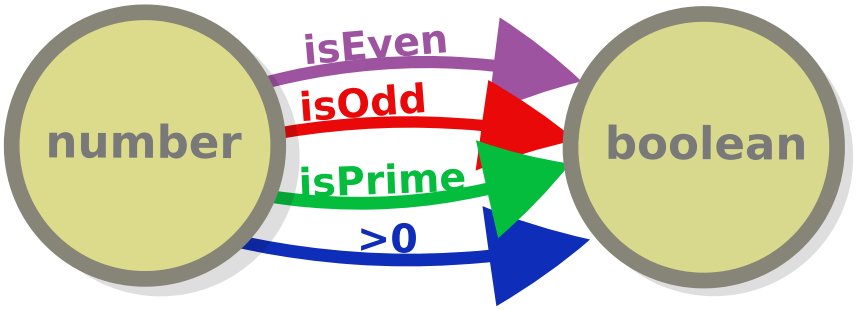


Wait a minute, we said that all sets form a category, but at the same time, any one set can be seen as a category in its own right (just one which has no morphisms). This is true and very characteristic of category theory — one structure can be examined from many different angles and may play many different roles, often in a recursive fashion.

This particular equivalence (a set as a category with no morphisms) is, however, rarely useful. Not because it's incorrect in any way, but rather because category theory is *all about the morphisms* — if the *arrows* in set theory are nothing but a connection between the sets that serve as their source and a destination, in category theory it's the *objects* that are nothing but a source and destination for the arrows that connect them to other objects. This is why, in the diagram above, the arrows, and not the objects, are colored: if you ask me, the category of sets should really be called *the category of functions*.

Speaking of which, note that objects in a category can be connected by multiple arrows and that having the same source and target sets does not in any way make arrows equivalent, as in set theory there are, for example, an infinite number of functions that go from number to boolean, and the fact that they have the same input type and the same output type (or the same *type*

signature, as we like to say) does not in any way make them equivalent to one another.

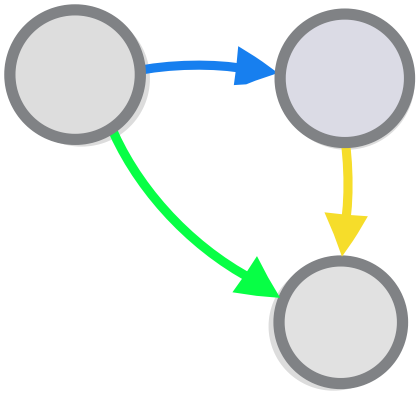


There are some types of categories that have only one morphism between two objects (in each direction), but we will talk about them in a later chapter.

Composition

The most important requirement for a structure to be called a category is that *two morphisms can make a third*, or in other words, that morphisms are *composable*.

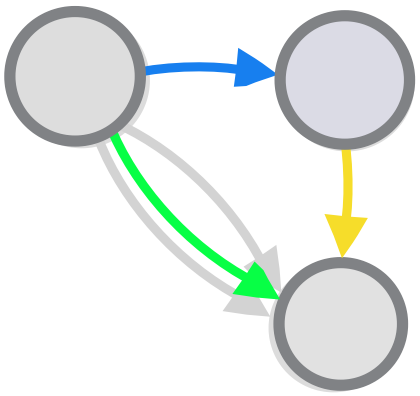
Given three objects and two successive arrows with between them, we can make a third arrow (in set theory, it is equivalent to the consecutive application of the first two).



Formally, this requirement sounds like this:

> **DEFINITION:**

> The composition operator is an operator, usually denoted with the symbol $\circ$, such that for any objects A, B and C, for each pair of morphisms $g : A \to B$ and $f : B \to C$, there exists a third morphism $(f \circ g) : A \to C$.

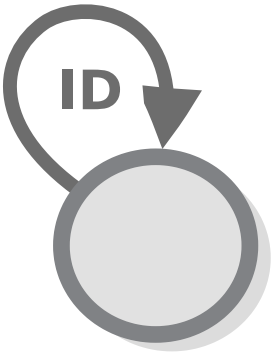


If you remember, in set theory, we picked functions, as opposed to the other types of relations because they are composable. Here we just invent the concept of a morphism and define it to be composable (in the same way as we invented the (co)products and later the empty and singleton set). Let's see where this definition gets us.

Note, that functional composition is read from right to left. e.g. applying g and then applying f is written $f \circ g$ and not the other way around. (You can think of it as a shortcut to $f(g(a))$). We can read $\circ$ as "after", e.g. f after g.

The law of identity

To have numbers, you have to have a zero. The zero of category theory is what we call the "identity morphism" for each object. In short, this is a morphism that doesn't do anything.



It's important to mark this morphism because there can be (let's again add this very important, and by now probably also very boring, reminder) many morphisms that go from one object to the same object (for example, in the category of sets, we deal with a multitude of functions that have the set of numbers as source and target, such as negate, square, $\mathrm{add\ \ one}$, and are not at all the identity morphism).

Wait, we had The way identity is formalized in an interesting way:

> **DEFINITION:**
>
> The identity morphisms of each object A, B, denoted $ID_A : A \to A$, $ID_B : B \to B$ etc. are such that for any $f : A \to B$ we have $f \circ ID_A = ID_B \circ f = f$

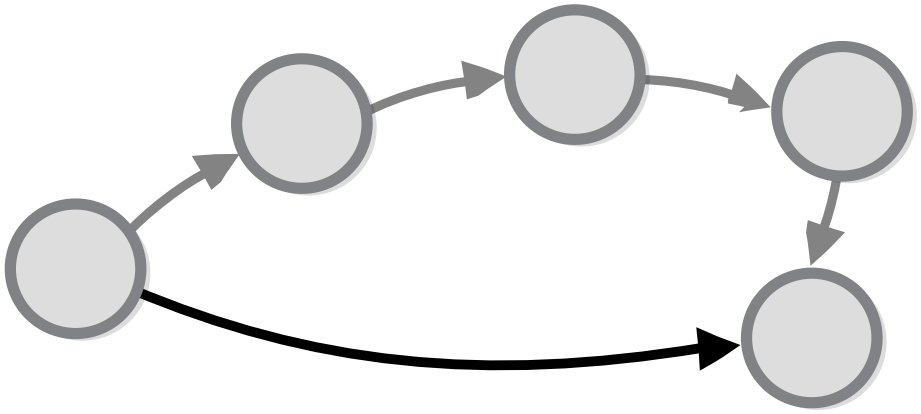
So they really "does nothing".

A structure must have an identity morphism for each object in order for it to be called a category — this is known as the *law of identity*.

Task 4: What is the identity morphism in the category of sets?

The law of associativity

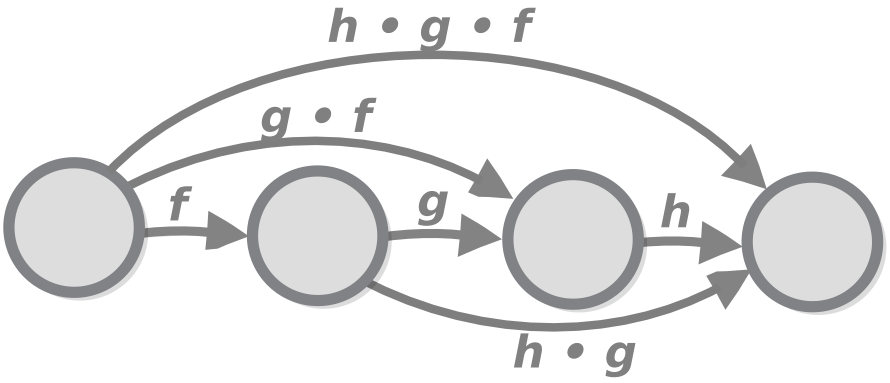
Composition is special not only because you can take any two morphisms with appropriate signatures and make a third, but because you can do so indefinitely, i.e. for each n successive arrows, each of which has as a source object the target object of the previous, we can draw one (exactly one) arrow that is equivalent to the consecutive application of all n arrows.



If we carefully review the definition above, we can see that it can be reduced to multiple applications of the following definition.

> **DEFINITION:**
>
> An operation is associative if given 3 sequential morphisms f g h, combining h and g with it and then combining the end result with f should be the same as combining h to the result of g and f: $(h \circ g) \circ f = h \circ (g \circ f)$).

This definition can be expressed using the following diagram, which would only commute if the formula is true (given that all our category-theoretic diagrams are commutative, we can say, in such cases, that the formula and the diagram are equivalent).



This formula (and the diagram) is the definition of a property called *associativity*. Being associative is required for functional composition to really be called functional composition (and thus for a category to really be called a category). It is also required for our diagrams to work, as diagrams can only

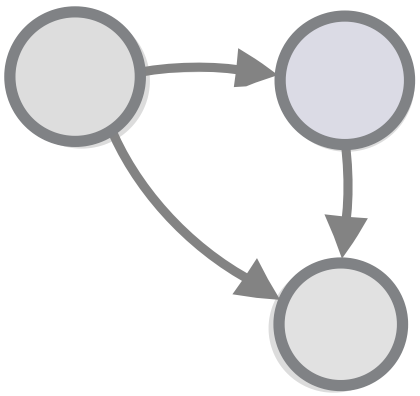
represent associative structures (imagine if the diagram above would not commute, that would be super weird).

Associativity is not just about diagrams. For example, when we express relations using formulas, associativity just means that brackets don't matter in our formulas (as evidenced by the definition $(h \circ g) \circ f = h \circ (g \circ f)$).

And it is not only about categories either, it is a property of many other operations on other types of objects as well e.g. if we look at numbers, we can see that the multiplication operation is associative e.g. $(1 \times 2) \times 3 = 1 \times (2 \times 3)$. While division is not $(1/2)/3 \neq 1/(2/3)$.

Commuting diagrams

The diagrams above use colours to illustrate the fact that the green morphism is equivalent to the other two (and not just some unrelated morphism), but in practice this notation is a little redundant, as the *only* reason to draw diagrams in the first place is to represent paths that are equivalent to each other. All other paths would just belong in different diagrams.



As we mentioned briefly in the last chapter, all diagrams that are like that (ones in which any two paths between two objects are equivalent to one another) are called *commutative diagrams* (or diagrams that *commute*). All diagrams in this book (except the incorrect ones, nudge nudge) commute.

More formally, a commuting diagram is a diagram in which given two objects a and b and two sequences of morphisms between those two objects, we can say that those sequences are equivalent.

The diagram above is one of the simplest commuting diagrams.

Despite the fact that all diagrams in books commute, in general, **not all diagrams commute**. That is, there are many morphisms with the same type signature that are not equivalent to one another.

Formal definition

For future reference, let's restate what a category is:

> **DEFINITION:**
>
> A category is a collection of *objects* (we can think of them as points) and *morphisms* (arrows) that go from one object to another, where:
>
> 1. Each object has to have an identity morphism.
> 2. There should be a way to compose two morphisms with an appropriate type signature into a third one, in a way that is associative.

This is it.

And, because categories behave as sets, many set-theoretic definitions are also valid for categories, for example, if we rewrite the definition of a set product, change "set" to "object" and "function" to "morphism", we get the general definition of a categorical product:

> **DEFINITION:**
>
> The product of A and B, denoted $A \times B$, is ~~a set~~ an object, such that:
>
> 1. There exists two "projection" ~~functions~~ morphisms $A \times B \to A$ and $A \times B \to B$.
> 2. For any impostor product I, that also has such projection ~~functions~~ morphisms ($I \to A$ and $I \to B$), there must also exist a unique ~~function~~ morphism with the type signature $g : I \to A \times B$, that converts the impostor product to the real product, such that the above two ~~functions~~ morphisms would be just the composition of g with the projections of the product.

So, we have been doing category theory from the first chapter, after all.

Addendum: Why are categories like that?

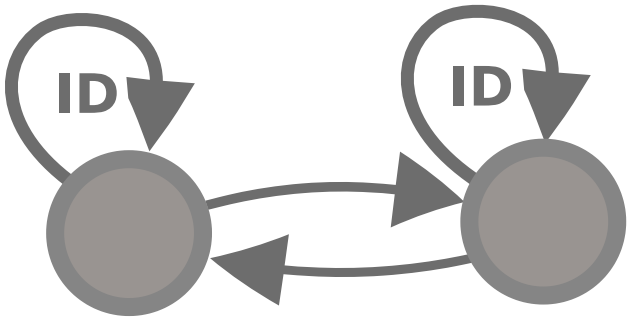
Why are categories defined by those two laws and not some other two (or one, three, four etc.). laws? From one standpoint, the answer to that seems obvious – we study categories because they *work*. I mean, look at how many applications there are... But at the same time, category theory is an abstract theory, so everything about it is kinda arbitrary: you can remove a law – and you get another theory that looks similar to category theory (although it might actually turn out to be quite different in practice). Or you can add one more law and get yet another theory (there are indeed such laws and such theories, and we will cover them later). So if this specific set of laws works better than any other, then this fact demands an explanation. Not a *mathematical* explanation (e.g. we cannot in any way *prove* that this theory is better than some other one), but an explanation nevertheless. What follows is *my* attempt to provide such an explanation, regarding the laws of *identity* and *associativity*.

Identity and isomorphisms

The reason the identity law is required is by far the more obvious one. Why do we need to have a morphism that does nothing? It's because morphisms are the basic building blocks of our language, we need the identity morphism to be able to speak properly. For example, once we have the concept of identity morphism defined, we can define a category-theoretic definition of an *isomorphism*, based on it (which is important, because the concept of an isomorphism is very important for category theory).

As we said in the previous chapter, an isomorphism between two objects (A and B) consists of two morphisms – ($A \to B$ and $B \to A$), such that their compositions are equivalent to the identity functions of the respective objects. Formally, objects A and B are isomorphic if there exist morphisms $f : A \to B$ and $g : B \to A$ such that $f \circ g = ID_B$ and $g \circ f = ID_A$.

And here is the same thing expressed with a commuting diagram.



Like the previous one, the diagram expresses the same (simple) fact as the formula, namely that going from one object (A or B) to the other and then back again to the starting object is the same as applying the identity morphism i.e. doing nothing.

Associativity and reductionism

Associativity — what does it mean and why is it there? In order to tackle this question, we must first talk about another concept — the concept of *reductionism*:

Reductionism is the idea that the behaviour of complex phenomena can be understood in terms of a number of *simpler* and more fundamental phenomena. In other words, that things keep getting simpler and simpler as they get "smaller" (or when they are viewed from a lower level). An example of reductionism is the idea that the behaviour of matter can be understood completely by studying the behaviours of its constituents i.e. atoms (the word means "undividable").

Whether the reductionist view is *universally valid*, i.e. whether it is possible to devise a *theory of everything* that describes the whole universe with a set of very simple laws, is a question over which we can argue until that universe's inevitable collapse. What is certain, though, is that *reductionism underpins all our understanding*, especially when it comes to science and mathematics — each scientific discipline is based on a set of simple *fundaments* (e.g. elementary particles in particle physics, chemical elements in chemistry etc.) on which it builds on its much more complex theories. And the reductionist view is captured by the law of associativity. And also by the closely-related law of commutativity, which we will examine in the next chapter.

Monoids etc

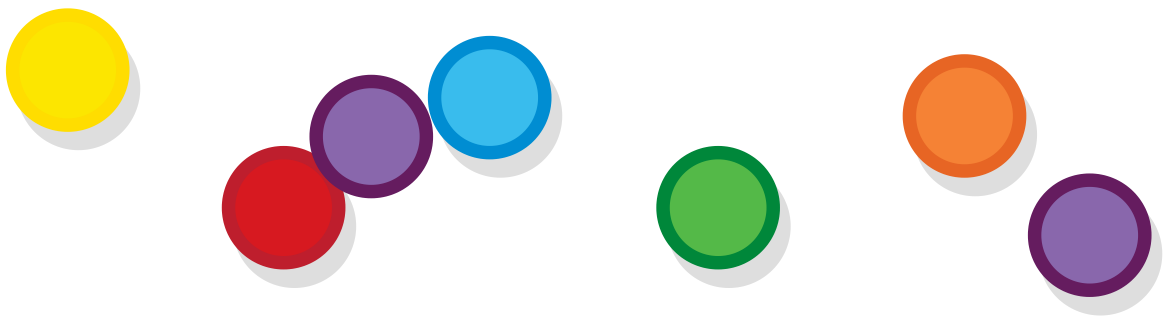
Since we are done with categories, let's look at some other structures that are also interesting — monoids.

What are monoids

Like categories, monoids/groups are abstract systems consisting of a set of elements and operations, however, the operations look different than the operations we have for categories. Here is the definition:

> **DEFINITION:**
>
> A monoid is defined by a collection/set of elements A (called the monoid's *underlying set*), together with an associative *monoid operation* — a rule for combining two elements that produces a third element one of the same kind — $A \circ A \to A$. Also, there should be an identity element I, such that $I \circ A = A$ and $A \circ I = A$.

Let's take our familiar colorful balls.



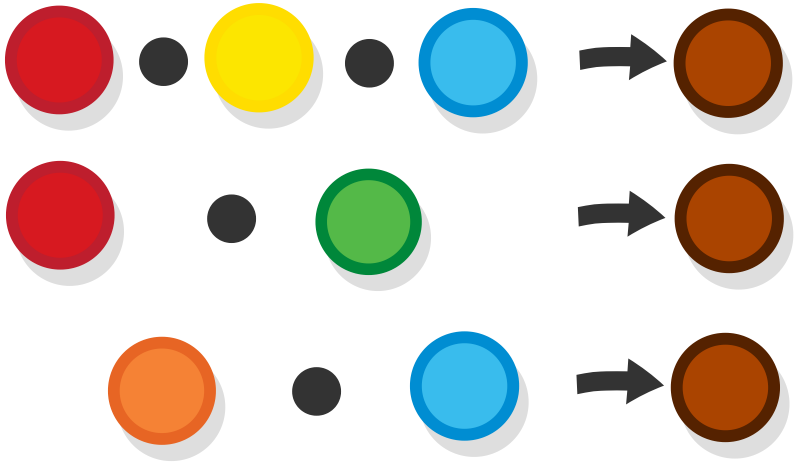
We can define a monoid based on this set by specifying an operation for "combining" two balls into one. An example of such an operation would be blending the colours of the balls as if we are mixing paint.



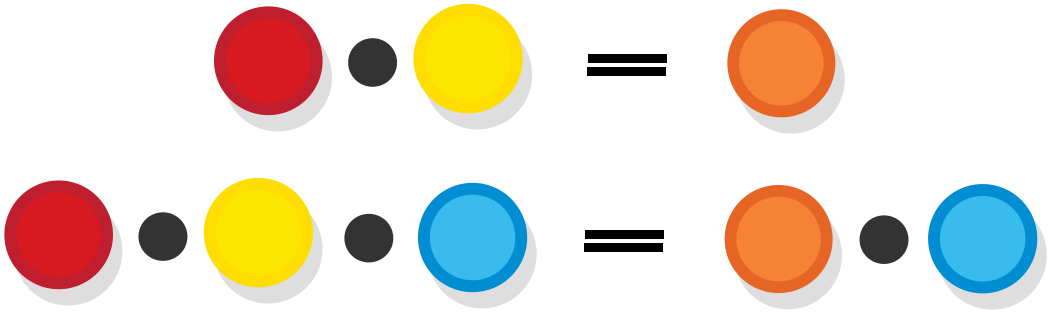
You can probably think of other ways to define a similar operation. This will help you realize that there can be many ways to create a monoid from a given set of set elements i.e. the monoid is not the set itself, it is the set *together with the operation*.

Associativity

The monoid operation should, like functional composition, be *associative* i.e. the way in which elements are grouped when applying the operation does not make any difference.



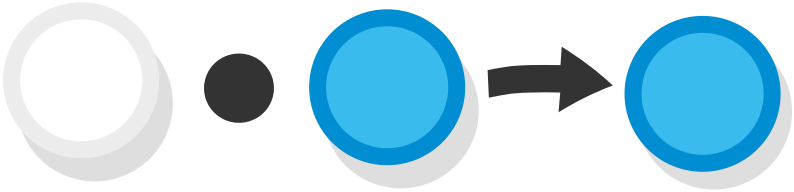
When an operation is associative, this means we can use all kinds of algebraic operations to any sequence of terms (or in other words to apply equation reasoning), like for example we can replace any element with a set of elements from which it is composed, or add a term that is present at both sides of an equation and retain the equality of the existing terms.



The identity element

Actually, not any (associative) operation for combining elements makes for a monoid (it makes for a *semigroup*, which is also a thing, but that's a separate topic). To be a monoid, a set must feature what is called an *identity element* of the operation, a concept of which you are already familiar from both sets and categories — it is an element that when combined with any other element gives back that same element (not the identity but the other one). Or simply $x \bullet i = x$ and $i \bullet x = x$ for any x.

In the case of our color-mixing monoid, the identity element is the white ball (or perhaps a transparent one, if we have one).



As you probably remember from the last chapter, functional composition is also associative and it also contains an identity element, so you might start suspecting that it forms a monoid in some way. This is indeed the case, but with one caveat, which we will talk about later.

Basic monoids

To keep the suspense, before we discuss the relationship between monoids and categories, we are going through see some simple examples of monoids.

Monoids and numbers

Mathematics is not only about numbers, however, numbers do tend to pop up in most of its areas, and monoids are no exception. The set of natural numbers $\mathbb{N}$ ($\{0, 1, 2, 3 \dots\}$) forms a monoid when combined with the all too familiar operation of addition (or *under* addition as it is traditionally said). This monoid is denoted $\langle \mathbb{N}, + \rangle$ (in general, all monoids are denoted by specifying the set and the operation, enclosed in angle brackets).

$$1 + 1 = 2$$

If you see a $1 + 1 = 2$ in your textbook you know you are either reading something very advanced, or very simple, although I am not really sure which of the two applies in the present case.

Anyways, the natural numbers also form a monoid under multiplication as well.

$$1 \times 1 = 1$$

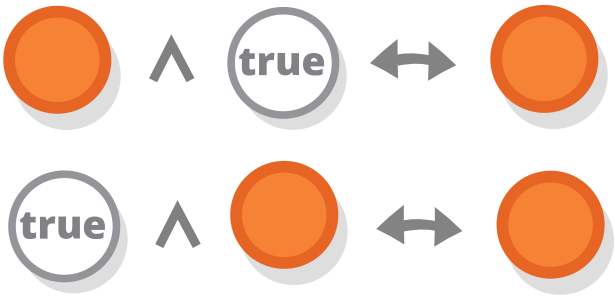
Task 1: Which are the identity elements of those monoids?

Task 2: Go through other mathematical operations and verify that they are monoidal.

Task 3: The natural numbers form a monoid under multiplication, but not a group. Find out why.

Monoids and boolean algebra

Thinking about operations that we covered, we may remember the boolean operations *and* and *or*. The operation *and* forms a monoid on the set, consisting of just two values $True, False$, in which $True$ is the identity element (the symbol $\wedge$ means *and*).



The operation *or* also forms a similar monoid.

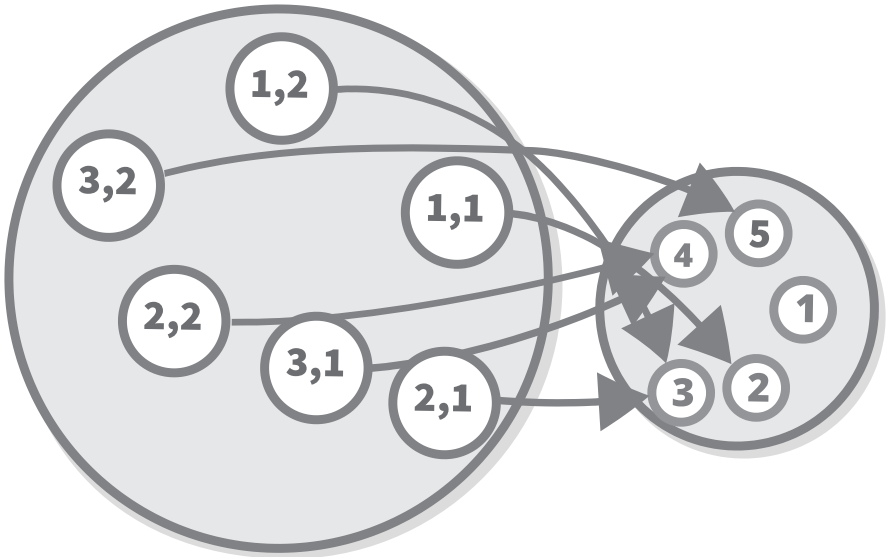
Task 4: Prove that **and** $\wedge$ is associative by expanding the formula $(A \wedge B) \wedge C = A \wedge (B \wedge C)$ with all possible values. Do the same for *or*.
Task 5: Which are the identity elements of the *or* operations?

Monoid operations in terms of sets

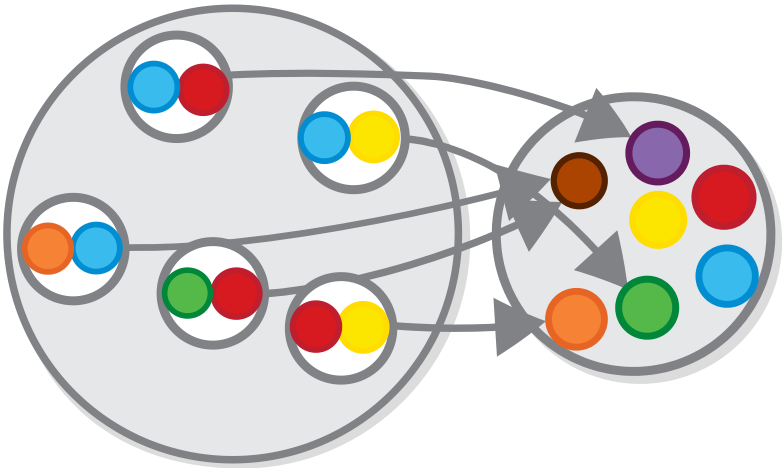
We now know what the monoid operation is, and we even saw some simple examples. However, we never defined the monoid rule/operation formally i.e. using the language of set theory with which we defined everything else. Can we do that? Of course we can – everything can be defined in terms of sets.

We said that a monoid consists of two things: a set (let's call it A), and a monoid operation that acts on that set. Since A is already defined in set theory (because it is just a set), all we have to do is define the monoid operation.

Defining the operation is not hard at all. Actually, we have already done it for the operation $+$ — in Chapter 2, we said that *addition* can be represented in set theory as a function that accepts a product of two numbers and returns a number (formally $+ : \mathbb{Z} \times \mathbb{Z} \to \mathbb{Z}$).



Every other monoid operation can also be represented in the same way — as a function that takes a pair of elements from the monoid's set and returns one other monoid element.



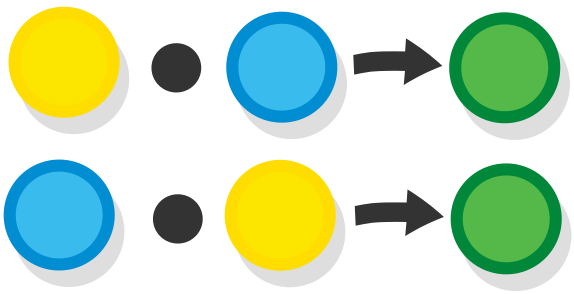
Formally, we can define a monoid from any set A, by defining an (associative) function with type signature $A \times A \to A$. That's it. Or to be precise, that is *one way* to define the monoid operation. And there is another way, which we will see next. Before that, let's examine some other types of structures.

Other monoid-like objects

Monoid operations obey two laws — they are *associative* and there exists an *identity element*. In some cases, we come across operations that also obey other laws that are also interesting. Imposing more (or less) rules to the way in which objects are combined results in the definition of other monoid-like structures.

Commutative (abelian) monoids

Looking at the monoid laws and the examples we gave so far, we observe that all of them obey one more rule (law) which we didn't specify — the order in which the operations are applied is irrelevant to the end result.



Such operations (ones for which combining a given set of elements yields the same result no matter which one is first and which one is second) are called *commutative* operations. Monoids with operations that are commutative are called *commutative monoids*.

As we said, addition is commutative as well — it does not matter whether I have given you 1 apple and then 2 more, or if I have given you 2 first and then 1 more.



All monoids that we examined so far are also *commutative*. We will see some non-commutative ones later.

Groups

A group is a monoid such that for each of its elements, there is another element which is the so-called "inverse" of the first one where the element and its inverse cancel each other out when applied one after the other. Plain-English definitions like this make you appreciate mathematical formulas more:

> **DEFINITION:**
>
> A group is a monoid in which every element A, has an inverse element, denoted usually as $-A$, such that $A \circ -A = I$ (where I is the identity element).

If we view *monoids* as a means of modelling the effect of applying a set of (associative) actions, we use *groups* to model the effects of actions which are also *reversible*.

A nice example of a group, which is related to a monoid we covered, is the set of integers under addition — the operation is again $(+)$, but the objects are the *integers* $\mathbb{Z}$, not the natural numbers $\mathbb{N}$ (so it's not $\{0, 1, 2, 3 \dots\}$, but $\{\dots -3, -2 - 1, 0, 1, 2, 3 \dots\}$). The negative numbers are added, as the natural numbers don't have inverses. The inverse of each number is its opposite number (positive numbers' inverse are negatives and vice versa).

In this instance, the above formula becomes $x + (-x) = 0$

The study of groups is a field that is much bigger than the theory of monoids (and perhaps bigger than category theory itself). And one of its biggest

branches is the study of “symmetry groups” which we will look into next.

Summary

Before we move on — the algebraic structures that we saw above can be summarized based on the laws that define them in this table:

	Semigroups	Monoids	Groups
Associativity	X	X	X
Identity		X	X
Invertability			X

And now on to symmetry groups.

Symmetry groups and group classifications

An interesting kind of groups/monoids are the groups of *symmetries* of geometric figures. Given some geometric figure, a symmetry is an action after which the figure is not displaced (e.g. it can fit into the same mold that it fitted before the action was applied).

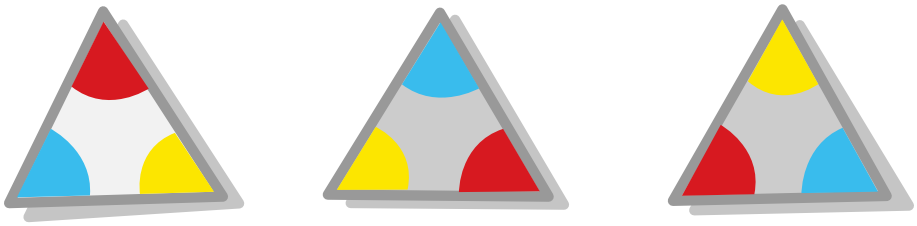
We won’t use the balls this time, because in terms of symmetries, they have just one position and hence just one action — the identity action (which is its own reverse, by the way).

Instead, let’s take this triangle, which, for our purposes, is the same as any other triangle. We are not interested in the triangle itself, but in its rotations. The only thing we need to make ourselves believe is that this is an “unrotated” triangle i.e. the one which represents the identity rotation.

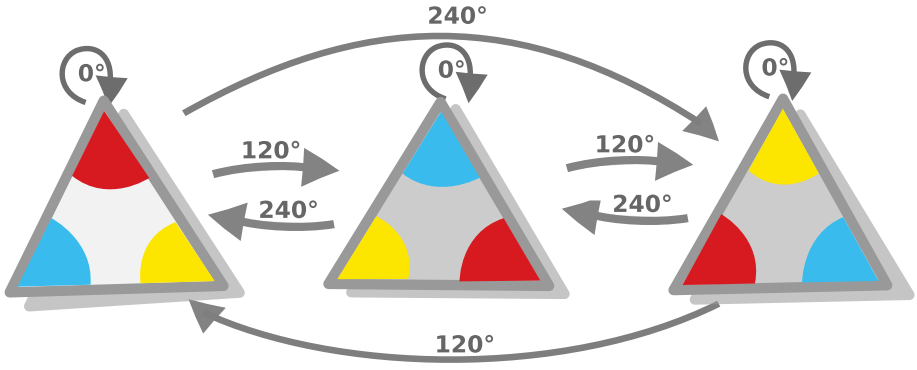


Groups of rotations

Let's first review the group of ways in which we can rotate our triangle i.e. its *rotation group*. A geometric figure can be rotated without displacement in positions equal to the number of its sides, so, for our triangle, there are 3 positions.



Connecting the dots (or the triangles in this case) shows us that there are just 3 possible rotations that get us from any state of the triangle to any other one — a *120-degree rotation* (i.e. flipping the triangle one time) and a *240-degree rotation* (i.e. flipping it twice, or equivalently, flipping it once in the opposite direction) and the identity action of 0-degree rotation.

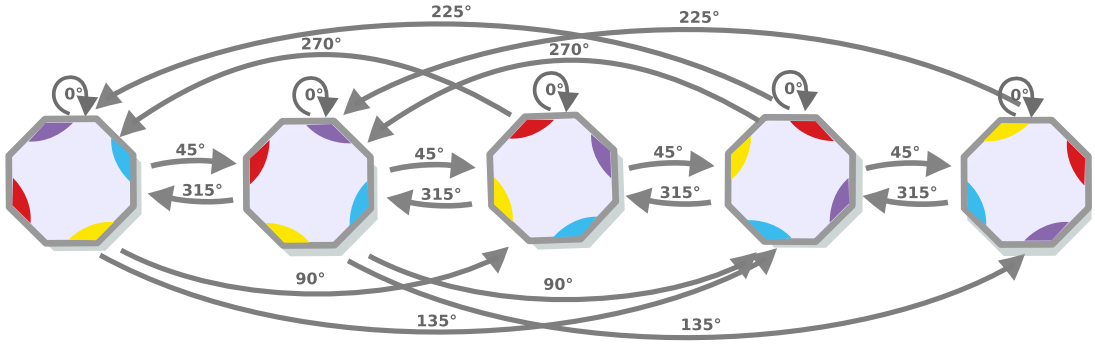


The rotations of a triangle form a monoid — the *rotations are objects* (of which the zero-degree rotation is the identity) and the monoid operation which combines two rotations into one is just the operation of performing the first rotation and then performing the second one.

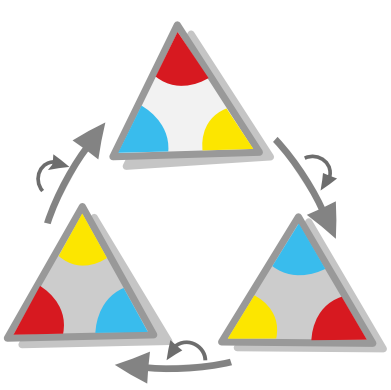
Note once again that the elements in the group are the *rotations*, not the triangles themselves, actually the group has nothing to do with triangles, as we shall see later.

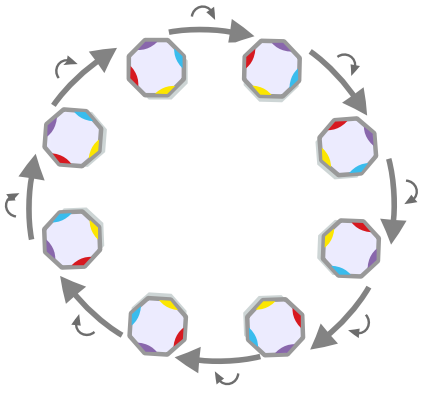
Cyclic groups/monoids

The diagram that enumerates all the rotations of a more complex geometrical figure looks quite messy at first.



But it gets much simpler to grasp if we notice the following: although our group has many rotations, and there are more still for figures with more sides (if I am not mistaken, the number of rotations is equal to the number of the sides), *all those rotations can be reduced to the repetitive application of just one rotation*, (for example, the 120-degree rotation for triangles and the 45-degree rotation for octagons). Let's make up a symbol for this rotation.

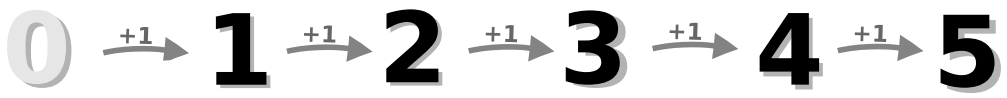




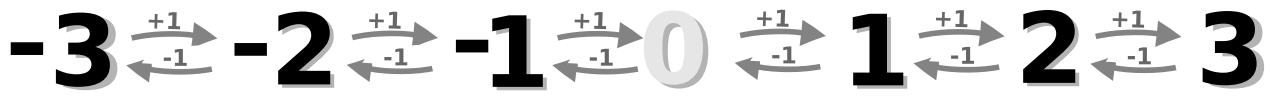
Symmetry groups that have such "main" rotation are called cyclic

> **DEFINITION:**
>
> Groups and monoids that have an object that is capable of generating all other objects by its repeated application, are called *cyclic groups*. The group's "main" object is called the *generator*.

All rotation groups/monoids are cyclic groups. Another example of a cyclic monoid is, yes, the natural numbers under addition, with $+1$ as the generator.



The *group* of integers under addition is cyclic too – here we can use $+1$ *or* -1 as the generator (as whichever of the two we choose, we would get the other one by applying the inverse law).



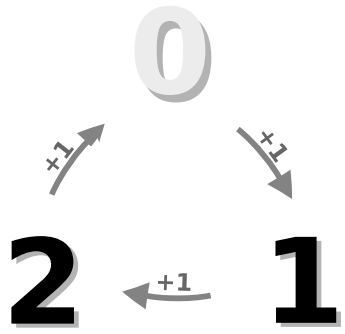
Wait, how can this be a cyclic group when there are clearly no cycles? This is because the integers are an *infinite* cyclic group.

A number-based example of a finite cyclic group is the group of integers under *modular arithmetic* (sometimes called "clock arithmetic"). Modular arithmetic's operation is based on a number called the modulus (let's take 12 for example). In it, each number is mapped to the *remainder of the integer division of that number and the modulus*.

For example: $1 \pmod{12} = 1$ (because $1/12 = 0$ with 1 remainder) $2 \pmod{12} = 2$ etc.

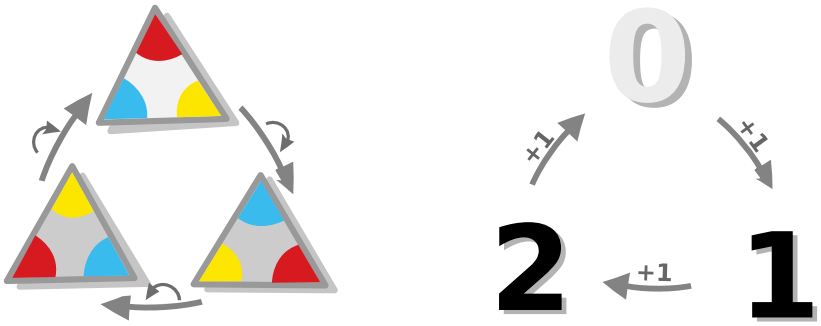
But $13 \pmod{12} = 1$ (as $13/12 = 1$ with 1 remainder) $14 \pmod{12} = 2$, $15 \pmod{12} = 3$ etc.

In effect, numbers "wrap around" forming a group with as many elements as the modulus number. For example, a group representation of modular arithmetic with modulus 3 has 3 elements.



All cyclic groups that have the same number of elements (or that are of the *same order*) are isomorphic to each other (careful readers might notice that we haven't yet defined what a group isomorphism is, even more careful readers might already have an idea about what it is).

For example, the group of rotations of the triangle is isomorphic to the group of integers under the addition with modulo 3.



All cyclic groups are *commutative* (or "abelian" as they are also called).

Task 6: Show that there are no other groups with 3 objects, other than Z_3.

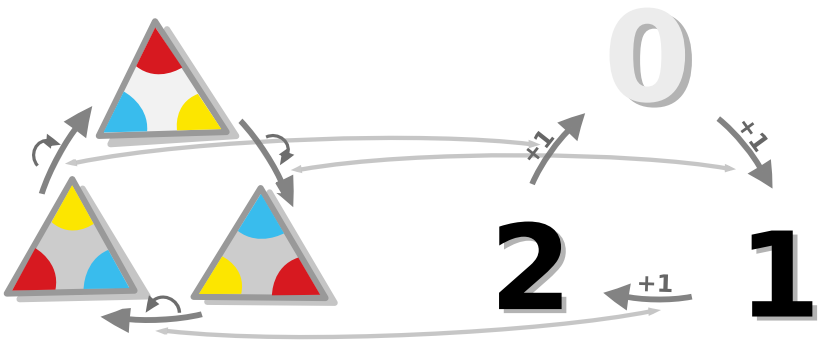
There are commutative groups that are not cyclic, but, as we shall see below, the concepts of cyclic groups and commutative groups are deeply related.

Group isomorphisms

We already mentioned group isomorphisms, but we didn't define what they are. Let's do that now:

> **DEFINITION:**
>
> An isomorphism between two groups is an isomorphism (f) between their respective sets of elements, such that for any a and b we have $f(a \circ b) = f(a) \circ f(b)$.

Visually, the diagrams of isomorphic groups have the same structure.



As in category theory, in group theory isomorphic groups are considered instances of one and the same group. For example, the one above is called Z_3.

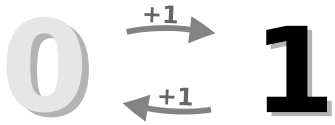
Finite groups

Like with sets, the concept of an isomorphism in group theory allows us to identify common finite groups.

The smallest group is just the trivial group Z_1 that has just one element.



The smallest non-trivial group is the group Z_2 which has two elements.



Z_2 is also known as the *boolean group*, due to the fact that it is isomorphic to the $True, False$ set under the operation that negates a given value.

Like Z_3, Z_1 and Z_2 are cyclic.

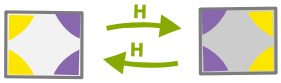
Group/monoid products

We already saw a lot of commutative groups that are also cyclic, but we didn't see any commutative groups that are *not* cyclic. So let's examine some of those like. Here, instead of looking into individual examples, we will present the general way in which commutative non-cyclic groups are produced — it is by uniting cyclic groups using the concept of *group product*.

> **DEFINITION:**
>
> Given any two groups, we can combine them to create a third group, comprised of all possible pairs of elements from the two groups and of the sum of all their actions.

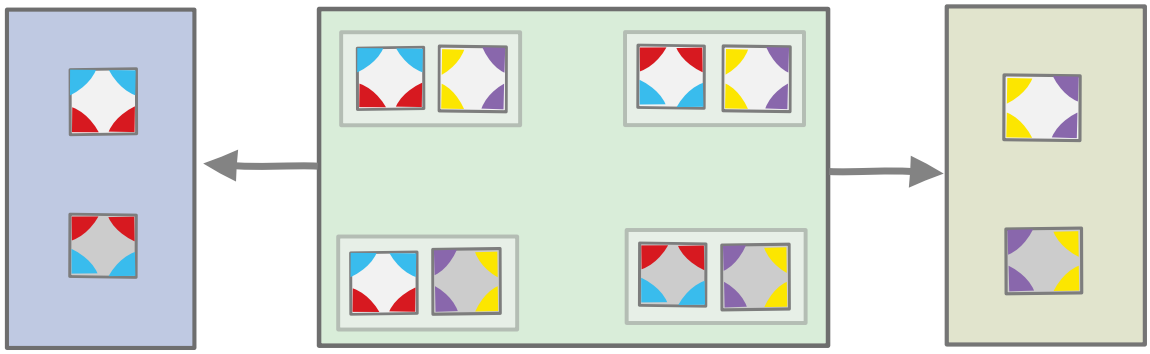
Let's see how the resulting group looks after taking the product of the following two groups (which, having just two elements and one operation, are both isomorphic to Z_2). To make it easier to imagine them, we can think of the first one as based on the vertical reflection of a figure and the second, as the horizontal reflection.



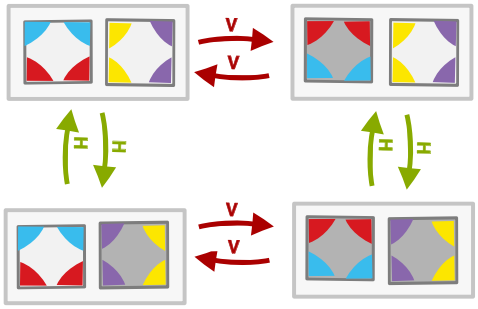


(again we have to pick an element of each group to represent the identity rotatiom, so, pretend that the left versions of the figures are the "unflipped" versions, while the right ones are flipped (although it can work the other way around too))

We get the set of elements of the new group by taking *the Cartesian product* of the set of elements of the first group and the set of elements of the second.

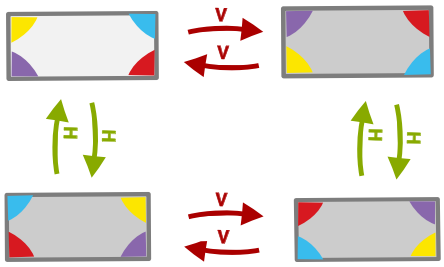


And the *actions* of a product group are comprised of the actions of the first group, combined with the actions of the second, where each action is applied only to the element that is a member of its original group, leaving the other element unchanged.



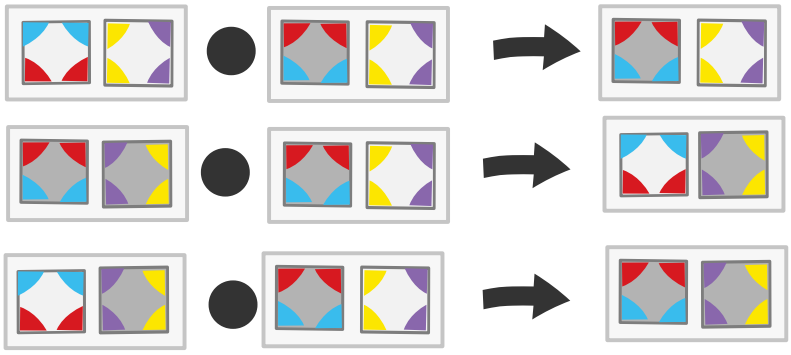
The product of the two groups presented is called the *Klein four-group* and it is the simplest *non-cyclic commutative* group.

Another way to present the Klein four-group is the *group of symmetries of a non-square rectangle*.



Task 7: Show that the two representations are isomorphic.

Here are some examples of how elements of the Klein four-group are combined.



(i.e. horizontal/vertical rotations cancel each other out, while a horizontal rotation doesn't cancel out a vertical one.)

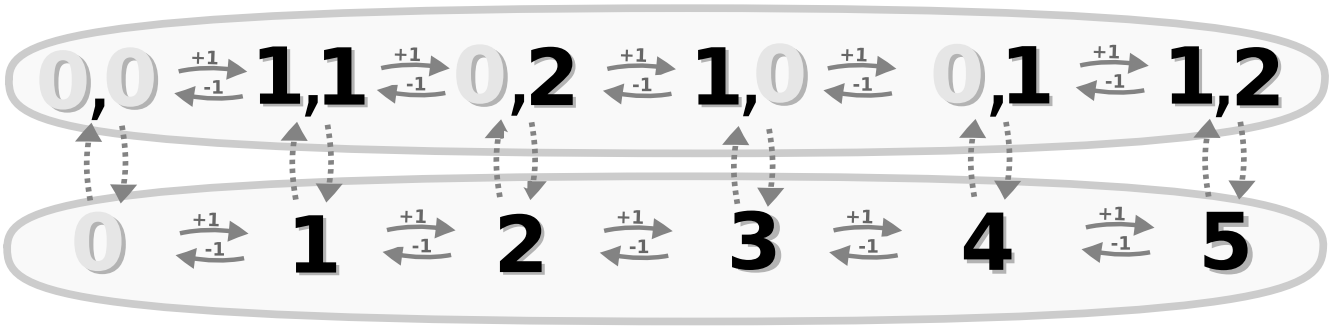
The Klein four-group is *non-cyclic* (because there are not one, but two generators) — vertical and horizontal spin. It is, however, still *commutative*, because the ordering of the actions still does not matter for the end result. Actually, the Klein four-group is the *smallest non-cyclic group*.

Cyclic product groups

In the previous chapter, we saw one *non-cyclic* product group (the Klein four-group), which was a product of *cyclic groups*. Most product groups (even the product of cyclic groups) would be non-cyclic, because it would have the generators of both groups that comprise it, i.e. even if the two original groups are cyclic and thus have 1 generator each, their product would still have 2 generators. But the product of two cyclic groups would still be cyclic if the number of elements of those groups (their *orders*) don't have a common divisor other than 1 (i.e. if they are *relatively prime numbers*).

So, if you combine two groups with orders that have some common divisor (as 2 and 2, which are both divided by 2), then, their product would not be cyclic. But, if you combine two groups with orders that are relatively prime, (like 2 and 3) you would get a cyclic group.

Furthermore, the product of two relatively prime groups would be isomorphic to a cyclic group of the same order, as the product of the orders of its components e.g. the product of Z_2 and Z_3 is isomorphic to the group Z_6 ($Z_2 \times Z_3 \cong Z_6$)



(the generator just adds 1 to each of the two groups)

This is a consequence of an ancient result, known as the *Chinese Remainder theorem*.

Commutative product groups

Product groups are *commutative*, provided that the *groups that form them* are commutative. We can see that this is true by noticing that, although there are multiple generators, each generator acts only on its own part of the group, so the generators don't interfere with each other.

Fundamental theorem of Finite abelian groups

Products provide one way to create non-cyclic commutative groups — by creating a product of two or more cyclic groups. The fundamental theory of finite abelian groups (or of finite *commutative* groups as we call them here) is a result that tells us that *this is the only way* to produce non-cyclic commutative groups i.e.

> **THEOREM:**
>
> All finite commutative groups are either cyclic or products of cyclic groups.

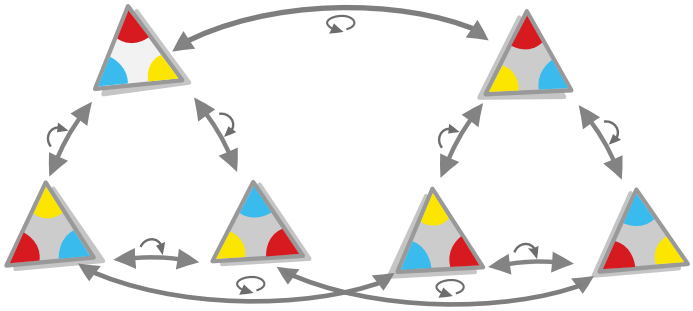
We can use this law to gain an intuitive understanding of what commutative groups are, but also to test whether a given group can be broken down to a product of more elementary groups.

Dihedral groups

Now, let's finally examine a non-commutative group — the group of rotations *and reflections* of a given geometrical figure. It is the same as the last one, but here besides the rotation action that we already saw (and its composite actions), we have the action of flipping the figure vertically, an operation which results in its mirror image:



Those two operations and their composites result in a group called $Dih3$ that is not commutative i.e. it is *non-commutative* (and is furthermore the *smallest* non-commutative group).



Task 8: Prove that this group is indeed not commutative.

Task 9: Besides having two main actions, what is the defining factor that makes this and any other group non-commutative?

Groups that represent the set of rotations and reflections of any 2D shape are called *dihedral groups*.

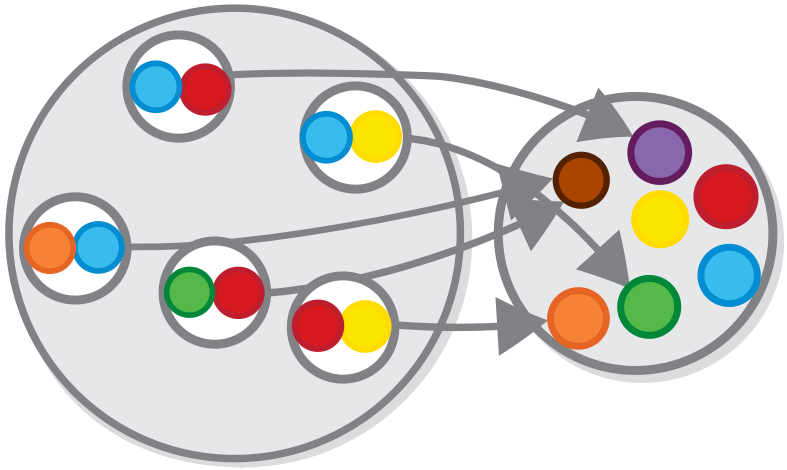
Groups/monoids as categories

Now it's the place for the grand reveal — *groups/monoids are categories*. More precisely, monoids are a *specific type of categories*, (and groups too).

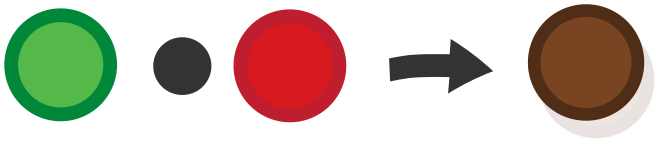
This is not to say that the definition that we examined, where we describe them as sets and binary operations, is a lie. It just says that there is an alternative, categorical definition, which is equivalent to it. Let's dive in.

Monoid elements as objects

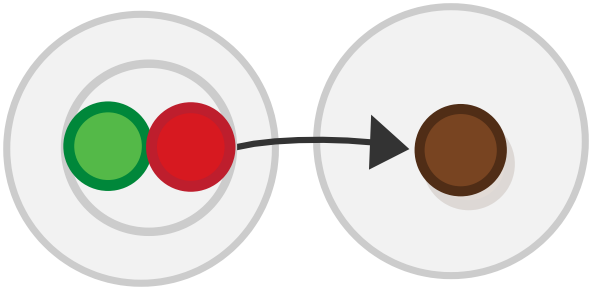
When we defined monoids, we presented their elements as *objects* and their operation — as a function/morphism that converts two objects into a third one. Then, we introduced a way for representing such operations using set theory — as functions that take a *pair* of elements from the monoid's set and return one other monoid element.



Under this correspondence, this specific mixing in the color-mixing monoid...



...corresponds to this specific *point* in the above function (point, being a mapping of a specific element of the set).

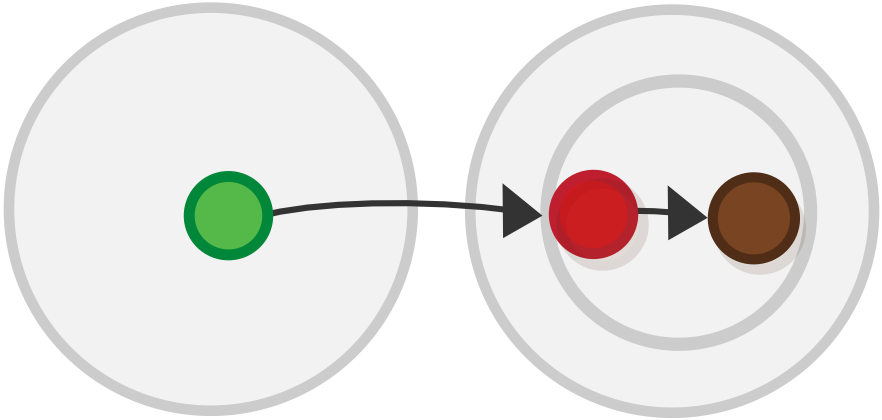


However, this is not the only way to represent multi-argument functions set-theoretically — there is another, equally interesting way, that doesn't rely on any data structures, but only on functions.

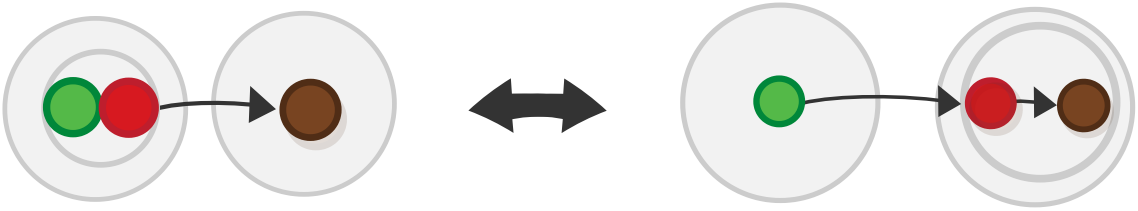
Monoid elements as morphisms

We saw that for some groups, like the groups of symmetries and rotations, the group elements can be understood not as objects but as *actions*. This is actually true for all other groups as well, e.g. the *red ball* in our color-blending monoid can be seen as the action of *adding the color red* to the mix, the number 2 in the monoid of addition can be seen as the operation $+2$ etc.

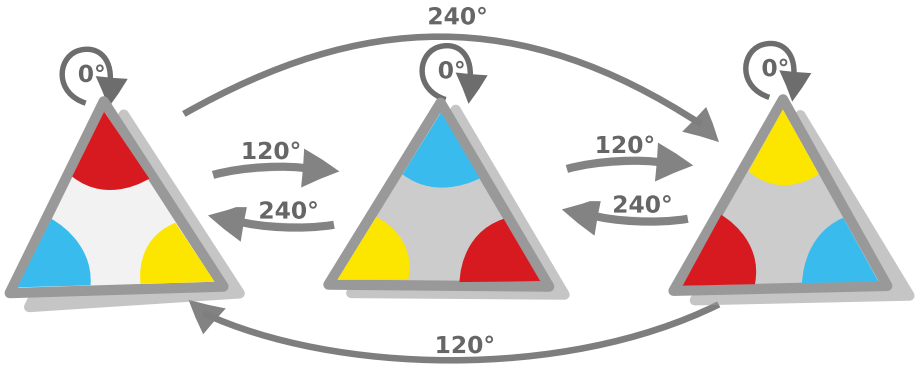
Formally, any function that takes a pair of objects, can be transformed to a function that takes one object and returns a *function* that takes the other one and returns the result.



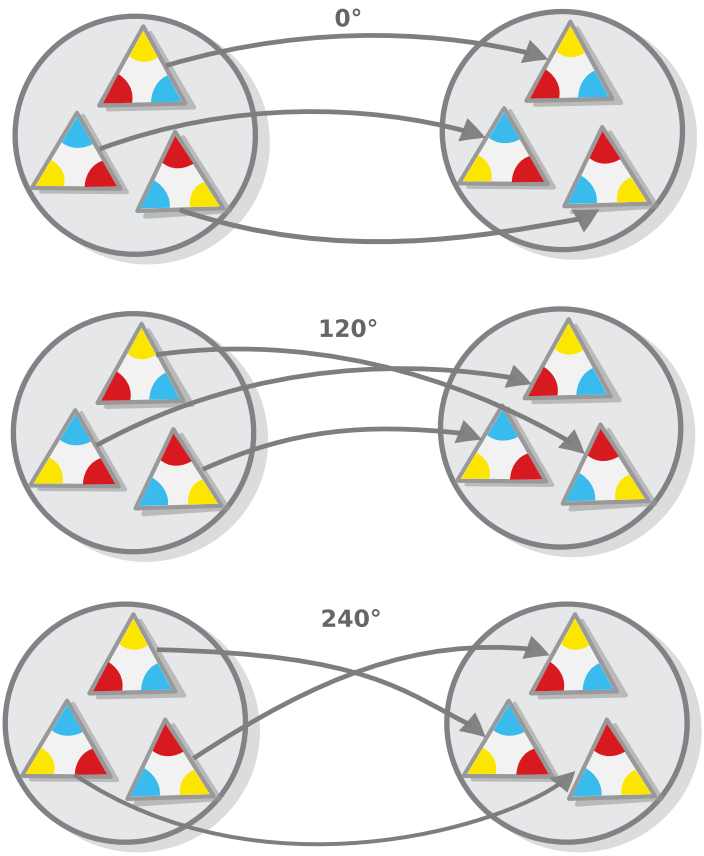
This transformation is called *currying* in the name of Haskell Curry, although it was invented some years earlier by Moses Schönfinkel (*Schönfinkelisation* didn't stick out for some reason). So, Schönfinkel discovered that the following two expressions are isomorphic.



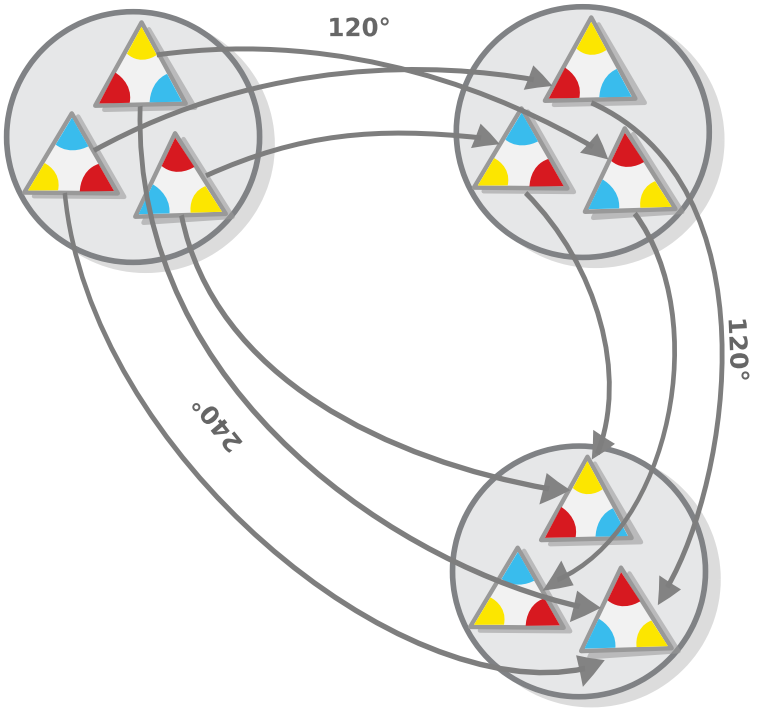
Let's take a step back and examine the groups/monoids that we covered so far in light of this equivalence e.g. let's examine the symmetric group Z_3:



The elements of this group can be viewed as functions which take a figure and rotate it a given amount of degrees.



And, we can represent the group operation itself as functional composition.



Formally, the 3 elements of Z_3 can be seen as 3 bijective (invertible) functions from a set of 3 elements to itself (in group-theoretic context, these kinds of functions are called *permutations*, by the way).

We can do the same for the addition monoid — numbers can be seen not as *quantities* (as in two apples, two oranges etc.), but as *operations*, (e.g. as the action of adding two to a given quantity).

Formally, the operation of the addition monoid, that we saw above has the following type signature.

$$+ : \mathbb{Z} \times \mathbb{Z} \to \mathbb{Z}$$

Because of the isomorphism we presented above, this function is equivalent to the following function.

$$+ : \mathbb{Z} \to (\mathbb{Z} \to \mathbb{Z})$$

When we apply an element of the monoid to that function (say 2), the result is the function $+2$ that adds 2 to a given number.

$$+2 : \mathbb{Z} \to \mathbb{Z}$$

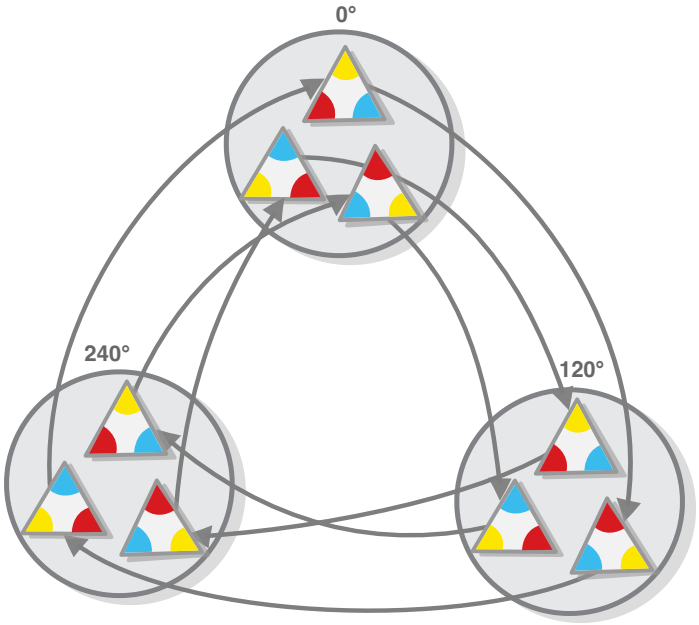
And because the monoid operation is always given in the context of a given monoid, we can view the element 2 and the function $+2$ as equivalent in the context of the monoid.

$$2 \cong +2$$

In other words, in addition to representing the monoid elements in the set as *objects* that are combined using a function, we can represent them as *functions* themselves.

Monoid operations as functional composition

As we said, when monoid elements are represented as functions, the monoid operation is represented as *functional composition*. The functions that represent the monoid elements have the same set as source and target, or the same *signature*, as we say (formally, they are of the type $A \to A$ for some A). Because of that, they all can be composed with one another, and the result of such compositions would also have the same signature.



This is true for all monoids, e.g. number functions can also be combined using functional composition.

$$+2 \circ +3 \cong +5$$

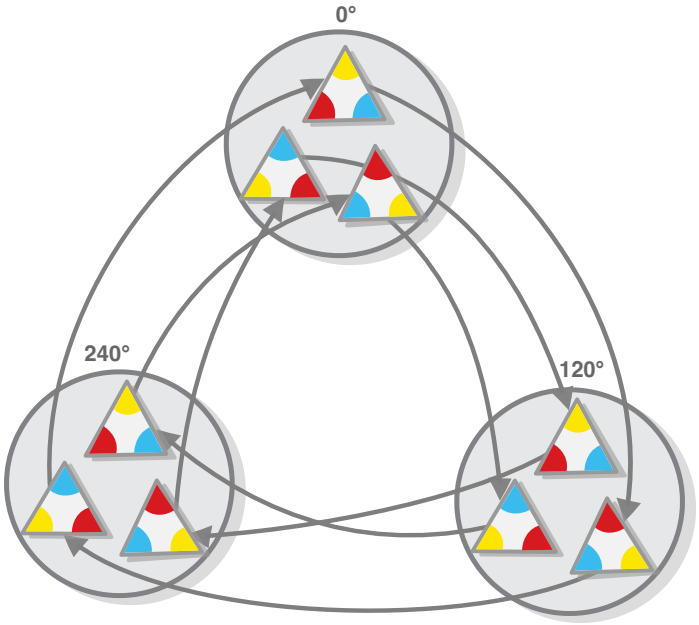
So, basically, the functions that represent the elements of a monoid also form a monoid, under the operation of functional composition (and the functions that represent the elements that form a group also form a group).

Task 10: Which are the identity elements of function groups?

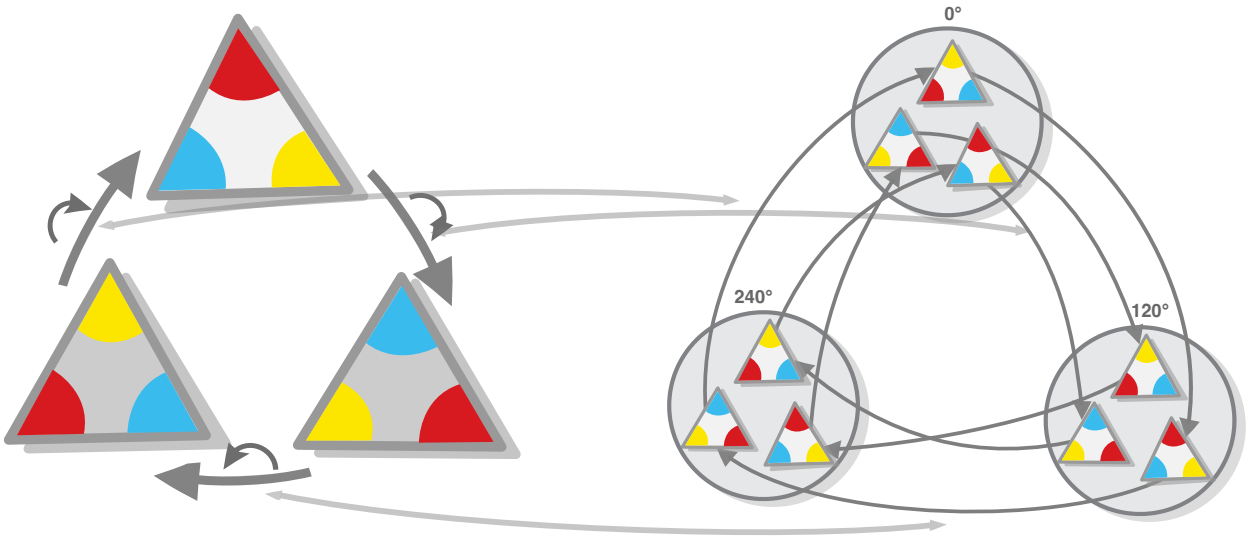
Task 11: Show that the functions representing inverse group elements are also inverse.

Cayley's theorem

Let's recap: in the previous section, we showed how the elements of every group/monoid correspond to functions from the monoid's underlying set to itself (AKA to permutations), and we said that those permutations make up a monoid of their own, under functional composition — the monoid of permutations, let's call it.



One thing that we *didn't say* in the prev section: every such permutation group/monoid is isomorphic to the monoid from which it is constructed.



This is a result known as the Cayley's theorem. In short:

> **THEOREM:**
>
> Any group is isomorphic to its corresponding permutation group.

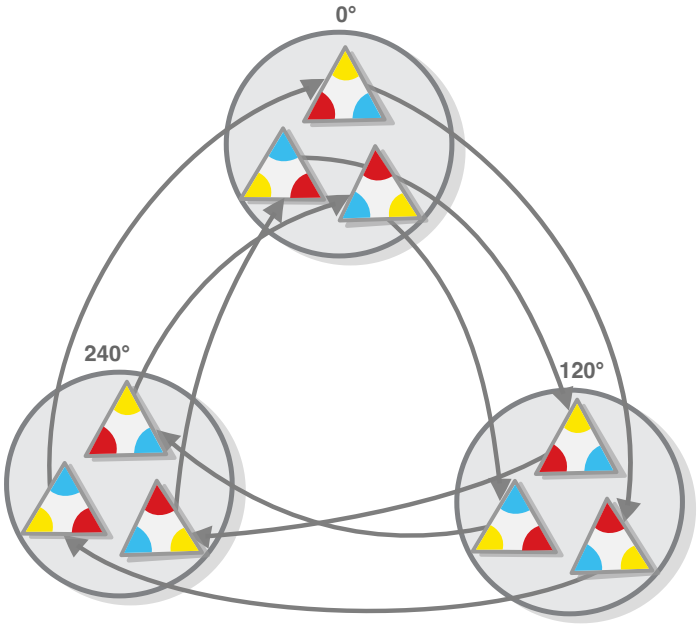
Or formally, if we use $Perm$ to denote the permutation group then Cayley's theorem states that $Perm(A) \cong A$ for any A.

Or in other words, representing the elements of a monoid/group as permutations actually yields a representation of the monoid itself (sometimes called its *standard representation*) e.g. a triangle is a figure such that if you flip it two times it would go back to the original position (and everything that you can flip two times and go back to the original position is a triangle).

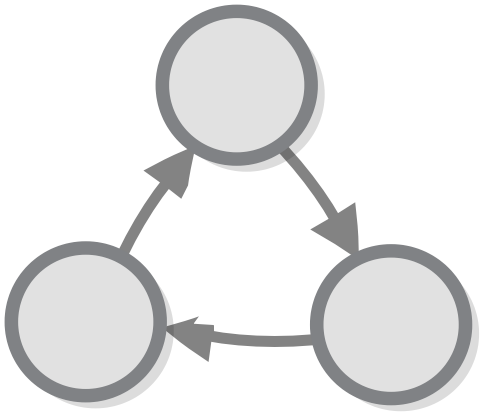
Cayley's theorem is a very important result, so the fact that it does not look impressive in this context only shows the power of the categorical framework (and how much we learned).

Monoids as categories

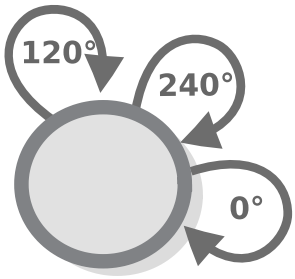
We saw that converting the monoid's elements to actions/functions yields an accurate representation of the monoid in terms of sets.



However, it is obvious that it is the *functions* of the monoids, not their sets that are important (after all, with monoids you have the same set everywhere) , so we can try depicting it as a categorical (external) diagram.



But wait, if the monoids' underlying *sets* correspond to *objects* in category theory, then the corresponding category would have just one object. And so the correct representation would involve just one point from which all arrows come and to which they go.



So a monoid, any monoid, can be seen as a *category with one object*

Formal definition

Let's check if that is really true, by reviewing the definition of a category:

> **DEFINITION:**
>
> A category is a collection of *objects* (we can think of them as points) and *morphisms* (arrows) that go from one object to another, where:
>
> 1. Each object has to have an identity morphism.
> 2. There should be a way to compose two morphisms with an appropriate type signature into a third one in a way that is associative.

Let's see what these laws imply for categories with one object:

> 1. Each object has to have an identity morphism.

For categories with just one object, there would also be one identity morphism. And monoids do have an identity *object*, which when viewed categorically corresponds to that identity morphism:

> 1. There should be a way to compose two morphisms with an appropriate type signature into a third one in a way that is associative.

But if the category has one object, all morphisms would have the same type signature (they would just be $A \to A$). So then *all morphisms would be composable with one another*. The monoid operation does exactly that – given any two objects (or two morphisms, if we use the categorical terminology), it creates a third.

We see that aside from the little-confusing fact that *monoid objects are morphisms* when viewed categorically, this describes exactly what monoids are.

> **DEFINITION:**
>
> A monoid, any monoid, can be seen as a *category with one object*—the morphisms of the category are the monoid elements, the identity morphism is the identity element and the monoid operation is composition of morphisms. The converse is also true: any category with one object can be seen as a monoid

Philosophically, defining a monoid as a one-object category corresponds to the view of monoids as a model of how a set of (associative) actions that are performed on a given object alter its state. Provided that the object's state is determined solely by the actions that are performed on it, we can leave it out of the equation and concentrate on how the actions are combined. And as per usual, the actions (and elements) can be anything, from mixing colors, to adding quantities to a given set of things etc.

Group/monoid presentations

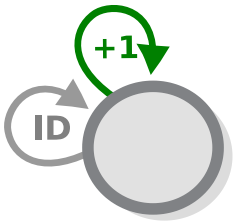
In the previous section, we proved that monoids are indeed equivalent to one-object categories. However, the implications of this statement still seem a bit baffling: Does this mean that all monoids and monoids (even ones with different underlying sets!) are kinda one and the same? The answer is that they are indeed similar, at least when we are viewing isomorphic monoids as one and the same monoid. The only differences between them can be traced in these two things:

1. The number of morphisms that they have.
2. The laws governing the composition of those morphisms.

Formally, the set of generators and laws that defines a given monoid is called the *presentation of a monoid* and every monoid can be defined by specifying its presentation. And this observation leads to a whole new way of defining a monoid/monoid.

Cyclic monoids

Let's imagine one specific set of categories: categories that, besides having one object, also have just one morphism (besides the identity).



Those category corresponds exactly to *cyclic monoids/monoids* (the morphism is the generator).

And the difference between all cyclic monoids/monoids is determined solely by the laws.

Z3

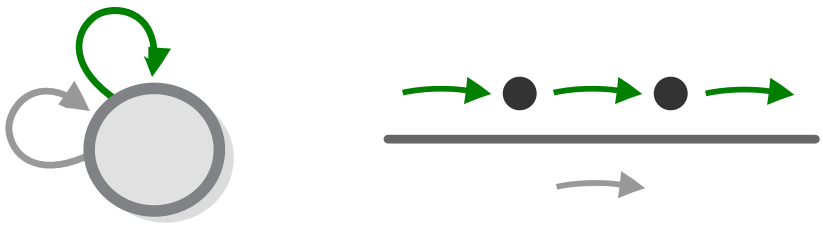
Let's turn our attention to the second component of the presentation — the laws describing the result of the *composition* of given two morphism.

In our case with cyclic monoids, we are talking about the result of composing the only morphism that forms the monoid with itself.

Here is one law that we may define:

> When you compose the generator with itself 3 times, you get identity morphism.

We can denote it like this:



So, what is the monoid that this law defines? To find out, we start composing the monoid generator with itself, and then applying the law, until we find all possible sequences of compositions.

0°



120°



240°



(because if we compose the morphism with itself one more time we will be back to the identity).

As you can already guess, this monoid is just our familiar monoid Z_3 – the monoid of triangle rotations, or the modular arithmetic with modulo 3.

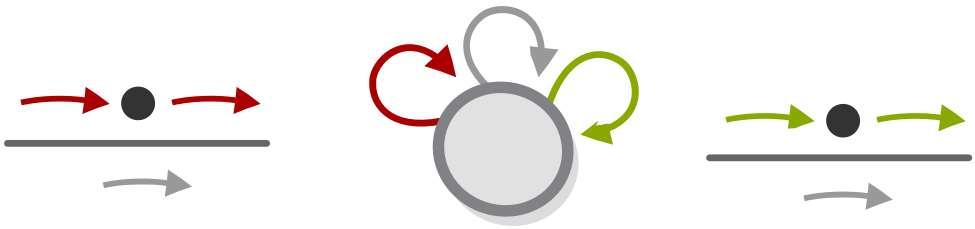
And what would happen if we reformulating the law so instead of 3 it says some other number n.

> When you compose the generator with itself n times, you get identity morphism.

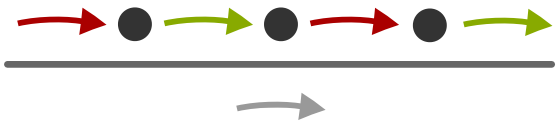
This would yield all other cyclic monoids: Z_1 Z_2 Z_3 etc...

Klein-four

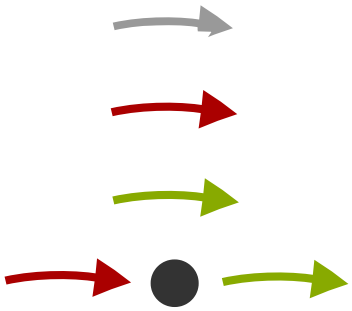
We can represent product monoids this way too. Let's take Klein four-monoid as an example, The Klein four-monoid has two generators that it inherits from the monoids that form it (which we considered as vertical and horizontal rotation of a non-square rectangle) each of which comes with one law.



To make the representation complete, we add the law for combining the two generators.



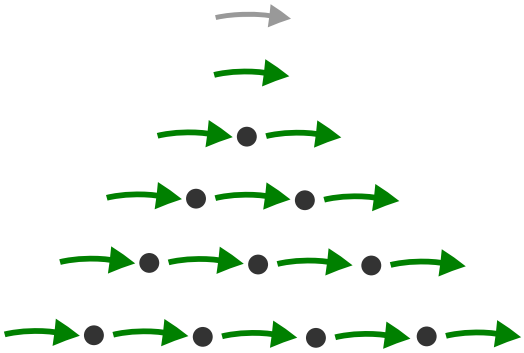
And then, if we start applying the two generators and applying the laws, we get the four elements of the monoid.



Free monoids

We saw how picking a different selection of laws gives rise to different types of monoids. But what would we get if we pick *no laws at all*? These monoids (we get a different one depending on the set of morphisms we pick) are called *free monoids*, as in "free from laws" (or as in, "you can upgrade the set of generators to a monoid for free").

The free monoid with just one generator is isomorphic to the monoid of natural numbers.



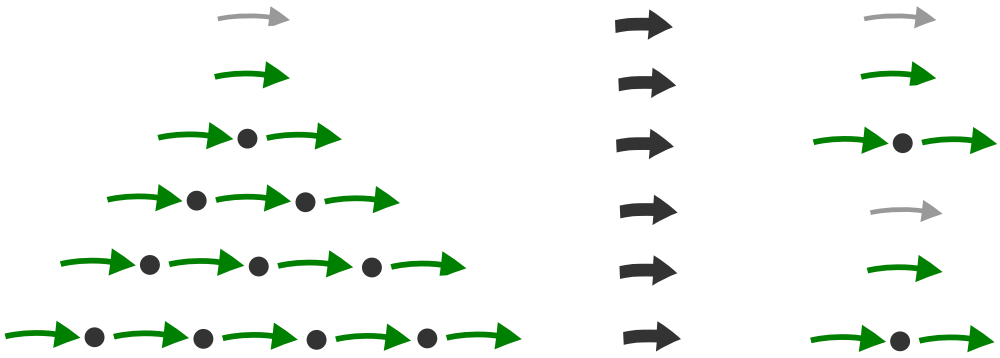
We can make a free monoid from the set of colorful balls — the monoid's elements would be sequences of all possible combinations of the balls.



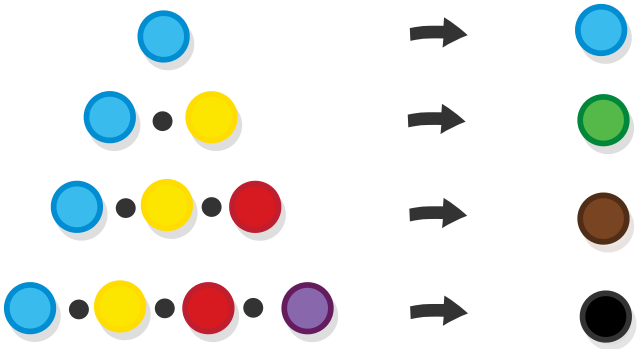
The universal property of free monoids

Free monoids a special one, in that you can define a function that converts it to any other monoid which has the same set of generators, By just applying the monoid's laws.

For example, if we take the free monoid with just one generator, and apply to it's elements the law of Z_3, we get... a function from it to Z_3,



And if we take the free monoid of balls, and we apply the laws of the color-mixing monoid, we would get a function from the free monoid of balls to the color-mixing monoid.



Task 14: Write up the laws of the color-mixing monoid.

If we put on our programmers' hat, we will see that the type of the free monoid under the set of generators `T` (which we can denote as `FreeMonoid<T>`) is isomorphic to the type `List<T>` and that the intuition behind the special property that we described above is actually very simple: keeping objects in a list allows you to convert them to any other structure i.e. when we want to perform some manipulation on a bunch of objects, but we don't know exactly what this manipulation is, we just keep a list of those objects until it's time to do it.

Orders

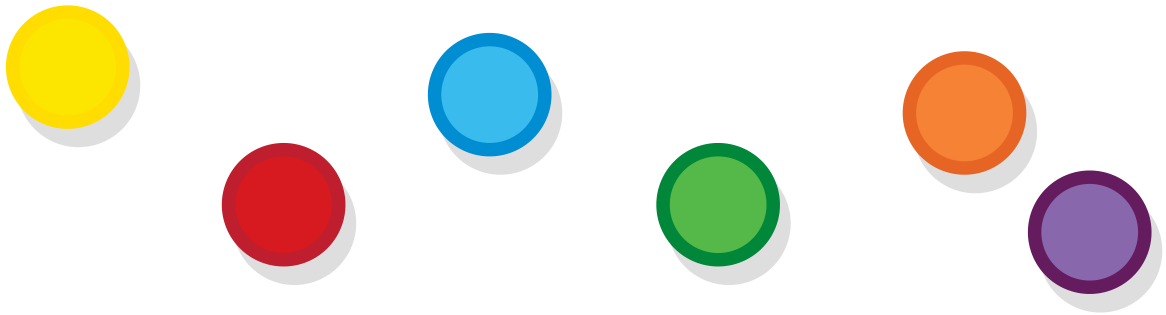
Given a set of objects, there can be numerous criteria, based on which to order them (depending on the objects themselves) — size, weight, age, alphabetical order etc.

However, currently we are not interested in the *criteria* that we can use to order objects, but in the *nature of the relationships* that define order. Of which there can be several types as well.

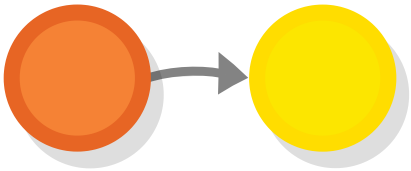
Mathematically, the order as a construct is represented (much like a monoid) by two components.

> **DEFINITION:**
>
> An order is a set of elements, together with a *binary relation* between these elements, denoted with $\leq$ ("bigger or equal to") that obeys certain laws.

We denote the elements of our set, as usual, like this.



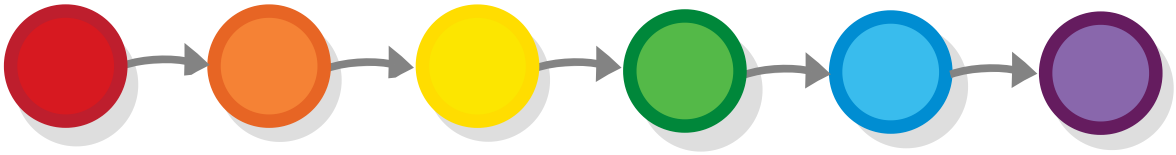
And the *binary relation* is a relation between two elements, which is often denoted with an arrow.



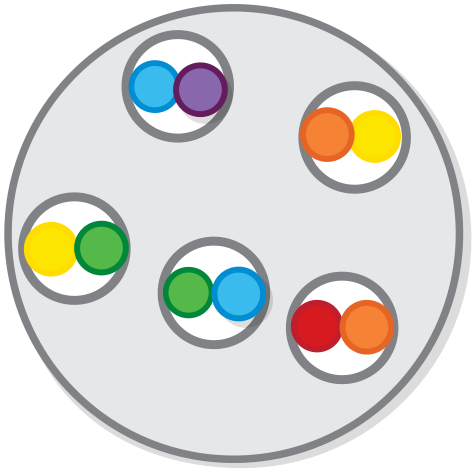
As for the laws, they are different depending on the type of order.

Linear order

Let's start with an example — the most straightforward type of order that you think of is *linear order* i.e. one in which every object has its place depending on every other object. In this case the ordering criteria is completely deterministic and leaves no room for ambiguity in terms of which element comes before which. For example, order of colors, sorted by the length of their light-waves (or by how they appear in the rainbow).



Using set theory, we can represent this order, as well as any other order, as a sets of pairs of the order's underlying set with itself (a subset of the product set).



And in programming, orders are defined by providing a function which, given two objects, tells us which one of them is “bigger” (comes first) and which one is “smaller”. It isn’t hard to see that this function defines a set of pairs (we are given a pair and we have to say whether or not it belongs to the set).

```
[1, 3, 2].sort((a, b) => {
  if (a > b) {
    return true
  } else {
    return false
  }
})
```

However (this is where it gets interesting) not all such functions (and not all sets of pairs) define orders. For such function to really define an order i.e. to have the same output every time, independent of how the objects were shuffled initially, it has to obey several rules.

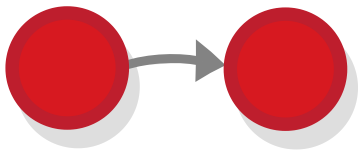
Incidentally, (or rather not incidentally at all), these rules are nearly equivalent to the mathematical laws that define the criteria of the order relationship i.e. those are the rules that define which element can point to which.

> **DEFINITION:**
>
> A linear order is a set of elements, together with a *binary relation* between these elements, that obeys the laws of reflexivity, transitivity, antisymmetry, totality.

Let's check what they are.

Reflexivity

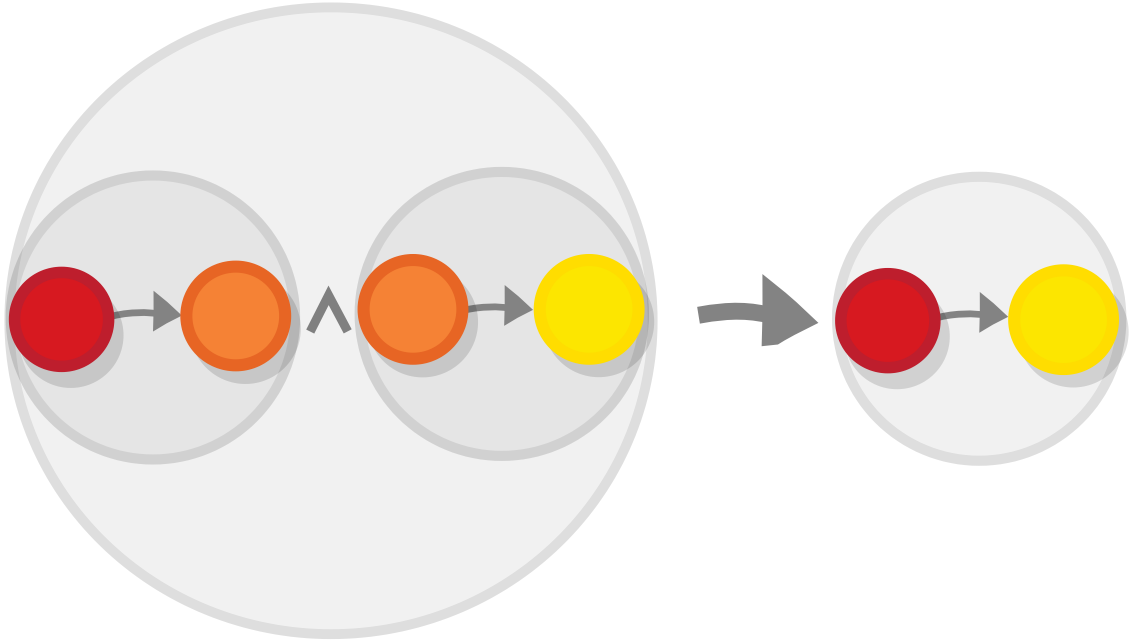
Let's get the boring law out of the way — each object has to be bigger or equal to itself, or $a \leq a$ for all a (the relationship between elements in an order is commonly denoted as $\leq$ in formulas, but it can also be represented with an arrow from first object to the second.)



This law only exist to cover the "base case": we can formulate it the opposite way too and say that each object should *not* have the relationship to itself, in which case we would have a relation than resembles *bigger than*, as opposed to *bigger or equal to* and a slightly different type of order, sometimes called a *strict* order.

Transitivity

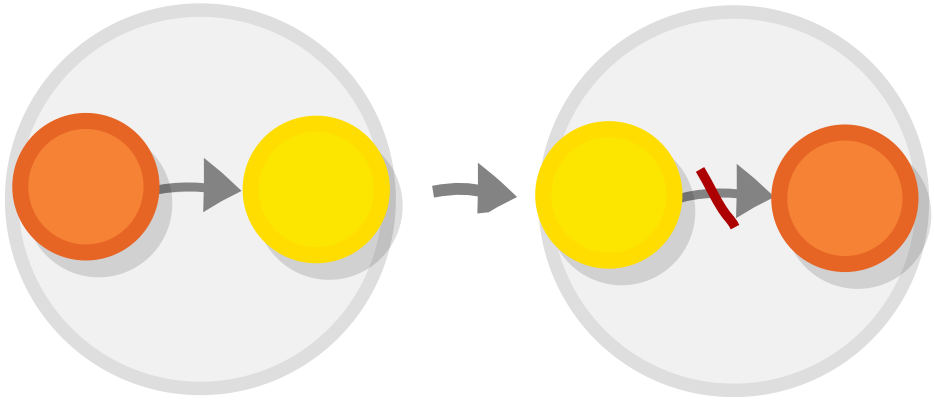
The second law is maybe the least obvious, (but probably the most essential) — it states that if object a is bigger than object b, it is automatically bigger than all objects that are smaller than b or $a \leq b \wedge b \leq c \rightarrow a \leq c$.



This is the law that to a large extend defines what an order is: if I am better at playing soccer than my grandmother, then I would also be better at it than my grandmother's friend, whom she beats, otherwise I wouldn't really be better than her.

Antisymmetry

The third law is called antisymmetry. It states that the function that defines the order should not give contradictory results (or in other words you have $x \leq y$ and $y \leq x$ only if $x = y$).

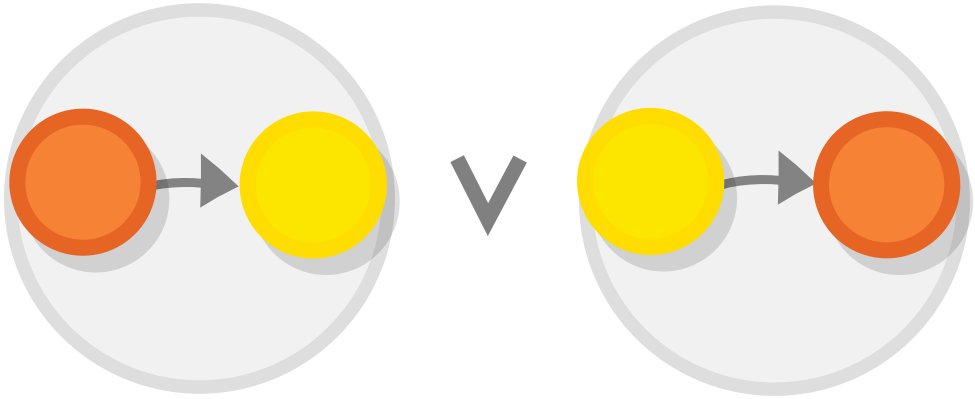


It also means that no ties are permitted — either I am better than my grandmother at soccer or she is better at it than me.

Totality

The last law is called *totality* (or *connexity*) and it mandates that all elements that belong to the order should be *comparable* ($a \leq b \vee b \leq a$). That is, for any two elements, one would always be "bigger" than the other.

By the way, the law of totality makes the reflexivity law redundant, as reflexivity is just a special case of totality when a and b are one and the same object, but I still want to present it for reasons that will become apparent soon.



Actually, here are the reasons: the law of totality can be removed. Orders, that don't follow the totality law are called *partial orders*, (and linear orders are also called *total orders*.)

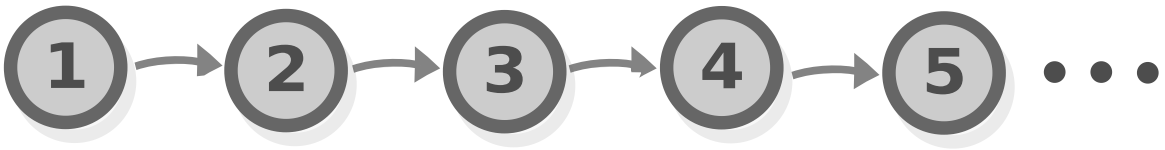
Task 1: Previously, we covered a relation that is pretty similar to this. Do you remember it? What is the difference?

Task 2: Think about some orders that you know about and figure out whether they are partial or total.

Partial orders are actually much more interesting than linear/total orders. But before we dive into them, let's say a few things about numbers.

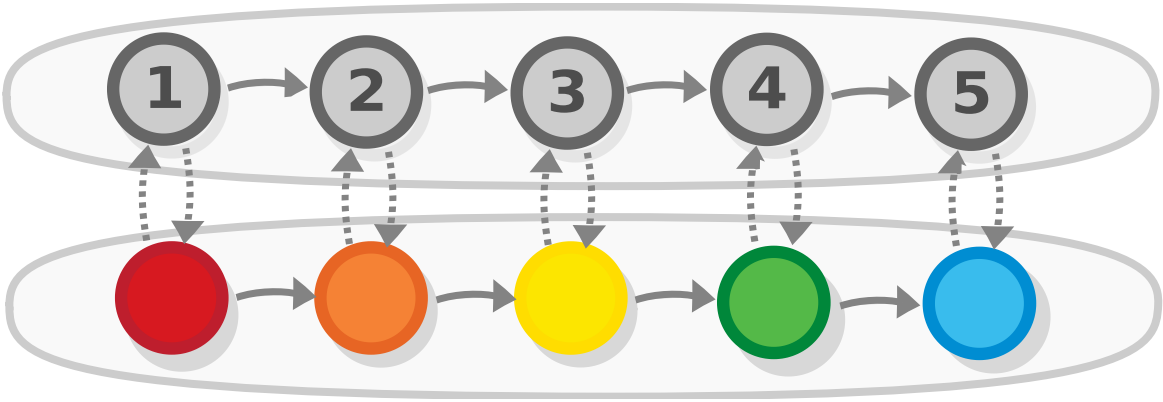
The order of natural numbers

Natural numbers form a linear order under the operation *bigger or equal to* (the symbol of which we have been using in our formulas.)



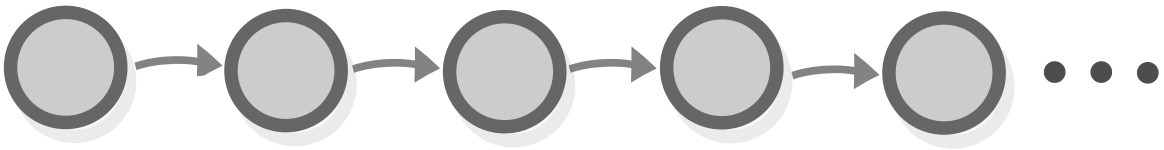
In many ways, natural numbers are the quintessential order – every finite order of objects is isomorphic to a subset of the order of numbers, as we can map the first element of any order to the number 1, the second one to the number 2 etc (and we can do the opposite operation as well).

If we think about it, this isomorphism is actually closer to the everyday notion of a linear order, than the one defined by the laws – when most people think of order, they aren't thinking of a *transitive*, *antisymmetric* and *total* relation, but are rather thinking about criteria based on which they can decide which object comes first, which comes second etc. So it's important to notice that the two notions are equivalent.



From the fact that any finite order of objects is isomorphic to the natural numbers, it also follows that all linear orders of the same magnitude are isomorphic to one another.

So, the linear order is simple, but it is also (and I think that this isomorphism proves it) the most *boring* order ever, especially when looked from a category-theoretic viewpoint – all finite linear orders (and most infinite ones) are just isomorphic to the natural numbers and so all of their diagrams look the same way.



However, this is not the case with partial orders that we will look into next.

Partial order

Law of totality does not look so "set in stone" as the rest of the laws i.e. we can probably think of some situations in which it does not apply. For example, if we aim to order all people based on soccer skills there are many ways in which we can rank a person compared to their friends their friend's friends etc. but there isn't a way to order groups of people who never played with one another.

Remove the law of totality from the laws of linear orders and we get a *partial order* (also a *partially-ordered set*, or *poset*).

> **DEFINITION:**
>
> An partial order is a set of elements, together with a *binary relation* between those elements, that obeys the laws of reflexivity, transitivity and antisymmetry.

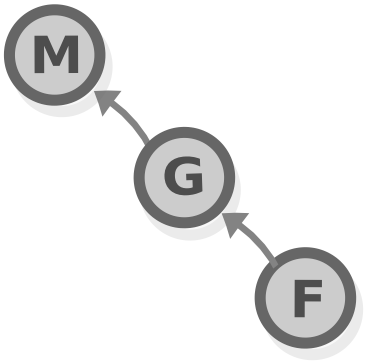
Every linear order is also a partial order (just as a group is still a monoid), but not the other way around.

We can even create an *order of orders*, based on which is more general.

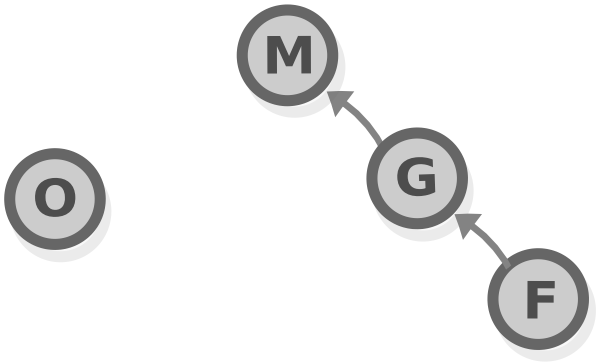
Partial orders are also related to the concept of an *equivalence relations* that we covered in chapter 1, except that *symmetry* law is replaced with *antisymmetry*.

If we revisit the example of the soccer players rank list, we can see that the first version that includes just **m**yself, my **g**randmother and her **f**riend is a

linear order.



However, including this **o**ther person whom none of us played yet, makes the hierarchy non-linear i.e. a partial order.

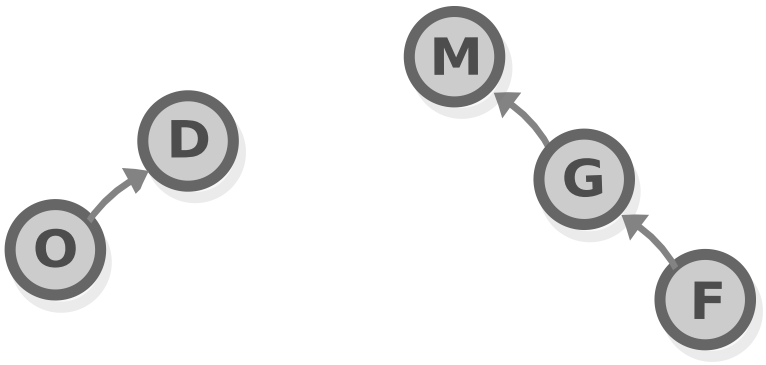


This is the main difference between partial and total orders — partial orders cannot provide us with a definite answer of the question who is better than who. But sometimes this is what we need — in sports, as well as in other domains, there isn't always an appropriate way to rate elements linearly.

Chains

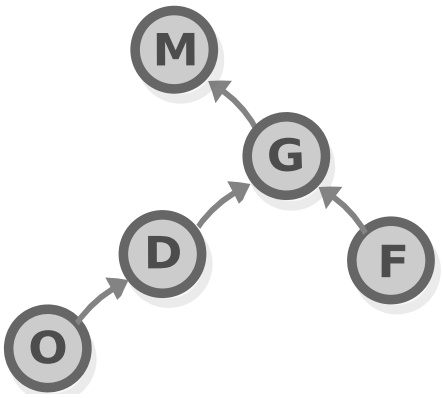
Before, we said that all linear orders can be represented by the same chain-like diagram, we can reverse this statement and say that all diagrams that look something different than the said diagram represent partial orders.

An example of this is a partial order that contains a bunch of linearly-ordered subsets, e.g. in our soccer example we can have separate groups of friends who play together and are ranked with each other, but not with anyone from other groups.



The different linear orders that make up the partial order are called *chains*. There are two chains in this diagram $m \to g \to f$ and $d \to o$.

The chains in an order don't have to be completely disconnected from each other in order for it to be partial. They can be connected as long as the connections are not all *one-to-one* i.e. ones when the last element from one chain is connected to the first element of the other one (this would effectively unite them into one chain.)

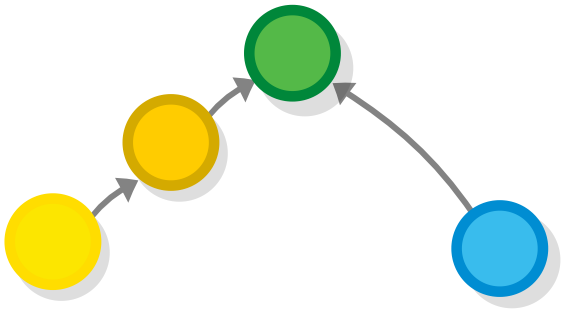


The above set is not linearly-ordered — although we know that $d \leq g$ and that $f \leq g$, the relationship between d and f is *not* known — any element can be bigger than the other one.

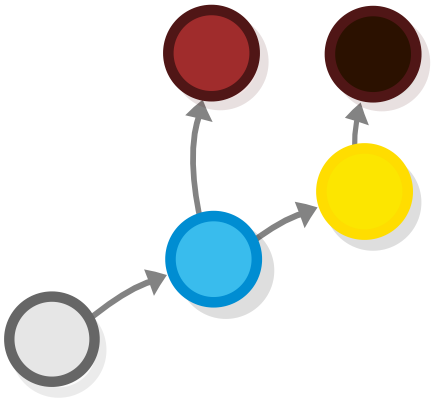
Greatest and least objects

Although partial orders don't give us a definitive answer to "Who is better than who?", some of them still can give us an answer to the more important question (in sports, as well as in other domains), namely "Who is number one?" i.e. who is the champion, the player who is better than anyone else. Or, more generally, the element that is bigger than all other elements.

> The *greatest element* of an order is an element a, such that we have we have $x \leq a$ for any other element x, Some (not all) partial orders do have such element — in our last diagram m is the greatest element, in this diagram, the green element is the biggest one.



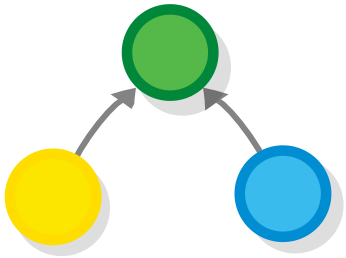
Sometimes we have more than one elements that are bigger than all other elements, in this case none of them is the greatest.



In addition to the greatest element, a partial order may also have a least (smallest) element, which is defined in the same way.

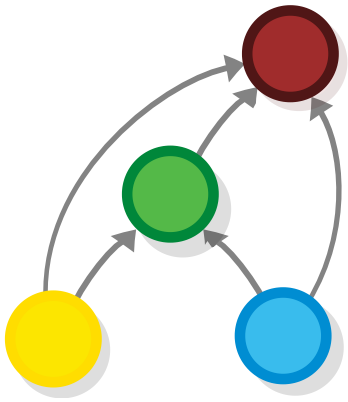
Joins

The *least upper bound* of two elements that are connected as part of an order is called the *join* of these elements, e.g. the green element is a join of the other two.

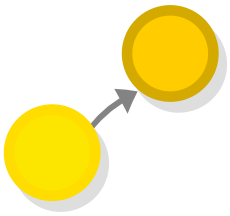


The join of a and b is the smallest element c that is bigger than then, formally:

> **DEFINITION:**
>
> The *join* of objects A and B is an object G, such that:
>
> 1. It is bigger than both of these objects, so $A \leq G$ and $B \leq G$.
> 2. It is smaller than any other object that is bigger than them, so for any other object P such that $P \leq A$ and $P \leq B$ then we should also have $G \leq P$.

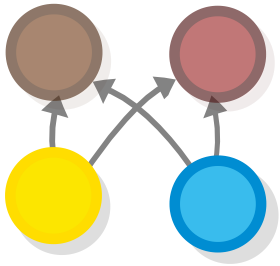


Given any two elements in which one is bigger than the other (e.g. $a \leq b$), the join is this bigger element (in this case b)

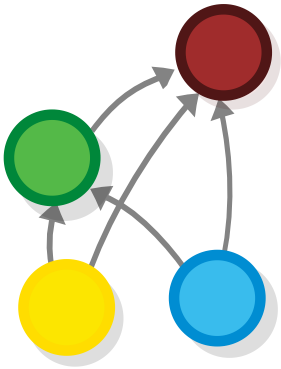


e.g. in a linear orders, the *join* of any two elements is just the bigger element.

Like with the greatest element, if two elements have several upper bounds that are equally big, then none of them is a *join* (a join must be unique).



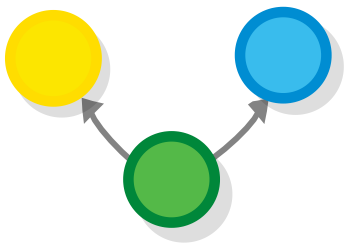
If, however, one of those elements is established as smaller than the rest of them, it immediately qualifies.



Task 3: Which concept in category theory reminds you of joins?

Meets

Given two elements, the biggest element that is smaller than both of them is called the *meet* of these elements.



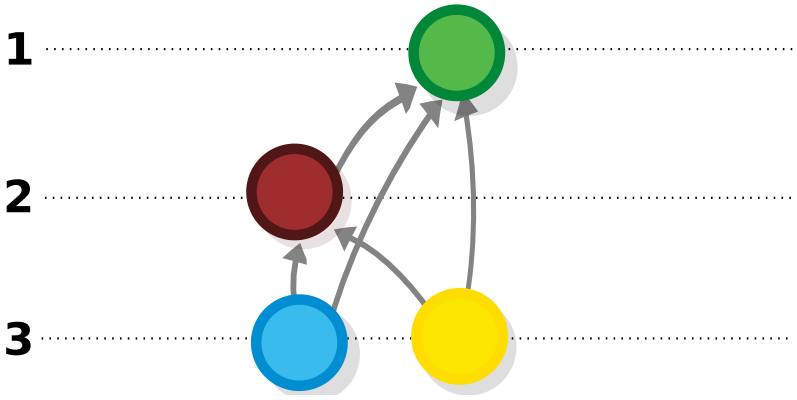
The same rules as for the joins apply, but in reverse.

Hasse diagrams

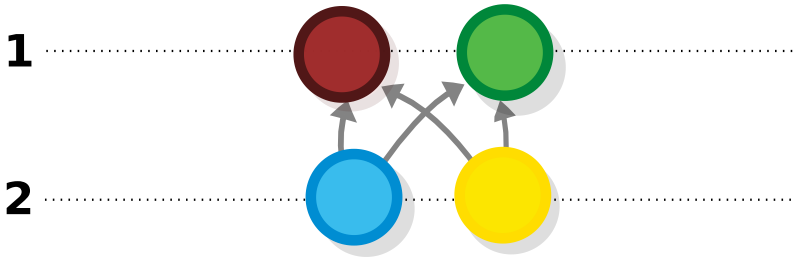
The diagrams that we use in this section are called “Hasse diagrams” and they work much like our usual diagrams, however they have an additional rule that is followed — “bigger” elements are always positioned above smaller ones.

In terms of arrows, the rule means that if you add an arrow to a point, the point *to* which the arrow points must always be above the one *from* which it points.

Hasse diagrams allow us to compare any two points by just seeing which one is above the other e.g. we can determine the *join* of two elements, by just identifying the elements that they connect to and see which one is lowest.



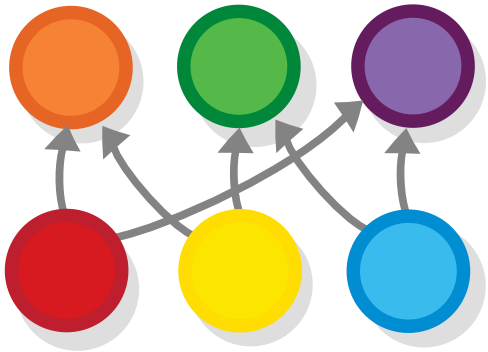
Likewize, we immediately see if two elements have no join.



Color-mixing partial order

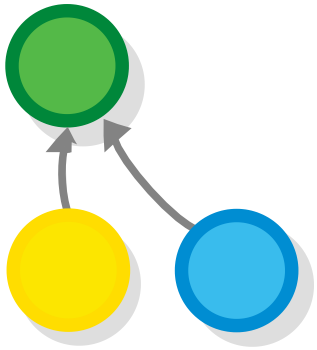
We all know many examples of total orders (any form of chart or ranking is a total order), but there are probably not so many obvious examples of partial orders that we can think of off the top of our head. So let's see some. This will gives us some context, and will help us understand what joins are.

To stay true to our form, let's revisit our color-mixing monoid and create a *color-mixing partial order* in which all colors point to colors that contain them.



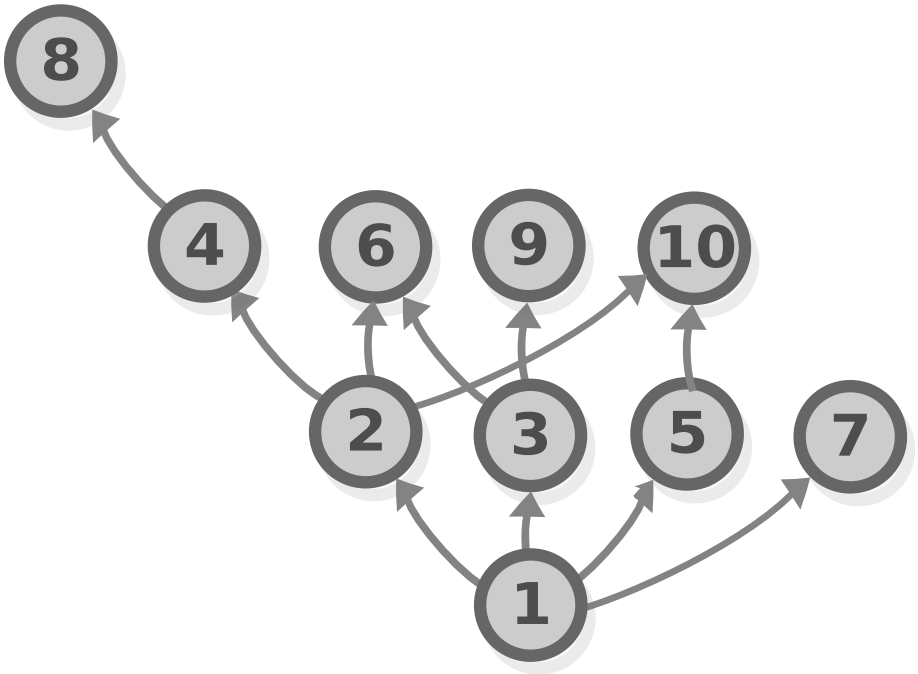
If you go through it, you will notice a curious property of the join

> **THEOREM:**
>
> In the color-mixing order, the join of any two colors is the color that they make up when mixed.



The partial order of numbers by division

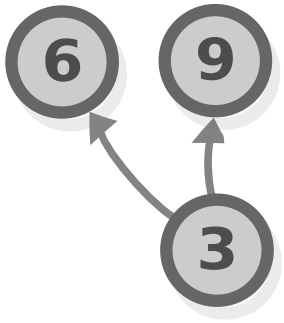
We saw that when we order numbers by "bigger or equal to", they form a linear order. But numbers can also form a partial order, for example they form a partial order if we order them by which divides which, i.e. if a divides b, then a is before b e.g. because $2 \times 5 = 10$, 2 and 5 come before 10 (but 3, for example, does not come before 10.)



And it so happens (actually for very good reason) that the join operation again corresponds to an operation that is relevant in the context of the objects:

> **THEOREM:**

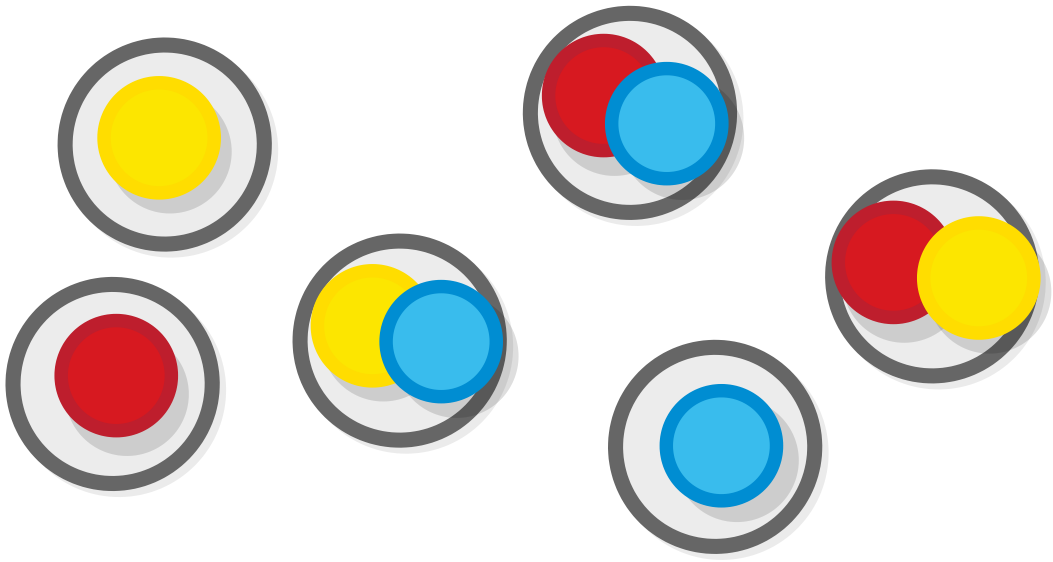
> In the partial order of numbers by division, the join of any two numbers is their *least common multiple*. And their *meet* is their *greatest common divisor*.



Let's dig a bit into why this happens.

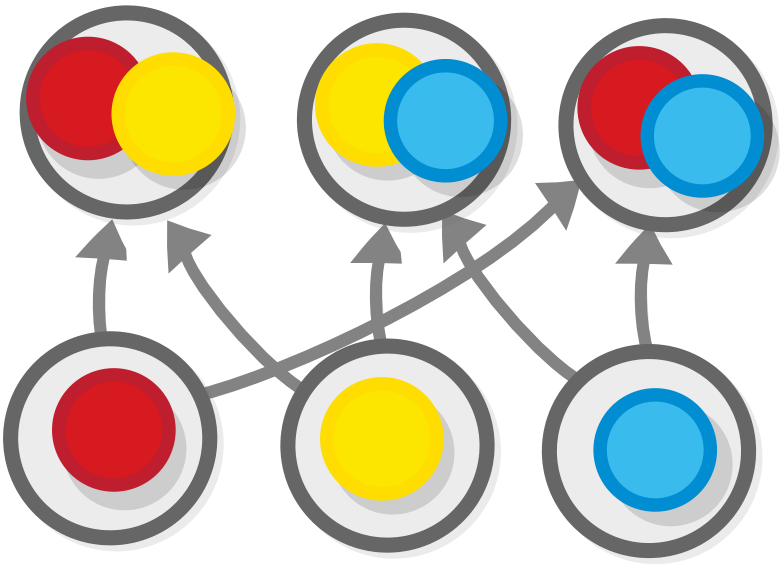
The inclusion partial order

Given a collection of sets containing a combination of a given set of elements...



...we can define what is called the *inclusion order* of those sets.

> **DEFINITION:**
>
> The *inclusion order* of a given collection of sets (usually sets that contain some common elements) is an order, based on the following binary relation: A comes before B if A *includes* B, or in other words if B is a *subset* of A.

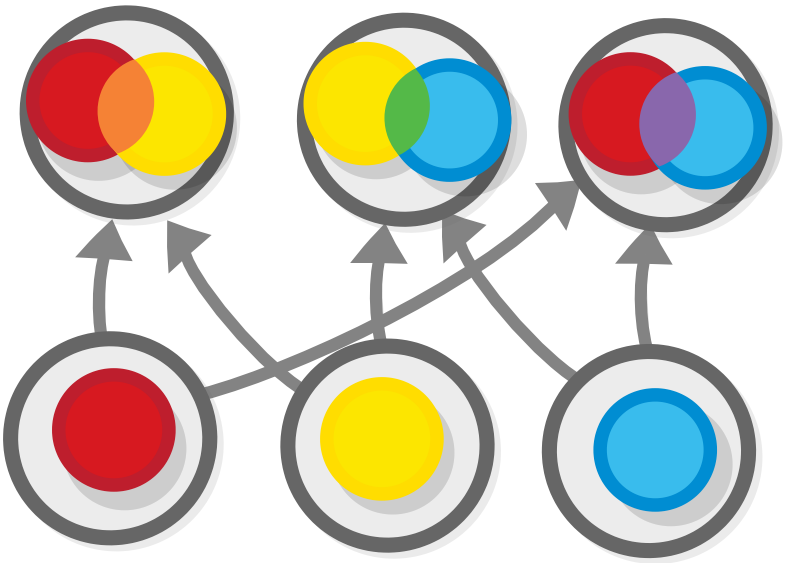


This means that...

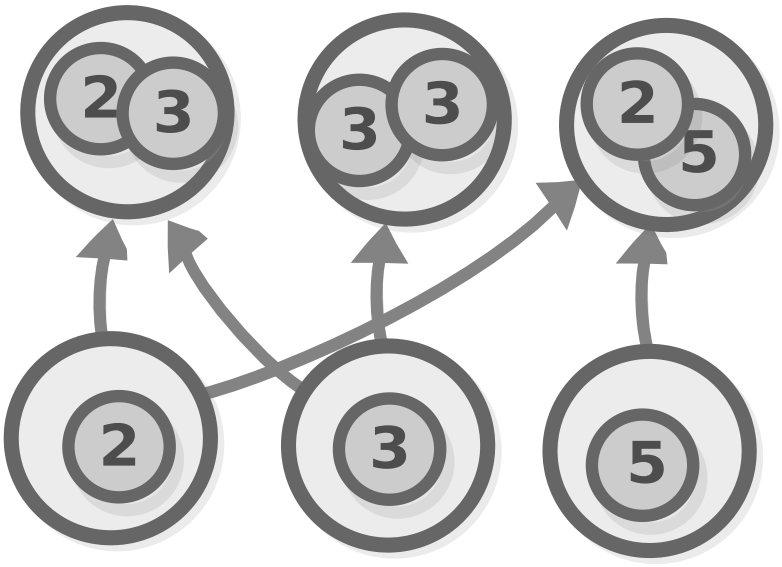
> **THEOREM:**
>
> the *join* operation of two sets in an inclusion order is their *union*, and the *meet* operation is their set *intersection*.

Birkhoff's representation theorem

This diagram might remind you of something — if we take the colors that are contained in each set and mix them into one color, we get the color-blending partial order that we saw earlier.



The order example with the number dividers is also isomorphic to an inclusion order, namely the inclusion order of all possible sets of *prime* numbers, including repeating ones (or alternatively the set of all *prime powers*). This is confirmed by the fundamental theory of arithmetic, which states that every number can be written as a product of primes in exactly one way.



So far, we saw two different partial orders, one based on color mixing, and one based on number division, that can be represented by the inclusion orders of all possible combinations of sets of some *basic elements* (the primary colors in the first case, and the prime numbers (or prime powers) in the second one.) Many other partial orders can be defined in this way. Which ones exactly, is a question that is answered by an amazing result called *Birkhoff's representation theorem*. They are the *finite* partial orders that meet the following two criteria:

1. All elements have *joins* and *meets*.
2. Those *meet* and *join* operations *distribute* over one another, that is if we denote joins as meets as $\vee$ or $\wedge$, then $x \vee (y \wedge z) = (x \vee y) \wedge (x \vee z)$.

The partial orders that meet the first criteria are called *lattices*. The ones that meet the second one are called *distributive lattices*. Let's write that down:

> Partial orders in which all elements have *joins* and *meets* is called a *lattice*. A lattice whose *meet* and *join* operations *distribute* over one

another is called a distributive lattice.

And the "prime" elements which we use to construct the inclusion order are the elements that are not the *join* of any other elements. They are also called *join-irreducible* elements.

So we may phrase the theorem like this:

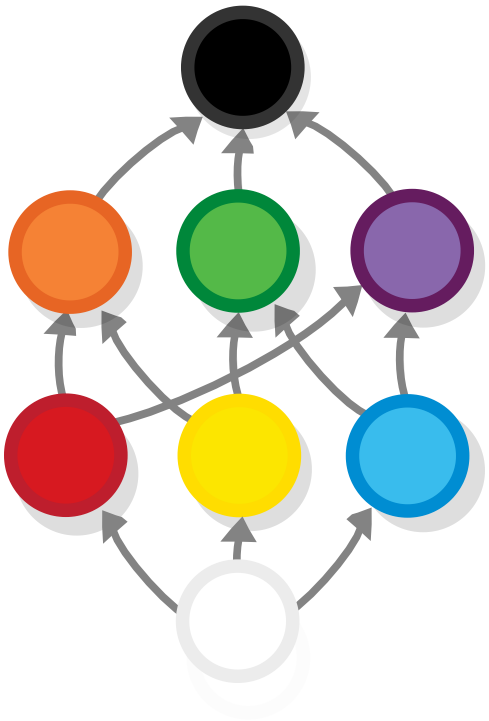
> **THEOREM:**
>
> Each distributive lattice is isomorphic to an inclusion order of its *join-irreducible* elements.

By the way, the partial orders that are *not* distributive lattices are also isomorphic to inclusion orders, it is just that they are isomorphic to inclusion orders that *do not contain all possible combinations* of elements.

Lattices

We will now talk more about *lattices* (the orders for which Birkhoff's theorem applies). Lattices are partial orders, in which every two elements have a *join* and a *meet*. So every lattice is also partial order, but not every partial order is a lattice (we will see even more members of this hierarchy).

Most partial orders that are created based on some sort of rule are distributive lattices, like for example the partial orders from the previous section are also distributive lattices when they are drawn in full, for example the color-mixing order.



Notice that we added the black ball at the top and the white one at the bottom. We did that because otherwise the top three elements wouldn't have a *join* element, and the bottom three wouldn't have a *meet*.

Bounded lattices

Our color-mixing lattice, has a *greatest element* (the black ball) and a *least element* (the white one). Lattices that have a least and greatest elements are called *bounded lattices*. It isn't hard to see that all finite lattices are also bounded.

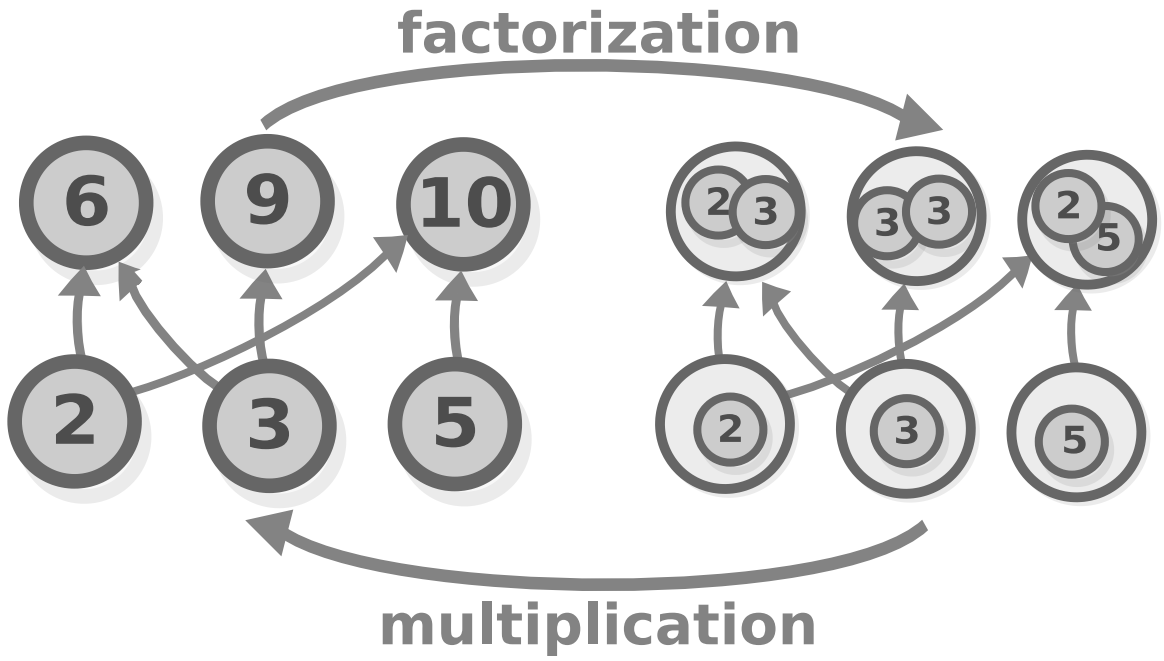
Task 4: Prove that all finite lattices are bounded.

Order isomorphisms

We mentioned order isomorphisms several times already so this is about time to elaborate on what they are.

Given two sets (we will use partial order of numbers by division and the prime inclusion order as an example) an isomorphism between them is comprised of the following two functions:

1. One function from the prime inclusion order, to the number order (which in this case is just the *multiplication* of all the elements in the set)
2. One function from the number order to the prime inclusion order (which is an operation called *prime factorization* of a number, consisting of finding the set of prime numbers that result in that number when multiplied with one another).



An order isomorphism is essentially an isomorphism between the orders' underlying sets (invertible function). However, besides their underlying sets, orders also have the arrows that connect them, so there is one more condition: in order for an invertible function to constitute an order isomorphism, it has to *respect those arrows*.

> **DEFINITION:**
>
> An isomorphism between two orders is an invertible function between their underlying sets, such that applying this function (let's call it F) to any two elements that have a certain order in one set (let's call them a and b) should result in two elements that have a corresponding order in the other set (i.e. $a \leq b$ if and only if $F(a) \leq F(b)$).

Such functions are called *order-preserving* functions.

Preorder

In the previous section, we saw how removing the law of *totality* from the laws of (linear) order produces a different (and somewhat more interesting) structure, called *partial order*. Now let's see what will happen if we remove another one of the laws, namely the *antisymmetry* law.

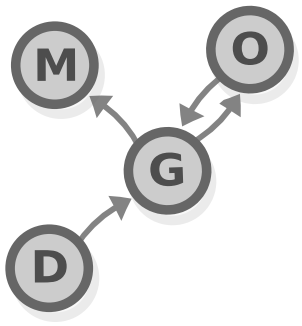
The antisymmetry law mandated that you cannot have an object that is at the same time smaller and bigger than another one. (or that $a \leq b \iff b \not\leq a$).

	Linear order	Partial order	Preorder
Element Comparability	$a \leq b$ or $b \leq a$	$a \leq b$ or $b \leq a$ or neither	$a \leq b$ or $b \leq a$ or neither or both
Reflexivity	X	X	X
Transitivity	X	X	X
Antisymmetry	X	X	-
Totality	X	-	-

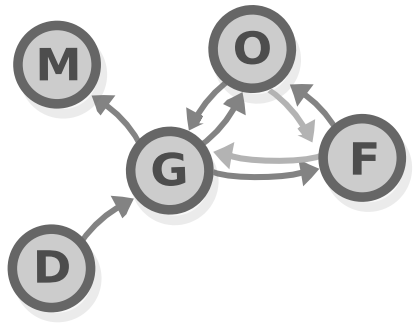
The result is a structure called a *preorder*:

> **DEFINITION:**
>
> An preorder is a set of elements, together with a *binary relation* between the elements of the set, that obeys the laws of reflexivity and transitivity.

Preorder is not exactly an order in the everyday sense — it can have arrows coming from any point to any other: if a partial order can be used to model who is better than who at soccer, then a preorder can be used to model who has beaten who, either directly (by playing him) or indirectly.



Preorders have just one law — *transitivity* $a \leq b \land b \leq c \to a \leq c$ (well, two, if we count *reflexivity*). The part about the indirect wins is a result of this law. Due to it, all indirect wins (ones that are wins not against the player directly, but against someone who had beat them) are added as a direct result of its application, as seen here (we show indirect wins in lighter tone).



And as a result of that, all "circle" relationships (e.g. where you have a weaker player beating a stronger one) result in just a bunch of objects that are all connected to one another.

All of that structure arises naturally from the simple law of transitivity.

Preorders and equivalence relations

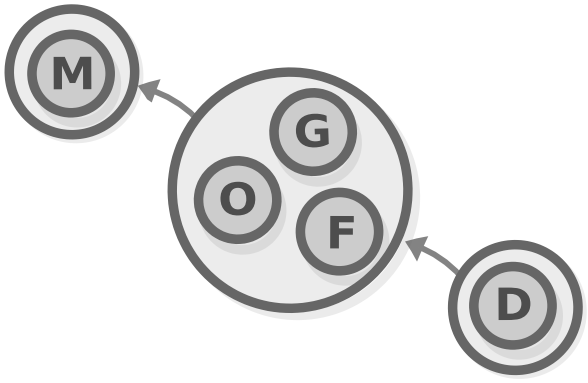
Preorders may be viewed as a middle-ground between *partial orders* and *equivalence relations*, as they are missing exactly the property on which those two structures differ — (anti)symmetry. Because of that, if we have a bunch of objects in a preorder that follow the law of *symmetry*, those objects form an equivalence relation. And if they follow the reverse law of *antisymmetry*, they form a partial order.

Equivalence relation	Preorder	Partial order
Reflexivity	Reflexivity	Reflexivity
Transitivity	Transitivity	Transitivity
Symmetry	-	Antisymmetry

In particular, any subset of objects that are connected with one another both ways (like in the example above) follows the *symmetry* requirement. So if we group all elements that have such connection, we would get a bunch of sets, all of which define different *equivalence relations* based on the preorder, called the preorder's *equivalence classes*.



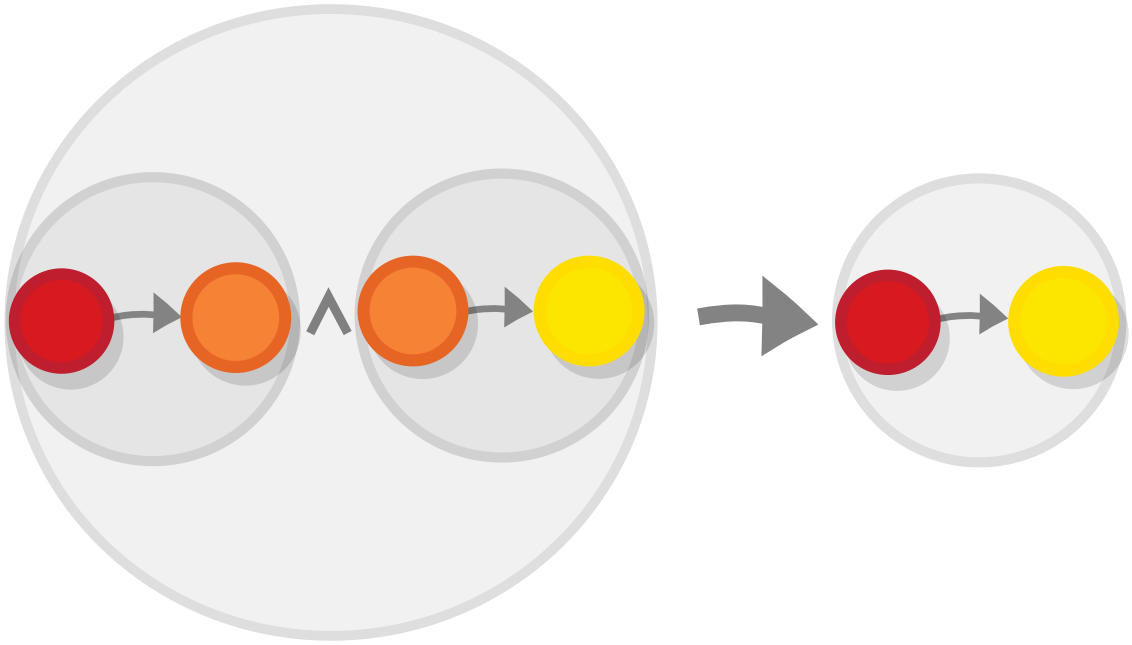
And, even more interestingly, if we transfer the preorder connections between the elements of these sets to connections between the sets themselves, these connections would follow the *antisymmetry* requirement, which means that they would form a *partial order*.



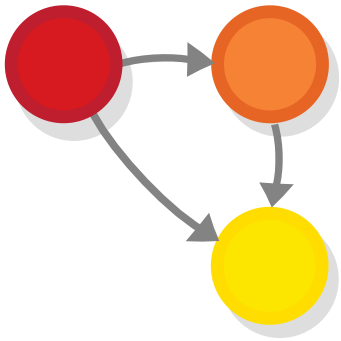
In short, for every preorder, we can define the *partial order of the equivalence classes of this preorder*.

Preorders as categories

We saw that preorders are a powerful concept, so let's take a deeper look at the law that governs them – the transitivity law. What this law tells us that if we have two pairs of relationship $a \leq b$ and $b \leq c$, then we automatically have a third one $a \leq c$.



In other words, the transitivity law tells us that the $\leq$ relationship composes i.e. if we view the "bigger than" relationship as a morphism we would see that the law of transitivity is actually the categorical definition of *composition*.



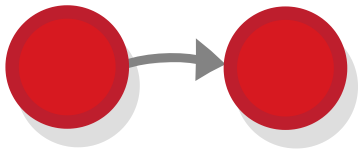
(we have to also verify that the relation is associative, but that's easy)

Formal definition

So, we suspect that preorders are categories, but is it really so? Let's review the definition of a category again.

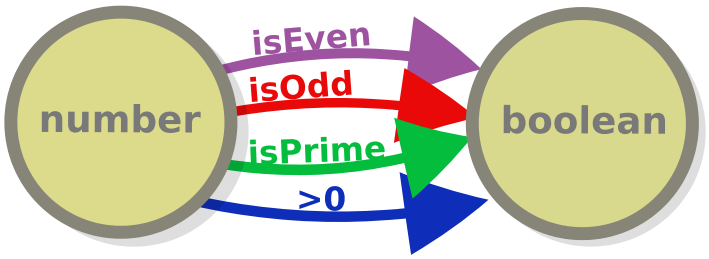
> **DEFINITION:**
>
> A category is a collection of *objects* (we can think of them as points) and *morphisms* (arrows) that go from one object to another, where:
>
> 1. Each object has to have the identity morphism.
> 2. There should be a way to compose two morphisms with an appropriate type signature into a third one in a way that is associative.

Looks like we have law number 2 covered, with transitivity. What about the identity law? We have it too, under the name *reflexivity*.

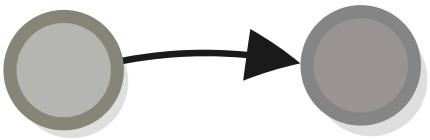


So it's official — preorders are categories (sounds kinda obvious, especially after we also saw that preorders can be reduced to sets and functions using the inclusion order, and sets and functions form a category in their own right).

Preorders are categories, but not all categories are preorders. Most categories have many different morphisms between given two objects. For example, in the category of sets where there are potentially infinite amount of functions from, say, the set of integers and the set of boolean values, as well as a lot of functions that go the other way around.



Whereas preorders, two object, whereas have *at most one morphism*, that is, we either have $A \leq B$ or we do not.



So, like a monoid is a category that has one object, an order is a category that has at most one *morphism* between two objects.

> **DEFINITION:**
>
> A preorder, any preorder, can be seen as a *category with at most one morphism between two given objects*— for any A and B, we say that if $A \leq B$ then a morphism $A \to B$ exists. The identity morphism exist because of reflexivity. The converse is also true: any category with no more than one morphism between two objects can be seen as a preorder.

An interesting fact that follows from they having at most one morphism between given two objects, is that in preorders *all diagrams commute automatically*.

Task 6: Prove this.

Partial orders and total orders as categories

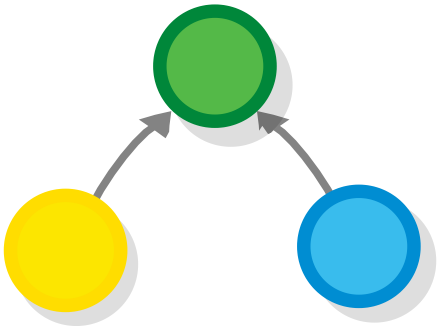
We said that partial orders and total orders are preorders. This means that they are categories as well.

Preorders in particular are what is known in category theory as *skeletal* categories — categories in which there are no isomorphic objects i.e. in which all isomorphic objects are identical.

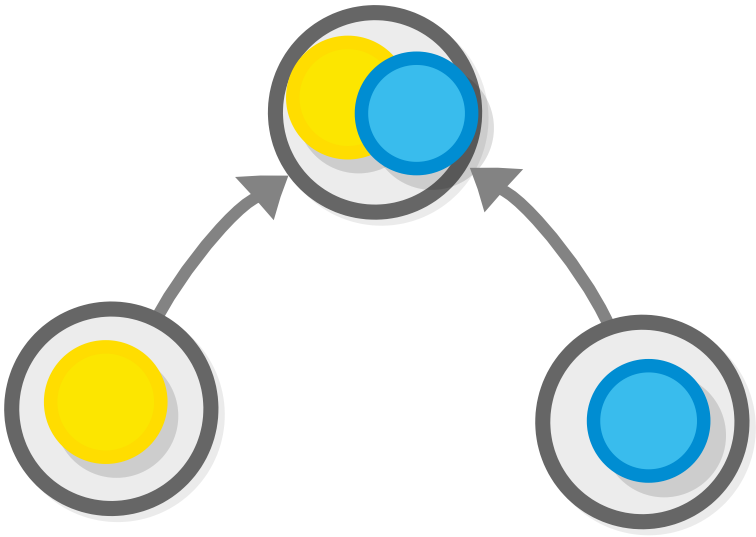
And total orders I guess we don't have a specific "categorical" name for them, but they are a certain type of categories as well.

Products and coproducts

While we are rehashing diagrams from the previous chapters, let's look at the diagram defining the *coproduct* of two objects in a category, from chapter 2.

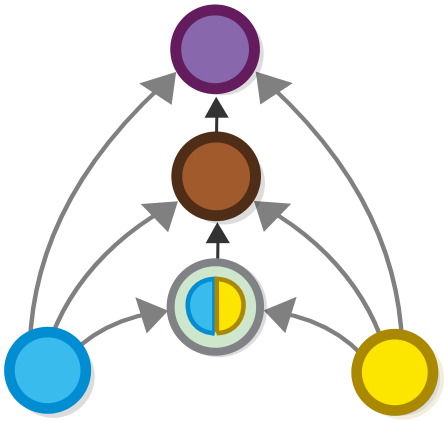


If you recall, this is an operation that corresponds to *set inclusion* in the category of sets.



But wait, wasn't there some other operation that that corresponded to set inclusion? Oh yes, the *join* operation in orders. And not merely that, but joins in orders are defined in the exact same way as the categorical coproducts.

> **DEFINITION:**
>
> The coproduct of A and B, denoted $A + B$, is an object, such that:
>
> 1. There exists two "projection" morphisms $A \to A + B$ and $B \to A + B$.
> 2. For any impostor coproduct I, that also has such projection morphisms ($A \to I$ and $B \to I$), there must also exist a unique morphism with the type signature $g : A + B \to I$, that converts the real coproduct to the impostor, such that the projections of the impostor would be just the composition of g with the projections of the product.



In the realm of orders, we define join as:

> **DEFINITION:**
>
> The *join* of objects A and B is an object G, such that:
>
> 1. It is bigger than both of these objects, so $A \leq G$ and $B \leq G$.
> 2. It is smaller than any other object that is bigger than them, so for any other object P such that $P \leq A$ and $P \leq B$ then we should also have $G \leq P$.

We can see that the two definitions, and their corresponding diagrams, are basically the same, we just replaced "bigger" with "has a unique morphism" (because in orders all morphisms are unique).

Speaking in category-theoretic terms, we can say that:

> **THEOREM:**
>
> The *categorical coproduct* in the *category of preorders* is the *join* operation.

Which of course means that *products* correspond to *meets* (duality).

Thin categories

In category-theoretic terms, orders (categories that have at most one morphism with a given type signature) are known as "thin" categories.

Thin categories are often used for exploring categorical concepts in a context that is easier to understand than in normal (non-thin) categories. For example, as we saw, understanding the *order-theoretic* concepts of meets and joins would help you better understand the *more general categorical* concepts of products and coproducts.

Thin categories are also helpful in contexts when we want to keep it simple and we aren't particularly interested in the differences between the morphisms that go from one object to another. We will see an example of that in the next chapter.

Logic

Now let's talk about one more *seemingly* unrelated topic just so we can "surprise" ourselves when we realize it's category theory. By the way, in this chapter there will be another surprise in addition to that, so don't fall asleep.

Also, I will not merely transport you to a different branch of mathematics, but to an entirely different discipline — *logic*.

What is logic

Logic is the science of the *possible*. As such, it is at the root of all other sciences, all of which are sciences of the *actual*, i.e. that which really exists. For example, if science explains how our universe works then logic is the part of the description which is also applicable to any other universe that is *possible to exist*. A scientific theory aims to be consistent with both itself and observations, while a logical theory only needs to be consistent with itself (and true regardless of observations).

So, we may say:

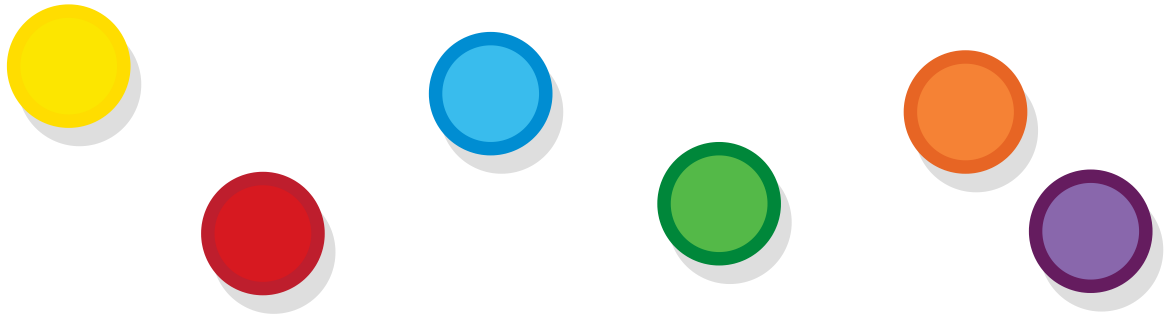
> **DEFINITION:**
>
> Logic studies the *rules* by which knowing one thing leads you to conclude (or *prove*) that some other thing is also true, regardless of the things' domain (e.g. scientific discipline) and by only referring to the form of the proof (i.e. "formally").
>
> On top of that, logic tries to organize those rules and arguments in *logical systems* (or *formal systems* as they are also called).

Seeing this description, we might think that the subject of logic is quite similar to the subject of set theory and category theory — instead of the word "formal" we used another similar word, namely "abstract", and instead of "logical system" we said "theory". This observation would be quite correct —

today most people agree that every mathematical theory is actually logic plus some additional definitions added to it. For example, part of the reason why *set theory* is so popular as a theory for the foundations of mathematics is that it can be defined by adding just one single primitive to the standard axioms of logic which we will see shortly — the binary relation that indicates *set membership*. Category theory is close to logic too, but in a quite different way, which we will understand later. So, let's begin.

Primary propositions

A consequence of logic being the science of the possible is that in order to do anything at all in it, we should have an initial set of propositions that we accept as true or false. These are also called "premises", "primary propositions" or "atomic propositions" as Wittgenstein dubbed them.



In the context of logic itself, these propositions are abstracted away (i.e. we are not concerned about them directly) and so they can be represented with the colorful balls that you are familiar with.

Composing propositions

At the heart of logic, as in category theory, is the concept of *composition* — if we have two or more propositions that are somehow related to one another, we can combine them into one using a *logical operators*, like "and", "or" and "implies/entails".

The results will be new propositions, which we might call *composite propositions* (to emphasize the fact that they are not primary).



Note that $\wedge$ is the symbol for *and* and $\vee$ is the symbol for *or* and $\rightarrow$ is the symbol for *follows*.

It is important to emphasize that propositions that are composed of several premises (symbolized by gray balls, containing some other balls) are not in any way different from "primary" propositions (single-color balls) and that they compose in the same way (although in the leftmost proposition the green ball is wrapped in a gray ball to make the diagram prettier).



Modus ponens

As an example of a proposition that contains multiple levels of nesting (and also as a great introduction of the subject of logic in its own right), consider one of the oldest (it was already known by Stoics at 3rd century B.C.) and most famous propositions ever, namely the *modus ponens*. Usually it is presented like this:

> **THEOREM:**
>
> If Socrates is human, then Socrates is mortal.
>
> But Socrates is human.

> So, Socrates is mortal.

Or we also can say:

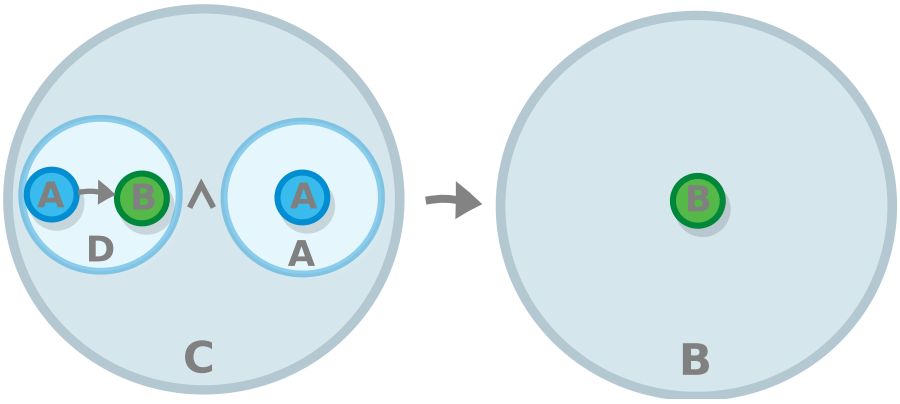
> **THEOREM:**
>
> If it rains, the ground gets wet.
>
> It rains.
>
> Therefore the ground gets wet.

You see the pattern:

> **DEFINITION:**
>
> Modus ponens is a proposition, comprised of two other propositions, denoted A and B, that states that if the proposition A is true, and that if A implies B $(A \to B)$, then B is true as well i.e. $(A \wedge (A \Rightarrow B)) \to B$.

In our first example, if we know that "Socrates is a human" (A) and that "humans are mortal"—or "being human implies being mortal" ($A \to B$), we also know that "Socrates is mortal" (B).

Here is how we can express the same thing with a diagram.



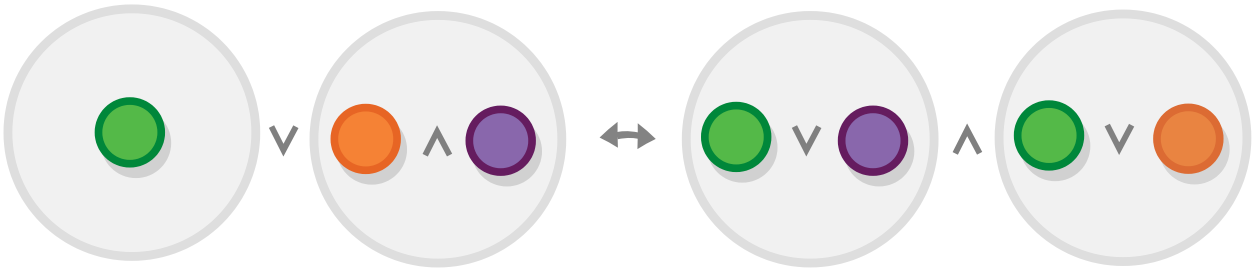
We can see that the modus ponens proposition is composed of two other propositions in a $implies$ relation, where the proposition B is primary, but the proposition which implies B is not primary (let's call that one C – so the whole proposition becomes $C \to B$.)

Going one more level down, we notice that the C propositions is itself composed of two propositions in an *and*, relationship – A and let's call the other one D (so $A \wedge D$), where D is itself composed of two propositions, this time in a $implies$ relationship – $A \to B$. But all of this is better visualized in the diagram.

Relations between logical operators

You might think that composition of logical propositions resembles the way in which two monoid objects are combined into one, using the monoid operation and, as we saw, some logical operations do form monoids.

However, unlike monoid/group theory, logic studies combinations of not just with one but with *many* logical operations and *the ways in which they relate to one another*, for example, in logic we are interested in the way "and" and "implies" operators relate to each other in "modus ponens". Or in the law of distributivity of *and* and *or* operations, which is represented by the following *tautology* (we will explain what that means later).



OK, we mentioned tautologies, now let's explain what they are.

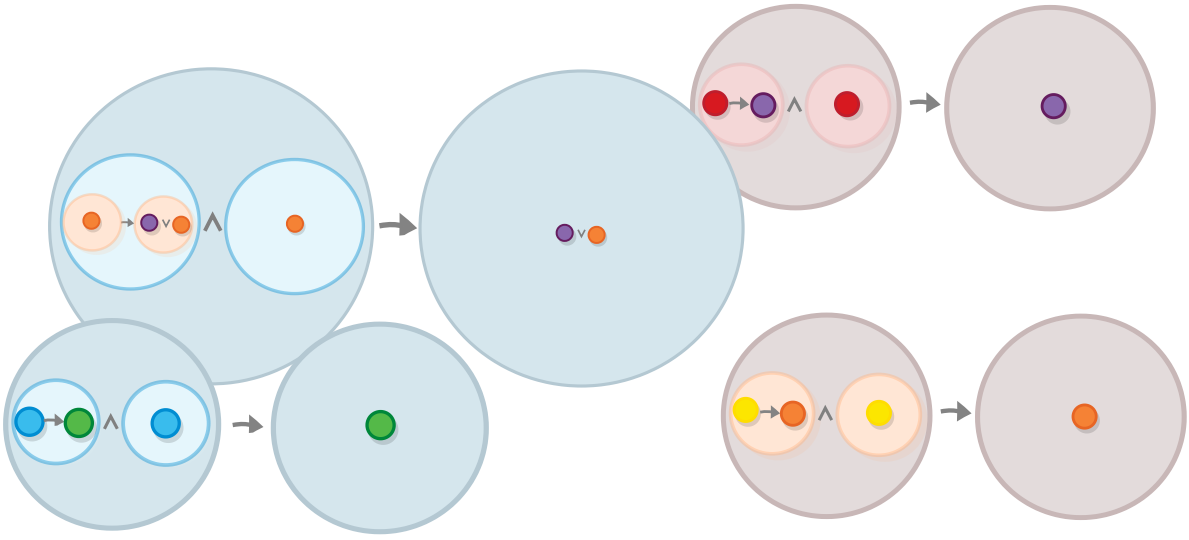
Tautologies

In most cases, we cannot tell whether a given composite proposition is true or false without knowing the values of the propositions that it is made of e.g. we cannot say if "A and B" or "A or B" is true, without knowing if A or B are true.



However, with propositions such as *modus ponens* we can: modus ponens is *always true*. Regardless of whether the propositions that form it (A and B) are true or false, the whole proposition signified by the formula ($(A \wedge (A \Rightarrow B)) \rightarrow B$) will always be true. If we want to be fancy, we can also say that it is *true in all models of the logical system*, a model being a set of real-world premises are taken to be signified by our propositions.

For example, our previous example will not stop being true if we *substitute* "Socrates" with any other name, nor if we substitute "mortal" for any other quality that humans possess.



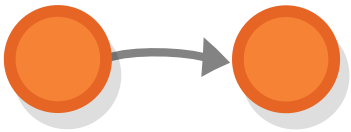
We call such propositions tautologies.

> **DEFINITION:**
>
> Propositions that are always true, regardless of are the value of the propositions that form them, are called *tautologies*.

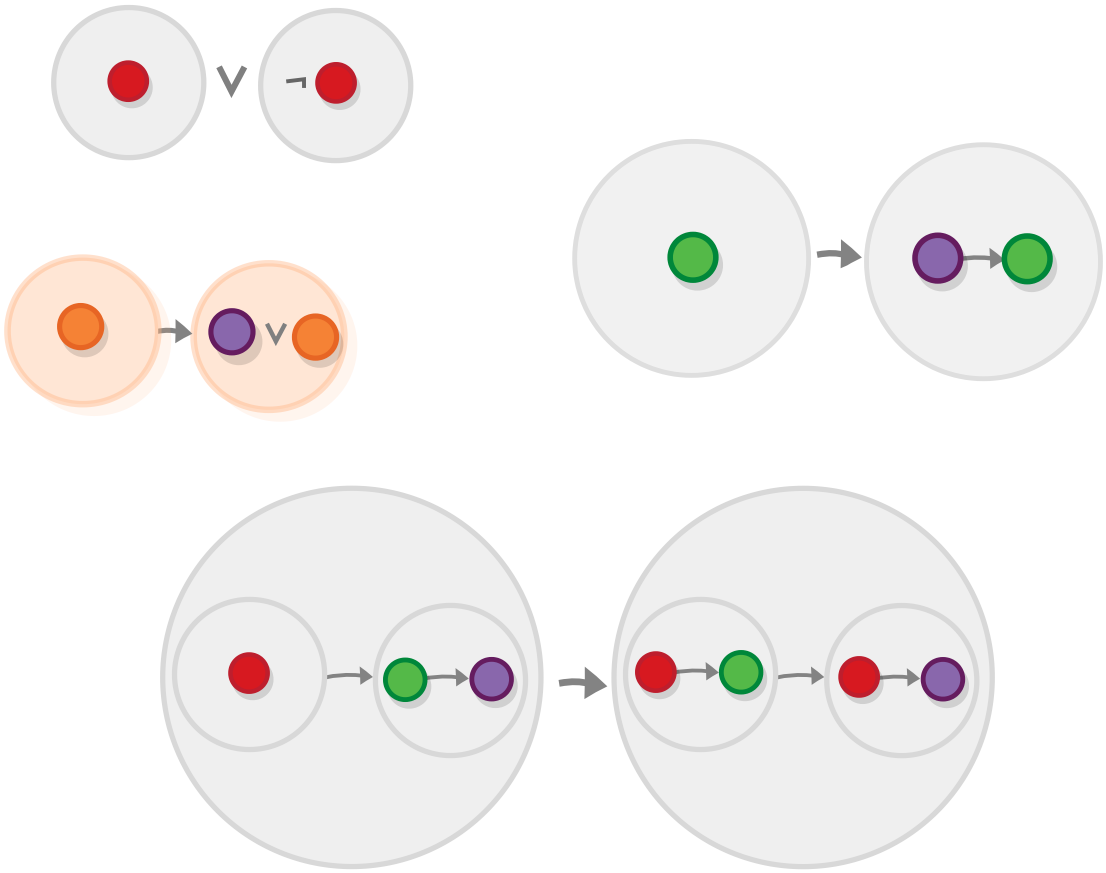
And their more-famous counterparts that are always false are called *contradictions*. You can turn each tautology into contradiction, or the other way around, by adding a "not".

The other statements, ones which may be true or false depending on the values of some other propositions are called "contingent statements". In logic, we don't care about contingent statements — after all, those are studied in all other sciences (and we are not like other sciences).

The simplest tautology is the so called law of identity, the statement that each proposition implies itself (e.g. "All bachelors are unmarried"). It may remind you of something.



Here are some more complex (less boring) tautologies (the symbol ¬ means "not"/negation.

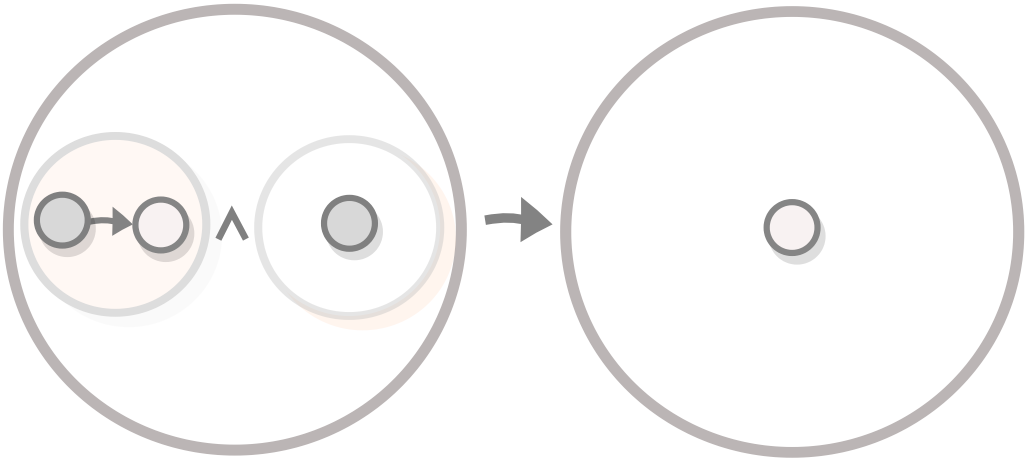


We will learn how to determine which propositions are a tautologies shortly, but first let's see why are tautologies important in the first place.

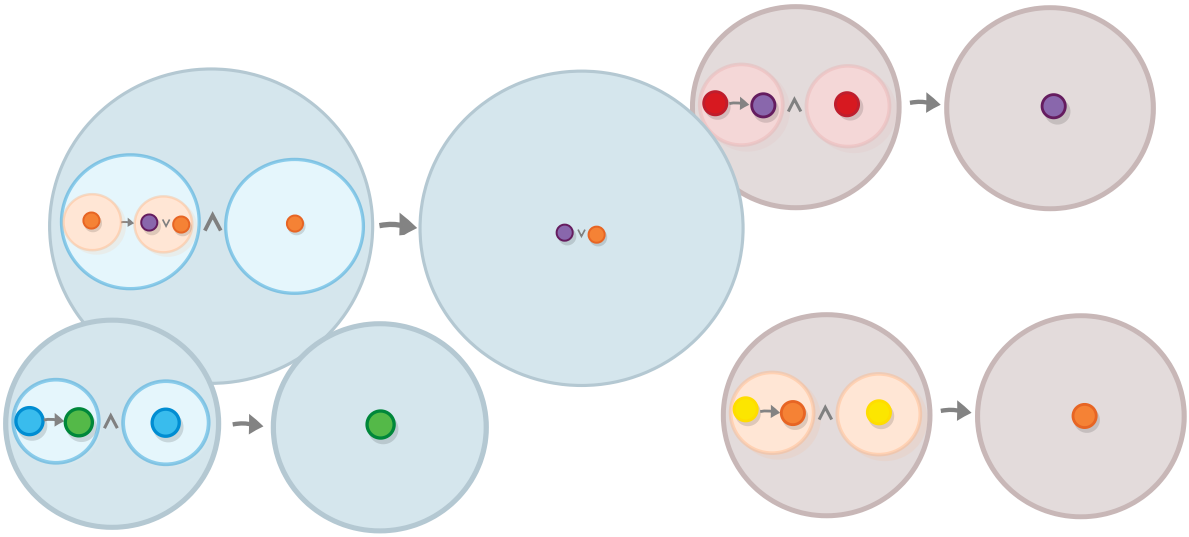
Axiom schemas/Rules of inference

Tautologies are useful because they are the basis of *axiom schemas/rules of inference*. And *axiom schemas* and *rules of inference* serve as starting point from which we can generate other true logical statements by means of substitution.

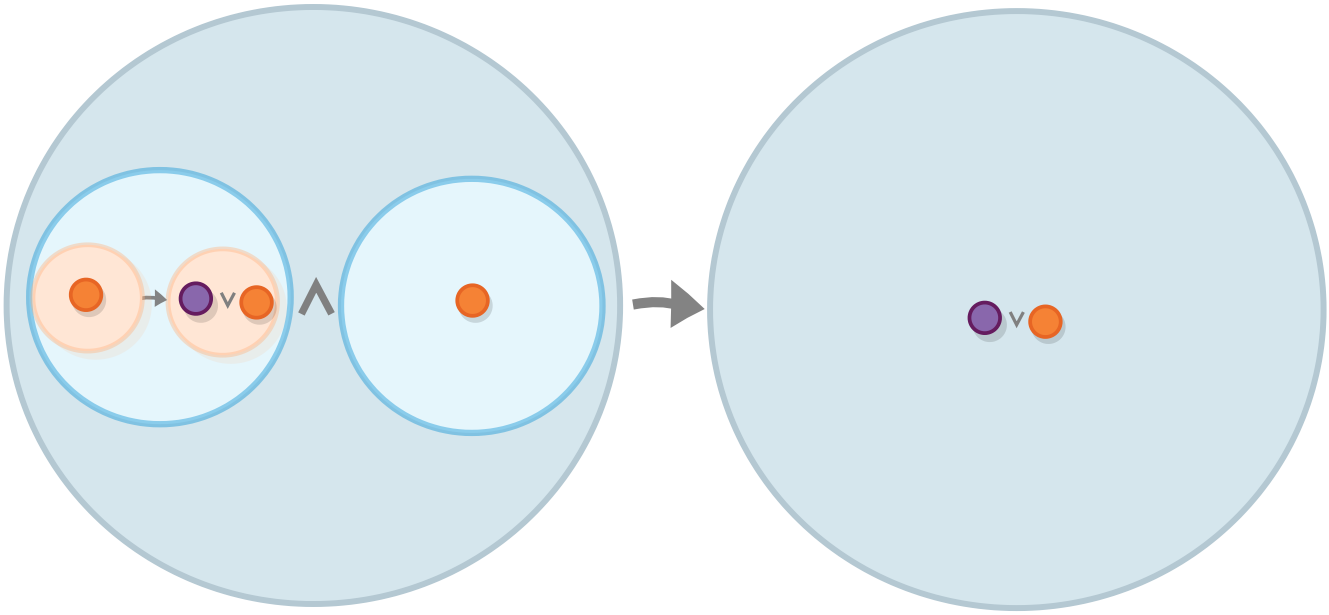
Realizing that the colors of the balls in modus ponens are superficial, we may want to represent the general structure (schema) of modus ponens that all of its variations share.



From then on, we can get to any modus-ponens proposition composed of primary proposition by just applying coloring.



Note that the propositions that we plug into the schema don't have to be primary. For example, having the proposition a (that is symbolized below by the orange ball) and the proposition stating that a implies $a \vee b$ (which is one of the tautologies that we saw above), we can plug those propositions into the *modus ponens* and prove that $a \vee b$ is true.



The basic structure of the proposition (the coloring book in our example) is called *axiom schema*. And the propositions that are produced by it are *axioms*.

> **DEFINITION:**
>
> An axiom schema is a formula (containing placeholders), from which we can derive propositions (by replacing those placeholders with propositions).

And *rules of inference* are almost the same thing as axiom schemas e.g. axiom schemas can be easily applied as rules of inference and the other way around.

Final note, in the previous chapter we repurposed one tautology (modus ponens) as an axiom schema. It is obvious that we can do the same thing for all other tautologies as well.

> **THEOREM:**
>
> Every tautology can be used as an axiom schema.

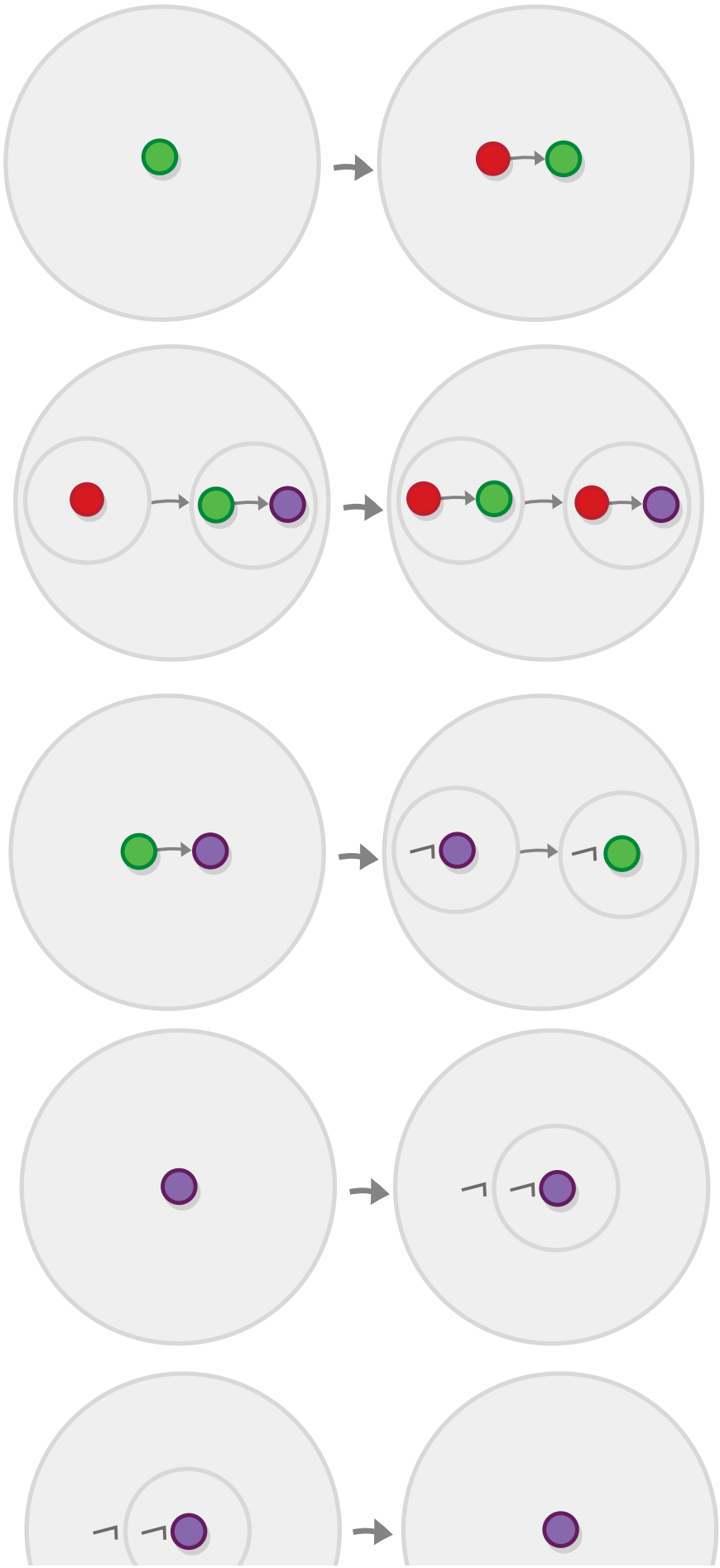
Logical systems

Knowing that we can use axiom schemas/rules of inference to generate new propositions, we might ask whether it is possible to create a small collection of such schemas/rules that is curated in such a way that it enables us to generate *all* possible propositions. You would be happy (although a little annoyed, I

imagine) to learn that there exist not only one, but many such collections. And yes, collections of this sort are what we call *logical systems*.

> **DEFINITION:**
>
> A logical system (known also as formal system) is a collection of axiom schemas/rules of inference such that by applying them we can produce all possible propositions.

Here is one such collection which consists of the following five axiom schemes *in addition to the inference rule modus ponens* (These are axiom schemes, even though we use colors).



Proving that this and other similar logical systems are complete (can really generate all other propositions) is due to Gödel and is known as "Gödel's completeness theorem" (Gödel is so important that I specifically searched for the "ö" letter so I can spell his name right).

Interpretations of logic

We now have an idea about how propositions, logical operators work. But we haven't actually *said what they are* (and in order to prove that they indeed work, we need to know what they are).

We haven't said this, because there are different definitions of what propositions and operators are, constituting different *interpretations* of logic. Now, we will look into two interpretations — one very old and the other, relatively recent. This would be a slight detour from our usual subject matter of points and arrows, but I assure you that it would be worth it. So let's start.

Classical logic. The truth-functional interpretation

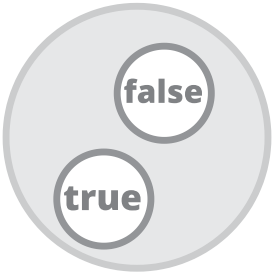
> Beyond the world that we inhabit and perceive every day, there exist the *world of forms* where reside all ideas and concepts that manifest themselves in the objects that we perceive e.g. beyond all the people that have ever lived, there lies the prototypical person, and we are people only insofar as we resemble that person, beyond all the things in the world that are strong, lies the ultimate concepts of strength, from which all of them borrow and this is true for every single category, e.g. if there is a cup, there is also "cupness". And although, as mere mortals, we live in the world of appearances and cannot perceive the world of forms, we can, through philosophy, "recollect" with it and know some of its features.

The above is a summary of a worldview that is due to the Greek philosopher Plato and is sometimes called Plato's *theory of forms*. Originally, the discipline of logic represents an effort to think and structure our thoughts in a way that they apply to this world of forms i.e. in a "formal" way. Today, this original paradigm of logic is known as "classical logic". Although it all started with Plato, most of it is due to the 20th century mathematician David Hilbert.

The existence of the world of forms implies that, even if there are many things that we, people, don't know and would not ever know, at least *somewhere out there* there exists an answer to every question. In logic, this translates to *the principle of bivalence* that states that *each proposition is either true or false*.



Due to this principle, propositions in classical logic can be aptly represented in set theory by the boolean set, which contains those two values.



Logical operators, then, are just our all-too-familiar functions.

DEFINITION:

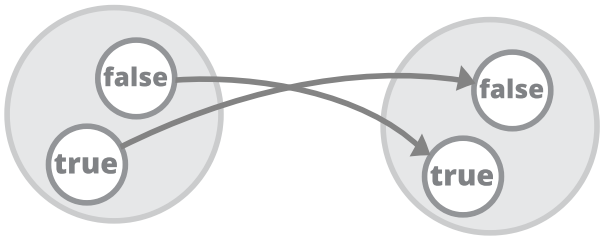
According to the classical interpretation of logic:

> - A *proposition* is something that is either true or false (a boolean value).
> - A *logical operator* is a *function* that takes a one or several boolean values and return another boolean value.

Let's review all logical operators in this semantic context.

The negation operation

Let's begin with the negation operation. Negation is a unary operation, which means that it is a function that takes just *one* argument and (like all other logical operators) returns one value, where both the arguments and the return type are boolean values.



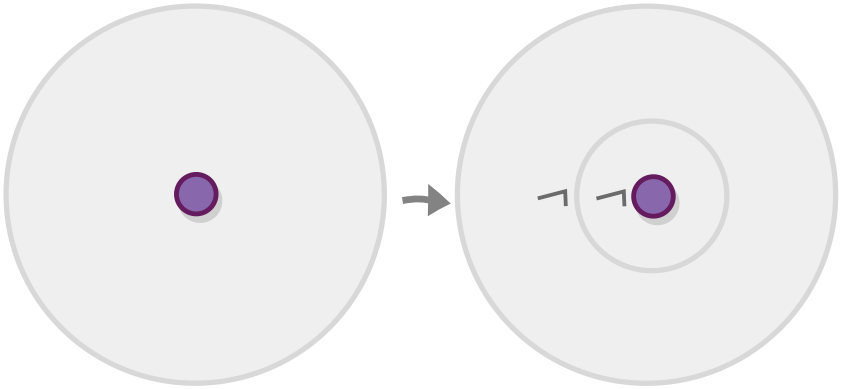
The same function can also be expressed in a slightly less-fancy way by this table.

p	¬p
True	False
False	True

Tables like this one are called *truth tables* and they are ubiquitous in classical logic. They can be used not only for defining operators but for proving results as well.

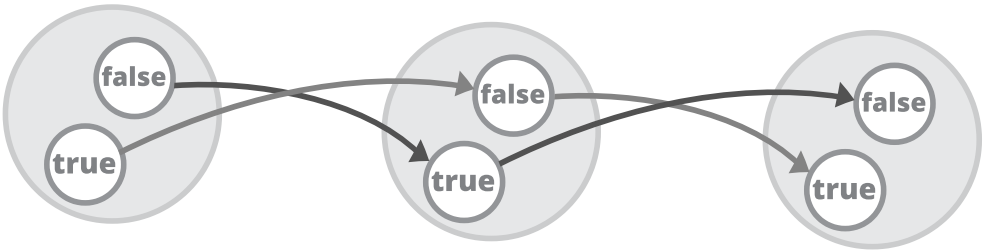
Proving results by truth tables

Having defined the negation operator, we are in position to prove the first of the axioms of the logical system we saw, namely the *double negation elimination*. In natural language, this axiom is equivalent to the observation that saying "I am *not unable* to do X" is the same as saying "I am *able* to do it".

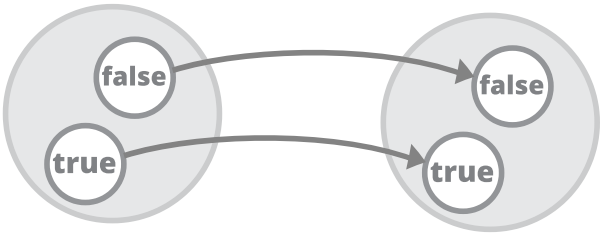


(despite its triviality, the double negation axiom is probably the most controversial result in logic, we will see why later.)

If we view logical operators as functions from and to the set of boolean values, than proving axioms involves composing several of those functions into one function and observing its output. More specifically, the proof of the formula above involves just composing the negation function with itself and verifying that it leaves us in the same place from which we started.



If we want to be formal about it, we might say that applying negation two times is equivalent to applying the *identity* function.



If we are tired of diagrams, we can represent the composition diagram above as table as well.

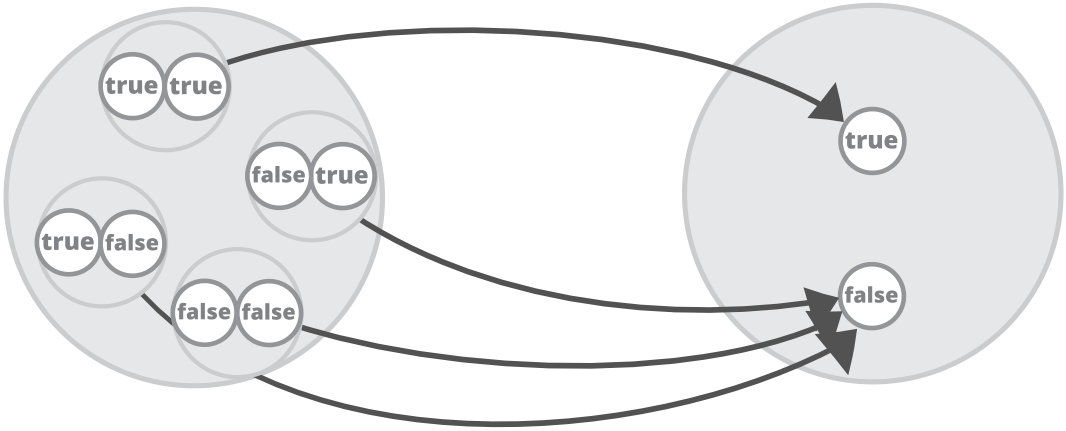
p	¬p	¬¬p
True	False	True
False	True	False

Each proposition in classical logic can be proved with such diagrams/tables.

The And and Or operations

OK, *you* know what *and* means and *I* know what it means, but what about those annoying people that want everything to be formally specified (nudge, nudge). Well we already know how we can satisfy them — we just have to construct the boolean function that represents *and*.

Because *and* is a *binary* operator, instead of a single value the function would accept a *pair* of boolean values.



Here is the equivalent truth-table (in which $\wedge$ is the symbol for *and*.)

p	q	p ∧ q
True	True	True
True	False	False
False	True	False
False	False	False

We can do the same for *or*, here is the table.

p	q	p ∨ q
True	True	True
True	False	True
False	True	True
False	False	False

Task 1: Draw the diagram for *or*.

Using those tables, we can also prove some axiom schemas we can use later:

- For *And*: $p \wedge q \to p$ and $p \wedge q \to q$ "If I am tired and hungry, this means that I am hungry".
- For *Or*: $p \to p \vee q$ and $q \to p \vee q$ "If I have a pen this means that I am either have a pen or a ruler".

The Implies operation

Let's now look into something less trivial: the *implies* operation, (also known as *material condition*). This operation binds two propositions in a way that the truth of the first one implies the truth of the second one (or that the first proposition is a *necessary condition* for the second.) You can read $p \to q$ as "if p is true, then q must also be true.

Implies is also a binary function — it is represented by a function from an ordered pair of boolean values, to a boolean value.

p	q	p → q
True	True	True
True	False	False
False	True	True
False	False	True

Now there are some aspects of this which are non-obvious so let's go through every case.

1. If p is true and q is also true, then p does imply q — obviously.
2. If p is true but q is false then q does not follow from p — cause q would have been true if it did.
3. If p is false but q is true, then p still does imply q. What the hell? Consider that by saying that p implies q we don't say that the two are 100% interdependent e.g. the claim that "drinking alcohol causes headache" does not mean that drinking is the only source of headaches.
4. And finally if p is false but q is false too, then p still does imply q (just some other day).

It might help you to remember that in classical logic $p \to q$ (p implies q) is true when $\neg p \vee q$ (either p is false or q is true.)

The If and only if operation

Now, let's review the operation that indicates that two propositions are equivalent (or, when one proposition is *a necessary and sufficient condition* for the other (which by itself implies that the reverse is also true.)) This operation yields true when the propositions have the same value.

p	q	p ↔ q
True	True	True
True	False	False
False	True	False
False	False	True

An interesting fact about the operation $A \leftrightarrow B$ is that it can be constructed using the *implies* operation — it is equivalent to each of the propositions

implying the other one

> For any P and Q, $P \leftrightarrow Q$ precisely when $P \to Q \wedge Q \to P$)

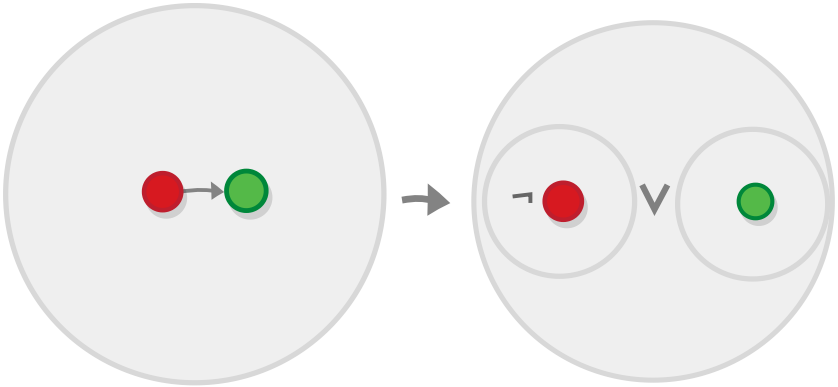
We can easily prove this by comparing the truth tables.

p	q	p → q	q → p	p → q ∧ q → p
True	True	True	True	True
True	False	False	True	False
False	True	True	False	False
False	False	True	True	True

Because of this, the equivalence operation is called “if and only if”, or “iff” for short.

Proving results by axioms/rules of inference

Let’s examine the formula, stating that $p \to q$ is the same as $\neg p \vee q$.

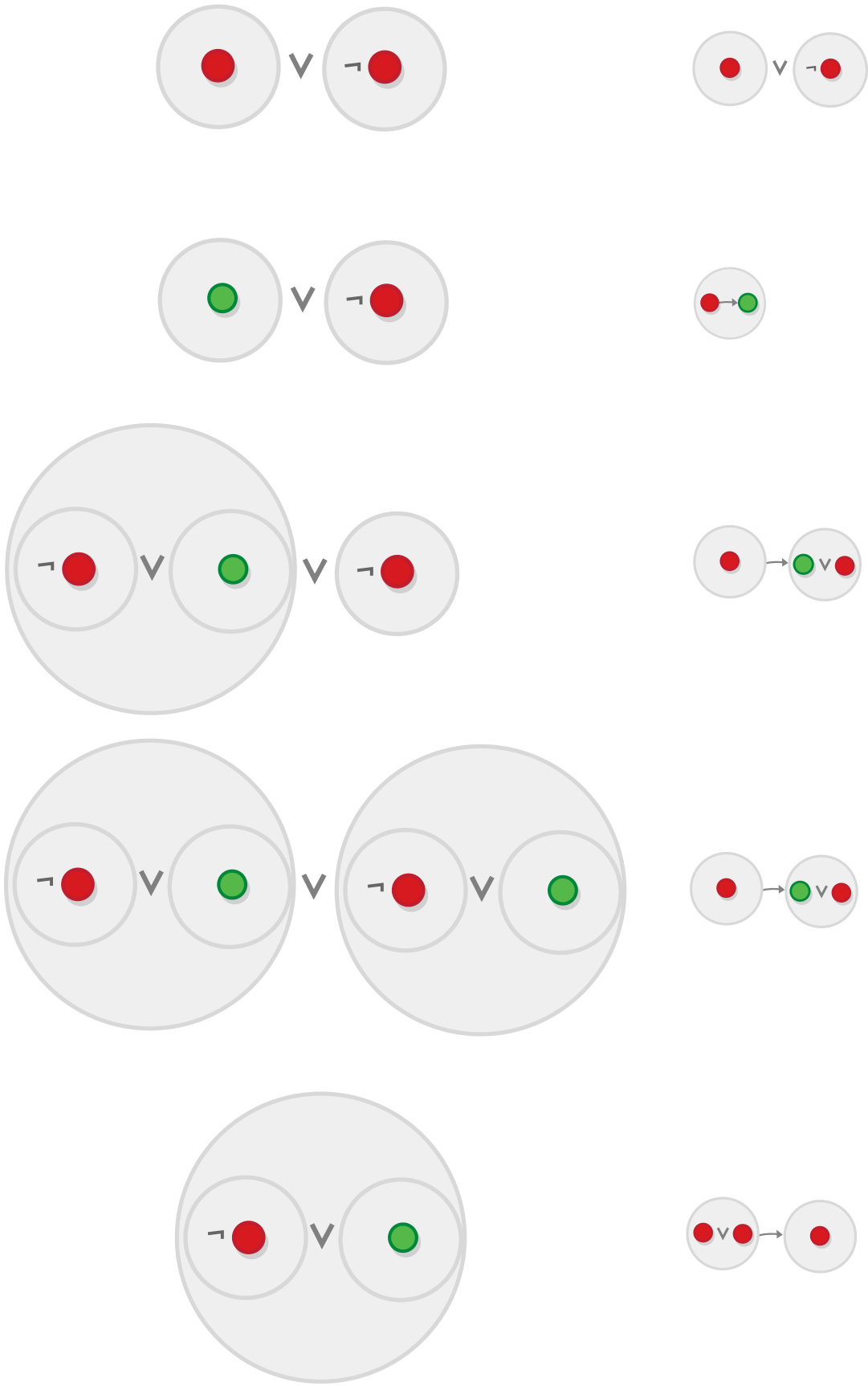


We can easily prove this by using truth tables.

p	q	p → q	¬p	q	¬p ∨ q
True	True	**True**	False	True	**True**
True	False	**False**	False	False	**False**
False	True	**True**	True	True	**True**
False	False	**True**	True	False	**True**

But it would be much more intuitive if we do it using axioms and rules of inference. To do so, we start with the formula we have ($p \to q$) plus the axiom schemas, and arrive at the formula we want to prove ($\neg p \vee q$).

Here is one way to do it. The formulas that are used at each step are specified at the right-hand side, the rule of inference is modus ponens.



Note that to really prove that the two formulas are equivalent we have to also do it the other way around (start with $(\neg p \vee q)$ and $(p \to q)$).

Intuitionistic logic. The BHK interpretation

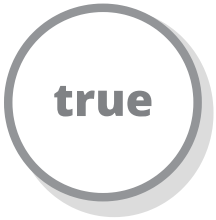
> [...] logic is life in the human brain; it may accompany life outside the brain but it can never guide it by virtue of its own power. — L.E.J. Brouwer

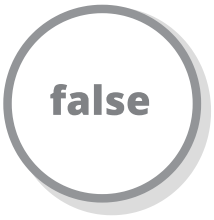
I don't know about you, but I feel that the classical truth-functional interpretation of logic (although it works and is correct in its own right) doesn't fit well the categorical framework that we are using here: It is too "low-level", it relies on manipulating the values of the propositions. According to it, the operations *and* and *or* are just 2 of the 16 possible binary logical operations and they are not really connected to each other (but we know that they actually are.)

For these and other reasons, in the 20th century a whole new school of logic was founded, called *intuitionistic logic*. If we view classical logic as based on *set theory*, then intuitionistic logic would be based on *category theory* and its related theories. If *classical logic* is based on Plato's theory of forms, then intuitionism began with a philosophical idea originating from Kant and Schopenhauer: the idea that the world as we experience it is largely predetermined of out perceptions of it. Thus without absolute standards for truth, a proof of a proposition becomes something that you *construct*, rather than something you discover.

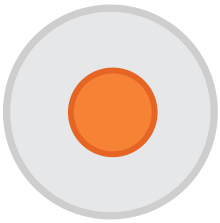
Classical and intuitionistic logic diverge from one another right from the start: because according to intuitionistic logic we are *constructing* proofs rather than *discovering* them as some universal truth, we are *off with the principle of bivalence*. That is, in intuitionistic logic we have no basis to claim that each statements is necessarily *true or false*. For example, there might be a statements that might not be provable not because they are false, but simply because they fall outside of the domain of a given logical system (the twin-prime conjecture is often given as an example for this.)

Anyway, intuitionistic logic is not bivalent, i.e. we cannot have all propositions reduced to true and false.





But there is one thing that we still *do* have — there are still propositions that are "true" in the sense that a proof for them is given — the primary propositions. So with some caveats (which we will see later) the bivalence between true and false proposition might be thought out as similar to the bivalence between the existence or absence of a proof for a given proposition — there either is a proof of it or there isn't.



This bivalence is at the heart of what is called the Brouwer-Heyting-Kolmogorov (BHK) interpretation of logic, something that we will look into next.

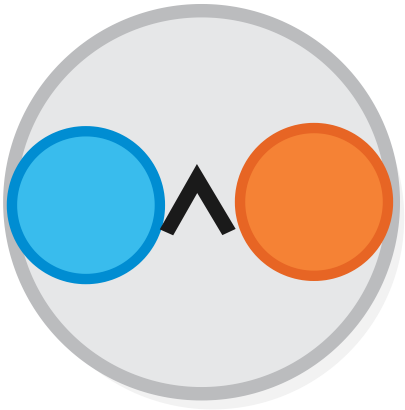
Here is a definition of the BHK interpretation (note that the BHK interpretation the main concept is not that of proposition, but that of proof).

> **DEFINITION:**
>
> According to the BHK interpretation of logic:
>
> - A *proposition* is something that has a proof.
> - A *logical operator* is a construction that creates proofs from other proofs.

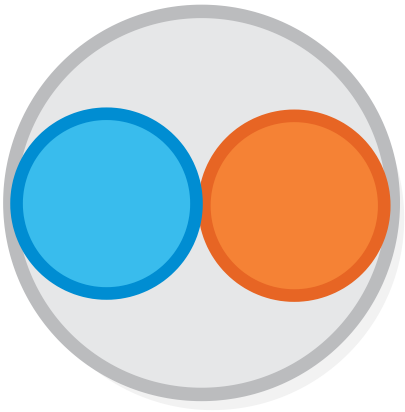
The original formulation of the BHK interpretation is not based on any particular mathematical theory. Here, we will first illustrate it using the language of set theory (just so we can abandon it a little later).

The And and Or operations

As the existence of a proof of a proposition is taken to mean that the proposition is true, the definitions of *and* is rather simple — the proof of that proposition $$A ∧ B$



...is just *a pair* containing a proof of A, and a proof of B i.e. *a product* of the two!



> **DEFINITION:**
>
> According to the BHK interpretation, a proof of $A \wedge B$ is a product of a proofs of A and B ($A \times B$).

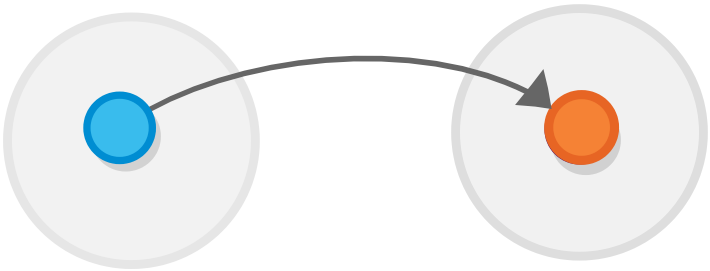
The principle for determining whether the proposition is true or false is similar to that of primary propositions — if the pair of proofs of A and B exist (i.e. if

both proofs exist) then the proof of $A \wedge B$ can be constructed (and so $A \wedge B$ is "true").

Task 2: What would be the **or** operation in this case?

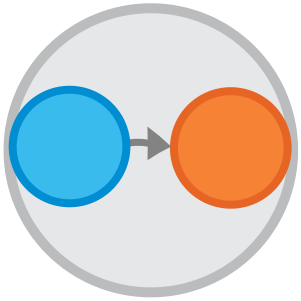
The Implies operation

Now for the punchline: in the BHK interpretation, the *implies* operation is just a *function* between proofs.



> **DEFINITION:**
>
> According to the BHK interpretation, saying that A implies B ($A \to B$), would just mean that there exist an arrow (function) that can convert a proof of A to a proof of B.

What is a proof that A implies B then? A proof that A implies B is just an element of the set of functions that go from A to B i.e. the *hom set* of $A \Rightarrow B$ – if this set is empty, then there is no proof (i.e. no way to convert a proof of A to a proof of B).

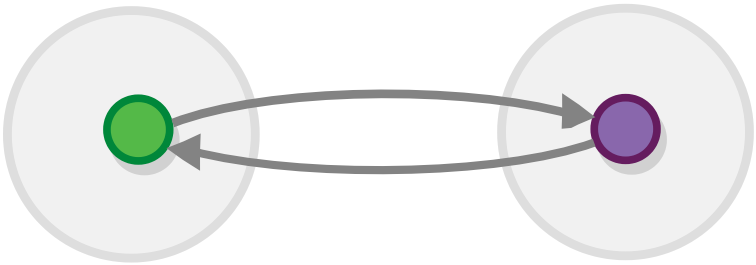


> **DEFINITION:**

> The set of proofs of $A \to B$ is the hom-set of A to B ($A \Rightarrow B$).

The If and only if operation

In the BHK interpretation we have no If and only if operation. But we have arrows.



In the section on classical logic, we proved that two propositions A and B are equivalent if A implies B and B implies A. But if the *implies* operation is just a function, then proposition are equivalent precisely when there are two functions, converting each of them to the other i.e. when the sets containing the propositions are *isomorphic*.

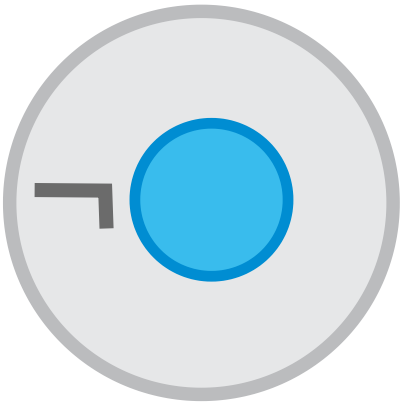
(Perhaps we should note that *not all set-theoretic functions are proofs*, only a designated set of them (which we call *canonical* functions) i.e. in set theory you can construct functions and isomorphisms between any pair of singleton sets, but that won't mean that all proofs are equivalent.)

The Negation operation

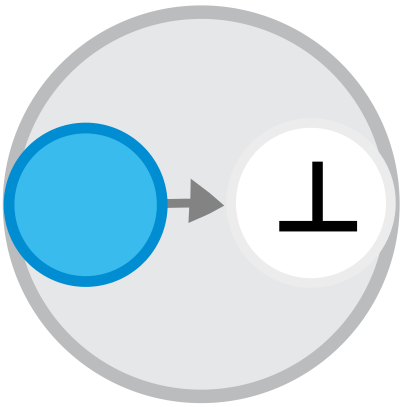
So according to BHK interpretation saying that A is true, means that that we possess a proof of A – simple enough. But it's a bit harder to express the fact that A is false: it is not enough to say that we *don't have a proof* of A (the fact that don't have it, doesn't mean it doesn't exist). Instead, we must show that claiming that A is true leads to a *contradiction*.

To express this, intuitionistic logic defines the constant $\bot$ which plays the role of *False* (also known as the "bottom value"). $\bot$ is defined as the proof of a formula that does not have any proofs. And the equivalent of false propositions

are the ones that imply that the bottom value is provable (which is a contradiction). So instead of...



...we can write:



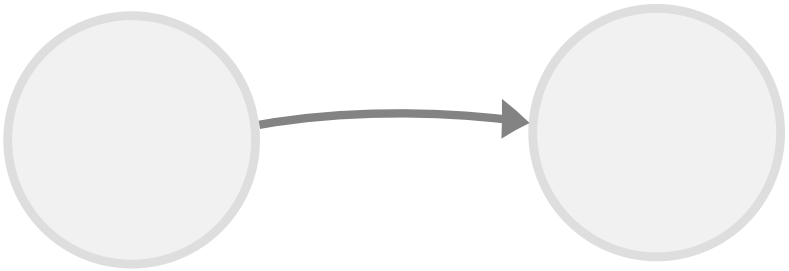
> **THEOREM:**
>
> According to the BHK interpretation, $\neg A$ can be read as $A \to \bot$

In set theory, the $\bot$ constant is expressed by the empty set.

And the observation that propositions that are connected to the bottom value are false is expressed by the fact that if a proposition is true, i.e. there exists a proof of it, then there can be no function from it to the empty set.



The only way for there to be such function is if the set of proofs of the proposition is empty as well.



Task 3: Look up the definition of function and verify that there cannot exist a function from any set *to the empty set*

Task 4: Look up the definition of function and verify that there does exist a function *from the empty set* to itself (in fact there exist a function from the empty set to any other set.

Logics as categories

Aside from being an alternative to classical logic, the BHK interpretation is interesting because it provides that higher-level view of logic, that we need in order to construct a interpretation of it based on category theory.

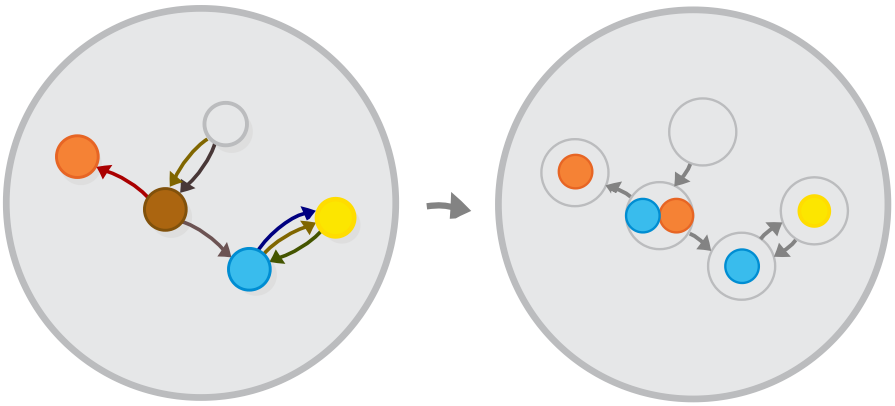
Such higher-level interpretations of logic are sometimes called *algebraic* interpretations, *algebraic* being an umbrella term describing all structures that

can be represented using category theory, like groups and orders.

So, you might suspect already:

> **DEFINITION:**
>
> Some categories can be seen as logical systems: Objects are propositions and morphisms are proofs.

But as usual there is a caveat — not all categories can be converted to logical systems, only some of them. So, to conclude our theorem, we will enumerate the criteria that a given category has to adhere to, in order for it to be "logical". These criteria have to guarantee that the category has an object that corresponds to every valid logical propositions and that no objects corresponds to an invalid ones.



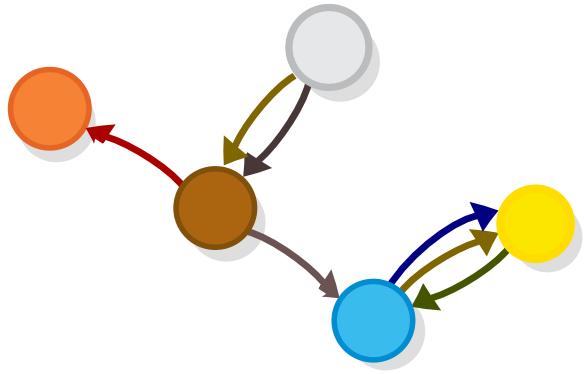
Categories that adhere to these criteria are called *bicartesian closed categories*. But before describing them them directly, we would start with a similar but simpler structures that we already examined — orders.

Task 5: There is a special types of programming languages called "proof assistants" that help you verify logical proofs. Install a proof assistant and try to see how it works. I recommend the Coq Tutorial by Mike Nahas for Coq/Roql, the Natural Numbers Game for Lean or the HoTT Game for Agda.

Task 6: We will concentrate on proving that some categories form logics. But meanwhile, you can prove that all logics form categories, using the definition of a category that we used in the previous chapter.

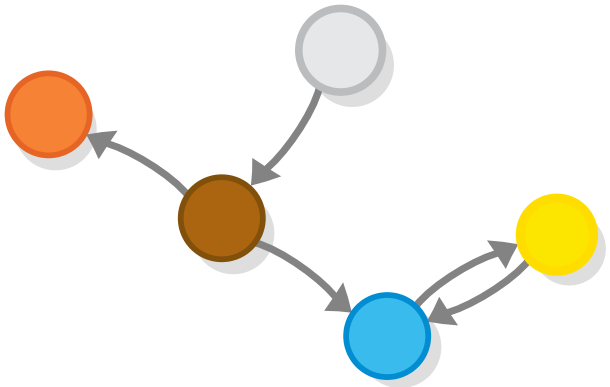
Logics as orders

So, we already saw that a logical system along with a set of primary propositions forms a category.

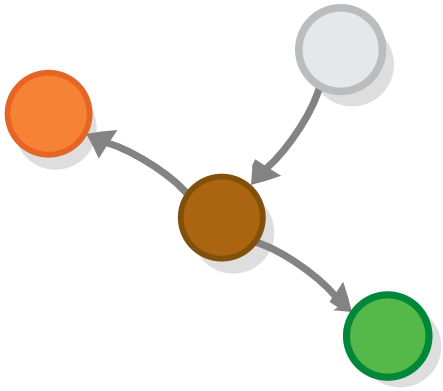


If we assume that there is only one way to go from proposition A, to proposition B (or there are many ways, but we are not interested in the difference between them), then logic is not only a category, but a *preorder*:

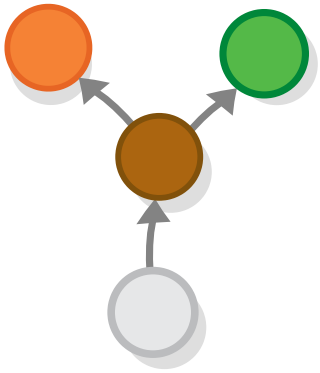
> **DEFINITION:**
>
> Some preorders can be seen as logical systems: elements are propositions and the relationship "bigger than" is taken to mean "implies", so ($A \to B$ is $A \leq B$).



Furthermore, if we count propositions that follow from each other (or sets of propositions that are proven by the same proof) as equivalent, then logic is a proper *partial order*.



And so it can be represented by a Hasse diagram, in which $A \to B$ only if A is below B in the diagram.

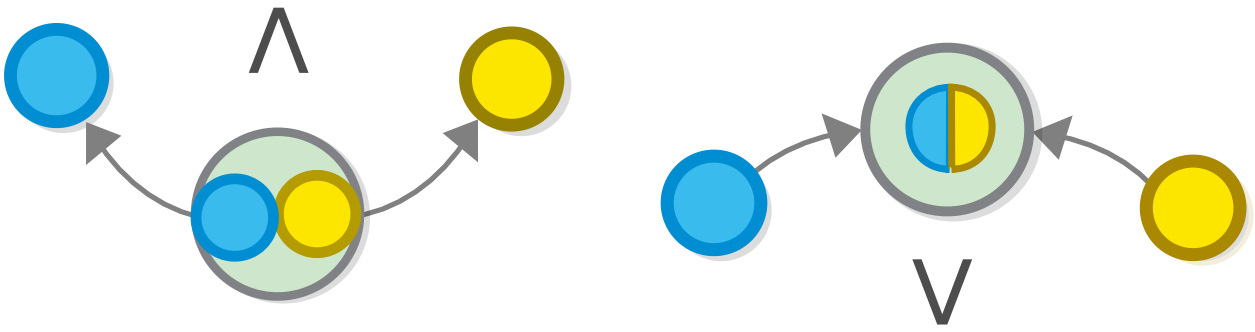


This is something quite characteristic of category theory – examining a concept in a more limited version of a category (in this case orders), in order to make things simpler for ourselves.

Now let's examine the question that we asked before – exactly which ~~categories~~ orders represent logic and what laws does an order have to obey so it is isomorphic to a logical system? We will attempt to answer this question as we examine the elements of logic again, this time in the context of orders.

And and Or operations

By now you probably realized that the *and* and *or* operations are the bread and butter of logic (although it's not clear which is which). As we saw, in the BHK interpretation those are represented by set *products* and *sums*. The equivalent constructs in the realm of order theory are *meets* and *joins* (in category-theoretic terms *products* and *coproducts*.)



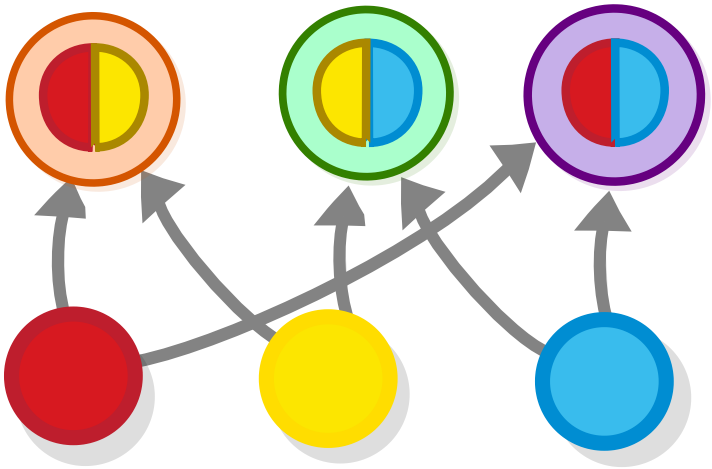
Logic allows you to combine any two propositions in and *and* or *or* relationship, so, in order for an order to be "logical" (to be a correct representation for a logical system,) *it has to have* $meet$ *and* $join$ *operations for all elements*. Incidentally we already know how such orders are called – they are called *lattices*.

> **DEFINITION:**
>
> An order which has meets and joins for all elements is called a lattice.

And there is one important law of the *and* and *or* operations, that is not always present in all lattices. It concerns the connection between the two, i.e. way that they distribute, over one another.

> **DEFINITION:**
>
> A lattice is distributive, if for every three objects A, B and C, we have $A \wedge (B \vee C) \cong (A \wedge B) \vee (A \wedge C)$.

Wait, where have we heard about distributive lattices before? In the previous chapter we said that they are isomorphic to *inclusion orders* i.e. orders of sets, that contain a given collection of elements, and that contain *all combinations* of a given set of elements. The fact that they popped up again is not coincidental – "logical" orders are isomorphic to inclusion orders. To understand why, you only need to think about the BHK interpretation – the elements which participate in the inclusion are our prime propositions. And the inclusions are all combinations of these elements, in an *or* relationship (for simplicity's sake, we are ignoring the *and* operation.)

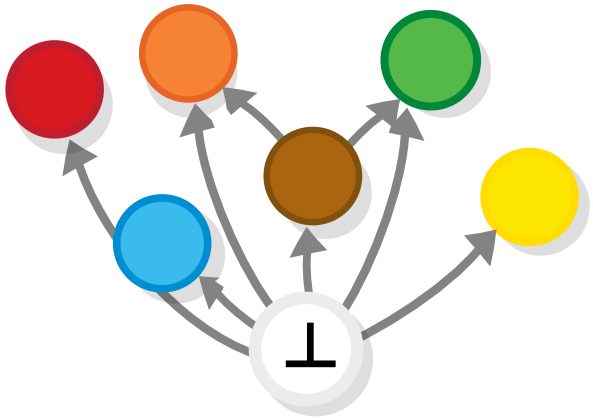


The *or* and *and* operations (or, more generally, the *coproduct* and the *product*) are, of course, categorically dual, which would explain why the symbols that represent them $\vee$ and $\wedge$ are the one and the same symbol, but flipped vertically. And even the symbol itself looks like a representation of the way the arrows converge (although it is probably not the case, as this symbol is used way before Hasse diagrams were a thing).

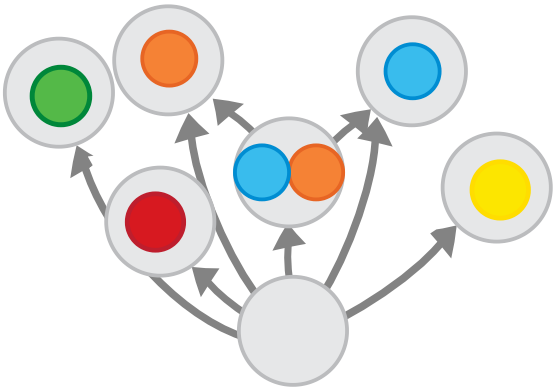
The negation operation

In order for a distributive lattice to represent a logical system, it has to also have objects that correspond to the values *True* and *False* (which are written $\top$ and $\bot$). But, to mandate that these objects exist, we must first find a way to specify what they are in order/category-theoretic terms.

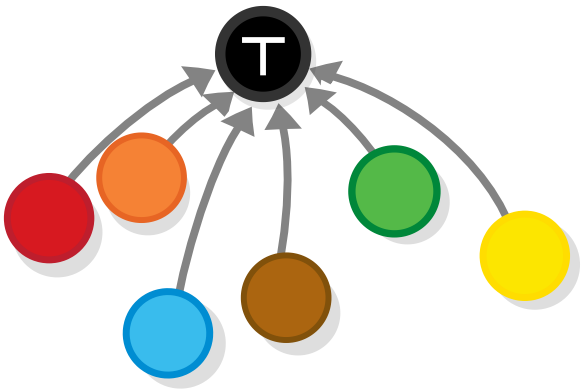
A well-known result in logic, called *the principle of explosion*, states that if we have a proof of *False* (which we write as $\bot$) i.e. if we have a statement "*False* is true" if we use the terminology of classical logic, then any and every other statement can be proven. And we also know that no true statement implies *False* (in fact in intuitionistic logic this is the definition of a true statement). Based on these criteria we know that the *False* object would look like this when compared to other objects:



Circling back to the BHK interpretation, we see that the empty set fits both of these conditions.

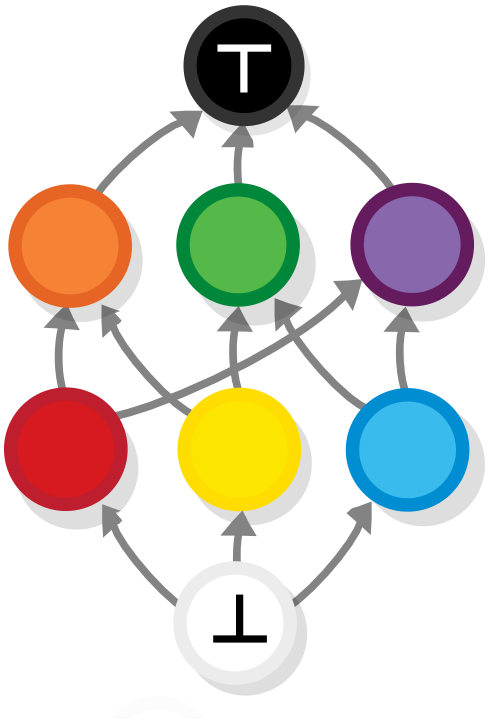


Conversely, the proof of *True* which we write as $\top$, expressing the statement that "*True* is true", is trivial and doesn't say anything, so *nothing follows from it*, but at the same time it follows from every other statement.



So *True* and *False* are just the *greatest* and *least* objects of our order (in category-theoretic terms *terminal* and *initial* object). This is another example of the categorical concept of duality – $\top$ and $\bot$ are dual to each other, which makes a lot of sense if you think about it, and also helps us remember their

symbols (although if you are like me, you'll spent a year before you stop wondering which one is which, every time I see them).



> **DEFINITION:**
>
> A lattice that has a least and greatest elements is called a *bounded lattice*.

So, to summarize, not only should our distributive lattice be *distributive*, but it also has to be *bounded* i.e. it has to have greatest and least elements (which play the roles of *True* and *False*).

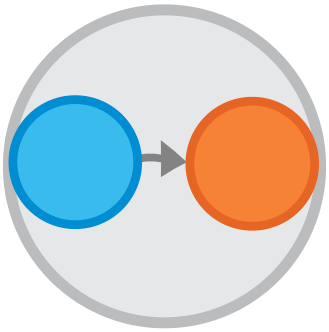
The implies operation

There is one final condition for our logic-representing lattice.

As we said, every lattice has representations of propositions implying one another (i.e. it has arrows).

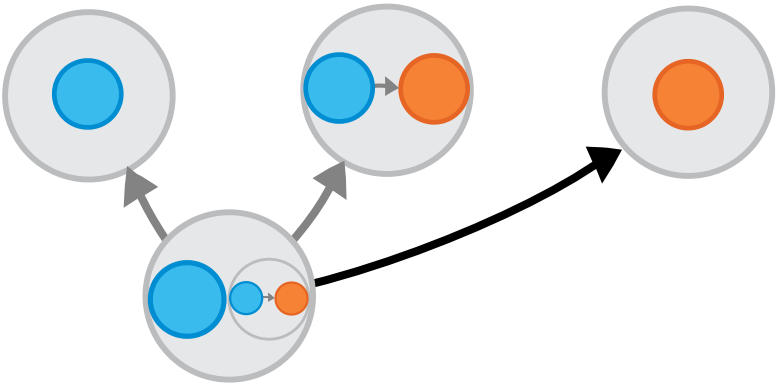


...but to really represents a logical system, a lattice it also has to have *implication objects* i.e. there needs to be a unique "implies object" $A \Rightarrow B$ for each pair of objects A and B, which represents the proposition A implies B.



In set theory, this object is just the "homomorphism set", the set of arrows, but here we are doing category theory, so we will describe this object in the categorical way: by defining a structure consisting of a of objects and arrows in which $A \Rightarrow B$ plays a part.

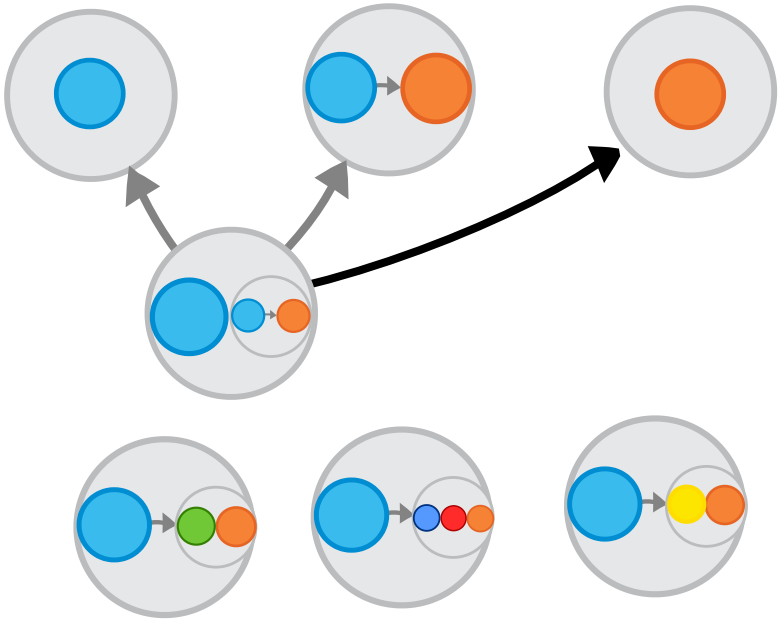
And this structure is actually a categorical reincarnation our favorite rule of inference, the *modus ponens*.



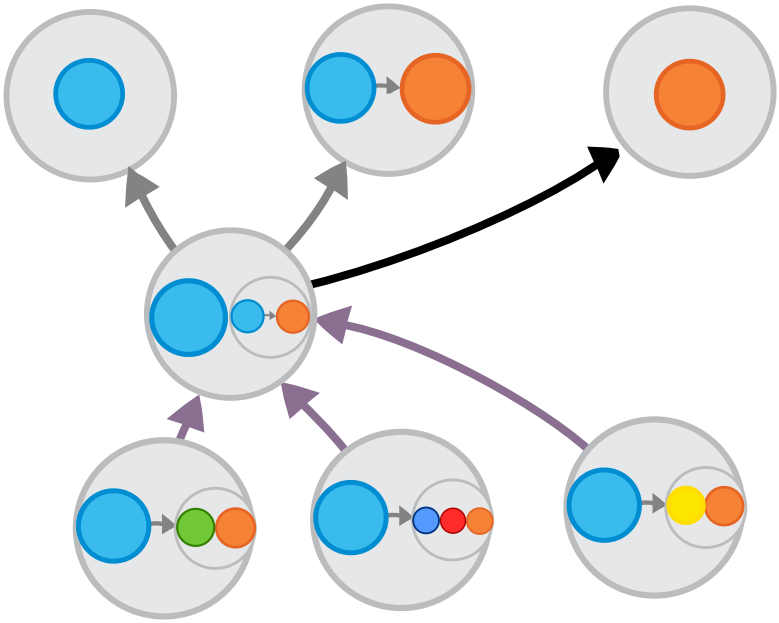
Modus ponens is the essence of the *implies* operation, and, because we already know how the operations that it contains (*and* and *implies*) are represented in our lattice, we can directly use it as a definition by saying that the object $A \to B$ is the one for which modus ponens rule holds.

> **DEFINITION:**
>
> The implication object $A \Rightarrow B$ is an object which is related to objects A and B in such a way that such that $A \wedge (A \Rightarrow B) \to B$.

This definition is not complete, however, because (as usual) $A \Rightarrow B$ is *not the only object* that fits in this formula. For example, the set $A \Rightarrow B \wedge C$ is also one such object, as is $A \Rightarrow B \wedge C \wedge D$



So how do we set apart the real object from all those "imposter" objects? If you remember the definitions of the *categorical product* (or of its equivalent for orders, the *meet* operation) you would already know where this is going: we recognize that $A \Rightarrow B$ is the upper *limit* of $A \Rightarrow B \wedge C$. So, $A \Rightarrow B \wedge C \wedge D$ and all other imposter formulas that can be in the place of X in $A \wedge X \to B$ are below it.



The relationship can be described in a variety of ways.

When we think of *orders*, we can say:

> **DEFINITION:**
>
> For any two elements in an order A and B, the *exponential element* $A \Rightarrow B$ (called also a *relative pseudo-complement* of A in respect to B) is the *biggest/topmost* object X such that the meet of X and A is smaller than B, so $(A \wedge X) \rightarrow B$ (i.e. $A \wedge (A \Rightarrow B) \rightarrow B$).

Logically, we say this:

> **DEFINITION:**
>
> For any propositions A and B, the *implication proposition* $A \Rightarrow B$ (called also *entailment*) is the most *trivial* proposition X for which the formula $A \wedge X \rightarrow B$ (i.e. $A \wedge (A \Rightarrow B) \rightarrow B$) is satisfied.

Finally, here is a general *categorical* definition:

> **DEFINITION:**
>
> For any objects A and B the *exponential object* (called also *internal homomorphism object*) denoted $A \Rightarrow B$ is object X such that:

1. The product of X and A is connected to B with a morphism, so $(A \times X) \to B$ (i.e. $A \times (A \Rightarrow B) \to B$).
2. For any impostor exponential object I, that also has such morphism, there must also exist a unique function (called universal morphism) with the type signature $g : I \to A \to B$, that converts the impostor exponential to the real exponential, such that the morphism connecting the impostor to B would be a result of the composition of g.

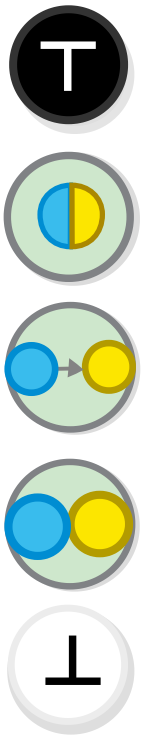
The existence of this implication object is the final condition for an order/lattice to be a representation of logic.

Note that this definition of implication object is valid specifically for intuitionistic logic. For classical logic, the definition of is simpler: because of the law of excluded middle there $A \Rightarrow B$ is just another way to spell $\neg A \vee B$.

Note that, as usual, we treat isomorphic objects as equal: there might be several objects that play the role of $A \Rightarrow B$, for some A and B, but they would be isomorphic to each other i.e. like meets and joins, implication object is defined *up to a (unique) isomorphism*.

Formal definition for orders

So, we talked about a lot of stuff, now it's time to lay the definitions. We saw that intuitionistic logic consists of the values *True* and *False* and the operations *and or* and *implies*.



As we said, the "logical" orders (those who account for all those conditions) have special names. They are called *Heyting algebras*.

> **DEFINITION:**
>
> An order that has joins/meets, greatest/least objects and a implication object is called a Heyting algebra.

And then we say...

> **DEFINITION:**
>
> The logical system of intuitionistic logic can be seen as a Heyting algebra—the "and" and "or" operations are the joins/meets, the values "True" and "False" are the greatest and least objects and the implication operation is the exponential object.

Formal definition for categories

We phrased the above definition in terms of *thin* categories (orders), but if we adjust the terminology, they will also be valid for all other categories as well.

> **DEFINITION:**

> A category that has products/coproducts, initial/terminal objects and an exponential objects is Bicartesian closed.

And then

> **THEOREM:**
>
> The logical system of intuitionistic logic can be seen as a Bicartesian Closed Category—the "and" and "or" operations are the product/coproducts, the values "True" and "False" are the initial/terminal objects and the implication operation is the exponential object.

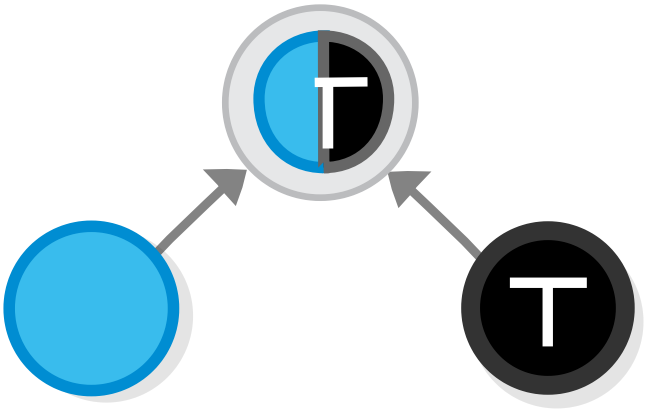
By the way, a lattice can follow the laws of *classical logic*, as well. it has to be *bounded* and *distributive* and in addition to that it has to be *complemented* which is to say that each proposition A, there exist an a unique proposition $\neg A$ (such that $A \vee \neg A = 1$ and $A \wedge \neg A = 0$). These lattices are called *boolean algebras*.

A taste of categorical logic

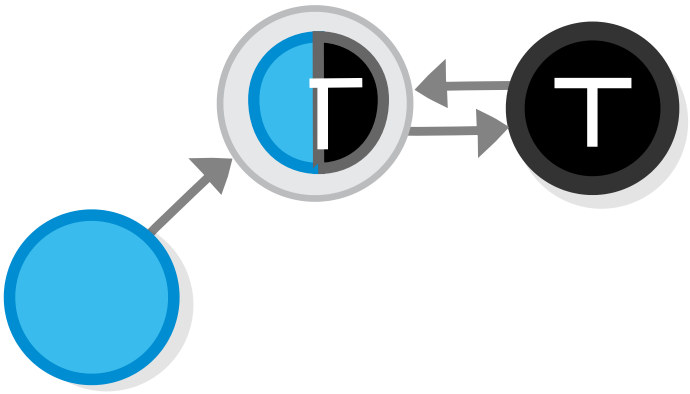
In the previous section we saw some definitions, here we will convince ourselves that they really capture the concept of logic correctly, by proving some results using categorical logic.

A or True is True.

The join (or least upper bound) of the *topmost* object $\top$ (which plays the role of the value *True*) and any other object that you can think of...



...is the $\top$ itself (or something isomorphic to it, which, as we said, is the same thing).



This follows trivially from the fact that the join of two objects must be bigger or equal than both of these objects, and by definition the only object that is bigger or equal to the $\top$ is $\top$ itself (this is because $\top$ (as any other object) is equal to itself).

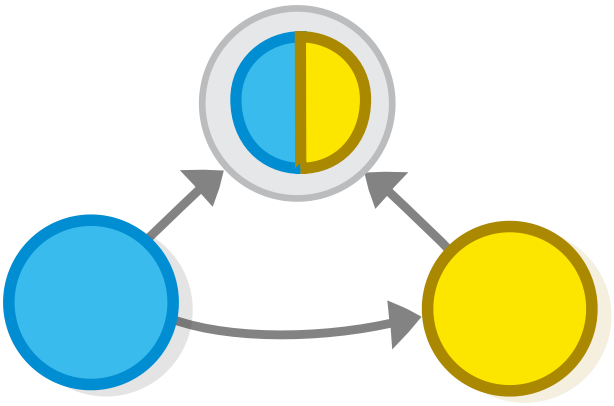
This diagram corresponds to the following logical statement $A \vee \top \cong \top$. So, in order to test if we worked properly, we have to check if this statement is a tautology (and hence a theorem). And indeed it is:

> **THEOREM:**
>
> For any object A or True is true, i.e. $A \vee \top = \top$.

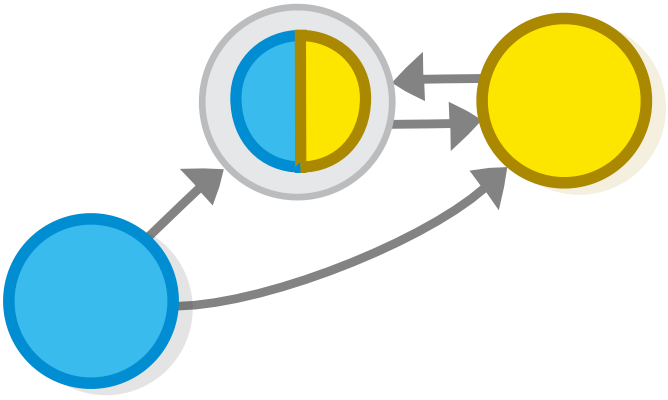
Task 7: Think of the dual situation, with False. What does it imply, logically?

If A implies B, A or B is equal to B

Let's try something else, take two objects A and B such that there is an arrow between them $A \to B$ and find their join.



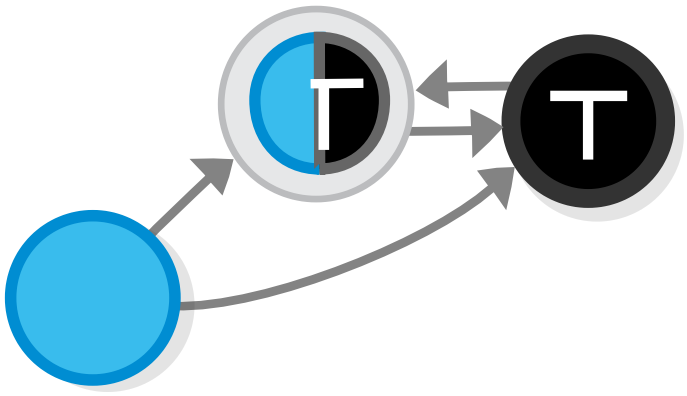
When we are looking for the join of two object, we are looking for the *least* upper bound i.e. the *lowest* object that is equal or bigger than both of them. So, any time we have two objects and one is higher than the other, their join would be (isomorphic to) the higher object.



In other words we have a new theorem (which can also be confirmed with truth tables and/or other axioms).

> **THEOREM:**
>
> For any objects A and B, if A implies B ($A \to B$), then A or B is B ($A \vee B = B$).

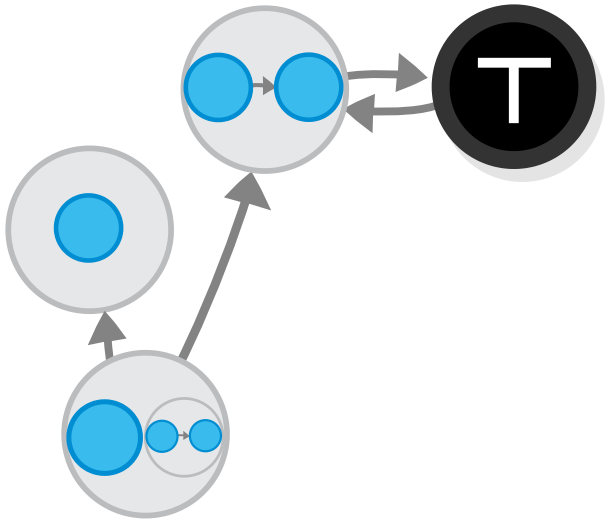
Note that this is actually a generalization of the previous result, which says that join between any random object and the $\top$ object is $\top$ itself (since for any object A, we always have $A \to \top$).



The law of identity

For our first example with implies, let's take the formula $A \Rightarrow B$ (note that we use a double arrow $\Rightarrow$ so as not to confuse $A \Rightarrow B$ (the object) with $A \to B$ (the statement that A implies B). And examine the case when A and B are the same object.

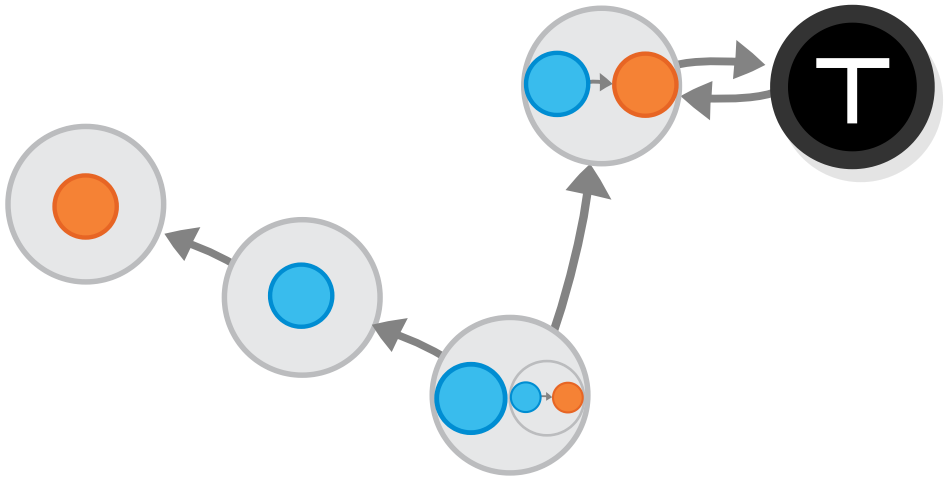
We said that, $A \Rightarrow B$ ($A \Rightarrow A$ in our case) is the topmost object X for which the criteria given by the formula $A \wedge X \to B$ is satisfied. But in this case, the formula is satisfied for any X, (because it evaluates to $(A \wedge A \Rightarrow A) \to A$, which is always true), i.e. the topmost object that satisfies it is... the topmost object there is i.e. (an object isomorphic to) $True$.



Does this make sense? Of course it does: in fact, we just proved one of the most famous laws in logic (called the law of identity, as per Aristotle):

> **THEOREM:**
>
> For any A, $A \to A$ is always true, i.e. everything implies itself (everything follows from itself).

And what happens if A implies B in any model, i.e. if $A \models B$ (semantic consequence)? In this case, A would be below B in our Hasse diagram (e.g. A is the blue ball and B is the orange one). Then the situation is somewhat similar to the previous case: $A \wedge X \to B$ will be true, no matter what X is (simply because A already implies B, by itself). And so $A \to B$ will again correspond to the $\top$ object.

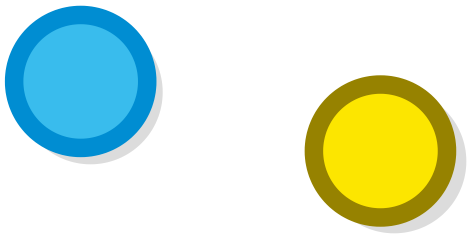


This is again a well-known result in logic:

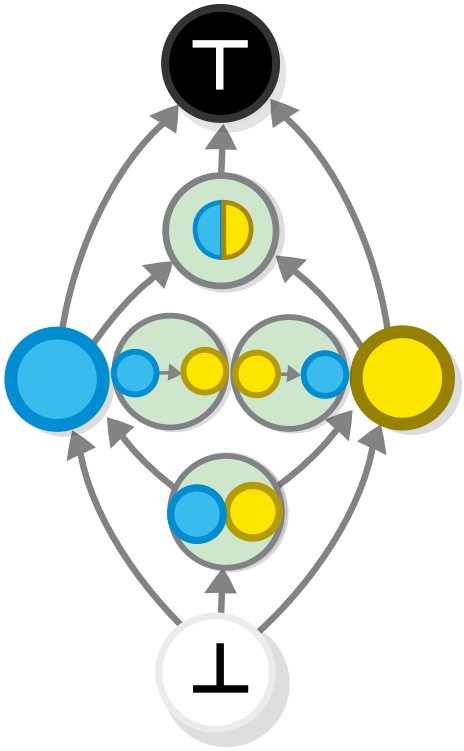
> **THEOREM:**
>
> (sometimes called deduction theorem) A implies B in any model, then ($A \models B$), then the statement $(A \Rightarrow B)$ will always be true.

Interlude: Free Heyting algebras - making ourselves a logic

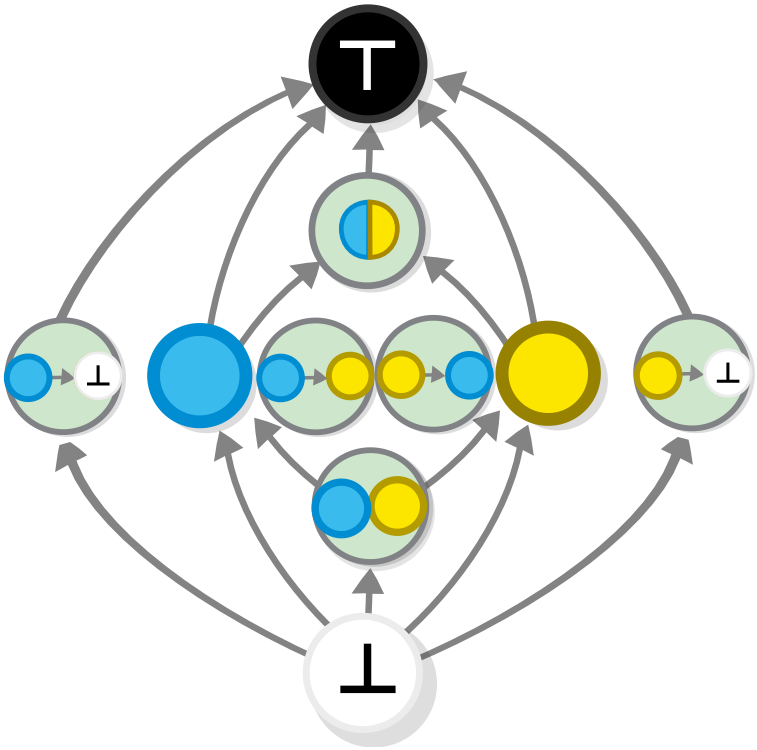
Once we know all this, doing logic is easy: first, we pick the primary propositions that we want to work with, those are the statements that depend on our problem domain (or, in this case, just our color preferences).



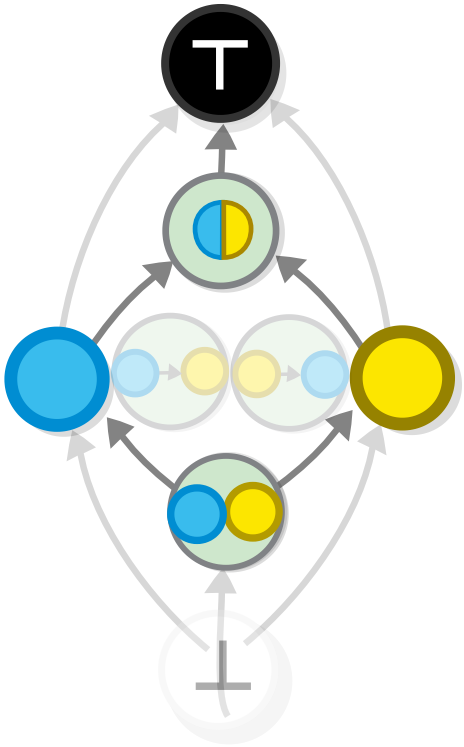
Then, depending of the flavor of logic that we selected, in this case intuitionistic logic, we start graphing the *composite propositions*, we have to have $A \wedge B$, $A \vee B$ for all As and Bs.



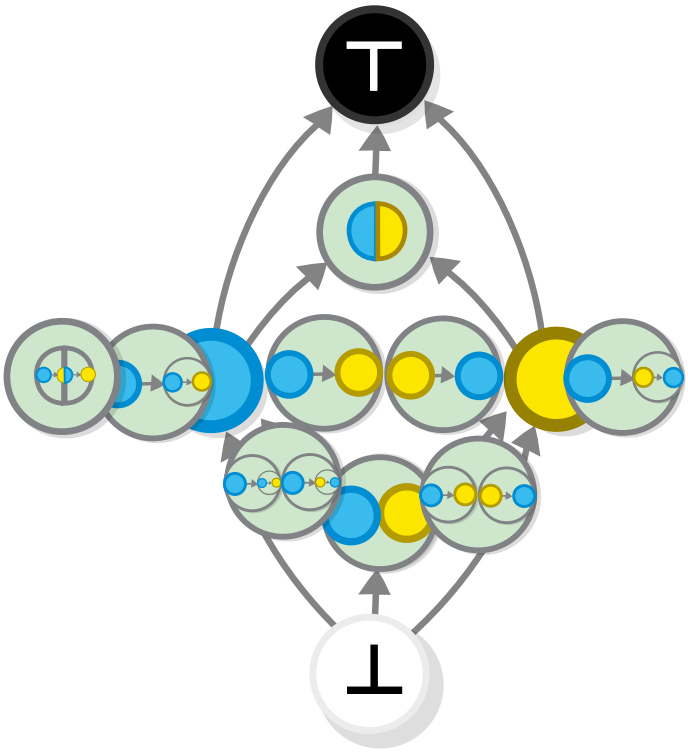
(By the way, we can just as well model Boolean algebra algebraically)



Then we are able to determine which propositions follow from any proposition by just following the path of the arrows coming from it.



Note that we also have to graph the compostite of the composite propositions, which will make our list infinite (drawing such diagrams is very hard and I can never be quite sure which is the correct place for each proposition, so please report me any errors you might see: I can send you a 100$ check, like Donald Knuth, but only if you promise not to cash it, as I am broke).



In general, doing logic is this — we start by the things that we already know and then we find the path that leads us to the things that we are interested in proving (or, depending on the viewpoint, we construct the proof by manipulating the proofs that we already have).

The only thing we are not generally able to do (in intuitionistic logic, specifically) is to prove that a given fact *cannot be reached* from on our path, i.e. that it cannot be proved from the axioms ("you cannot prove a negative").

Types

In this chapter we will talk about types. This might be disappointing for you, if you expected to learn about as many *new* categories as possible (which you don't even suspect are really categories till the unexpected reveal)—we've been talking about the category of types in a given programming language ever since the first chapter, and we already know how they form a category. However, types are not just about programming languages. And they are more than just another category. They are also at the heart of a mathematical theory known as *type theory*.

Type theory is an alternative to set theory, as well as category theory itself, as a foundational language of mathematics, and it is as powerful a tool as any of those formalisms.

Sets, Types and Russell's paradox

We started talking about sets again. Most books about category theory (and mathematics in general) begin with sets, and often go back to sets. Even in a book about category theory, like this one, the standard definitions of most mathematical objects involve sets. Indeed, upon hearing the definition about monoids being one-object categories, a person who only knows about sets might say:

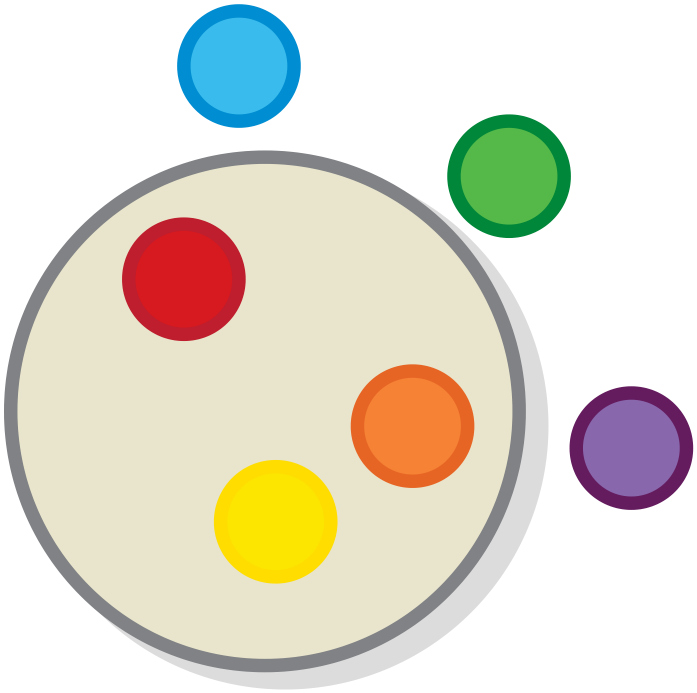
> "Forget that! Have you seen a set? It's the same thing, but you also have this binary operation."

Or for *orders* as being categories with one morphism:

> "Have you seen a set? It's the same thing, but some elements are bigger than others."

The reason for the prevalence of this "set-centric" viewpoint is actually trivial: *sets are simple to understand*, especially when we are operating on the conceptual level that is customary for introductory materials.

We all, for example, group together a set of supplies that are needed for a given activity, (e.g. a *protractor*, a *compass*, and a *pencil* for the math class, or *paper*, cans of *paint* and *brushes* when drawing) so as not to forget some of them. Or we group people that often hang out together as this or that company. And so, when we draw a circle around a few things, everyone knows what we are talking about.

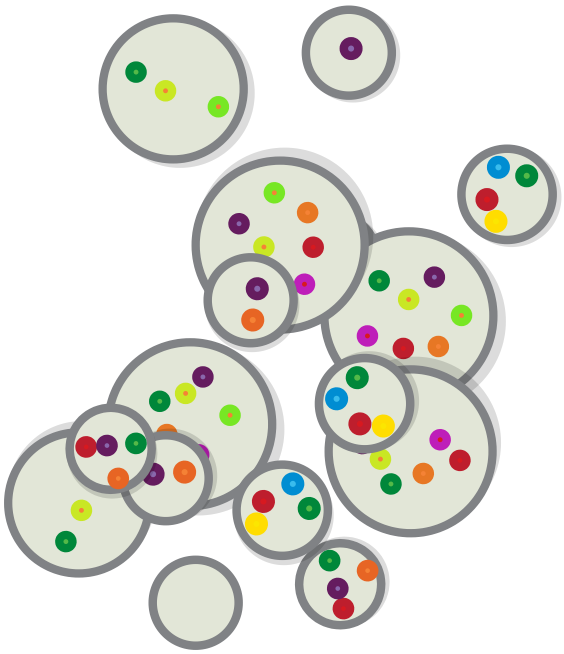


However, this initial understanding of sets is somewhat *too simple*, (or *naive*, as mathematicians call it), as, when it is examined closely, it leads to a bunch of paradoxes which are not easy to resolve, the most famous of which is Russell's paradox.

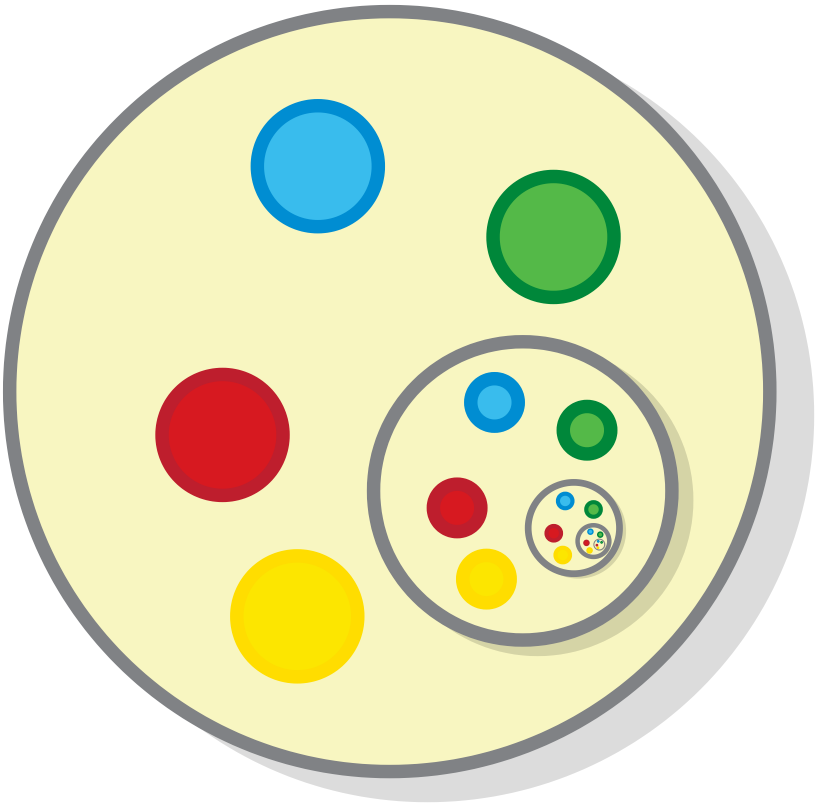
Russell's paradox

Besides being interesting in its own right, Russell's paradox is one of the motivations for creating type theory, so we will start this chapter by understanding how and why it occurs.

Most sets that we saw (like the empty set and singleton sets) do not contain *themselves*.

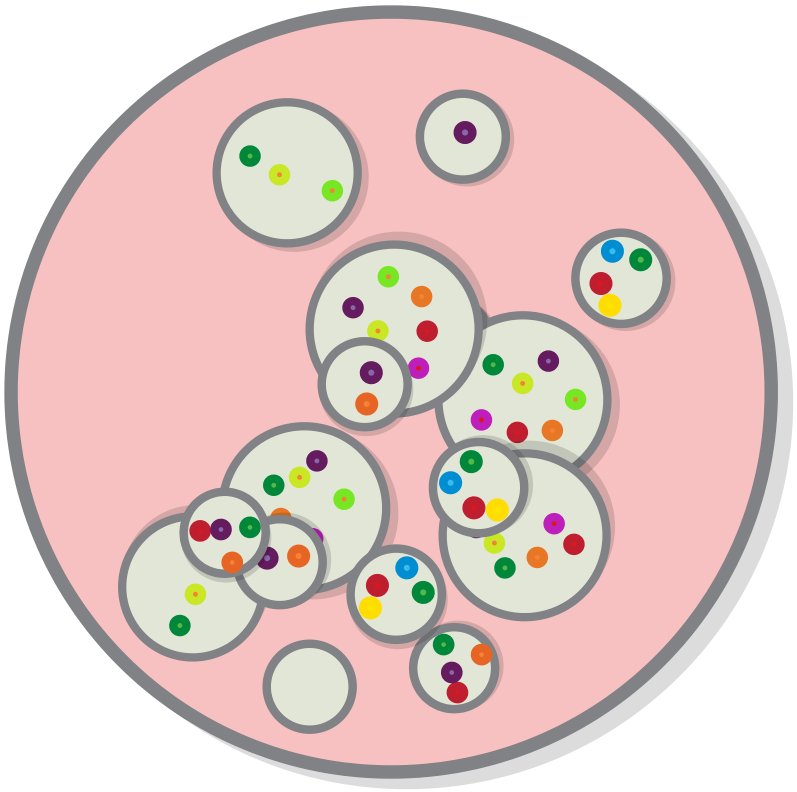


However, as the elements of sets are again sets, *a set can contain itself*.

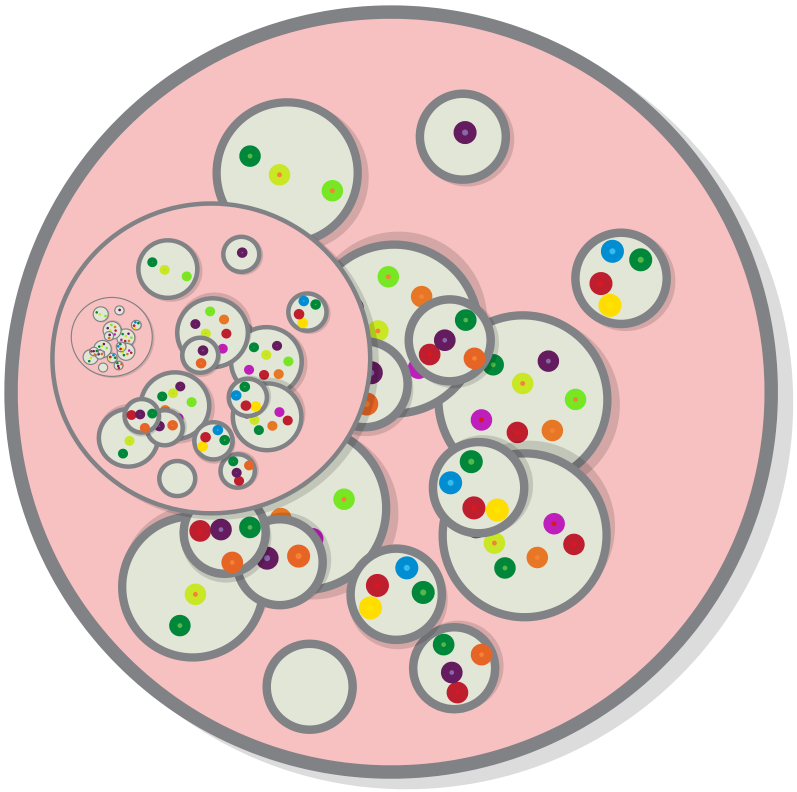


This ability is the root cause of Russell's paradox.

The paradox occurs when we try to visualize *the set of all sets that do not contain themselves*. In the original set notation, it can be defined, as the set such that it contains all sets x such that x is not a member of x (or $x \mid x \notin x$).



However, there is something wrong with this picture — if we look at the definition, we recognize that the set that we just defined *also does not contain itself* and therefore it belongs there as well.



Hmm, something is not quite right here either — because of the new adjustments that we made, our set now *contains itself*.

And removing the set, so it's no longer an element of itself would just take us back to where we started, so we have no way to go — this is Russell's paradox.

Resolving the paradox with sets

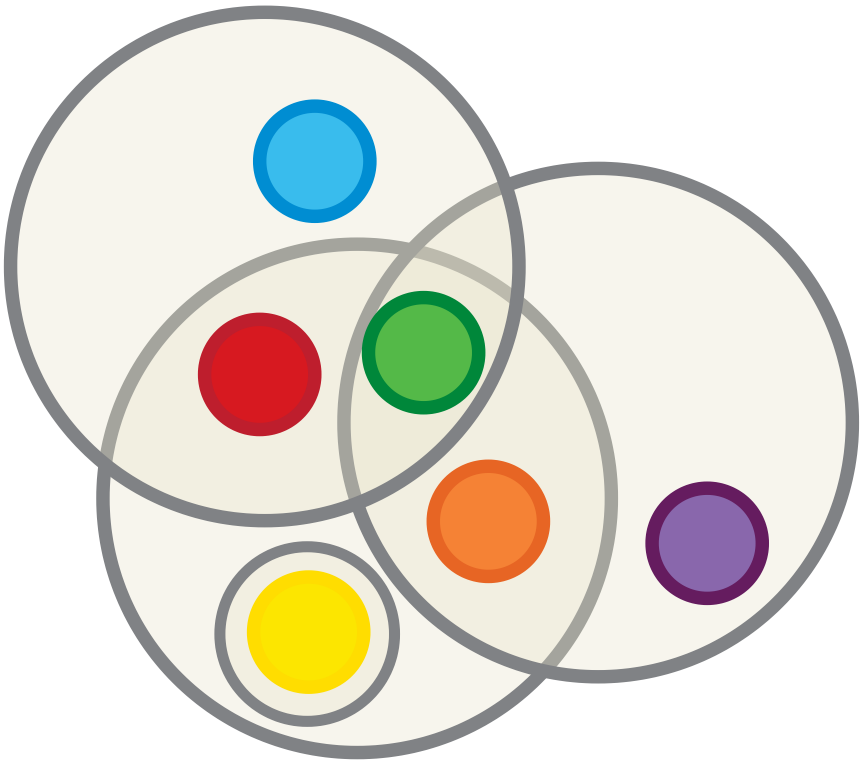
The set of sets that do not contain themselves doesn't sound like a very useful set. And it really isn't — in fact, I haven't seen it mentioned for any other reason, other than the construction of Russell's paradox. So, most people's initial reaction when learning about Russell's paradox would be something like this:

> "Wait, can't we just add some rules that say that you cannot draw the set of sets that don't contain themselves?"

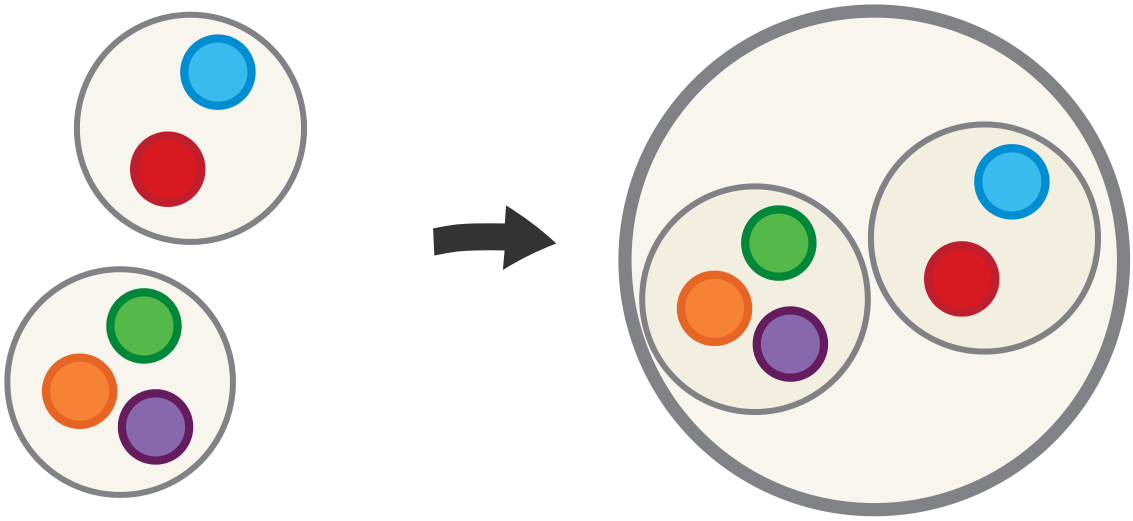
This was exactly what the mathematicians Ernst Zermelo and Abraham Fraenkel set out to do (no pun intended). And the extra rules they added led to a new

definition of set theory, known as *Zermelo–Fraenkel set theory*, or *ZFC* (the *C* at the end is a separate story) which is a version of set theory that is free of paradoxes. ZFC was a success, and it is still in use today, however it compromises one of the best features that sets have, namely their *simplicity*.

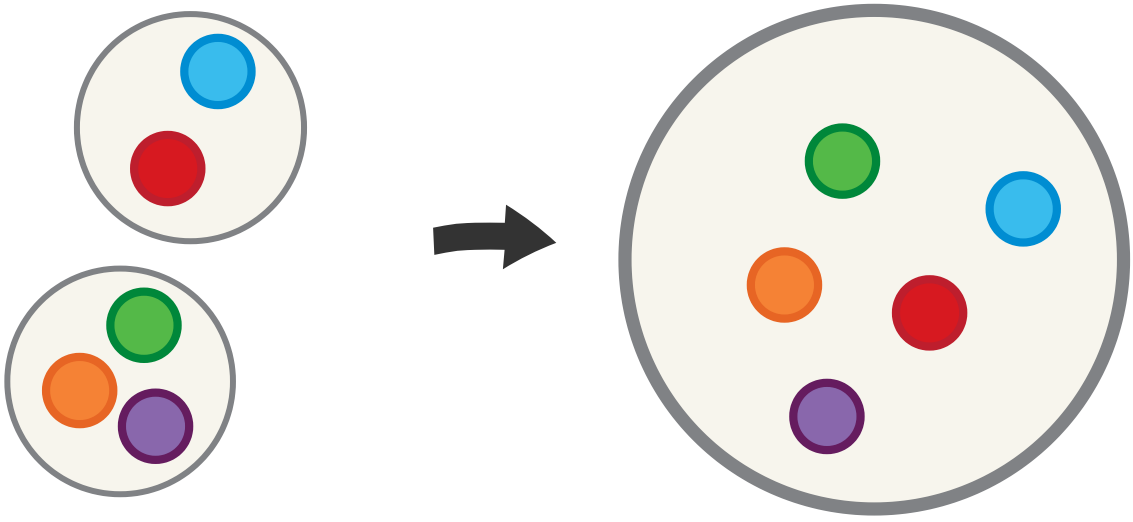
What do we mean by that? Well, the original formulation of set theory (which is nowadays called *naive* set theory) was based on just one (rather vague) rule/axiom: "Given a property P, there exists a set, containing all objects that have this property" i.e. any bunch of objects can form a set.



In contrast, ZFC is defined by a larger number of (more restrictive) axioms, as for example, the *axiom of pairing*, which states that given any two sets, there exists a set which contains them as elements.



...or *the axiom of union*, that states that if you have two sets you also have the set that contains all their elements.



There are a total of about 8 such axioms (depending on the flavour of the theory). They are curated in a way that allows us to construct all sets that are interesting, without being able to construct the infamous set that contains itself. However, accepting ZFC would mean accepting that set theory is not as simple and straightforward, as it looks like.

Indeed, it is more complex than category theory, and more complex than the other theory which we will learn about in a minute...

Resolving the paradox with types

While Zermelo was working on refining the axioms of set theory in order to avert Russell's paradox, Russell himself took a different route towards solving his paradox and decided to ditch sets altogether, and develop an entirely new mathematical concept that is free of paradoxes *by design* - one where you don't need to patch things up with extra axioms to avoid having illogical constructions. And so, in 1908, the same year in which Zermelo published the first version of ZFC, Russell came up with his *theory of types*.

Type theory is not at all similar to set theory, but it is at the same time, not entirely different from it, as the concepts of *types* and *terms* are clearly reminiscent of the concepts of *sets* and *elements*.

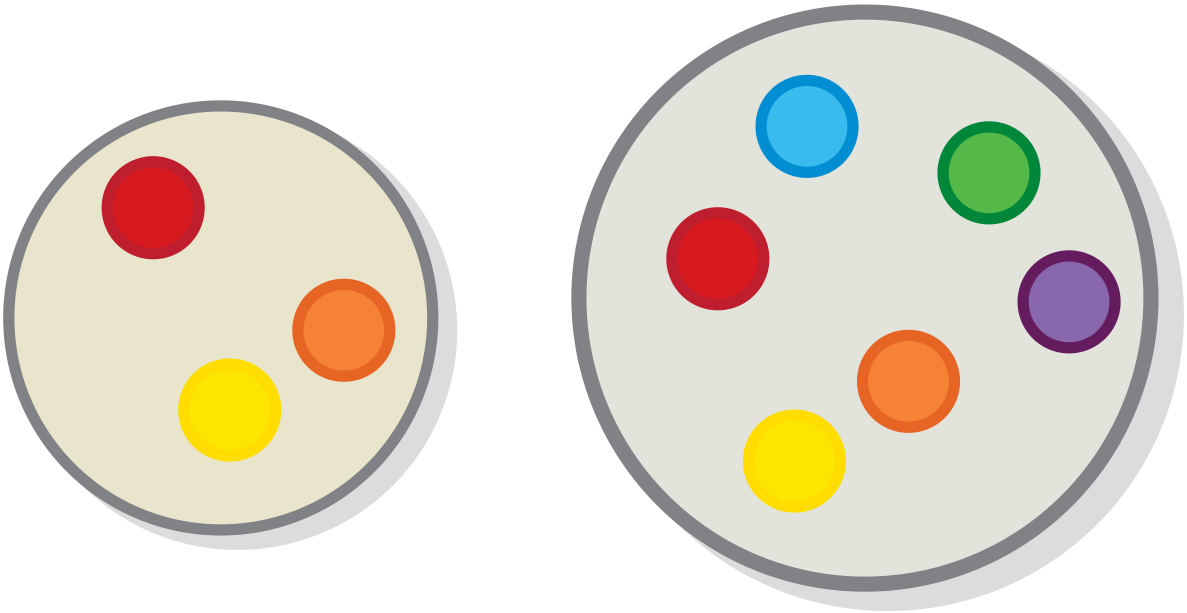
Theory	Set theory	Type Theory
	Element	Term
Belongs to a	Set	Type
Notation	$a \in A$	$a : A$

The biggest difference, between the two, when it comes to *structure* is that terms are bound to their types.

So, while in set theory, one element can be a member of many sets

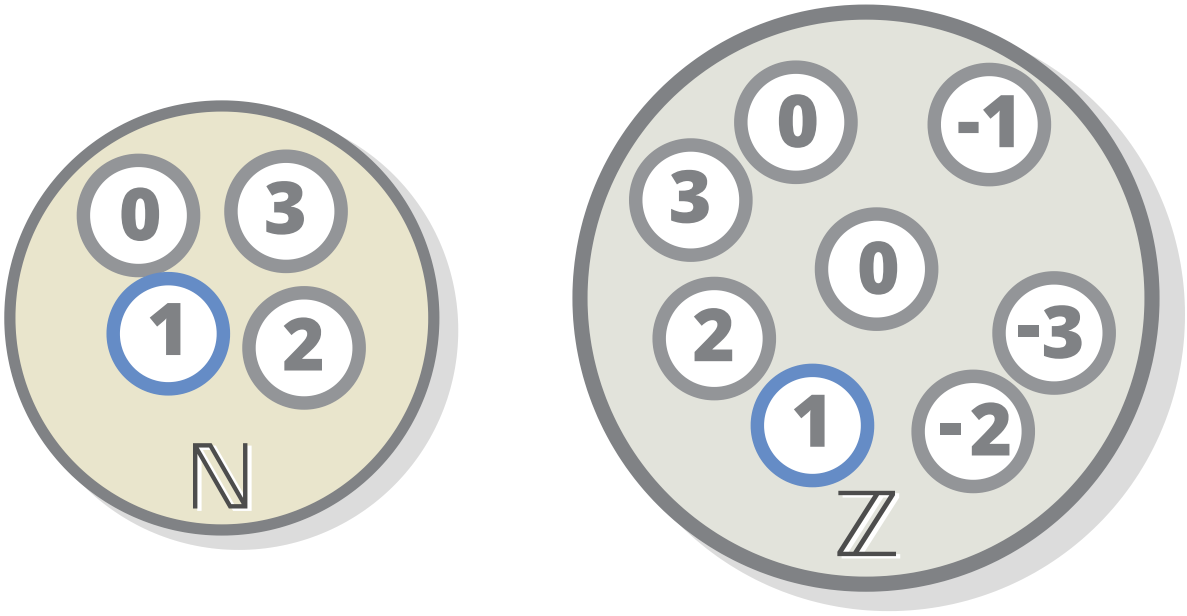


In type theory, a term can have only one type. (note that the red ball in the small circle is different from the red ball in the bigger circle)

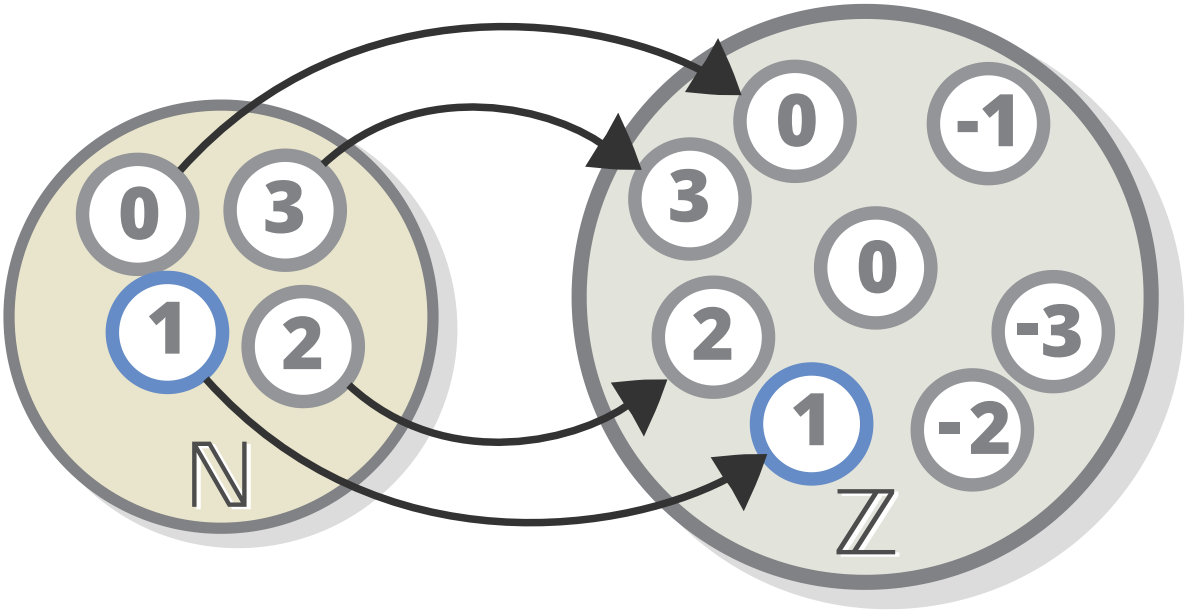


Due to this law, types cannot contain themselves, so Russell's paradox, is entirely avoided.

The law may sound weird e.g. because a term can only belong to one type, in type theory, the natural number 1 is denoted as $1 : \mathbb{N}$ and it is an entirely separate object from the integer 1 (denoted as $1 : \mathbb{Z}$)



It only starts to make some sense once we realize that we can always convert one version of the value to the other, using the image function that we learned about in the first chapter.



As you would see shortly, the concept of types has to do a lot with the concept of functions.

What is type theory

> "Every propositional function ϕ(x)—so it is contended—has, in addition to its range of truth, a range of significance, i.e. a range within which x must lie if ϕ(x) is to be a proposition at all, whether true or false. This is the first point in the theory of types; the second point is that ranges of significance form types, i.e. if x belongs to the range of significance of ϕ(x), then there is a class of objects, the *type* of x, all of which must also belong to the range of significance of ϕ(x)" — Bertrand Russell - Principles of Mathematics

In the last section, we almost fell in the trap of explaining types as something that are "like sets, but... " (e.g. they are like sets, but a term can only be a member of one type). However, while it may be technically true, any such explanation would not be at all appropriate, as, while types started as alternative to sets, they actually ended up being quite different. So, thinking in terms of sets won't get you far. Indeed, if we take the proverbial set

theorist from the previous section, and ask them about types, their truthful response would have to be:

> "Have you seen a set? Well, it has nothing to do with it."

So let's see how we define a type theory in its own right.

But first...

Long disclaimer

Before we begin, let's get this long disclaimer out of the way:

Notice that in the last sentence we said *a* type theory, not "type theory" or "the type theory". This is because there are not one, but many different (albeit related) formulations of type theory that are, confusingly, called type *theories* (and, less confusingly, *type systems*), such as *Simply-typed lambda calculus* or *Polymorphic Lambda calculus*. For this reason, it makes sense to speak about *a* type theory.

Have I confused you enough? No?

In some contexts, the term "type theory" (uncountable) refers to the whole field of study of type theories, just like category theory is the study of categories. But, (take a deep breath) you can sometimes think of the different type systems as "different versions of type theory" and so, when people talk about a given set of features that are common to all type systems, they sometimes use the term "type theory" to refer to any random type system that has these features.

What are types?

Anyhow, let's get back to our subject (however we want to call it). As we said, type theory was born out of Russell's search for a way to define all collections of objects that are *interesting*, without accidentally defining collections that lead us astray (e.g. to his eponymous paradox), and without having to make up a multitude of additional axioms (a-la ZFC).

He thought a lot (at least I imagine he did) and he managed to devise a formal system that fits all these criteria, based on a revolutionary new idea... which is

basically the same idea that is at the heart of category theory (I don't know why he never got credit for being a category theory pioneer). The idea is the following: The *interesting* collections, the collections that we want to talk about in the first place, are the *collections that are the source and target of functions*. So, we might say.

> **DEFINITION:**
>
> A type is something that can be the source and/or target of an arrow.

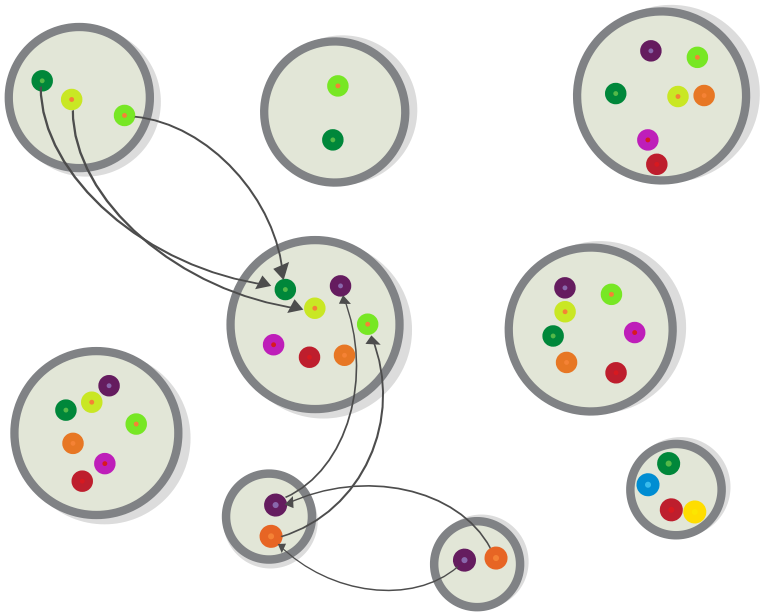
(To make the definition more general, we use the more general term — "arrow", but you can think of arrows as functions for now.)

Let's think again about the set of all sets that don't contain themselves. Besides being the cause of Russell paradox, this set is quite useless (unless we count causing paradoxes as useful). And if we dig into it, we eventually discover why: there are no (interesting) functions from any other set to this set, so *we cannot get to it* from anywhere. And, conversely, we cannot get anywhere from it (there are no functions where it is the source either). This set is like an oasis at the center of the desert... or perhaps a little desert in the center of big oasis... Contact me if you can think of some good metaphor.

Building types

We saw that type theory is not so different from set theory when it comes to *structure that it produces* — all types (at least on the first level) are sets, although not all sets are types. And all functions are... well functions. However, type theory is very different from set theory when it comes to *the way the structure comes about*, in the same way as the intuitionistic approach to logic is different from the classical approach (by the way, if this metaphor made the connection between type theory and intuitionistic logic too obvious for you, do me a favor, please don't mention it and act surprised when we make it explicit).

In set theory, (and especially in its naive version) all possible sets and functions are *already there from the start*, as the Platonic world of forms. What we do is merely exploring the ones that interests us.



In type theory, we start with a space that is empty.

[diagram omitted]

From there, we have to build our types. One by one. With our bare hands (OK, we do have some cool mathematical tools that assist us).

Type formation, term introduction, term elimination

> "In general, we can think of data as defined by some collection of selectors and constructors, together with specified conditions that these procedures must fulfill in order to be a valid representation." — Harold Abelson, Gerald Jay Sussman, Julie Sussman — Structure and Interpretation of Computer Programs

Before introducing the specific formulae for building types, I want to elaborate on the general idea. In the last section, we said that a type is something that can be the source and/or target of an arrow. This definition may seem a bit vague, but it is trivial when we look at how types are defined in computer programming. It is obvious, even when viewed through the lens of traditional imperative languages, that the definition of a type consists of the definitions of rules for constructing functions (and more generally arrows).

```
class MyType<A> {

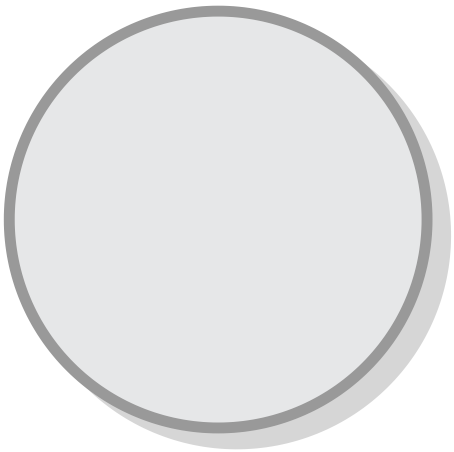
  a: A;
  constructor(a) {
    this.a = a;
  }

  getA() {
    return this.a;
  }

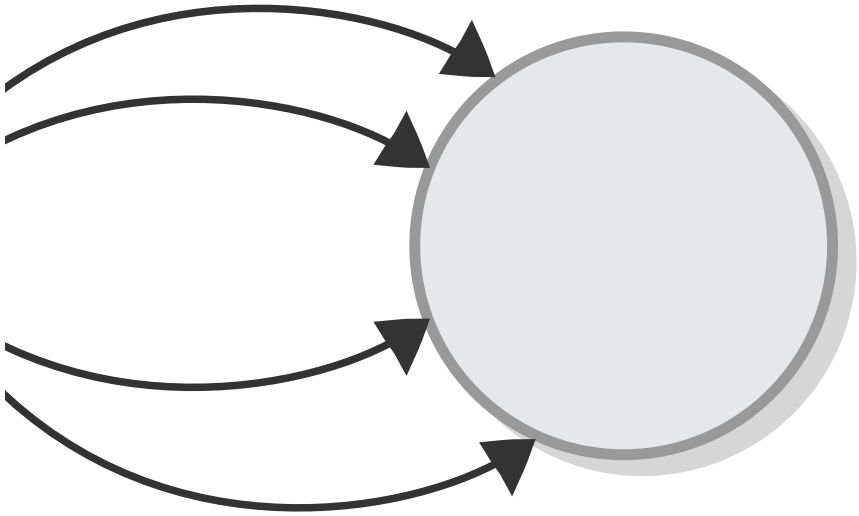
}
```

What kinds of rules? We can categorize them in three groups.

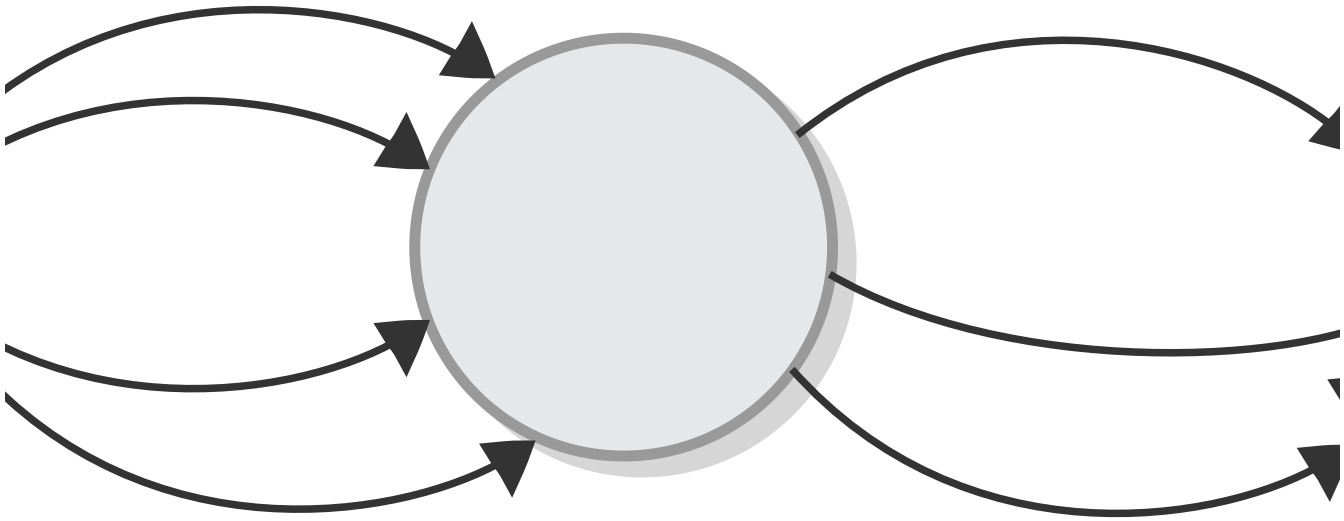
First off, a type has to have a *definition* which specifies what it is. Note that this arrow is different from what we perceive as an arrow — it is not a value-level arrow (going from one type to the other), but is a type-level arrow (from one category of types, to another). This is known as a *type formation rule*.



Next up, a type has to have at least one arrow pointing to the new type. This is known as a *term introduction rule* ("term" being the word for "value"). In programming, it is called a *constructor*, and it is a value-level arrow (e.g. function).



Finally, as we don't want to construct types just for the sake of constructing new types, a type has to have at least one arrow coming *from* this new type. This is value-level arrow (function) known as a *term elimination rule* (as if we are eliminating the type by replacing it with the result of the method).



In summary

> **DEFINITION:**
>
> A type is defined by defining these three arrows:
>
> 1. One type-level arrow (type formation).
> 2. At least one value-level arrow for which the type is the target (term introduction).
> 3. At least one value-level arrow for which it is the source (term elimination).

OK, I think we went too far in trying to define type theory without actually defining type theory, so we will proceed with the formulas... after our second long disclaimer.

Picking a theory (another long disclaimer)

As we said in the first long disclaimer, there is not one, but many type theories. So if we want to do type theory, we have first pick *a* type theory (if this sentence confuses you, read the first disclaimer again).

Picking a type theory (or a type system let's call it), also involves picking a *language* that this theory is described in terms of. When hearing about language, programmers would probably think of the popular feature-rich programming languages like TypeScript or Java. *Type theorists*, on the other hand, have different preferences — since they are interested in the type system, not the language, they don't really care about the features, and so the language of choice of most of them is the simplest, most minimal language that is possible to exist, namely *Lambda Calculus*. If you haven't heard about it, this is language that only has (anonymous) functions and nothing else.

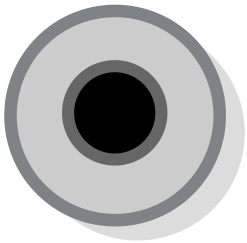
To please both parties, (and annoy them both, at the same time), we will go with a language that is somewhere in between — namely (a subset of) *Haskell*. This will not make much difference in terms of the theory, as Haskell is based on Lambda calculus, but will make things easier for programmers: unlike Lambda Calculus, which only has functions, Haskell supports defining product constructors as a primitive (which itself makes no difference from a formal standpoint, as we can easily go from products to functions via currying and uncurrying).

Also, last but not least, Haskell constructors and functions can have names (believe me, this helps).

Since we are picking Haskell, we will work in the type theory/type system of Haskell. This is a type system, discovered by Jean-Yves Girard in 1972, called *Polymorphic Lambda Calculus* or *System F*.

The Unit type

We start with an empty space, when nothing is defined, except for the singleton type, known as the $Unit$ type in Haskell.



A type which has only one value, which we can use as a starting point.

> **DEFINITION:**
>
> The $Unit$ type (1) is the type with one value.

The Lambda type

There is actually one more prerequisite which is not so easy to explain: namely the *arrow types* for each pair of objects, known as the *Lambda* types. In Lambda Calculus arrows between types are types as well (a feature sometimes called "first-class functions" in programming context).

> **DEFINITION:**
>
> For any types A and B, there is a type $A \to B$, called the *Lambda* type of A and B, which has all arrows that connect A and B as values.

We won't go into more detail here, because the Lambda type is defined and works in the same way as the implication object in intuitionistic logic, as defined in the previous chapter. As you will learn, it also has the same role.

Base types. The boolean type

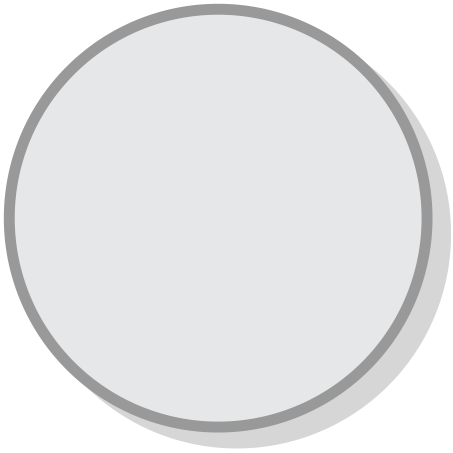
Once we have a starting point we can define some types. But how? Let's start with base types, like the *booleans*. For them, the process is quite simple, because we can just straight out *list out their values*.

> **DEFINITION:**
>
> $$\begin{aligned} \text{Bool} &: \text{Type} \\ \text{True} &: \text{Bool} \\ \text{False} &: \text{Bool} \end{aligned}$$

Let's go through this definition:

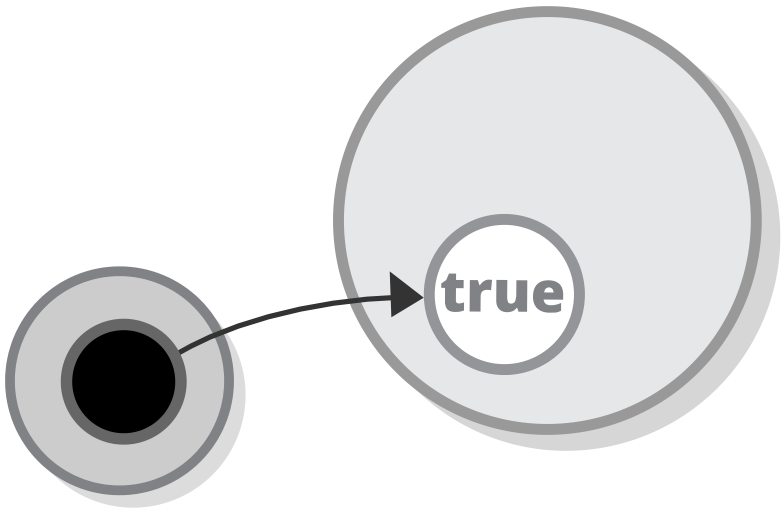
Type formation

First, $Bool : Type$, says that there exists a type that we call "Bool".

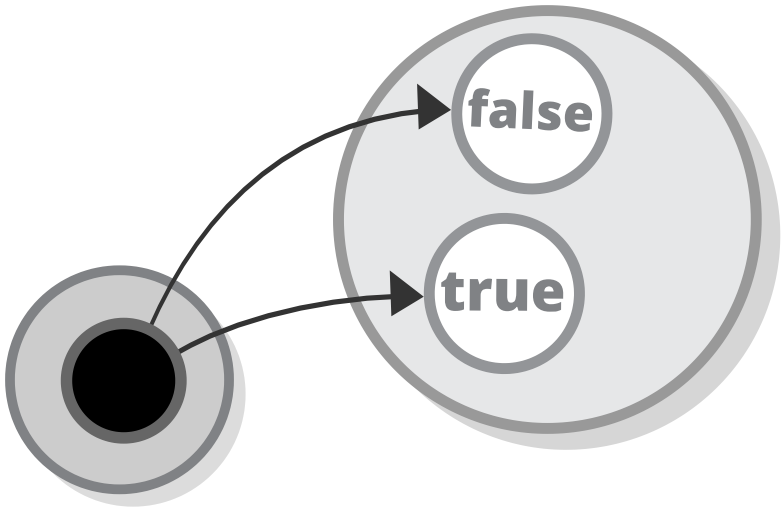


Term introduction

Then, $True : Bool$ says that "$True$ is a boolean" i.e. it adds one value to this newly created datatype. In the diagram, we will represent that as an arrow from the singleton type, known as the $Unit$ type in Haskell, as per the Elementary Theory of the Category of Sets from chapter 2.



And $False$: $Bool$ creates another such value.



Et voila, we have just defined a type!

Term elimination

Wait, scratch that. We actually haven't defined a type. Or rather we have defined one, but it is quite useless. For it would only be useful once we define at least one arrow, coming *from* it (otherwise, it will just be a one-way street). For the Booleans, this function is usually called $ifElse$.

DEFINITION:

$$\begin{aligned} \text{ifElse} : \forall a.\ \text{Bool} \to a \to a \to a \\ \text{ifElse}\ True\ a\ b\ &=\ a \\ \text{ifElse}\ False\ a\ b\ &=\ b \end{aligned}$$

You can see that the functions in Haskell are pretty rudimentary to define — you just map each individual value of one type, to the value of another one.

Here are some expressions which use the function accompanied with indications of what they return (-- is Haskell's comment syntax)

```
ifElse True 1 2 --1
ifElse False 1 2 --2
```

Isomorphisms between types

But why (with the risk of repeating myself) does this exact type has to be the Boolean type? What is stopping our colleague Bobby who always wants to do everything their way, to define their own version of Boolean and using it in their project.

DEFINITION:

$$\begin{aligned} \text{BobbysBool} :&\ \text{Type} \\ \text{BobbysTrue} :&\ \text{BobbysBool} \\ \text{BobbysFalse} :&\ \text{BobbysBool} \end{aligned}$$

The answer is "nothing". But that is not a huge deal — we can just whip up a function to convert their Bool to ours:

DEFINITION:

$$\begin{aligned} convert\ BobbysBool &\to Bool \\ convert\ BobbysTrue\ &=\ True \\ convert\ BobbysFalse\ &=\ False \end{aligned}$$

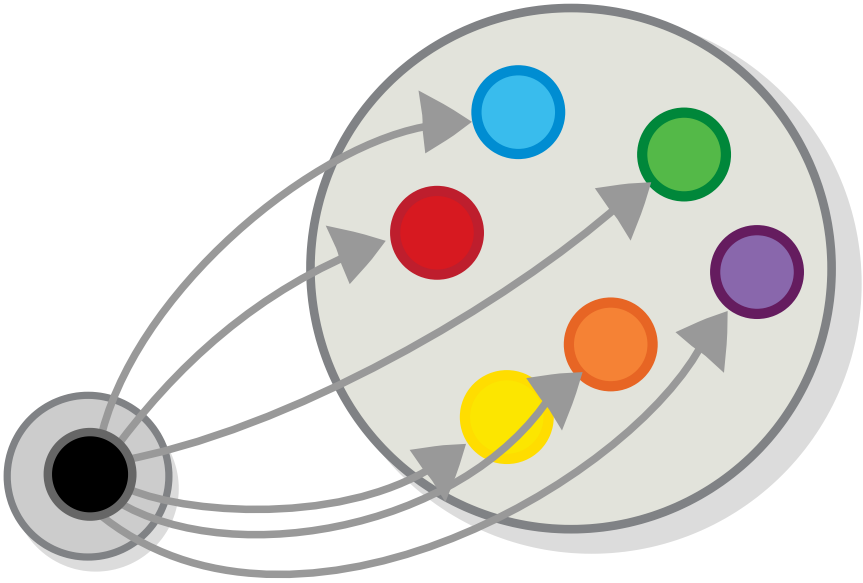
This function is also reversible. Which means that the two types are isomorphic i.e. they are one and the same type, *up to a (unique) isomorphism*.

Other base types

Almost forgot: in the same way as we constructed the Booleans, we can construct any other finite/base types, such as the type of balls.

DEFINITION:

$$\begin{aligned} \text{Ball} &: \text{Type} \\ \text{OrangeBall} &: \text{Ball} \\ \text{RedBall} &: \text{Ball} \\ \text{YellowBall} &: \text{Ball} \end{aligned}$$



Polymorphic types. The Maybe type

Now, we will define the type that is known in Haskell as, $Maybe$ (and what in other languages is usually called $Option$). If you haven't encountered it, the Haskell documentation provides a very good description:

> The Maybe type encapsulates an optional value. A value of type Maybe a either contains a value of type a (represented as $Just[a]$), or it is empty (represented as $Nothing$). Using $Maybe$ is a good way to deal with errors or exceptional cases without resorting to drastic measures such as error.

But, once you learn to read it, the type definition, by itself is clear enough:

DEFINITION:

$$\begin{aligned} \text{Maybe} &: \text{Type} \to \text{Type} \\ \text{Nothing} &: \forall a.\ \text{Maybe}[a] \\ \text{Just} &: \forall a.\ a \to \text{Maybe}[a] \end{aligned}$$

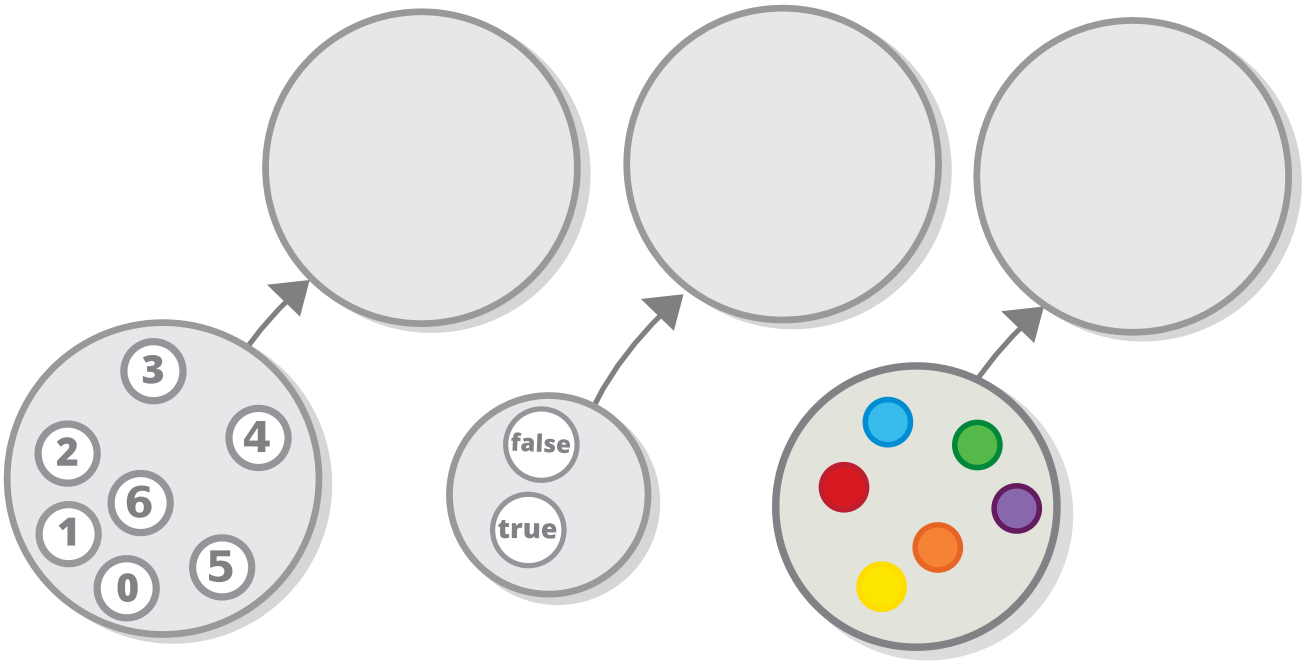
Type formation

$Maybe$ looks a lot like $Bool$, but, unlike $Bool$, $Maybe$ is a *polymorphic* type, as we can tell by looking at the *type formation rule*

DEFINITION:

$$\text{Maybe} : \text{Type} \to \text{Type}$$

Maybe is polymorphic i.e. there is not just one $Maybe$, but many $Maybe$'s – one for each type a . Polymorphic types are arrows from the universe of types, to itself.



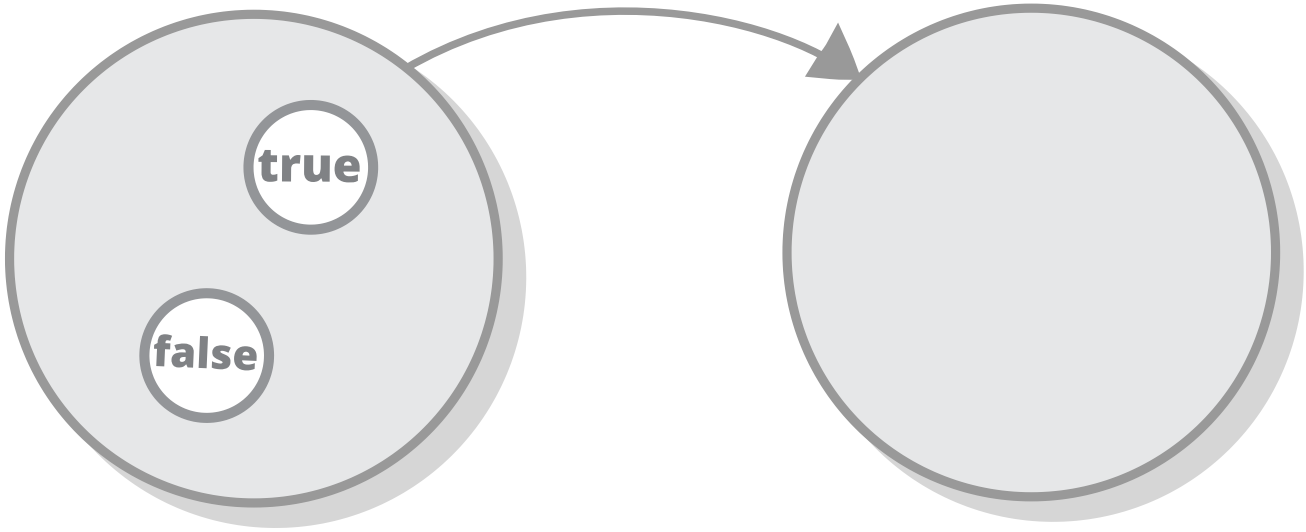
This is why the kind of $Maybe$ is $Type \to Type$, while $Bool$ is just a $Type$.

Let's take just the type $Bool$ as an example. Applying it to the type-level arrow, we get

DEFINITION:

$$\text{MaybeBool} : \text{Type}$$

i.e. because $Bool$ is a type, then $Maybe[Bool]$ is also a type.



Term introduction: Nothing

Now, it's time to fill our type.

The first line is

DEFINITION:

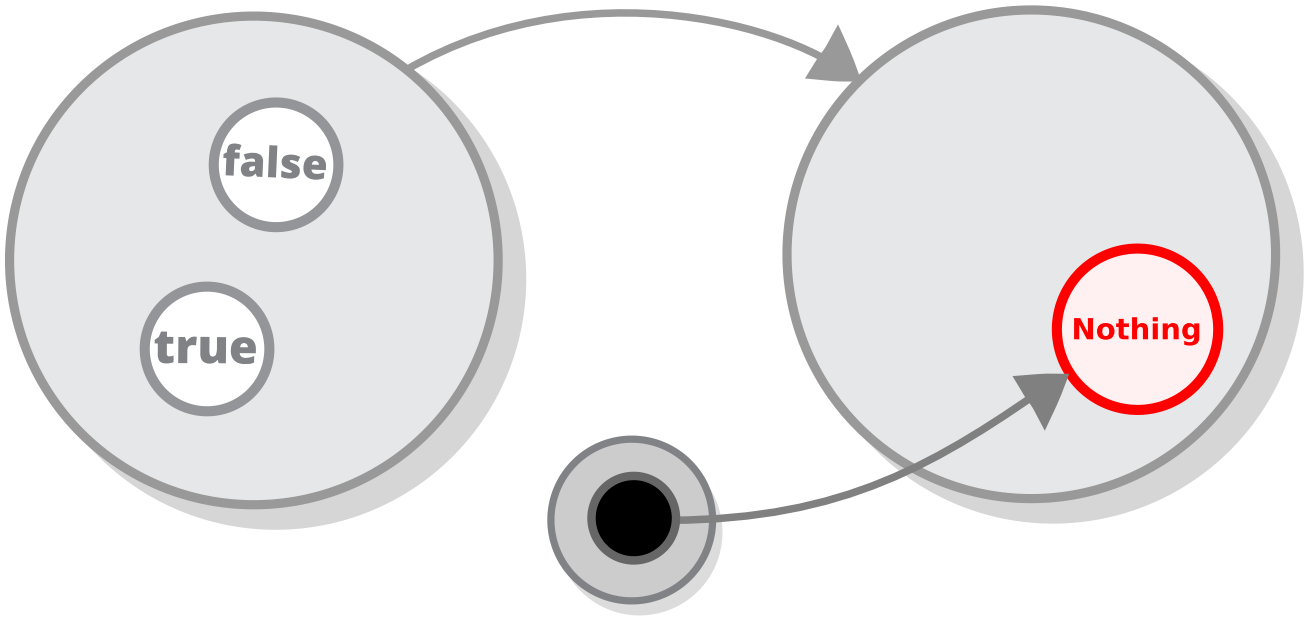
$$\text{Nothing} : \forall a.\ \text{Maybe}[a]$$

Or if we take a to be $Bool$, it is just:

DEFINITION:

$$\text{Nothing} : \text{Maybe}[Bool]$$

It says that there is a value called $Nothing$ for all $Maybe$ types (that's what $\forall$ means - "for all").



Term introduction: Just

Of course there would be no point in having many $Maybe$s if they all are the same. That's where the second line comes.

DEFINITION:

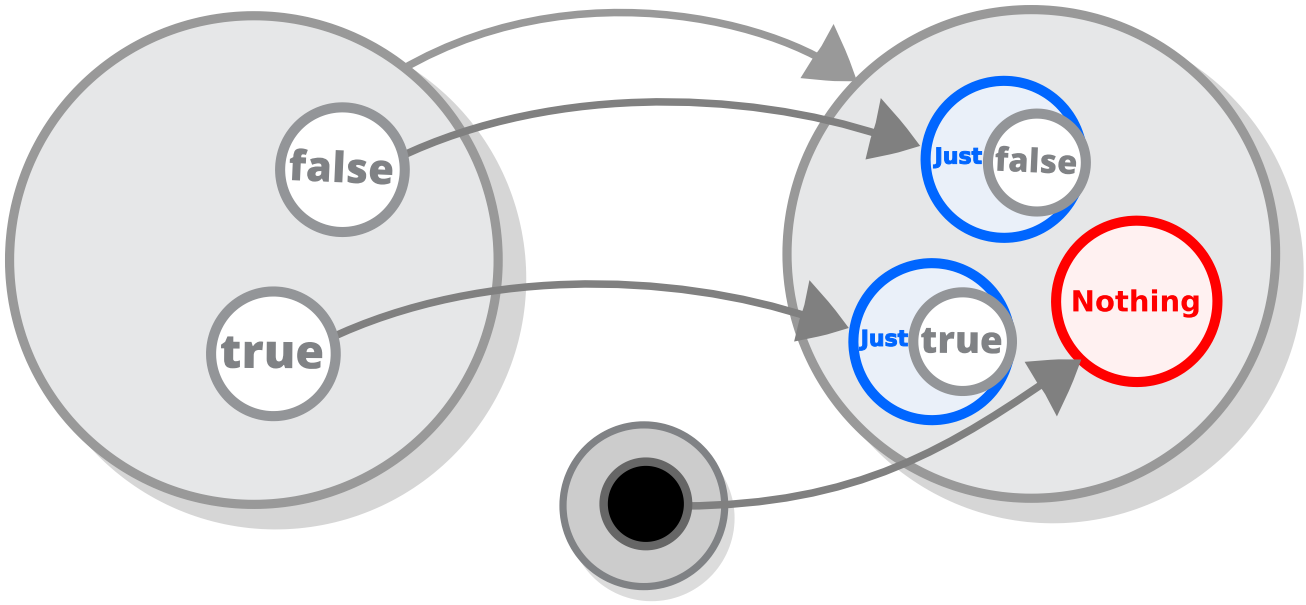
$$\text{Just} : \forall a.\ a \to \text{Maybe}[a]$$

or for booleans

DEFINITION:

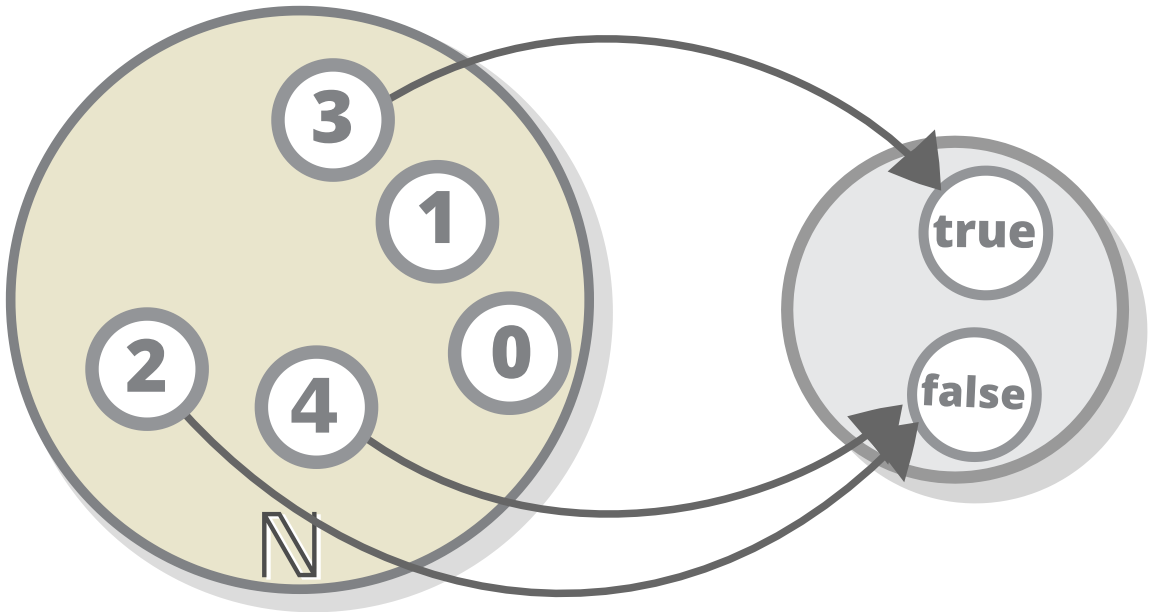
$$\text{Just} : \ Bool \to \text{Maybe}[Bool]$$

The constructor $Just$ represents an arrow from type a to type $Maybe[a]$ e.g. from $Boolean$ to $Maybe[Boolean]$.

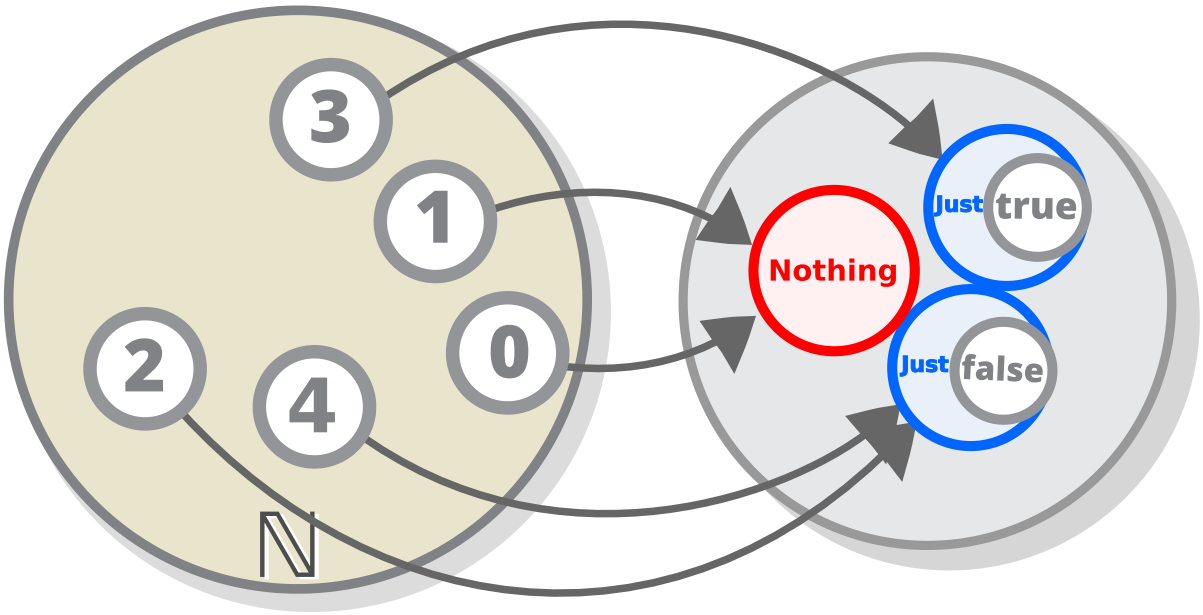


Using Maybe

The $Maybe$ type is used for handling errors i.e. for defining *partial functions*. Let's say we want to define a function that does not have an arrow for all values in the source. Does this mean that this function cannot be defined?



No, we just have to wrap the target type in $Maybe$ and it becomes a regular function.



Term elimination

To close the case, we define one function for deconstructing/eliminating the type maybe i.e. to convert it to something else, by using a function for converting its underlying type.

DEFINITION:

$$
\begin{aligned}
maybe : \forall\ a\ b.\ b\ \to (a \to b) \to Maybe[a] \to b \\
maybe\ n\ f\ Nothing\ &= \ n \\
maybe\ n\ f\ Just[x]\ &= \ f\ x
\end{aligned}
$$

Notice that this function defines an arrow from type $Maybe[a]$ to any type b, provided that a function $a \to b$, and a value of b is provided.

Inductive types. The natural number type.

Learning mathematics can feel overwhelming at first: you might not know how to proceed with such huge, even infinite, body of knowledge. But, it turns out the answer is simple: you start off knowing 0 things. Then, you learn 1 theory - congrats, you have learned your first theory and so you would know a total

of 1 theories. Then, you learn 1 more theory and you would already know a total of 2 theories. Then learn 1 more theory and then 1 more and, given enough time and dedication, you may learn all theories.

This argument applies not only to mathematical theories, but to everything else that is "countable", so to say. This is because it is the basis of the mathematical definition of natural numbers, as famously synthesized in the 19th century by the Italian mathematician Giuseppe Peano

1. 0 is a natural number.
2. If n is a natural number, $n + 1$ is a natural number.

Or as Haskellians say:

DEFINITION:

$$\begin{aligned} \mathbb{N} &: \mathrm{Type} \\ \mathrm{Zero} &: \mathbb{N} \\ \mathrm{Succ} &: \mathbb{N} \to \mathbb{N} \end{aligned}$$

Let's follow the arrows.

Type formation

The first line indicates that the natural numbers type is a normal non-polymorphic, or "monomorphic" type.

DEFINITION:

$$\mathbb{N} : \mathrm{Type}$$



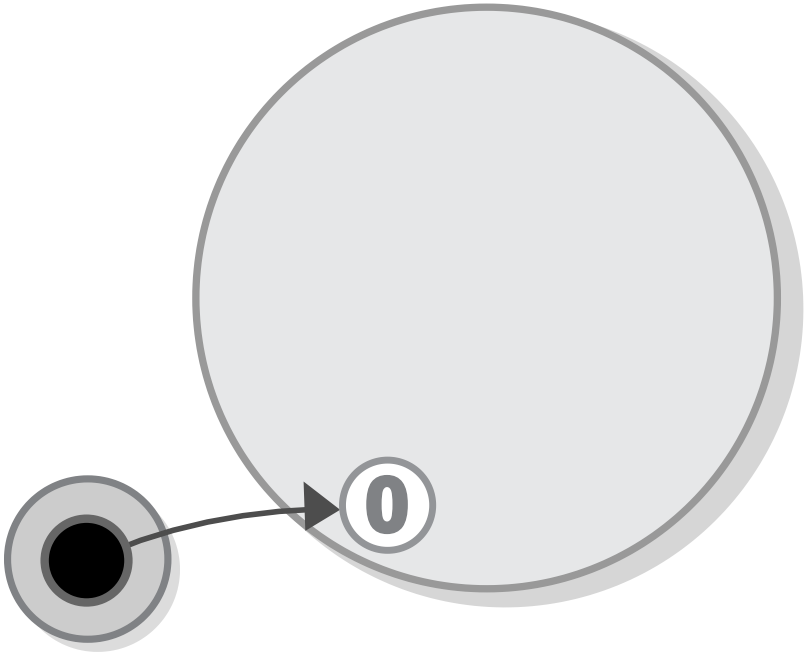
Term introduction: Zero

The first rule is also trivial.

DEFINITION:

$$\text{Zero} : \mathbb{N}$$

It allows us to construct one value, called zero



i.e. it is a *mot à mot* repetition of Peano's first axiom.

> 1. 0 is a natural number.

Term introduction: Successors

The second rule is more interesting.

DEFINITION:

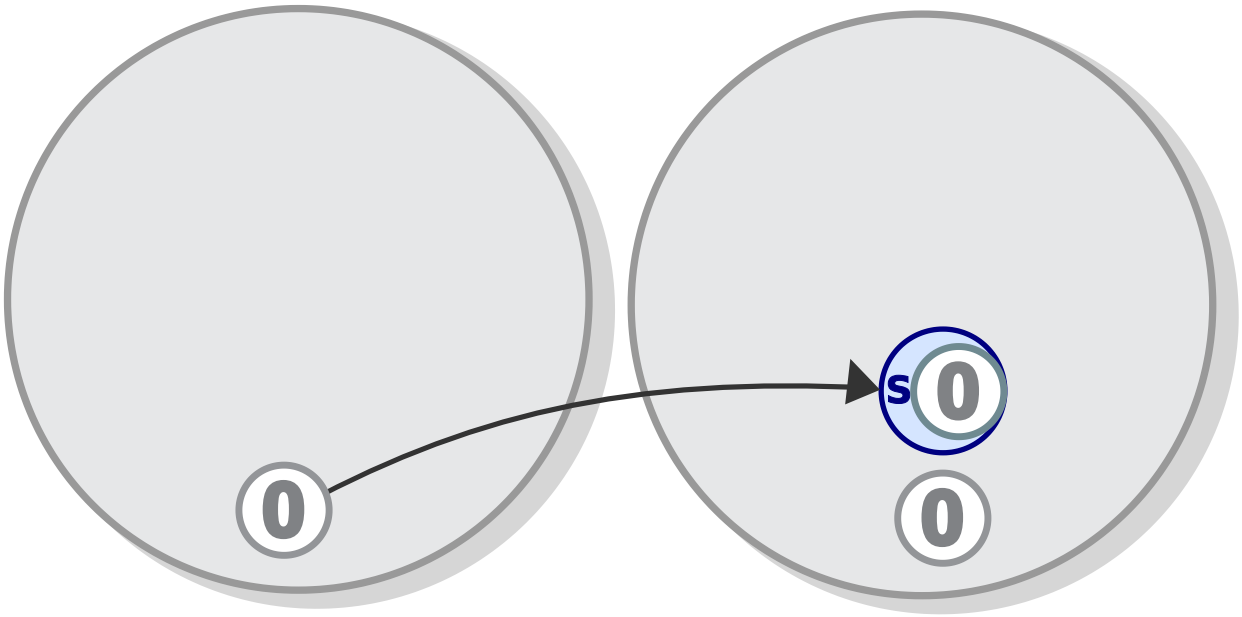
$$\text{Succ}: \mathbb{N} \to \mathbb{N}$$

It says that there is a constructor, called "Successor" $Succ$ (or +1, as we would call it) i.e. this is the equivalent of

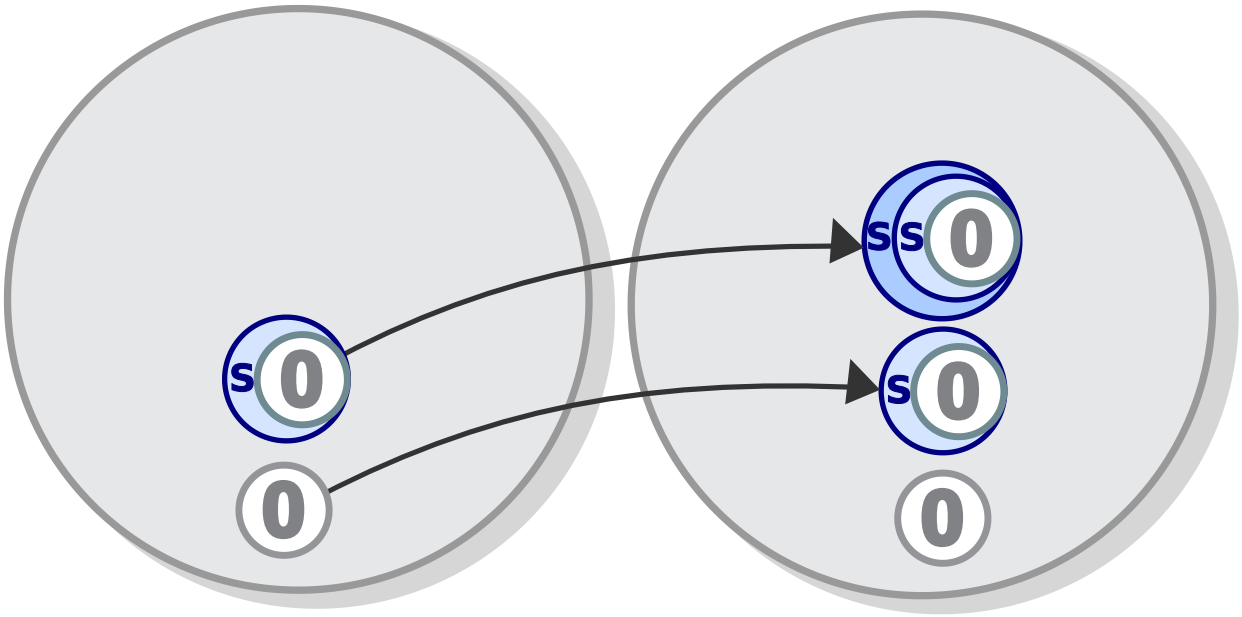
> 1. If n is a natural number, $n + 1$ is a natural number.

$Succ$ is an arrow from the type of the natural numbers to itself which means that given one natural number, $Succ$ constructs another one.

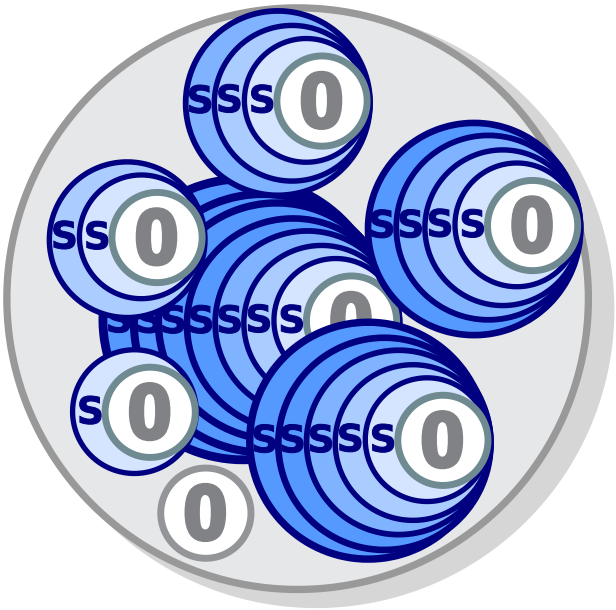
But right now we have just one term (value) of the natural numbers type: $Zero$. We draw the $Succ$ arrow and construct another one, $Succ\ Zero$ (known in some contexts as 1.



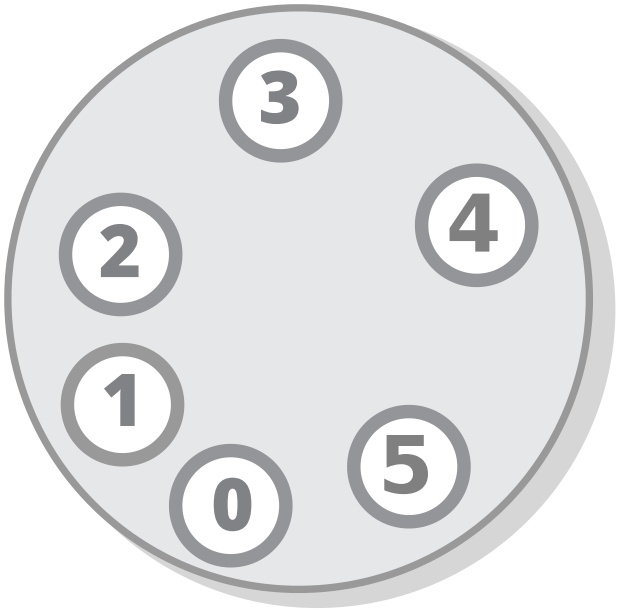
And now, we have one more value so we have to draw one more $Succ$ arrow. This time the result is $Succ\ Succ\ Zero$ i.e. two.



And we go on like this, *ad infinitum*, creating an endless chain of values.



Hm, this notation is a bit clunky, if only there were a better way to represent such values... Oh, wait.



And this is how you define an *inductive* type (or a *recursive* type, we can also call it).

Term elimination

Wait, there are also elimination rules, I always forget elimination rules, here they are.

> **DEFINITION:**
>
> $$\begin{aligned} foldNat &: \mathbb{N} \to a \to (a \to a) \to a \\ foldNat\ Zero\ z\ s &= z \\ foldNat\ (Succ\ a)\ z\ s &= s\ (foldNat\ a\ z\ s) \end{aligned}$$

This allows us, for example, to convert our `Nats` to the normal Haskell `Nats`:

```
foldNat (Succ (Succ Zero)) 0 (+ 1) -- 2
```

Any other canonical function that converts natural numbers to other types can also be defined using the elimination rule.

Composite types. The list type.

The landscape of types would be a really... flat place, without the *composite types*. Those are the types that allow you to unite several values of other types, into one.

The ultimate composite type is the list. The linked list , specifically, is a thing of beauty:

DEFINITION:

$$\begin{aligned} \text{List} &: \text{Type} \to \text{Type} \\ \text{Nil} &: \forall a.\ \text{List}[a] \\ \text{Cons} &: \forall a.\ a \to \text{List}[a] \to \text{List}[a] \end{aligned}$$

Let's unpack:

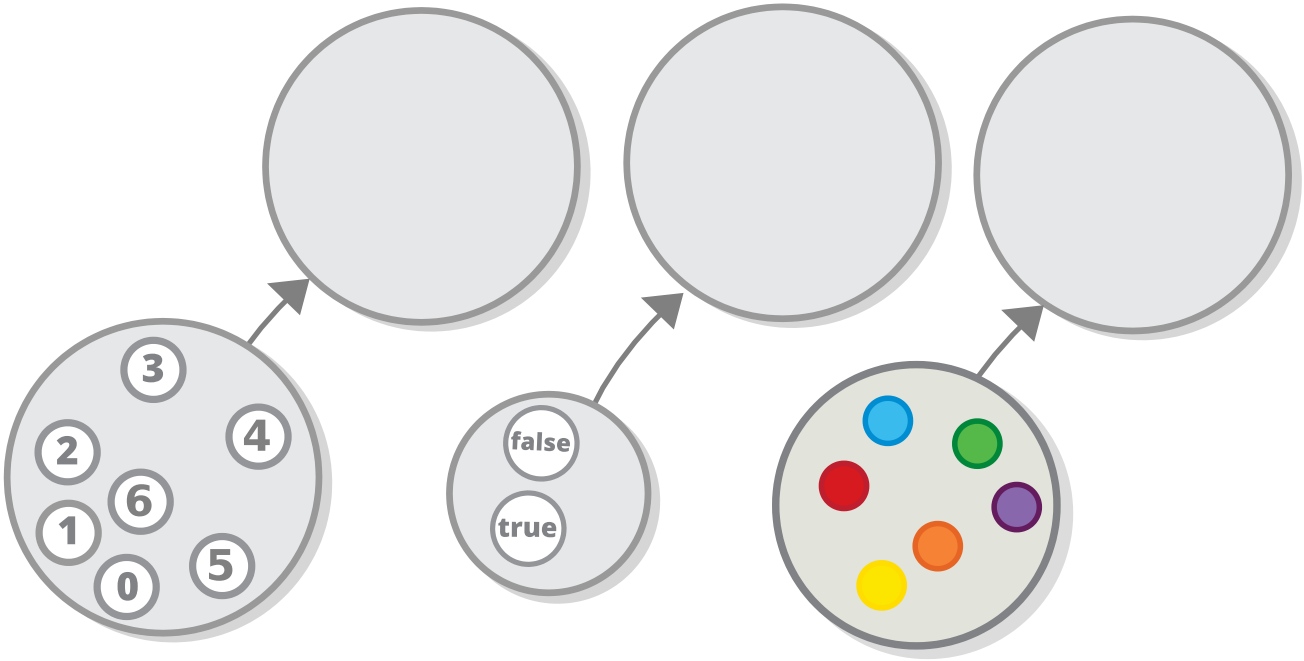
Type formation

The type formation rule tells us that $List$ (like $Maybe$) is a polymorphic type.

DEFINITION:

$$\text{List} : \text{Type} \to \text{Type}$$

This means, that there is not one, but many $List$ types, such as $List[Nat]$ $List[Bool]$ etc (infinitely many, if you consider lists of lists (of lists)). Those are usually read as "List of natural numbers", "List of Booleans" etc.

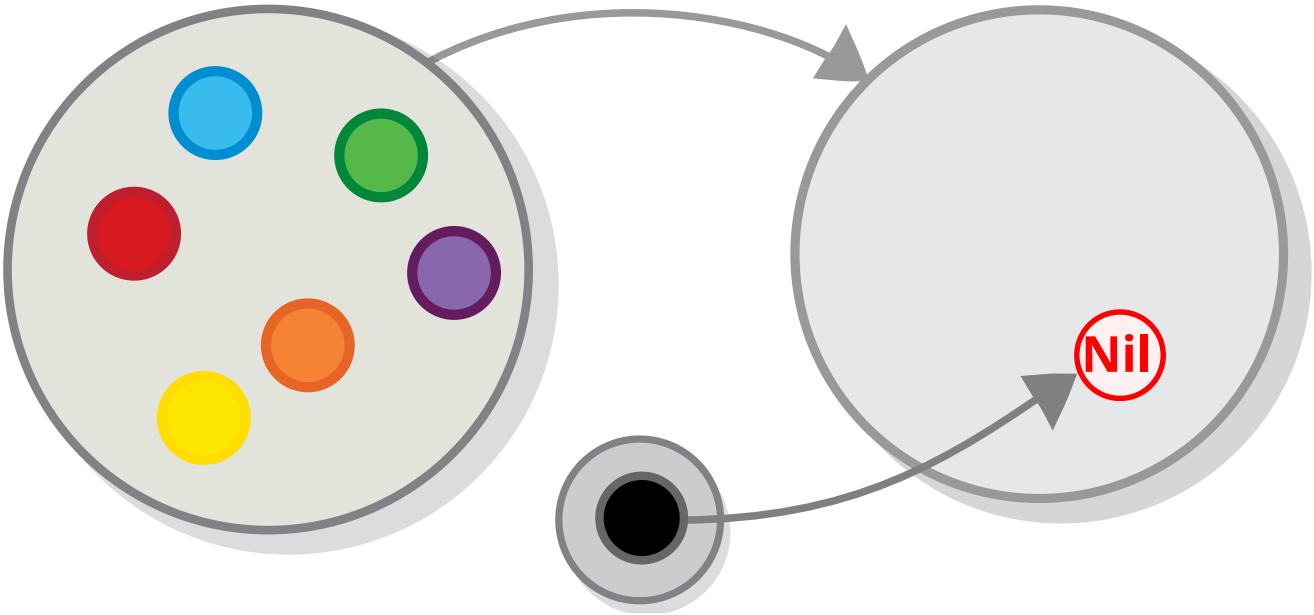


Term introduction: Nil

Now, let's check the constructors. The first defines a static value, one for each list, representing the empty list.

DEFINITION:

$$\text{Nil} : \ \forall a.\ \text{List}[a]$$



We will call this value Nil (native Haskell lists use the `[]` symbol).

Term introduction: Cons

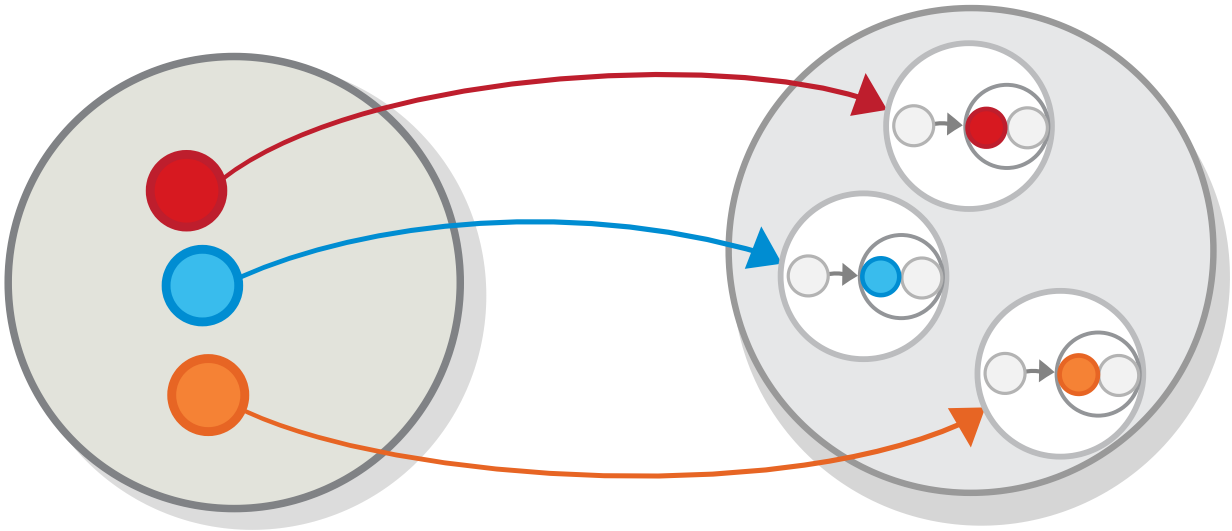
And now for the more interesting part: $Cons$, our second term introduction rule, ($Cons$ is short for construct, by the way) can be viewed as the operation of adding the value a to a list (and returning that list).

DEFINITION:

$$\text{Cons}: \ \forall a.\ a \to \text{List}[a] \to \text{List}[a]$$

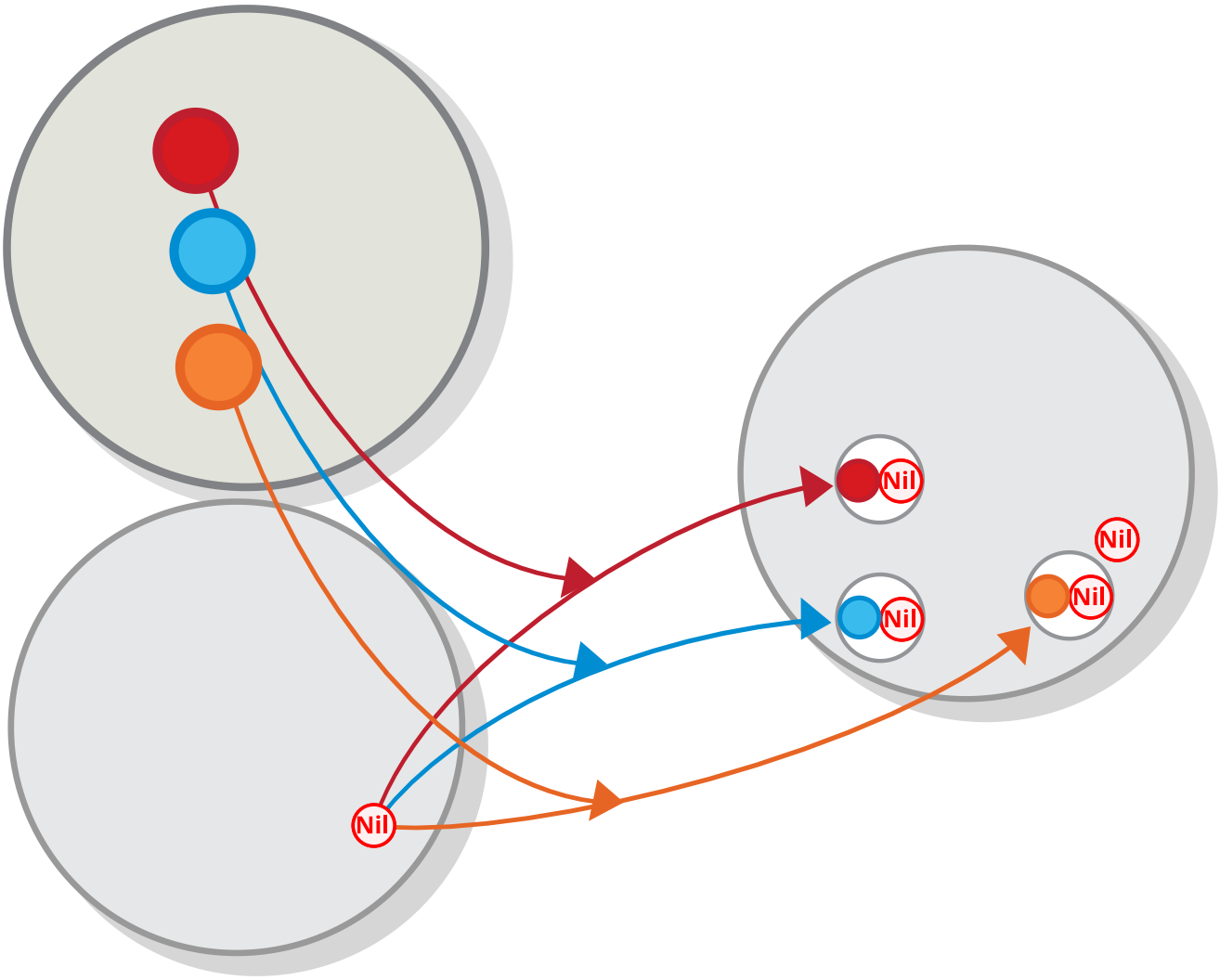
On first glance, $Cons$ looks pretty similar to the inductive $Succ$ constructor that we saw. And indeed, like $Succ$, $Cons$ is an inductive/recursive constructor that generates an infinite amount of terms.

However, unlike $Succ$, which has signature $X \to X$ (i.e. for each X, there is another one), $Cons$ has a signature $a \to (X \to X)$ – there is one List constructor for *every value of the type* a. We can visualize $Cons$ as an arrow, which points to another arrow.

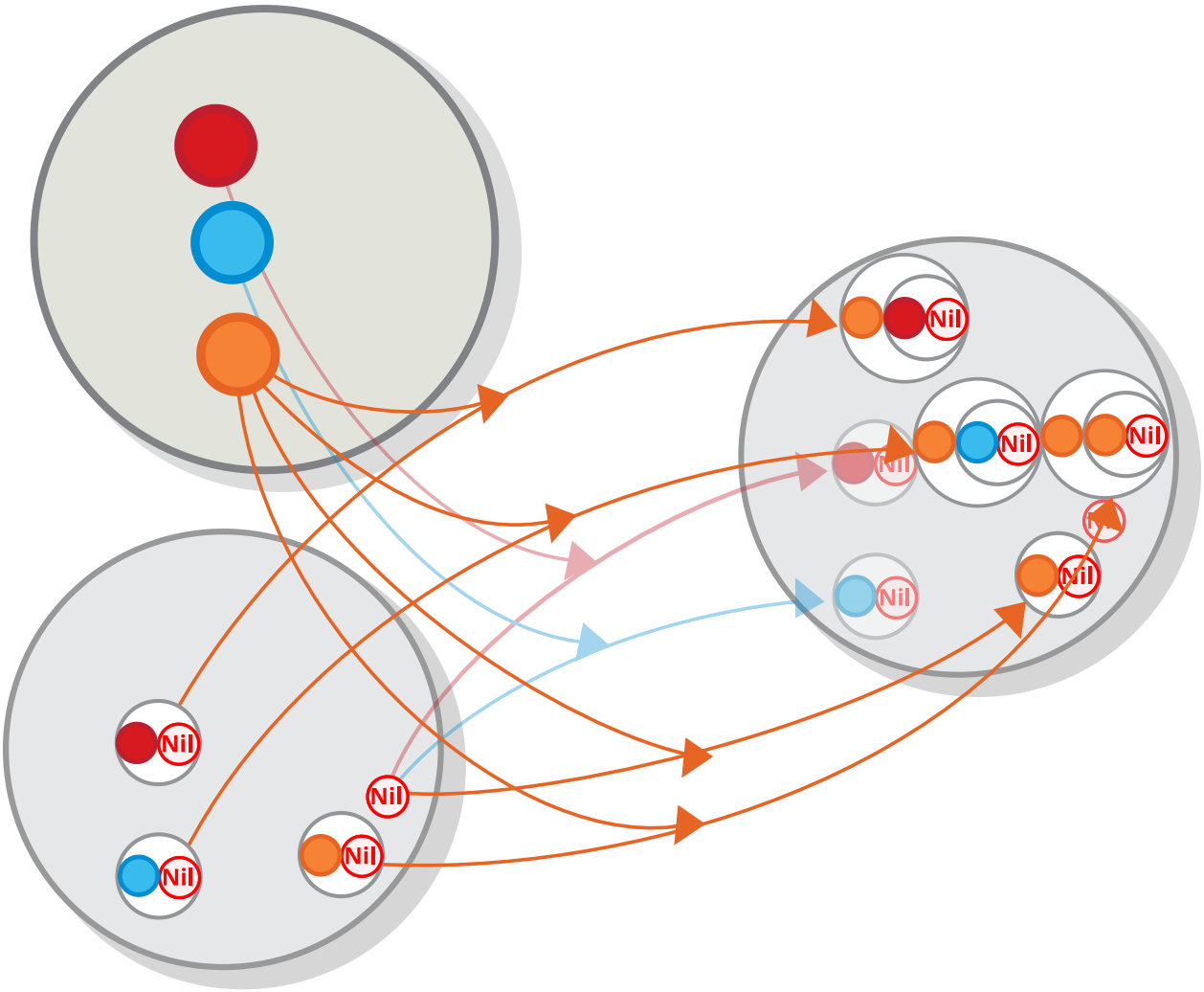


Why are we able to have arrows coming from other arrows? Perhaps this is the place to remind you that in Lambda calculus arrows between types are types as well.

So, let's start plotting these arrows, starting with the base value Nil.



As we said, the $List$ type is inductive i.e. every arrow that you draw generates more arrows (here, we only draw *part* of them (the ones that come from the orange ball)).



The result is a type with values that are... well, *lists* of other values,

Term elimination

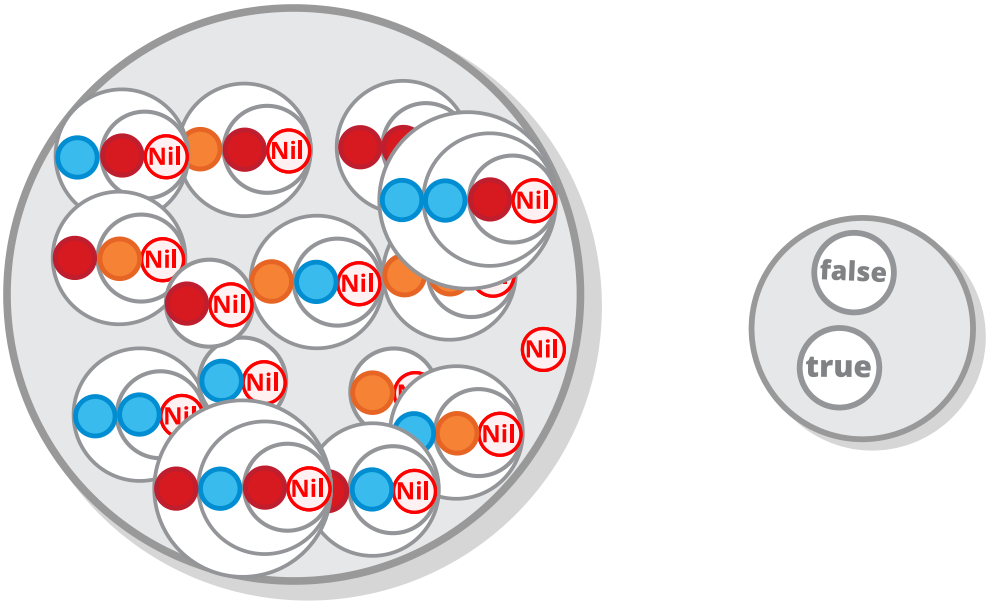
Now, let's write the term elimination rule.

DEFINITION:

$$
\begin{aligned}
foldList \ : \ (b \to a \to b) \to b \to List[a] \to b \\
foldList \ f \ z Nil = z \\
foldList \ f \ z (Cons \ x \ xs) = foldList \ f \ (f \ z \ x) \ xs
\end{aligned}
$$

This rule is also the most useful function for manipulating lists.

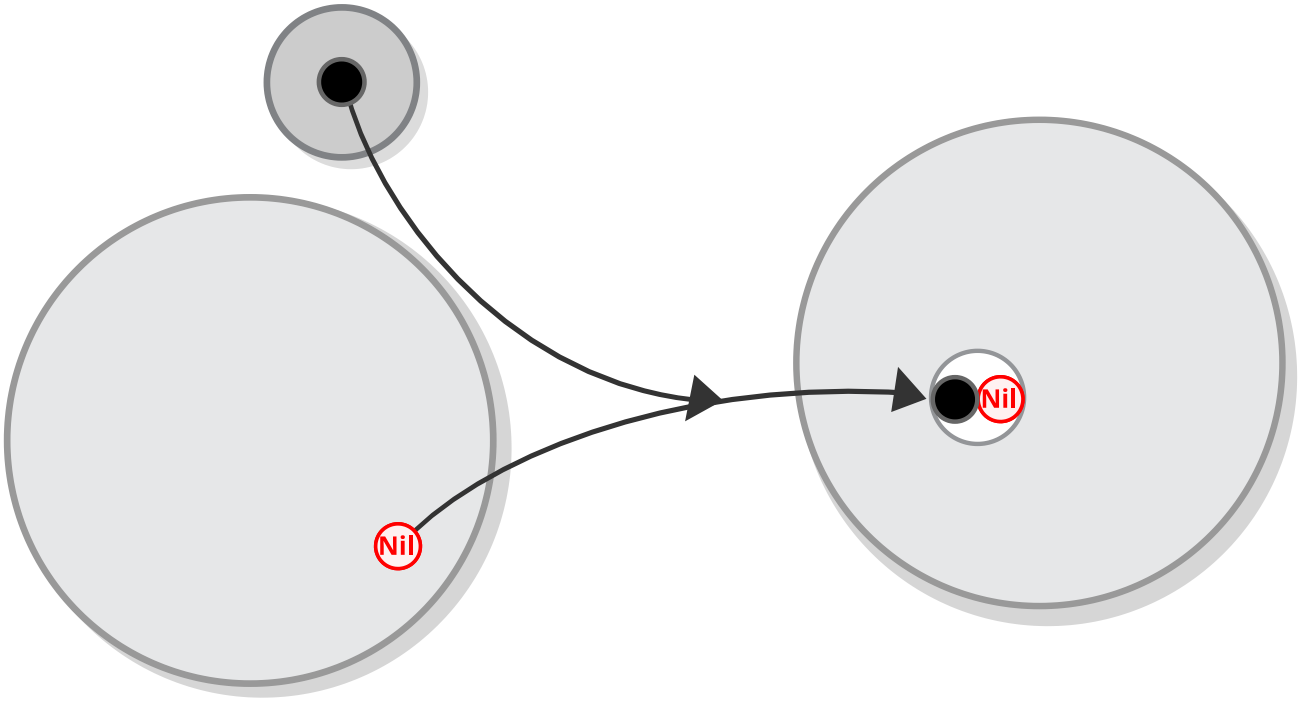
Task 1: There is a certain mapping from $List$ to $Boolean$ which is so useful, that some dialects of Lisp have no $Boolean$ type at all and rely just on this mapping. Draw it.



Task 2: Define this mapping (between `List` and `Bool`) in Haskell. Define it once by writing a function from scratch, and twice, with using the `foldList` function.

```
f :: (Bool -> a -> Bool)
f = undefined
foldList f False
```

Task 3: I present to you the type $ListUnit$ (where $Unit$ is the singleton type, (a type with one value). Draw the values of $ListUnit$ until you run out of space.



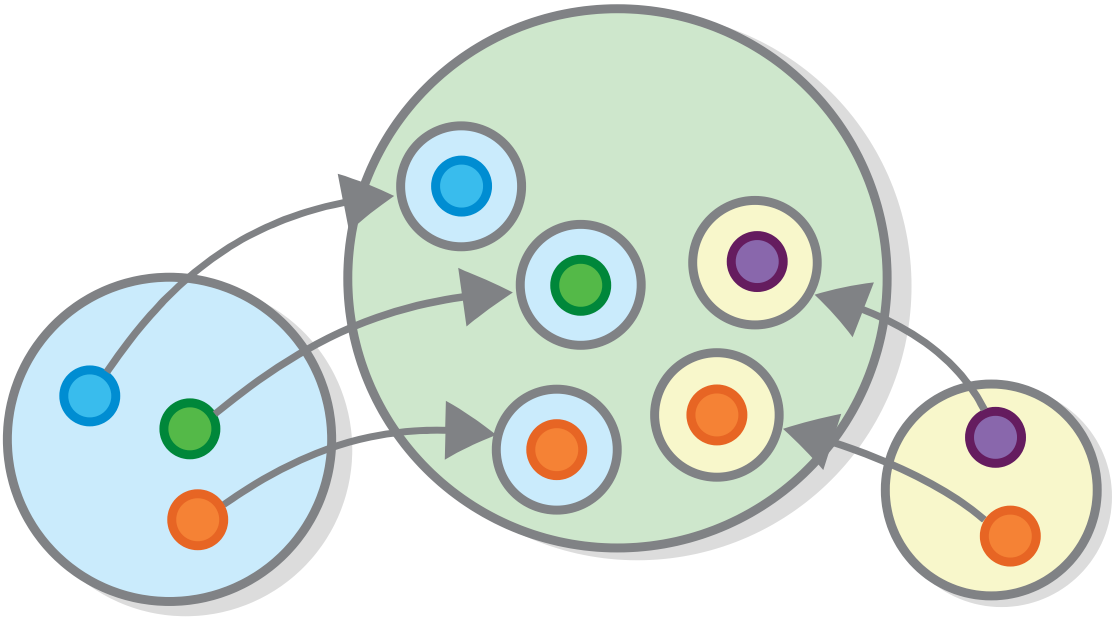
Task 4: The $ListUnit$ type is actually isomorphic to another type that we reviewed here. Find out which.

Positive and negative types. Either and Tuples.

Now, we will quickly present two more types, (hm... I have the feeling that I actually have seen those before).

Either

The $Either$ type is an interesting one.



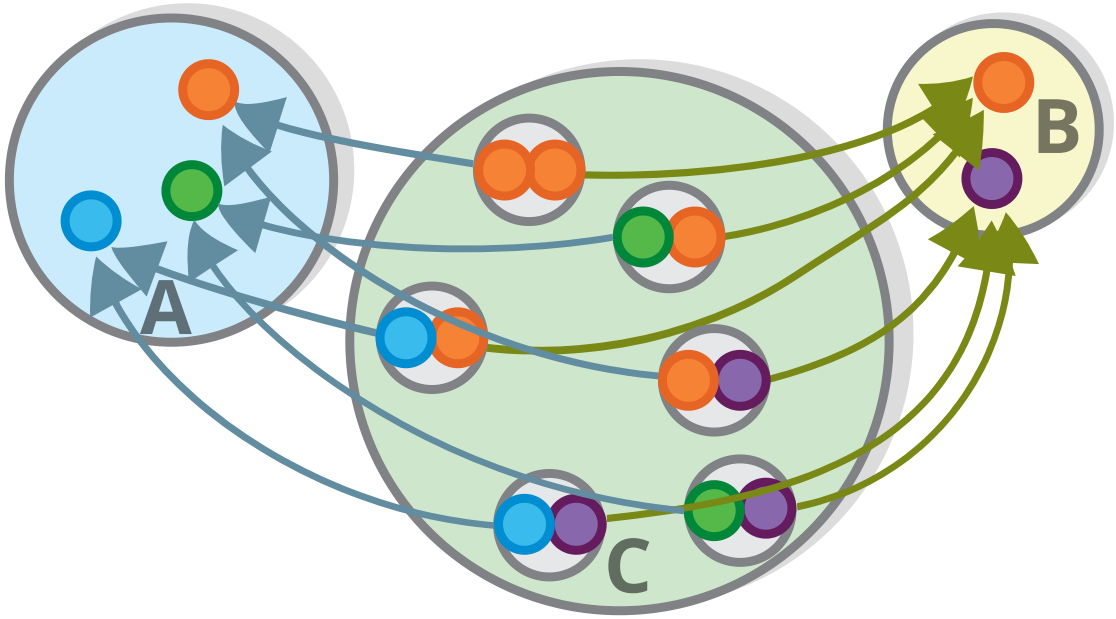
It is a type that is parametrized by two types a and b, and has two constructors/term introduction rules — one constructor, called $Left$, that takes a value of a. And another one, called $Right$ that takes a b. Here is the definition of Either (excluding the term elimination rule).

DEFINITION:

$$\begin{aligned} \text{Either} &: \text{Type} \to \text{Type} \to \text{Type} \\ \text{Left} &: \forall a\, b.\ a \to \text{Either}[a\ b] \\ \text{Right} &: \forall a\, b.\ b \to \text{Either}[a\ b] \end{aligned}$$

Tuple

The next type that we will introduce is the $Tuple$ type, which is also parametrized by a and b, but each value of it contains *both* a value of a and a value of b.



Here we will do something different — instead of the definition, we will directly present the type elimination rules.

DEFINITION:

$$
\begin{aligned}
first &: \forall\, a\ b.\, Tuple[a\ b] \to a \\
second &: \forall\, a\ b.\, Tuple[a\ b] \to b
\end{aligned}
$$

Task 5: Write a constructor of Tuple. Write a `fold` function for Either.

Positive and negative types

The $Either$ type is uniquely defined by its introduction rules i.e. the elimination rules can be derived from the introduction rules.

DEFINITION:

$$
\begin{aligned}
\text{Left} &: \forall a\ b.\ a \to \text{Either}[a\ b] \\
\text{Right} &: \forall a\ b.\ b \to \text{Either}[a\ b]
\end{aligned}
$$

$Tuple$, on the other hand, is defined by its elimination rules i.e. the introduction rules can be derived from them:

DEFINITION:

$$first \ : \ \forall \ a \ b. Tuple[a \ b] \to a$$
$$second \ : \ \forall \ a \ b. Tuple[a \ b] \to b$$

Types that, like $Either$, are defined by their introduction rules are called *positive types*. Types that are defined by their elimination are *negative*. All types that we saw so far (except $Tuple$) are positive.

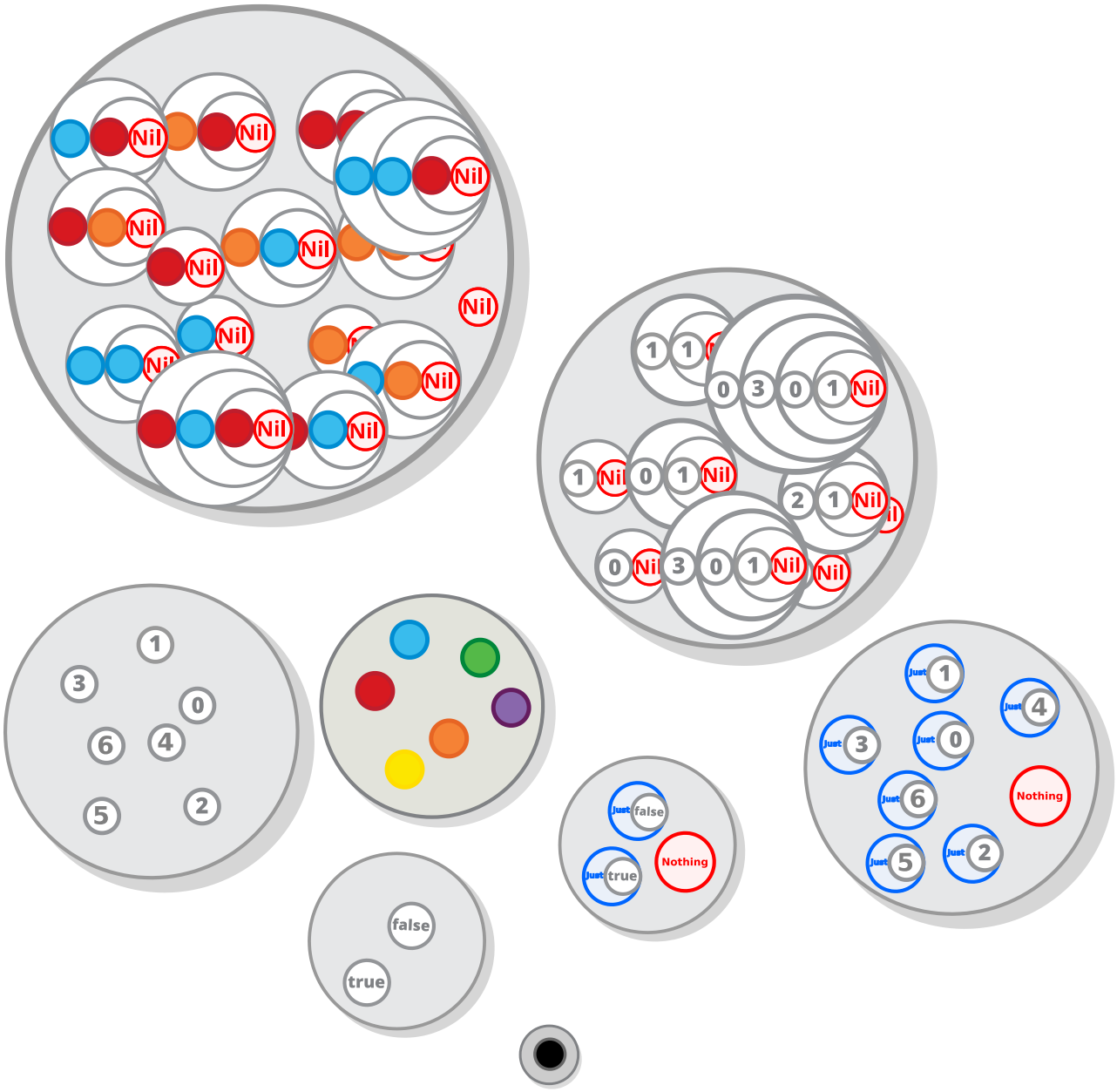
Positive and negative types are dual to each other.

Positive/negative types correspond to the categorical concepts of limit/colimit, but we will learn more about those later.

Task 6: Besides $Tuple$, there is one very important negative type, which we covered in this chapter (and in various other places).

Conclusion

In this section, we started from almost nothing – just the $Unit$ type. Then we defined a lot of stuff, very quickly.



One can see that some types that programmers use are still missing, but those can be defined in much the same way as the types we already defined: e.g. $char$ is just a base type, $string$ is just $List[char]$ etc.

Church encoding: From Haskell to Lambda Calculus

In the previous section, we did define a lot of stuff, very quickly, But we relied on Haskell's Generalized Algebraic Datatypes (GADTs) and it is not obvious that

it is possible to achieve the same things with just functions. However, it *is* possible: there exists a mechanism for encoding every type as a function, known as *Church encoding*, named after the creator of Lambda Calculus, Alonzo Church.

Base types. the Boolean type

Consider the Boolean type, which we defined like this:

DEFINITION:

$$\begin{aligned} \text{Bool} &: \text{Type} \\ \text{True} &: \text{Bool} \\ \text{False} &: \text{Bool} \end{aligned}$$

Here is a version of the same thing, with just functions.

DEFINITION:

$$\begin{aligned} type\ \text{Bool} &= \forall a.\, a \to a \to a \\ false &: \text{Bool} \\ false\ a\ b &= b \\ true &: \text{Bool} \\ true\ a\ b &= a \end{aligned}$$

Here $Bool$ is just a shorthand for the function $\forall a.\, a \to a \to a$ which accepts two values of type a for all a and returns another one. We can see that under this definition, $True$ is a function that returns the first a, and $False$ is a function that returns the second one.

Don't believe that these can function as booleans? Here is an implementation of the $ifElse$ function:

DEFINITION:

$$\begin{aligned} ifElse &: \forall a.\, \text{Bool} \to a \to a \to a \\ ifElse\ v\ a\ b\ &= v\ a\ b \end{aligned}$$

The implementation is trivial, because the datatype itself is doing the work. This is one of the main principle behind the "Church encodings" of datatypes as

they are called — the datatype encodes the term elimination rule.

Here is how you would use this:

```
ifElse true "True" "False" -- "True"
ifElse false "True" "False" -- "False"
```

Polymorphic types. The Maybe type

The Boolean type is, of course, a basic type. So, let's consider the $Maybe$ type, which is more complex:

DEFINITION:

$$\begin{aligned} \text{Maybe} &: \text{Type} \to \text{Type} \\ \text{Nothing} &: \forall a.\ \text{Maybe}[a] \\ \text{Just} &: \forall a.\ a \to \text{Maybe}[a] \end{aligned}$$

$Maybe$ is more complex, because it can *contain another value* in itself (with the $Just$ constructor). Here is where we learn another important principle of Church encoding: using curried functions to hold values.

DEFINITION:

$$\begin{aligned} type\ Maybe[a] &= \forall m.\, m \to (a \to m) \to m \\ nothing &: Maybe[a] \\ nothing\ n\ j &= n \\ just &: a \to Maybe[a] \\ just\ val\ n\ j &= (j\ val) \end{aligned}$$

The general principle

By now, you have probably seen that the Church-encoding of the type has the same constructors as the "normal" type — the difference is only that it accepts them as parameters.

If we have a type which is like $Maybe$, but with just one value (like the 1 type)...

DEFINITION:

$$\text{Nothing} : \ \forall a.\ \text{Maybe}[a]$$

...its Church encoding would be...

DEFINITION:

$$\forall m.\, m \to m$$

(This is indeed the Church encoding of the 1 type)

And then, if we extend the type to also have the $Just$ constructor...

DEFINITION:

$$\begin{aligned} \text{Nothing} &: \ \forall a.\ \text{Maybe}[a] \\ \text{Just} &: \ \forall a.\ a \to \text{Maybe}[a] \end{aligned}$$

...the encoding is extended like this...

DEFINITION:

$$\forall m.\, m \to (a \to m) \to m$$

This is why the signature of the Church-encoded type is almost identical as the signature of the $fold$ function, used to eliminate it:

DEFINITION:

$$foldMaybe : \forall a\ m.\, m \to (a \to m) \to Maybe[a] \to m$$

and the $fold$ function itself is trivial. Generally, the Church encoding of the type is basically a curried $fold$ function, a function, which, given the same arguments as the "folding" functions, produces the same results.

The general principle of Church encoding, is that a type is fully characterized by what you can do with it, so instead of providing constructors for types, which we later eliminate with the $fold$, we skip straight to the $fold$.

Task 7: Write a Church-encoded version of the natural numbers type

Polymorphic lambda calculus - Formal definition

So far, we saw how Lambda Calculus *works*. Now, we are about to see how it is defined formally. The answer is that, like all type systems, it is defined by *typing rules*. And what are typing rules? Well, basically they are also arrows. (Surprised?)

Natural deduction

Yes, Haskell's typing rules are indeed arrows, but they are defined in a language that is different from Haskell, called *natural deduction*. Natural deduction is like Haskell, but it uses a syntax where the premise and the conclusion are separated by a horizontal dash, e.g. instead of...

```
[a \to b]
```

...we write...

```
[\frac{a}{b}]
```

Aside from that, there is no big difference between natural deduction and Haskell. Take, for example the boolean type. We defined it in Haskell like this:

DEFINITION:

$$\begin{aligned} \text{Bool} &: \text{Type} \\ \text{True} &: \text{Bool} \\ \text{False} &: \text{Bool} \end{aligned}$$

If we want to define it using natural deduction, so it is one of the "primitive" types that are part of the type system itself (as it is defined in most programming languages), it would look like this.

DEFINITION:

$$\frac{}{\text{Bool} :: \text{Type}}$$

This means that there is a type called Bool (technically, this is not a typing rule, but a "kinding" rule (and thus the double-colon)).

DEFINITION:

$$\frac{}{\mathrm{True} : \mathrm{Bool}}$$

DEFINITION:

$$\frac{}{\mathrm{False} : \mathrm{Bool}}$$

True and False are Booleans.

Oh I forgot, in natural deduction it is permitted to have conclusions without premises.

Task 8: Define the elimination rule for Booleans, using natural deduction.

Contexts

Is this *too* simple? Let's add the concept of the typing *context* (or typing *environment*) to the mix.

Here's the deal: Types and variables have to be stored *somewhere*. So, given a bunch of values (e.g. x, y, z etc.) and a bunch of types (e.g. A, B, C etc.), a context is a *set* (Oops, I did it again) of all variables and their types e.g. $(x, A), (y, B), (z, C) \ldots$.

We usually denote the context with the letter Γ, and we use the $\vdash$ symbol to denote something that follows from that context (oh, no not another arrow) e.g. $\Gamma \vdash a : b$ means that in the context Γ, there is a variable a that has the type b.).

So, when we consider the context, the above definition becomes

DEFINITION:

$$\frac{}{\Gamma \vdash \mathrm{Bool} :: \mathrm{Type}}$$

i.e. the context includes the type Bool

DEFINITION:

$$\frac{}{\Gamma \vdash \mathrm{True} : \mathrm{Bool}}$$

i.e. the context includes the value True of type Bool

DEFINITION:

$$\frac{}{\Gamma \vdash \mathrm{False} : \mathrm{Bool}}$$

i.e. the context includes the value False of type Bool

Thus, we straight away define the Boolean type to be part of the context.

Value-level arrows

With that, we list the axioms of Lambda Calculus, which contain nothing more than the definition of the type of value-level arrows (functions).

There are several typing rules that we have to define, starting with the trivial rule *Var*, that states the following: if we previously said that x has type A, then x has type A.

DEFINITION:

Variable typing rule (Var)

$$\frac{x : A \in \Gamma}{\Gamma \vdash x : A}$$

Now, we proceed to define the types of the arrows.

We start with the type formation rule (or the *kinding* rule).

DEFINITION:

Lambda type formation rule

$$\frac{\Gamma \vdash A :: Type, \Gamma \vdash B :: Type}{\Gamma \vdash A \to B :: Type}$$

And then the two typing rules. One is the *term introduction* for lambda terms, which is called *abstraction* (or *Abs*).

DEFINITION:

Lambda term introduction rule (Abs)

$$\frac{\Gamma, x : A \vdash y : B}{\Gamma \vdash \lambda z : A \to B}$$

(i.e. if we have a way given a value x of type A to obtain a value y of type B, then we have ourselves a function $A \to B$).

And there is also term elimination for lambdas, i.e. function *application* (*App*).

DEFINITION:

Lambda term elimination rule (App)

$$\frac{\Gamma \vdash z : A \to B, \Gamma \vdash x : A}{\Gamma \vdash zx : B}$$

To understand how those rules work, let's take the function $length : string \to int$ as an example.

DEFINITION:

Term introduction of "length"

$$\frac{\Gamma, x : string \vdash y : int}{\Gamma \vdash \lambda length : string \to int}$$

THEOREM:

Term application of "length"

$$\frac{\Gamma \vdash length : string \to int, \Gamma \vdash x : string}{\Gamma \vdash length\ x : int}$$

Those rules are all you need to define value-level arrows.

Simply-typed Lambda Calculus

The rules we reviewed so far don't define Polymorphic Lambda Calculus, but they define a simpler type system aptly called *Simply-typed Lambda Calculus*.

> **DEFINITION:**
>
> The *simply-typed Lambda Calculus* (STLC) is the type system which only has one type constructor — function (lambda), with typing rules *Abs* and *App* (for term introduction and elimination). It also has the *Var* typing rule.

Polymorphism

Simply-typed lambda calculus is just like Polymorphic Lambda Calculus, except that... it is not polymorphic i.e. we cannot define the polymorphic types like $Maybe$.

> **DEFINITION:**
>
> $$\begin{aligned} \text{Maybe} &: \text{Type} \to \text{Type} \\ \text{Nothing} &: \forall a.\ \text{Maybe}[a] \\ \text{Just} &: \forall a.\ a \to \text{Maybe}[a] \end{aligned}$$

The best we can do is to define a separate versions of the type, e.g. one which works just for int.

> **DEFINITION:**
>
> $$\begin{aligned} \text{MaybeInt} &: \text{Type} \\ \text{NoInt} &: \text{MaybeInt} \\ \text{JustInt} &: \text{int} \to \text{MaybeInt} \end{aligned}$$

And one for string

> **DEFINITION:**
>
> $$\begin{aligned} \text{MaybeString} &: \text{Type} \\ \text{NoString} &: \text{MaybeString} \\ \text{JustString} &: \text{string} \to \text{MaybeString} \end{aligned}$$

Furthermore, in STLC we cannot define *functions* that work for polymorphic $Maybe$ (regardless of the type they are holding), so we have to redefine not only the types, but all functions that use them.

To combat this problem, and to ascend ourselves from *Simply-typed* to *Polymorphic* Lambda Calculus (AKA System F), we define type-level arrows.

But, actually we should talk about Kinds first...

Kinds

In the expression "$x : A$" , "A" denotes the type of the value. But then what is $Type$ in the Expression "$A :: Type$"? We cannot say that $Type$ is a *type*, cause we will see Russell, lurking behind our back, with his eponymous paradox. In Lambda Calculus, it is resolved in the following way:

- **Values** have **types** (which are annotated with single-colon - :)
- **Types** have types-of-types i.e. **kinds** (which are annotated with a double-colon - ::.
- **Kinds** have... OK, let's stop here for now...

This means that besides a type system and typing rules, we have a *kind-system* and *kinding rules*. But please, don't throw this book out of the window - the kinding system for both STLC is pretty easy to define: there is just one kind, that we call Type (sometimes it is marked with a $*$).

> **DEFINITION:**
>
> Kinding rule in STLC
>
> $$\frac{}{\Gamma \vdash Type}$$

And then the type definition rules are defined like this (i.e. everything is of kind Type:

> **THEOREM:**
>
> $$\frac{}{\Gamma \vdash \mathrm{Bool} :: \mathrm{Type}}$$

And for Polymorphic Lambda Calculus, we would see in the next section.

Type-level arrows

We started defining STLC by defining value-level *variables*, using the trivial *Var* typing rule,

> **DEFINITION:**
>
> Variable typing rule (Var)
>
> $$\frac{x : A \in \Gamma}{\Gamma \vdash x : A}$$

In Polymorphic Lambda Calculus we also have *type-level variables*, which are defined with a similar rule.

> **DEFINITION:**
>
> Type Variable kinding rule (TVar)
>
> $$\frac{A :: K \in \Gamma}{\Gamma \vdash A :: K}$$

Now, let's proceed with defining the type of the arrows themselves.

In Polymorphic Lambda Calculus, as in STLC, we have value-level arrows that convert values to other values...

> **DEFINITION:**
>
> Lambda type formation rule
>
> $$\frac{\Gamma \vdash A :: Type, \Gamma \vdash B :: Type}{\Gamma \vdash A \to B :: Type}$$

And, we also have *type-level* arrows that convert types to other types. They are defined with this kinding rule:

> **DEFINITION:**
>
> Type-level lambda kind formation rule

$$\frac{\Gamma, (\alpha :: A) \vdash (B :: Type)}{\Gamma \vdash \forall (\alpha :: A).\, (B :: Type)}$$

For example, for the $Maybe$ type, this rule would say

DEFINITION:

Type-level lambda type formation rule for Maybe

$$\frac{\Gamma, (\alpha :: Type) \vdash Maybe[\alpha] :: Type}{\Gamma \vdash \forall (\alpha :: Type).\, Maybe[\alpha] :: Type}$$

Polymorphic functions

The more interesting (and harder) part of polymorphism is augmenting value-level arrows to work with polymorphic types i.e. to have functions which accept a type as an argument, in addition to a value.

For example, if we work in the context of STLC and we use the $MaybeString$ type that we defined in the last section (and that works only with strings), we can define a function with the following type signature

DEFINITION:

A monomorphic function

$$z :: string \to MaybeString$$

Using the capabilities of Polymorphic Lambda Calculus, we can abstract the type $String$ (with the rule *TAbs* to build a polymorphic function, which looks like this.

DEFINITION:

A polymorphic function, using *TAbs*

$$z' :: \forall \alpha.\, \alpha \to Maybe\ \alpha$$

And then, we use *TApp* to apply the type parameter $String$ to the abstract function to get our original function.

> **DEFINITION:**
>
> A monomorphic function, using *TApp*
>
> $$z = z'[String]$$

(This is not a real Haskell syntax, as Haskell does type application automatically – you just provide the value and the language deduces the type from it).

Here are the typing rules of polymorphic functions themselves:

> **DEFINITION:**
>
> *Type abstraction* (or *TAbs*)
>
> $$\frac{\Gamma, (\alpha :: A) \vdash z : C}{\Gamma \vdash (\Lambda \alpha :: A.\, z) : \forall (\alpha :: A).\, C}$$

> **DEFINITION:**
>
> *Type application* (*TApp*)
>
> $$\frac{\Gamma \vdash z' : \forall (\alpha :: A).\, C, \Gamma \vdash (X :: A)}{\Gamma \vdash z'[X] : C[\alpha := X]}$$

And with this, we conclude the definition of System F.

> **DEFINITION:**
>
> The *polymorphic Lambda Calculus* (System F) is the type system which only has one type constructor – polymorphic lambda, with typing rules *Abs* and *App* (for introduction and elimination of terms) and kinding rules *TAbs* and *TApp* (for introduction and elimination of types). It also has the *Var* typing rule. and *TVar* kinding rule.

Now we are ready to see how it relates to our main subject matter.

Type systems as Categories

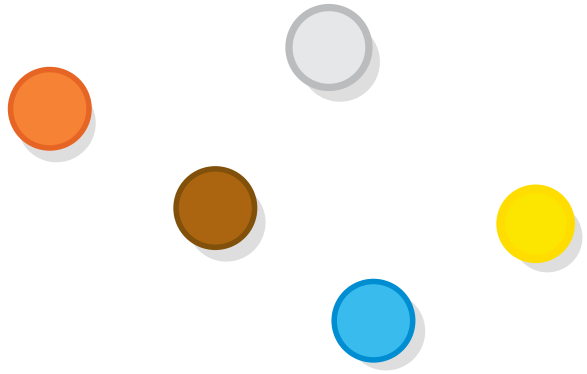
We already drew some parallels between type theory and category theory, but there is more than just mere parallels: when viewed through the proper angle,

type systems *are* a certain type of categories. And, more: we already know which one!

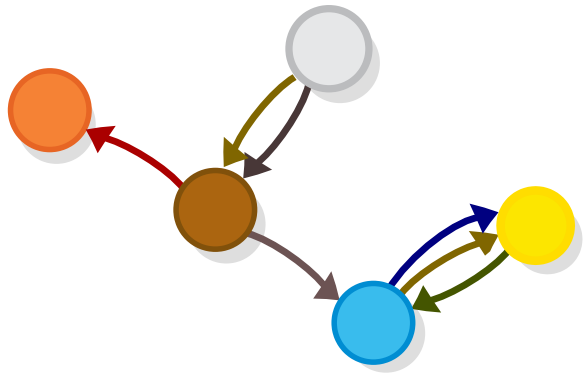
Types as objects arrows as morphisms

Let's start from the basics.

Every type is an object.



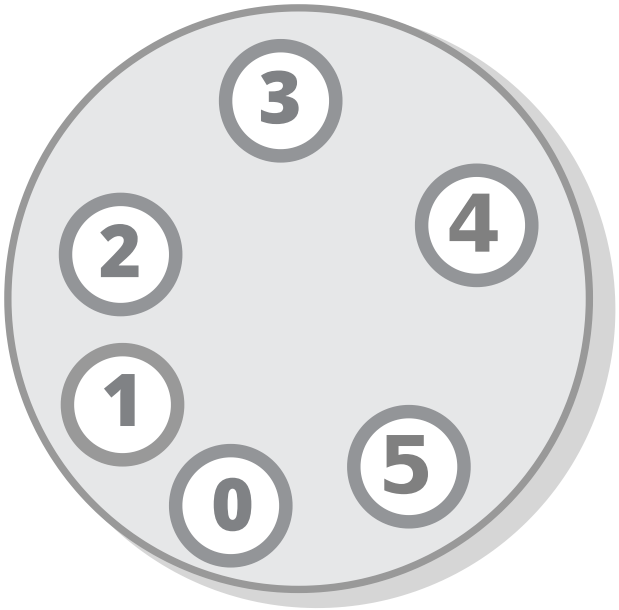
And every value-level arrow (function) is a morphism.



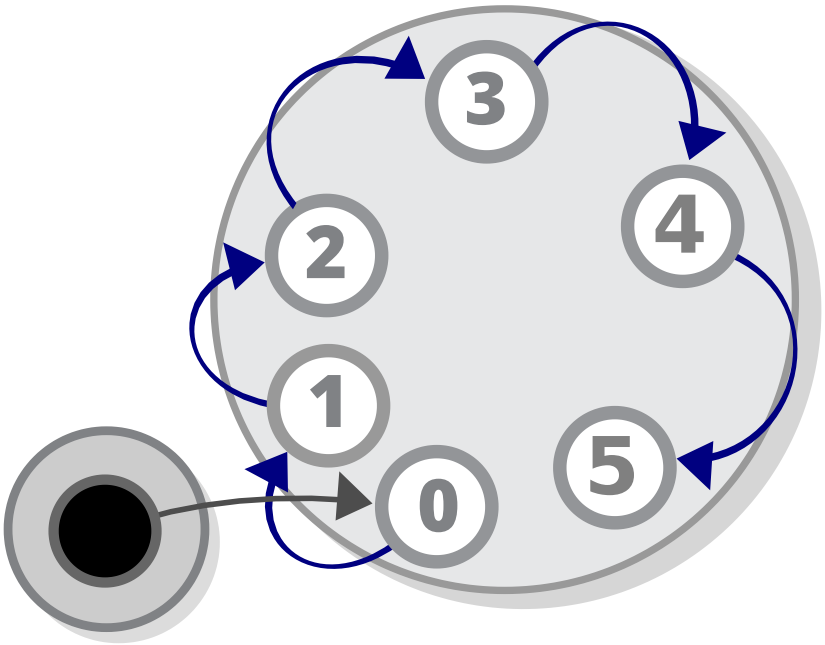
And now for something not so trivial — values.

Values as morphisms too

We said that category theory is all about arrows. Here, we seemingly turned away from this, and we started drawing values and internal diagrams again, as for examples the natural numbers type.

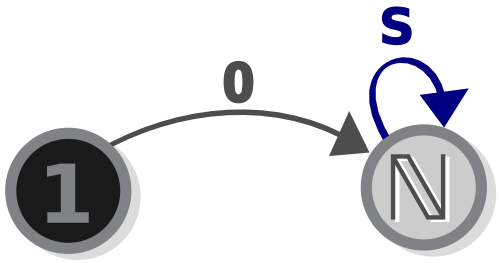


But there is no discrepancy. We said that in type theory, "the only values are the ones which are sources and targets of arrows". In the case of natural numbers, it is the $successor$ arrow, and the $zero$ arrow.



This means that *values are actually just another way to represent arrows*.

For example, the type of natural numbers be represented externally like this.



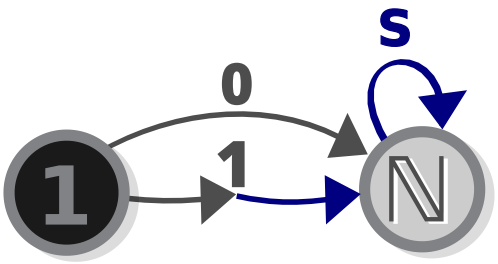
This is where we go back to a simple theorem about set/type elements that we learned in the second chapter:

> **THEOREM:**
>
> Each element of any set X is isomorphic to a function $1 \to X$ (where 1 means the singleton set).

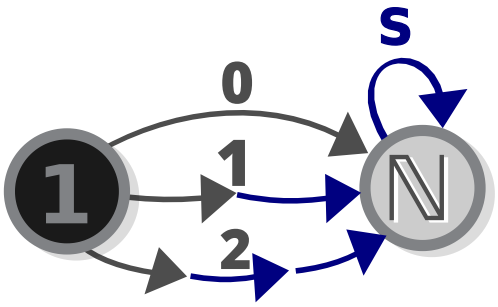
So, arrow from $1 \to X$ for some type X is equivalent to a *value* of X when we view it as a set.

This means that the *arrow* from $1 \to \mathbb{N}$ which we call 0 is isomorphic to the *value* 0.

Besides $1 \to \mathbb{N}$, we have one more arrow $s : \mathbb{N} \to \mathbb{N}$. So, what happens when we combine the two? We get another $1 \to \mathbb{N}$ arrow – $s \circ 0$, the successor of 0, i.e. 1.



And if we do this again, we get $s \circ s \circ 0$.



We can see that it plays out in the same way as the old definition. Rather than going *back* to values, we went *full circle* and discovered (or rediscovered, since we knew it from the Elementary Theory of the Category of Sets) that values are just a more convenient way to draw arrows.

Simply-typed Lambda calculus as a Cartesian Closed Category

OK, we got it: if we view types as objects and arrows *and values* as morphisms, the entire type theory/type system can be viewed as a category. But which category? To understand, we review the special features that Lambda Calculus has: we know that it has the Lambda type, AKA the function type, so we need a category with a function type. To define it, you also need the terminal object (1 object) and products, so we are searching for a category has to has those. Surprisingly, we already know which category has all those features. We examined it in the previous chapter, when we covered logic.

> **THEOREM:**
>
> Simply-typed Lambda Calculus can be seen as a Cartesian Closed Category—the tuple and either types are the "and" and "or" operations, the Unit and Empty types are the values "True" and "False" and the Lambda type is the exponential object.

Untyped Lambda calculus as a Cartesian Closed Monoid

We won't cover it in depth, but I think that it is worth mentioning that there is also such a thing as a *Untyped* Lambda Calculus. This is a language that only has one type — function and every function accepts every other, as an argument.

So, how does Untyped Lambda Calculus fit in our picture? It fits perfectly: it corresponds to Cartesian Closed Monoids (C-monoids for short). Those are cartesian closed categories with only one object.

Polymorphic Lambda calculus as...

We established that value-level arrows correspond *to morphisms*.

But what about *type-level arrows* (AKA polymorphic types)? What do they correspond to?

We will get on with this in the next chapter!

Curry-Howard-Lambek correspondence

Before we close this off, let's think about the following: why do Lambda Calculus and Intuitionistic logic correspond to almost the same type of categories — Cartesian Closed.

To understand, we look into the correspondence between logics and categories that we studied in the previous chapter.

> **THEOREM:**
>
> The logical system of intuitionistic logic can be seen as a Bicartesian Closed Category—the "and" and "or" operations are the product/coproducts, the values "True" and "False" are the initial/terminal objects and the implication operation is the exponential object.

This is almost the exact same definition that we now have about types (the only difference is that Lambda Calculus can work without coproducts and an initial object, so it's a Cartesian Closed category, whereas Intuitionistic logic needs those, so it's *Bi*cartesian).

This means that the similarities between Intuitionistic Logic and Lambda Calculus don't end with both of them being "categorical". They are actually one and the same thing:

- Logical propositions can be viewed as *types*.
- A proof of a given proposition is nothing but a value of the corresponding type. - The Lambda type is the implication object.
- Functional application (App) is *modus ponens*.

Intuitionistic logic	Simply-typed Lambda calculus	Cartesian closed category
Proposition	Type	Object
Implication	Arrow	Morphism
Primary proposition	Value of type A	Morphism 1 → A (global element)
Implication object (A → B)	Lambda type (A → B)	Exponential object B^A
And (A ∧ B)	Tuple (A × B)	Product A × B
Or (A ∨ B)	Either (A + B)	Coproduct A + B

Intuitionistic logic	Simply-typed Lambda calculus	Cartesian closed category
True ($\top$)	Unit type	Terminal object (1)
False ($\bot$)	Empty type	Initial object (0)
Negation A → $\bot$	Function A → Empty	Morphism A → 0

In short, in the last chapter, we talked about the correspondence (known as Curry-Howard-Lambek correspondence) between Intuitionistic logic and Cartesian Closed Categories and now we are adding a third branch of the correspondence — Lambda Calculus.

Appendix: definitions in Haskell

I thought a lot about whether to provide the definitions of the basic types in executable Haskell, or as formulas (i.e. monospaced or modern). At the end, I decided that formulas are better, but all of them are indeed Haskell, with several caveats:

First, we want to start from a blank state. In Haskell we can do that by removing the standard library, (called "Prelude") which is typically imported implicitly.

```
{-# LANGUAGE NoImplicitPrelude #-}
```

And we use one more extension, that would allow us to write type definitions that are a bit more explicit.

```
{-# LANGUAGE GADTs, NoImplicitPrelude #-}
```

And that is pretty much it. That and using `forall` instead of the $\forall$ symbol.

```
{-# LANGUAGE GADTs,  NoImplicitPrelude #-}
import qualified Prelude as P

data Bool where
  True :: Bool
  False :: Bool
  deriving (P.Show)
```

```
ifElse :: forall a. Bool -> a -> a -> a
ifElse True a b = a
ifElse False a b = b

data Either a b where
  Left   :: forall a b. a -> Either a b
  Right  :: forall a b. b -> Either a b
  deriving (P.Show)

foldEither :: forall a b c. (a -> c) -> (b -> c) -> Either a b ->
c
foldEither fa fb (Left  val) = fa val
foldEither fa fb (Right val) = fb val

data Tuple a b where
  Tuple :: forall a b. a -> b -> Tuple a b
  deriving (P.Show)

first  :: Tuple a b -> a
first    (Tuple a b) = a

second :: Tuple a b -> b
second   (Tuple a b) = b

data Maybe a where
  Nothing :: forall a. Maybe a
  Just :: forall a. a -> Maybe a
  deriving (P.Show)

foldMaybe :: forall a b.  b -> (a -> b) -> Maybe a -> b
foldMaybe n _ Nothing  = n
foldMaybe _ f (Just x) = f x

data List a where
  Nil :: forall a. List a
  Cons :: forall a. a -> List a -> List a
  deriving (P.Show)
```

```
foldList :: (b -> a -> b) -> b -> List a -> b
foldList f z Nil = z
foldList f z (Cons x xs) = foldList f (f z x) xs

data Nat where
  Zero :: Nat
  Succ :: Nat -> Nat
  deriving (P.Show)

foldNat :: Nat -> a -> (a -> a) -> a
foldNat Zero z s = z
foldNat (Succ a) z s  = s (foldNat a z s)

plus :: Nat -> Nat -> Nat
plus Zero n = n
plus (Succ m) n = Succ (plus m n)

not :: Bool -> Bool
not False = True
not True = False

main = do
  P.print (ifElse True 1 2) --1
  P.print (ifElse False 1 2) --2
  P.print (foldNat (Succ (Succ Zero)) 0 (P.+ 1)) -- 2
```

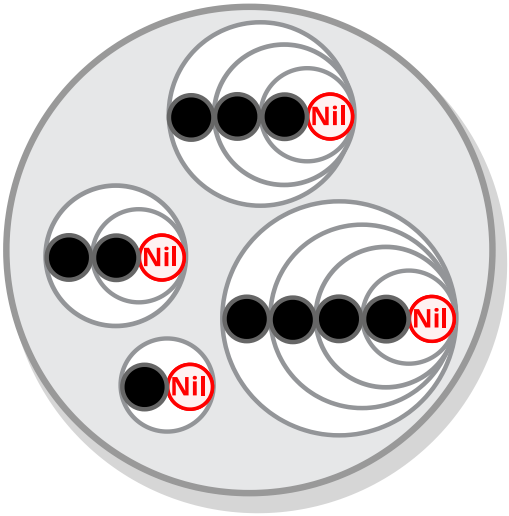
Answers

Task 1: There is a certain mapping from `List` to `Boolean` which is very intuitive. So intuitive, that some dialects of Lisp have no `Boolean` type at all and rely just on this mapping. Try to guess this mapping.

Task 2: Define this mapping (between `List` and `Bool`) in Haskell. Define it once by writing a function from scratch, and twice, with using the `foldList` function.

Task 3: I present to you the type `List Unit` where `Unit` is the singleton type, known as 1 (a type with one value). Draw the values of `List Unit` until you run out of space.

OK, here they are (we will draw just the end result, not the arrows).



Task 4: The `List Unit` type is actually isomorphic to another type that we reviewed here. Find out which.

It is isomorphic to the type of Natural numbers (`Nat`). `List` and `Nat` are both inductive types, the only difference between the two is that the inductive constructor of `List Cons` is parametrized by a type i.e. there is one constructor for each value of the type, whereas `Nat` has just one `Succ` constructor. Therefore, a list of a type that has just one value, like `Unit`, is isomorphic to `Nat`.

Task 5: Write a constructor of Tuple. Write a `fold` function for Either.

First the constructor for tuple. It is very straightforward, in order to be able to have functions that output an `a` and a `b`, you have to *input* an `a` and a `b` in the constructor.

```
data Tuple a b where
  Tuple :: forall a b. a -> b -> Tuple a b
```

The `fold` function of `Either` is also straightforward, although it may not appear so from a first glance: As the name suggest, an `Either a b` is either an

a or a b, so to convert `Either a b -> c`, you have to provide `(a -> c)` and `(b -> c)`.

```
foldEither :: forall a b c. (a -> c) -> (b -> c) -> Either a b ->
c
foldEither fa fb (Left  val) = fa val
foldEither fa fb (Right val) = fb val
```

Task 6: Besides `Tuple`, there is one very important negative type, which we will cover in this chapter (and in various other places).

It is the *function* type. We can think of functions as "objects that can be evaluated", which means that, as Tuples, they are characterized by their term elimination rule: a function `a -> b` (together with a value `a`) can be reduced to a value `b`.

Task 7: Write a Church-encoded version of the natural numbers type

Here it is

[

$$type\ \mathbb{N} = \forall b.\ b \to (b \to b) \to b \qquad zero : \mathbb{N} \qquad z\ s = z \qquad succ : \mathbb{N} \to \mathbb{N} \qquad suc$$

]

[

$$foldNat : \forall a\ b.\ b \to (b \to b) \to \mathbb{N} \to b \qquad foldNat\ z\ s\ n = n\ z\ s$$

]

Task 8: Define the elimination rule for Booleans, using natural deduction.

The elimination rule(the $ifElse$ function) expressed as a typing rule is this:

[\frac{\Gamma \vdash b : \mathrm{Bool} \qquad \Gamma \vdash x : a \qquad \Gamma \vdash y : a}{\Gamma \vdash \mathrm{ifElse}\ b\ x\ y : a}]

Given a boolean b, and two values of some type a, we can produce a value of type a.

Functors

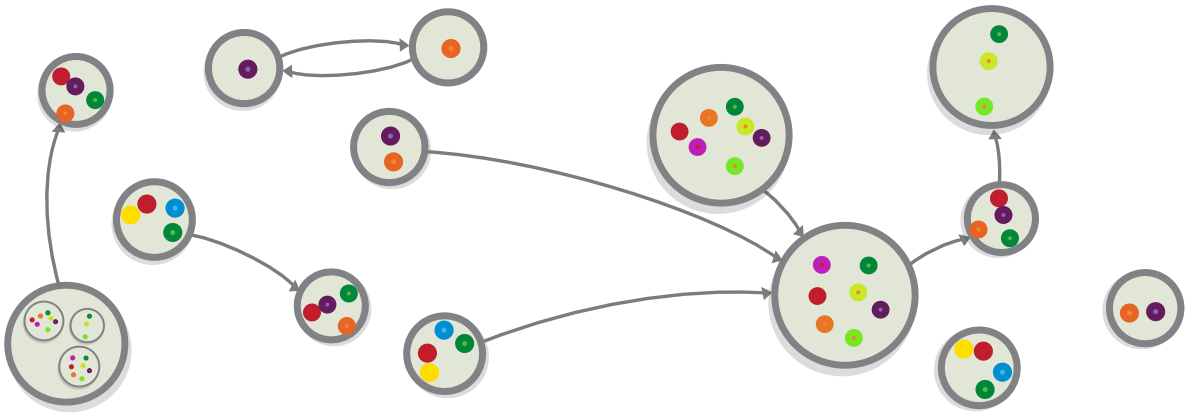
From this chapter on, we will change the tactic a bit (as I am sure you are tired of jumping through different subjects) and we will dive at full throttle into the world of categories, using the structures that we saw so far as context. This will allow us to generalize some of the concepts that we examined in these structures and thus make them (the concepts) valid for all categories.

Categories we saw so far

So far, we saw many different categories and category types. Let's review them once more:

The category of sets

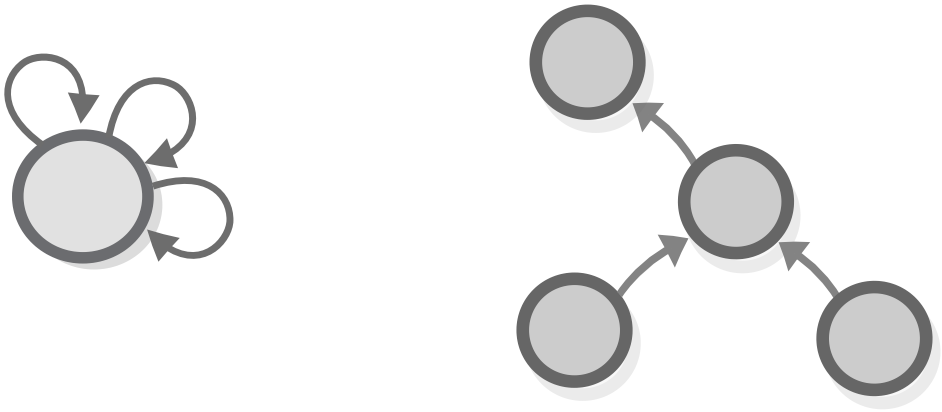
We began by reviewing the mother of all categories — *the category of sets*.



We also saw that it contains within itself many other categories, such as the category of types (actually different categories of types, but let's think about the one we learned about (System F) for now).

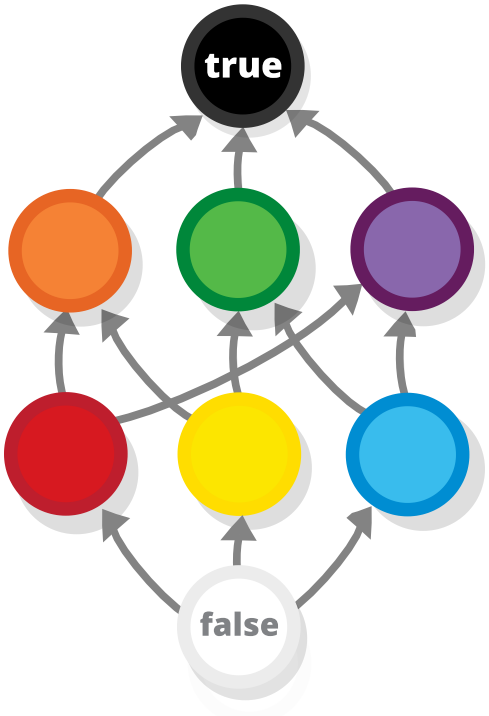
Special types of categories

We also learned about other algebraic objects that turned out to be just *special types of categories*, like categories that have just one *object* (monoids, groups) and categories that have only one *morphism* between any two objects (preorders, partial orders).



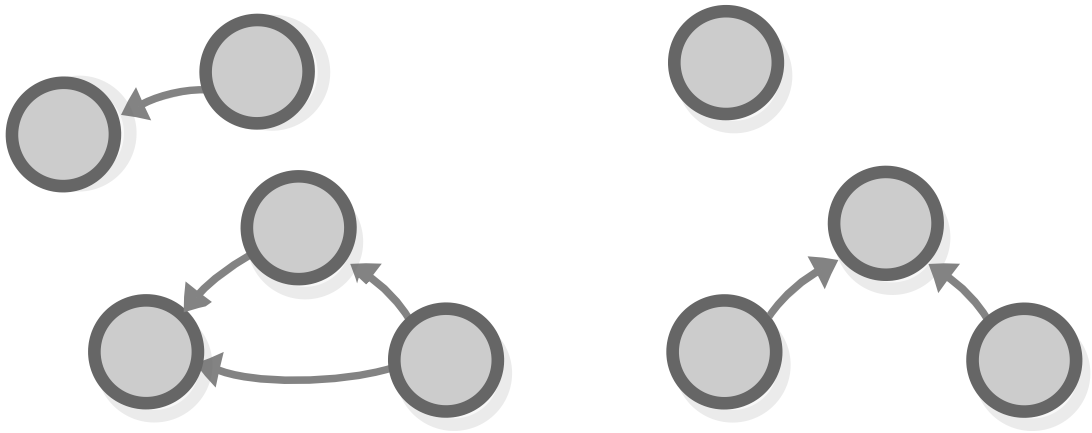
Other categories

We also defined a lot of *categories based on different concepts*, like the ones based on logics/types, but also some "less-serious ones", as for example the color-mixing partial order/category.



Finite categories

And most importantly, we saw some categories that are *completely made up*, such as my soccer player hierarchy. Those are formally called *finite categories*.



Although they are not useful by themselves, the idea behind them is important — we can draw any combination of points and arrows and call it a category, in the same way that we can construct a set out of every combination of objects.

Examining some finite categories

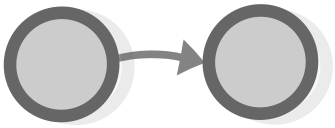
For future reference, let's see some important finite categories.

The simplest category is 0 (enjoy the minimalism of this diagram).

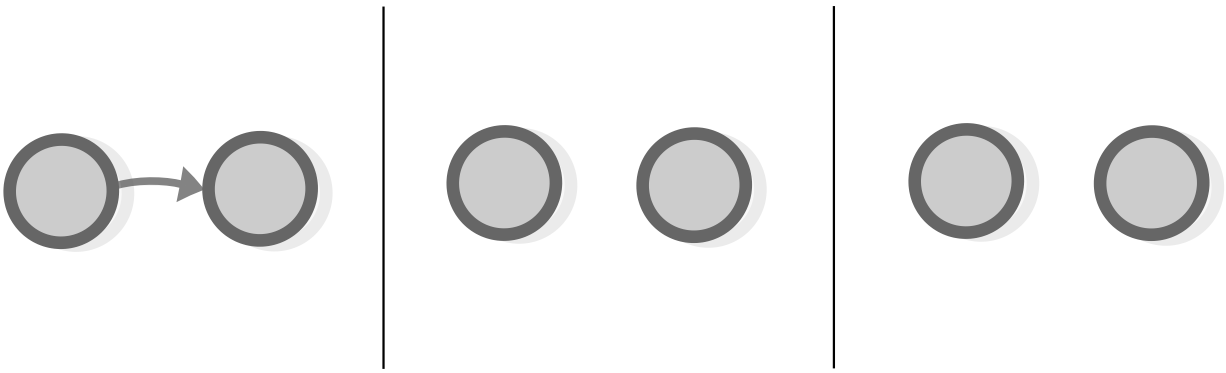
The next simplest category is 1 — it is comprised of one object and no morphisms besides its identity morphism (which we don't draw, as usual)



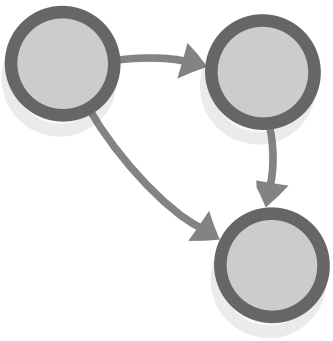
If we increment the number of objects to two, we see a couple of more interesting categories, like for example the category 2 containing two objects and one morphism.



Task 1: There are just two more categories that have 2 objects and at most one morphism between two objects, draw them.



And finally the category $\mathbf{3}$ has 3 objects and also 3 morphisms (one of which is the composition of the other two).

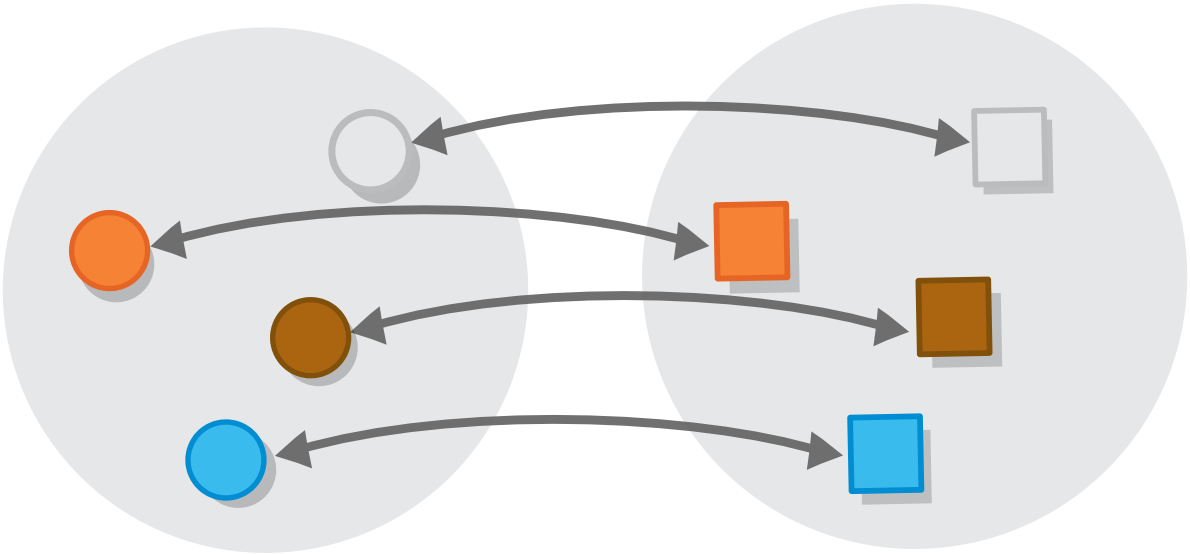


Categorical isomorphisms

Many of the categories that we saw are similar to one another, as for example, both the color-mixing order and categories that represent logic have a *greatest* and a *least* object. To pinpoint such similarities, and understand what they mean, it is useful to have formal ways to connect categories with one another. The simplest type of such connection is the good old isomorphism.

Set isomorphisms

In chapter 1 we talked about *set isomorphisms*, which establish an equivalence between two sets. In case you have forgotten, a set isomorphism is a *two-way function* between two sets.

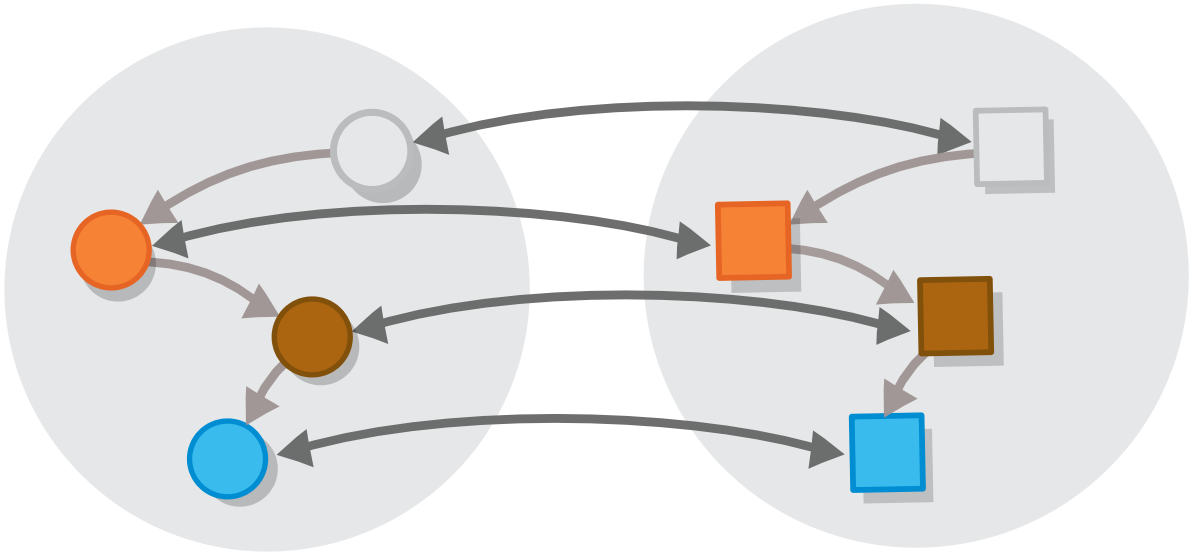


It can alternatively be viewed as two "twin" functions such that each of which equals identity, when composed with the other one. Formally:

> **DEFINITION:**
>
> Two sets A and B are isomorphic (or $A \cong B$) if there exist functions $f : A \to B$ and its reverse $g : B \to A$, such that $f \circ g = ID_A$ and $g \circ f = ID_B$.

Order isomorphisms

Then, in chapter 4, we encountered *order isomorphisms* and we saw that they are like set isomorphisms, but with one extra condition — aside from just being there, the functions that define the isomorphism have to preserve the order of the objects



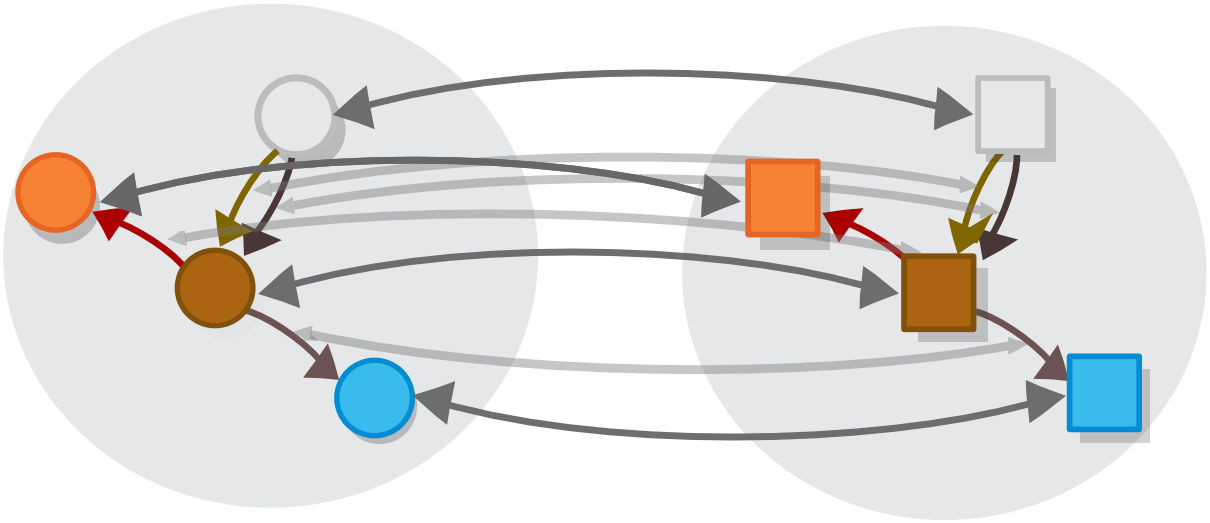
e.g. the greatest object of one order should be connected to the greatest object of the other one, the least object of one order should be connected to the least object of the other one, and same for all objects that are in between. Formally.

> **DEFINITION:**
>
> An isomorphism between two orders is an invertible function between their underlying sets, such that applying this function (let's call it F) to any two elements that have a certain order in one set (let's call them a and b) should result in two elements that have a corresponding order in the other set (i.e. $a \leq b$ if and only if $F(a) \leq F(b)$).

Categorical isomorphisms

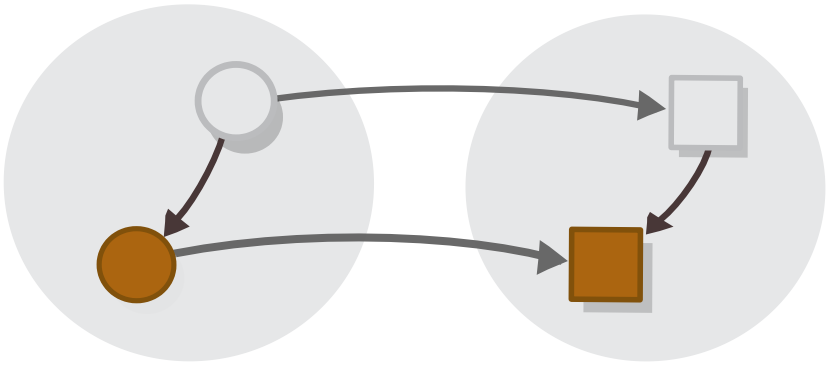
Now, we will generalize the definition of an order isomorphism, so it also applies to all other categories (i.e. to categories that may have more than one morphism between two objects):

> **DEFINITION:**
>
> Given two categories, an isomorphism between them is an invertible mapping between the underlying sets of objects, *and* an invertible mapping between the morphisms that connect them, which maps each morphism from one category to a morphism *with the same signature*.



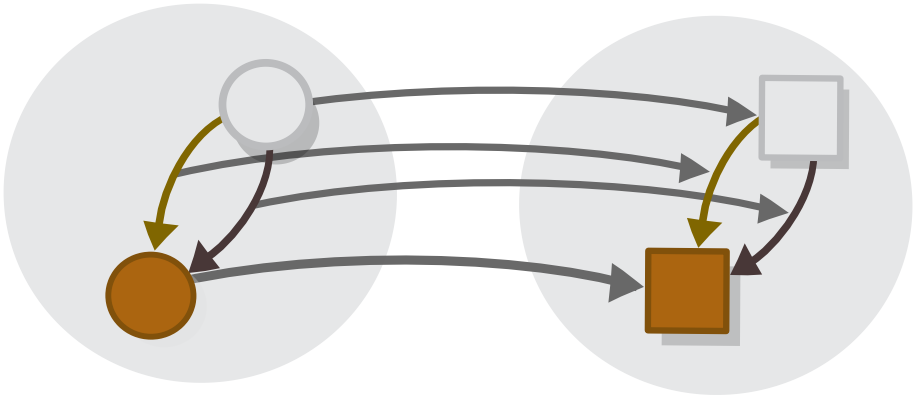
After examining this definition closely, we realize that, although it *sounds* a bit more complex (and *looks* a bit messier) than the one we have for orders *it is actually the same thing*.

It is just that the so-called "morphism mapping" between categories that have just one morphism for any two objects are trivial, and so we can omit them.



Task 2: What is the morphism mapping for orders?

However, when we can have more than one morphism between two given objects, we need to make sure that each morphism in the source category has a corresponding morphism in the target one, and for this reason we need not only a mapping between the categories' objects, but one between their morphisms.



By the way, what we just did (taking a concept that is defined for a more narrow structure (orders) and redefining it for a more broad one (categories)) is called *generalizing* of the concept.

The problem with categorical isomorphisms

By examining them more closely, we realize that categorical isomorphisms are not so hard to define. However there is another issue with them, namely that they *don't capture the essence of what categorical equality should be*. I have devised a very good and intuitive explanation why is it the case, that this ~~margin~~ section is too narrow to contain. So we will leave it for the next

chapter, where we will also devise a more apt way to define a *two-way connection* between categories.

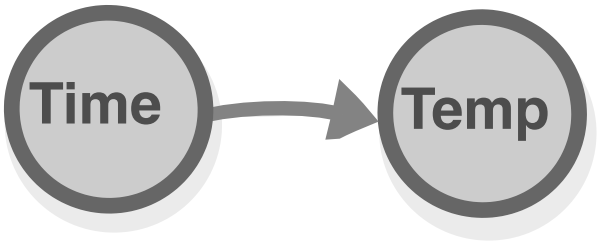
But first, we need to examine *one-way connections* between them, i.e. *functors*.

PS: Categorical isomorphisms are also *very rare in practice* — the only one that comes to mind is the Curry-Howard-Lambek isomorphism from the previous chapter. That's because if two categories are isomorphic then there is no reason at all to treat them as different categories — they are one and the same.

What are functors

The logician Rudolf Carnap coined the term "functor" as part of his project to formalize the syntax for the natural languages such as English in order to create a precise way for us to talk about science. Originally, a functor meant a word or phrase whose meaning can be customized by combining it with a numerical value, such as the phrase "the temperature at x o'clock", which has a different meaning depending on the value of x.

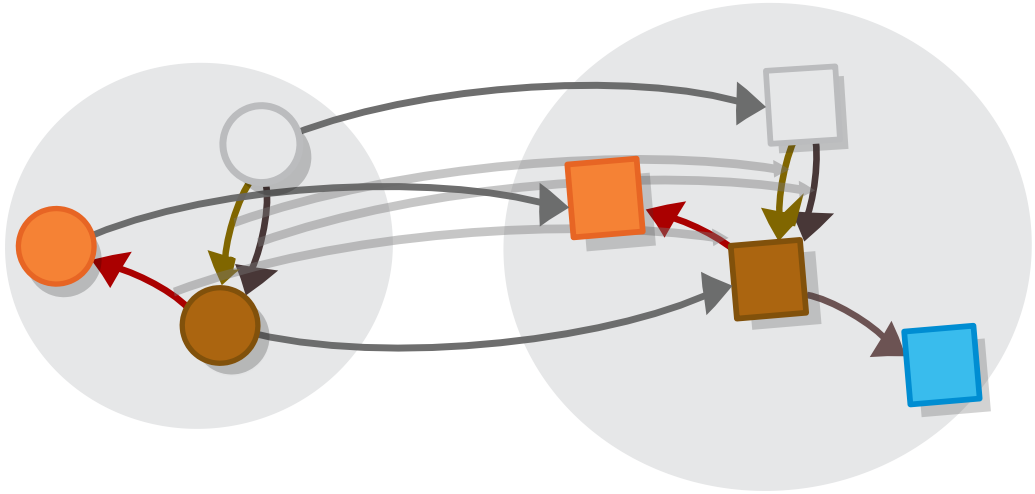
In other words, a functor is a phrase that *acts as a function*, only not a function between sets, but one between *linguistic concepts* (such as times and temperature).



Later, one of the inventors of category theory Sanders Mac Lane borrowed the word, to describe a something that *acts as function between categories*, which he defined in the following way:

> **DEFINITION:**

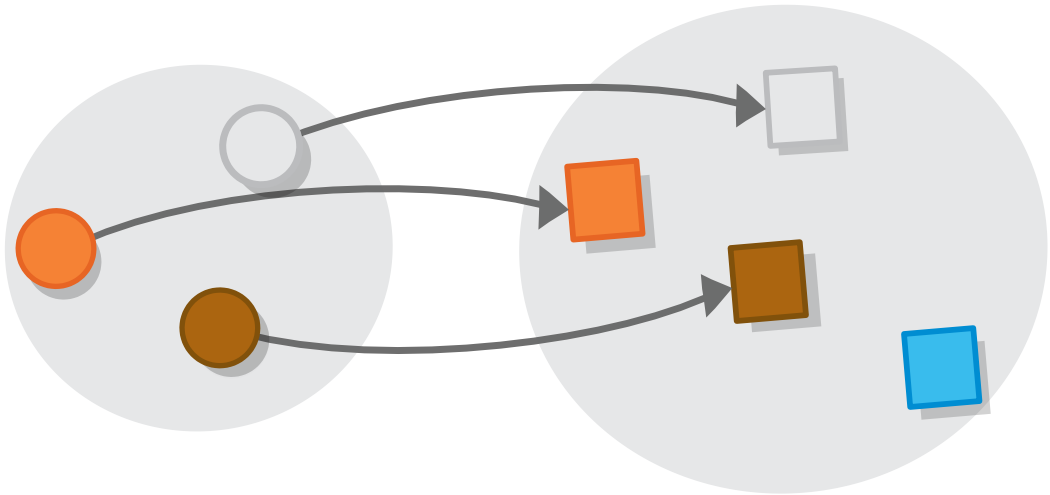
> A functor between two categories (let's call them A and B) consists of two mappings — a mapping that maps each *object* in A to an object in B and a mapping that maps each *morphism* between any objects in A to a morphism between objects in B, in a way that *preserves the structure* of the category.



Now let's unpack this definition by going through each of its components.

Object mapping

In the definition above, we use the word "mapping" to avoid misusing the word "function" for something that isn't exactly a function. But in this particular case, calling the mapping a function would barely be a misuse — if we forget about morphisms and treat the source and target categories as sets, the object mapping is nothing but a regular old function.

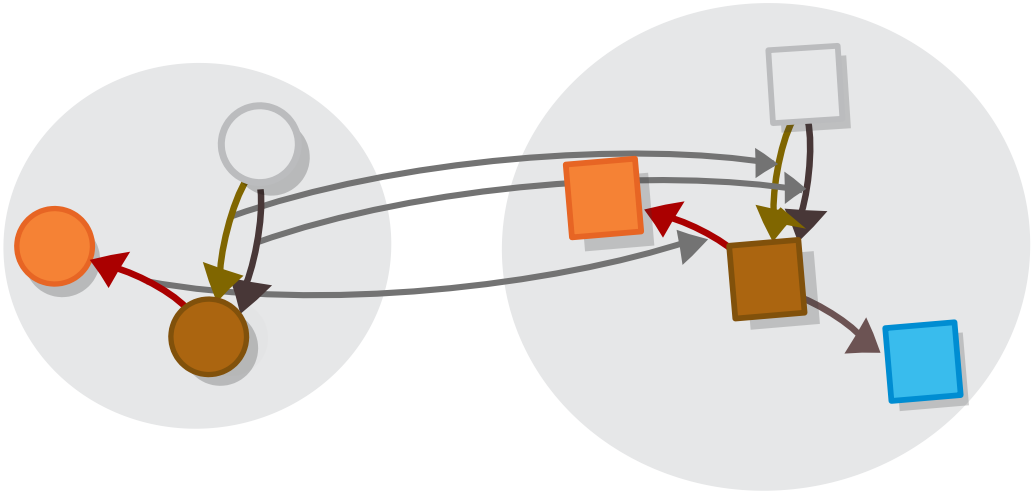


A more formal definition of object mapping involves the concept of an *underlying set* of a category: Given a category A, the underlying set of A is a set that has the objects of A as elements. Utilizing this concept, we say that the object mapping of a functor between two categories is *a function between their underlying sets*. The definition of a function is still the same:

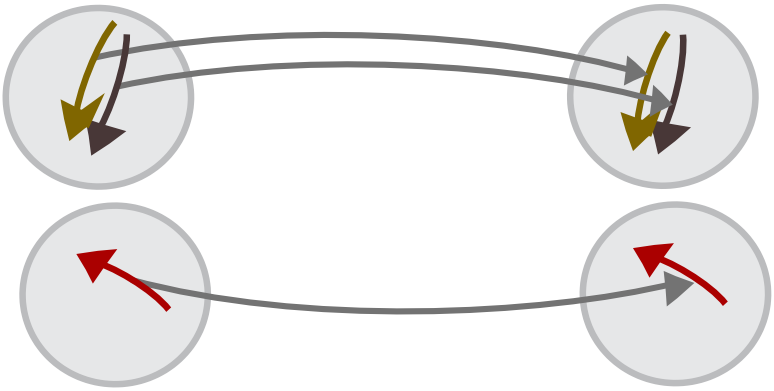
> **DEFINITION:**
>
> A function is a relationship between two sets that matches each element of one set, called the *source set* of the function, with exactly one element from another set, called the *target set* of the function.

Morphism mapping

The second mapping that forms the functor is a mapping between the categories' morphisms. This mapping resembles a function as well, but with the added requirement that each morphism in A a given source and target must be mapped to a morphism with the corresponding source and target in B, as per the object mapping.



A more formal definition of a morphism mapping involves the concept of the *homomorphism set*: this is a set that contains all morphisms that go between two given objects in a given category. When utilizing this concept, we say that a mapping between the morphisms of two categories consists of a *set of functions between their respective homomorphism sets*.



Notice how the concepts of *homomorphism set* and of *underlying set* allowed us to "escape" to set theory when defining categorical concepts and define everything using functions?

Functor laws

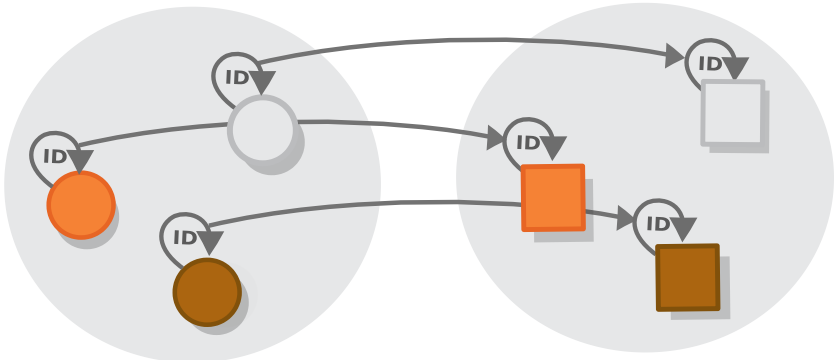
So these are the two mappings (one between objects and one between morphisms) that constitute a functor. But not every pair of such two mappings is a functor. As we said, in addition to existing, the mappings should *preserve the structure* of the source category into the target category. To see what that means, we revisit the definition of a category from chapter 2:

> **DEFINITION:**
>
> A category is a collection of *objects* (we can think of them as points) and *morphisms* (arrows) that go from one object to another, where:
>
> 1. Each object has to have the identity morphism.
> 2. There should be a way to compose two morphisms with an appropriate type signature into a third one, in a way that is associative.

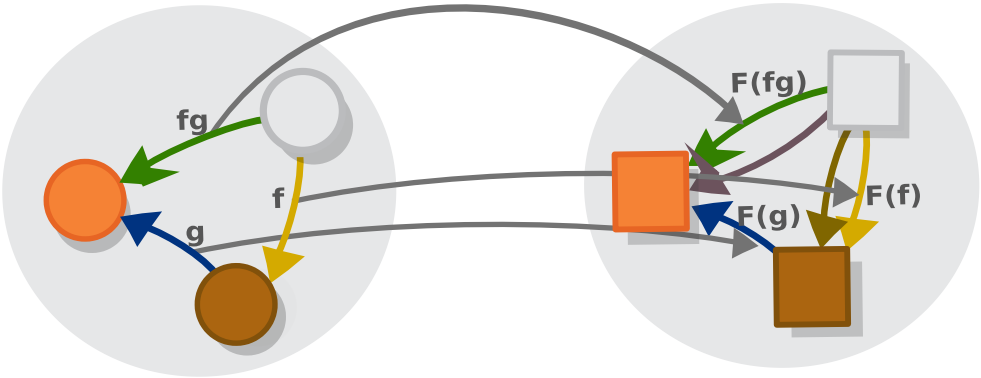
So this definition translates to the following two *functor laws*

> **DEFINITION:**
>
> A functor between two categories (let's call them A and B) consists of two mappings — a mapping that maps each *object* in A to an object in B and a mapping that maps each *morphism* between any objects in A to a morphism between objects in B, in a way that *preserves the structure* of the category:

1. Functions between morphisms should *preserve identities* i.e. all identity morphisms should be mapped to other identity morphisms.



2. Functors should also *preserve composition* i.e. for any two morphisms f and g, the morphism that corresponds to their composition $F(g \bullet f)$ in the source category should be mapped to the morphism that corresponds to the composition of their counterparts in the target category, so $F(g \bullet f) = F(g) \bullet F(f)$.



And these laws conclude the definition of functors — a simple but, as we will see shortly, very powerful concept.

To see *why* is it so powerful, let's check some examples.

Functors in everyday language

There is a common figure of speech (which is used all the time in this book) which goes like this:

> If a is like Fa, then b is like Fb.

Or "a is related to Fa, in the same way as b is related to Fb," e.g. "If schools are like corporations, then teachers are like bosses".

This figure of speech is nothing but a way to describe a functor in our day-to-day language: what we mean by it is that there is a certain set of connections (or category-theory terms a "morphisms") between schools and teachers, that is similar to the connections between corporations and bosses i.e. that there is some kind of structure-preserving map that connects the category of school-related things, to the category of work-related things which maps schools (a) to corporations (Fa) and teacher (b) to bosses (Fb), and which is such that the connections between schools and teachers ($a \to b$) are mapped to the connections between corporations and bosses ($Fa \to Fb$).

Diagrams are functors

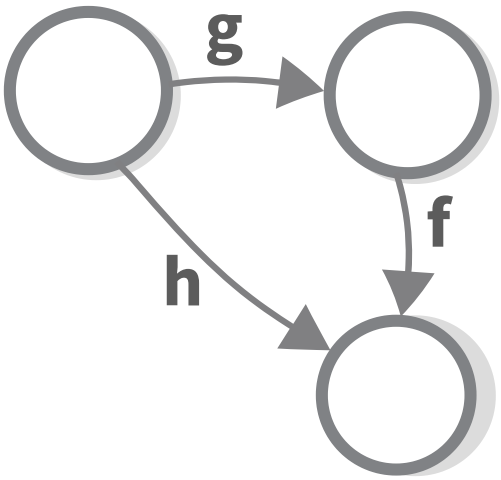
> "A sign is something by knowing which we know something more." — Charles Sanders Peirce

We will start with an example of a functor that is very *meta* — the diagrams/illustrations in this book.

You might have noticed that diagrams play a special role in category theory — while in other disciplines their function is merely complementary i.e. they only show what is already defined in another way, here *the diagrams themselves serve as definitions*.

For example, in chapter 1 we presented the following definition of functional composition.

> **DEFINITION:**
>
> The composition of two functions f and g is a third function h defined in such a way that this diagram commutes.

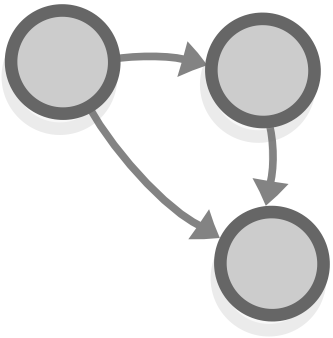


We all see the benefit of defining stuff by means of diagrams as opposed to writing lengthy definitions like

> "Suppose you have three objects a, b and c and two morphisms $f : b \to c$ and $g : a \to b$..."

However, it (defining stuff by means of diagrams) presents a problem — definitions in mathematics are supposed to be formal, so if we want to use diagrams as definitions we must first *formalize the definition of a diagram itself*.

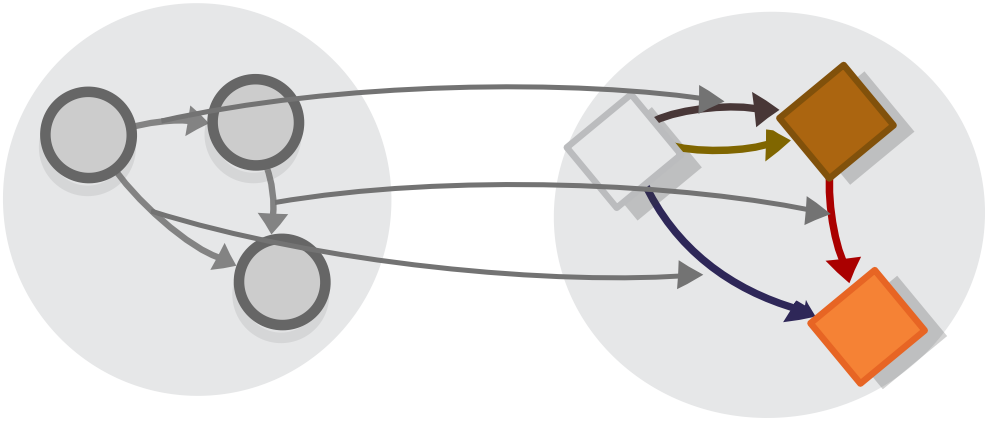
So how can we do that? One key observation is that diagrams look as finite categories, as, for example, the above definition looks in the same way as the category $\mathbf{3}$.



However, this is only part of the story as finite categories are just structures, whereas diagrams are *signs*. They are "something by knowing which we know

something more.", as Peirce famously put it (or "...which can be used in order to tell a lie", in the words of Umberto Eco).

For this reason, aside from a finite category that encodes the diagram's structure, the definition of a diagram must also include a way for "interpreting" this category in some other context i.e. they must include *functors*.



This is how the concept of functors allows us to formalize the notion of diagrams:

> **DEFINITION:**
>
> A *diagram* is comprised of one finite category (called an *index category*) and a functor from it to some other category.

If you know about semiotics, you may view the source and target categories of the functor as *signifier* and *signified*.

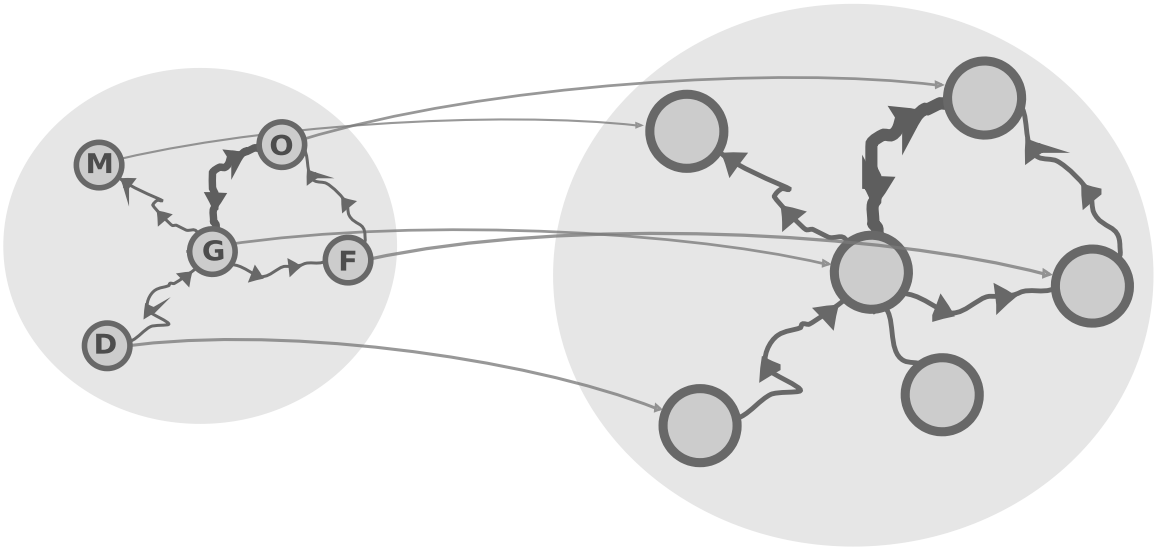
And so, you can already see that the concept of a functor plays a very important role in category theory. Because of it, diagrams in category theory can be *specified formally* i.e. they are categorical objects *per se*.

You might even say that they are categorical objects *par excellence* (TODO: remove that last joke).

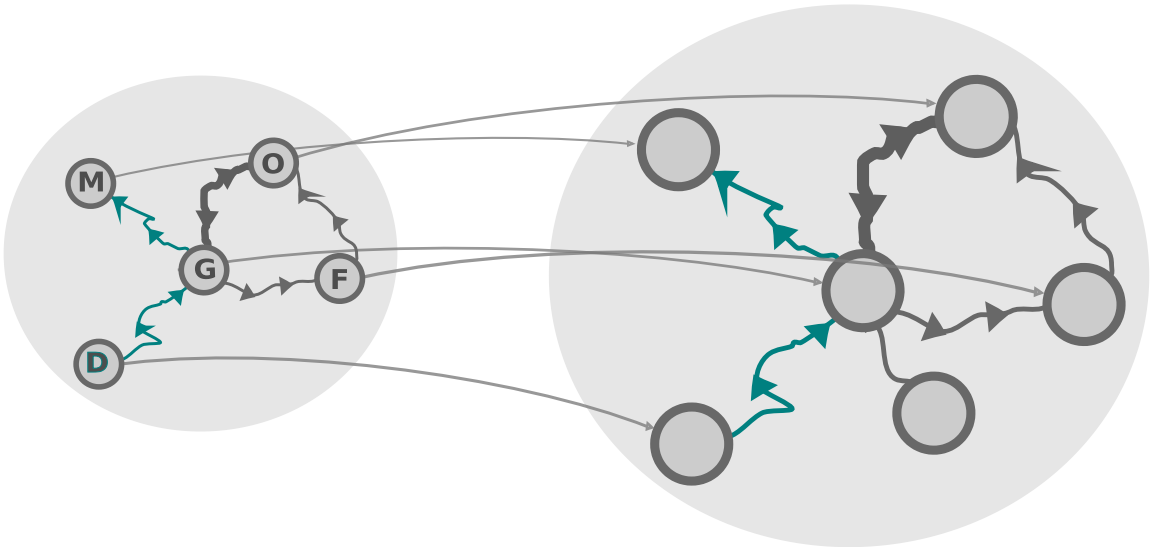
Maps are functors

> A map is not the territory it represents, but, if correct, it has a similar structure to the territory, which accounts for its usefulness. — Alfred Korzybski

Functors are sometimes called “maps” for a good reason — maps, like all other diagrams, are functors. If we consider some space, containing cities and roads that we travel by, as a category, in which the cities are objects and roads between them are morphisms, then a road map can be viewed as category that represent some region of that space, together with a functor that maps the objects in the map to real-world objects.



In maps, morphisms that are a result of composition are often not displayed, but we use them all the time — they are called *routes*. And the law of preserving composition tells us that every route that we create on a map corresponds to a real-world route.



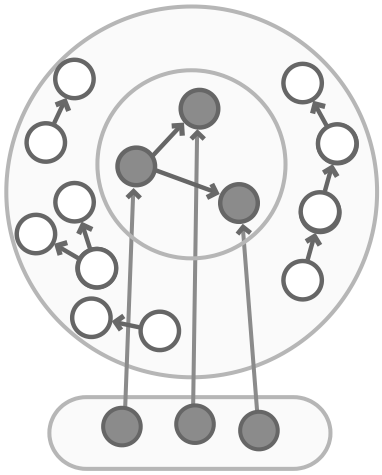
Notice that in order to be a functor, a map does not have to list *all* roads that exist in real life, and *all* traveling options (“the map is not the territory”), the

only requirement is that *the roads that it lists should be actual* — this is a characteristic shared by all many-to-one relationships (i.e. functions).

Human perception might be functorial

As we saw, in addition to category theory, functors appear in many disciplines that study the human mind, such as logic, linguistics, semiotics and the like. I thought about why is it so in a blog post that I wrote. My response to that question is that human perception, human thinking, is itself functorial: to perceive the world around us, we are going through a bunch of functors that go from more raw "low-level" mental models to more abstract "high-level" ones.

We may say that perception starts with raw sensory data. From it, we go, (using a functor) to a category containing some basic model of how the world works, mapping the objects we are seeing to some concepts that we formed in our mind.



Then we are connecting this model to another, even more abstract model (or models), which provide us with a higher-level overview of the situation that we are in, again using functorial connection (technically these are functors from subcategories).



You can view this as a progression of connections that go from simpler to more abstract (i.e. from categories with less morphisms to categories with more morphisms).

These connections are functorial in nature, because they work purely in terms of *structure*: the idea of a given object has nothing in common with the object itself (e.g. the idea of a biscuit isn't round, sweet etc). What ideas have in common with the objects of representation is that the connections they have between one another mimic some connections a real-life biscuit has with other objects.

All this is, of course, just a speculation, but how else would we be capable of forming thoughts, e.g. to imagine a person riding a bike and bumping in a tree, when no part of our brains (nor the impulses they create) resembles in any way people/bikes/trees?

The answer is structure — thoughts have the structure of the situation, that's why they refer to it (although they cannot refer to any of the elements by itself).

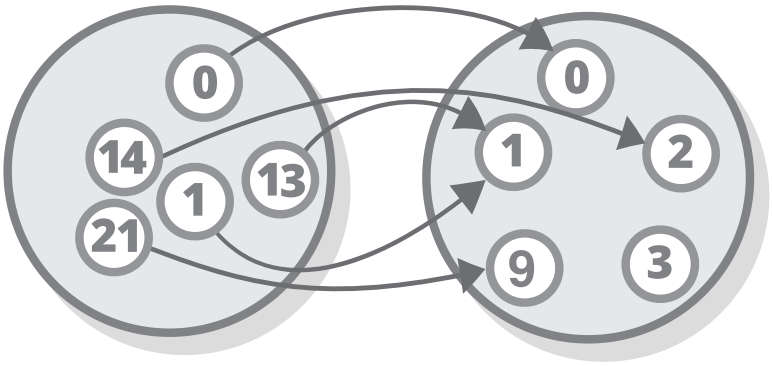
Functors in monoids

So, after this slight detour, we will return to our usual modus operandi.

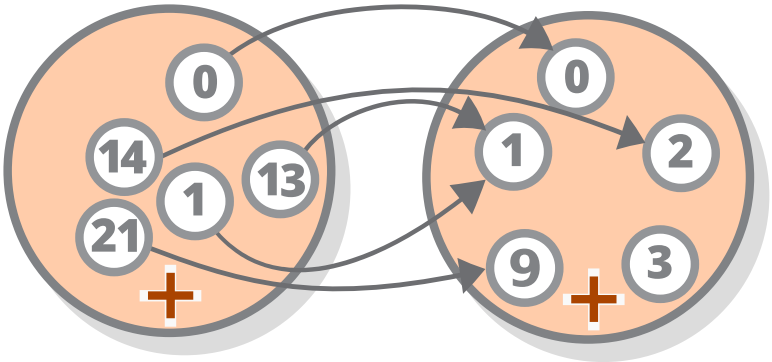
Hey, do you know that in group theory, there is this cool thing called *group homomorphism*.

> **DEFINITION:**
>
> *Group homomorphism* (or *monoid homomorphism* when we are talking about monoids) is a function F between the groups'/monoids' underlying sets that preserves the group operation i.e. $F(a * b) = F(a) + F(b)$ (where $+$ and $*$ are the group's operators).

So, for example, If the time of the day right now is 00:00 o'clock (or 12 PM) then what would the time be after n hours? The answer to this question can be expressed as a function with the set of integers as source and target.



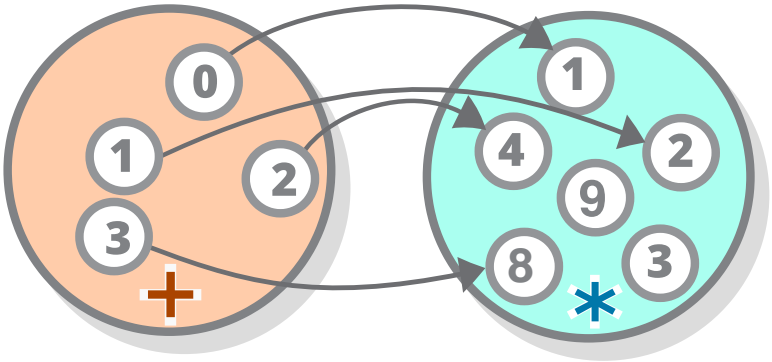
This function is interesting — it preserves the operation of (modular) addition: if, 13 hours from now the time will be 1 o'clock and if 14 hours from now it will be 2 o'clock, then the time after (13 + 14) hours will be (1 + 2) o'clock.



Or to put it formally, if we call it (the function) F, then we have the following equation: $F(a + b) = F(a) + F(b)$ (where $+$ in the right-hand side of the

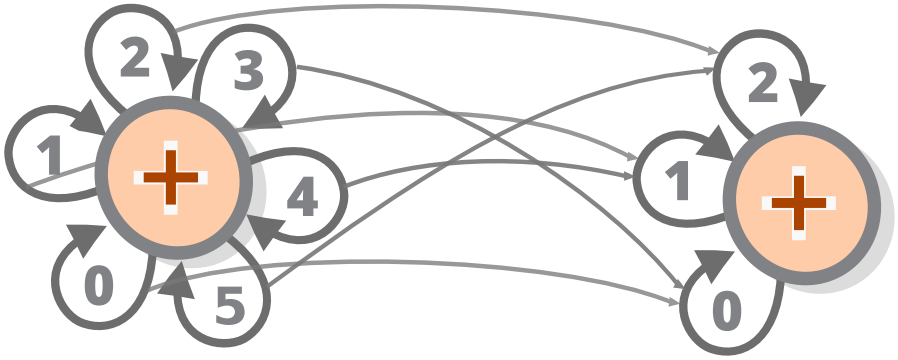
equation means modular addition). Because this equation holds, the F function is a *group homomorphism* between the group of integers under addition and the group of modulo arithmetic with base 11 under modular addition (where you can replace 11 with any other number).

The groups don't have to be so similar for there to be a homomorphism between them. Take, for example, the function that maps any number n to 2 to the *power of* n, so $n \to 2^n$ (here, again, you can replace 2 with any other number). This function gives a rise to a group homomorphism between the group of integers under addition and the integers under multiplication, or $F(a+b) = F(a) \times F(b)$.



Wait, what were we talking about, again? Oh yeah:

Group homomorphisms are functors. To see why, we switch to the category-theoretic representation of groups and revisit our first example and (to make the diagram simpler, we use $mod2$ instead of $mod11$).



THEOREM:

> When we view groups/monoid as one-object categories, a group/monoid homomorphism is just a functor between these categories.

Let's see if that is the case.

Object mapping

Groups/monoids have just one object when viewed as categories, so there is also only one possible object mapping between any couple of groups/monoids — one that maps the (one and only) object of the source group to the object of the target group (not depicted in the diagram).

Morphism mapping

Because of the above, the morphism mapping is the only relevant component of the group homomorphism. In the category-theoretic perspective, group objects (like 1 and 2 3 etc.) correspond to morphisms (like $+1$, $+2$ $+3$ etc.) and so the morphism mapping is just mapping between the group's objects, as we can see in the diagram.

Functor laws

The first functor law trivial, it just says that the one and only identity object of the source group (which corresponds to the identity morphism of its one and only object) should be mapped to the one and only identity object of the target group For groups, this even follows immediately from the second law and for monoids, it has to be added as an extra condition.

And if we remember that the group operation of combining two objects corresponds to *functional composition* if we view groups as categories, we realize that the group homomorphism equation $F(a + b) = F(a) \times F(b)$ is just a formulation of the second functor law: $F(g \bullet f) = F(g) \bullet F(f)$.

And many algebraic operations satisfy this equation, for example the functor law for the group homomorphism between $n \to 2^n$ is just the famous algebraic rule, stating that $g^a g^b = g^{a+b}$.

Task 3: Prove that the first functor law (preservation of identities) of groups can be proven from the second law. Note that this is valid for groups, but not

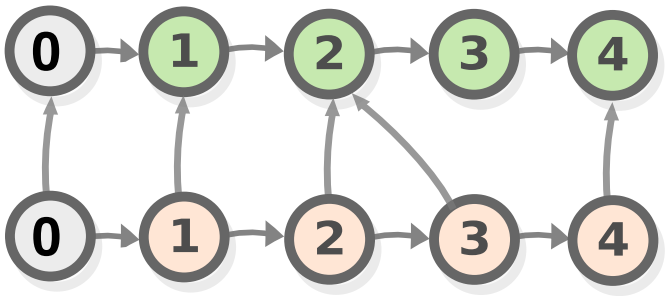
monoids.

Functors in orders

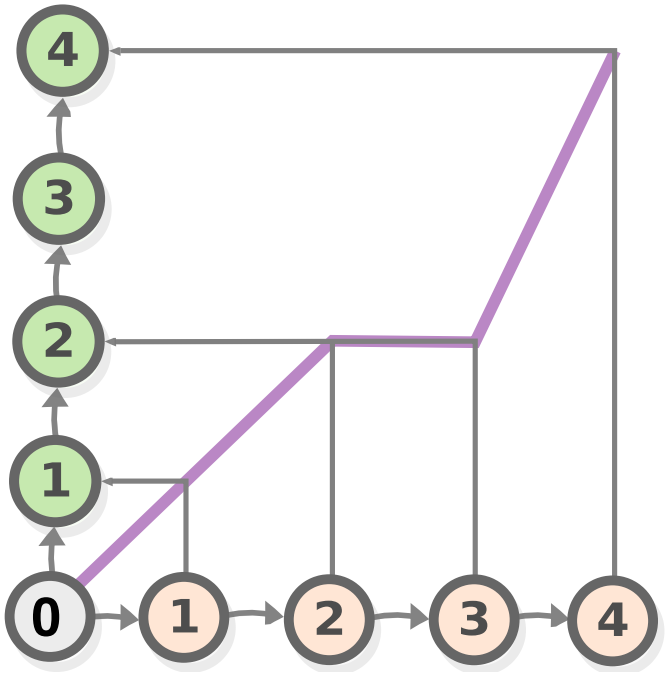
And now let's talk about a concept that is completely unrelated to functors, nudge-nudge (hey, bad jokes are better than no jokes at all, right?) In the theory of orders, we have the concept of functions between orders (which is unsurprising, given that orders, like monoids/groups, are based on sets) and one very interesting type of such function, which has applications in calculus and analysis, is a *monotonic function* (also called *monotone map*).

> **DEFINITION:**
>
> A monotonic function is a function between two orders that *preserves the order of the objects in the source order, in the target order. So, a function F is monotonic when for every a and b in the source order, if $a \leq b$ then $F(a) \leq F(b)$.

For example, the function that maps the current time to the distance traveled by some object is monotonic because the distance traveled increases (or stays the same) as time increases.



If we plot this or any other monotonic function on a line graph, we see that it goes just one direction (i.e. just up or just down).



Now we are about to prove that monotonic functions are functors too, ready?

> **THEOREM:** When we view order as thin categories, monotone maps are functors.

Object mapping

Like with categories, the object mapping of an order is represented by a function between the orders' underlying sets.

Morphism mapping

With monoids, the object mapping component of functors was trivial. Here is the reverse: the morphism mapping is trivial - given a morphism between two objects from the source order, we map that morphism to the morphism between their corresponding objects in the target order. The fact that the monotonic function respects the order of the elements, ensures that the latter morphism exists.

Functor laws

It is not hard to see that monotone maps obey the first functor law as identities are the only morphisms that go between a given object and itself.

And the second law ($F(g \circ f) = F(g) \circ F(f)$) also follows trivially — we just have to prove that if function F preserves order, i.e. that if

[a ≤ b \to F(a) ≤ F(b)]

and

[b ≤ c \to F(b) ≤ F(c)]

then obviously

[F(a) ≤ F(c)]

Task 4: Show why the law holds.

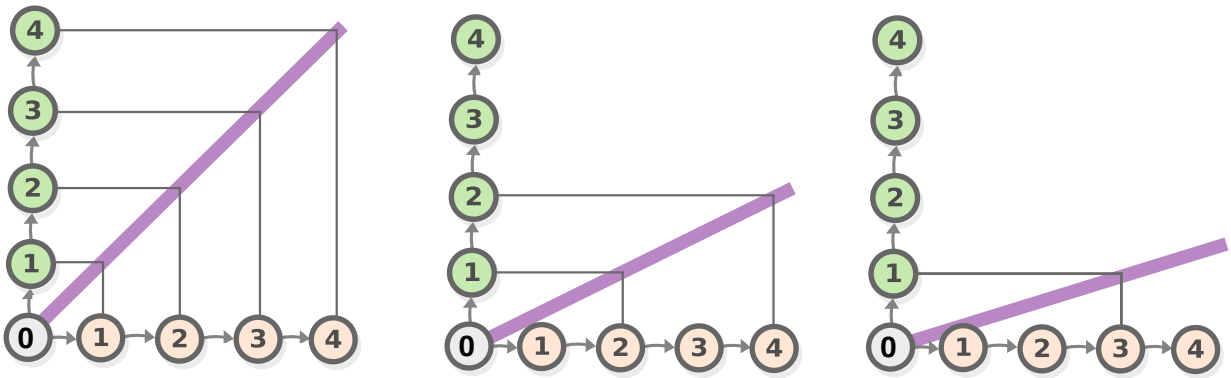
Linear functions

OK, enough with this abstract nonsense, let's talk about "normal" functions — ones between numbers.

In calculus, there is this concept of *linear functions* (also called "degree one polynomials"):

> **DEFINITION:**
>
> Linear functions are functions of the form $f(x) = xa$ that contain no operations other than multiplying the argument by some constant (designated as a in the example).

But if we start plotting some such functions we will realize that there is another way to describe them — their graphs are always comprised of straight lines.

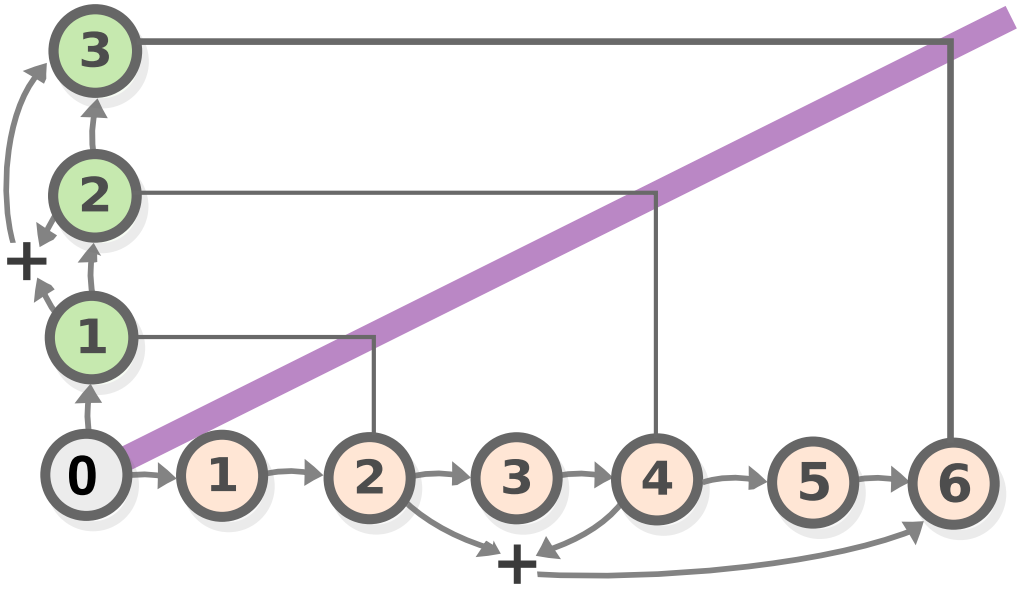


Task 5: Why are the graphs of linear functions comprised of straight lines?

Another interesting property of these functions is that most of them *preserve* addition, that is for any x and y, you have $f(x) + f(y) = f(x + y)$. We already know that this equation is equivalent to the second functor law. So:

> **THEOREM:**
>
> Linear functions are just functors between the monoid of natural numbers under addition and itself.

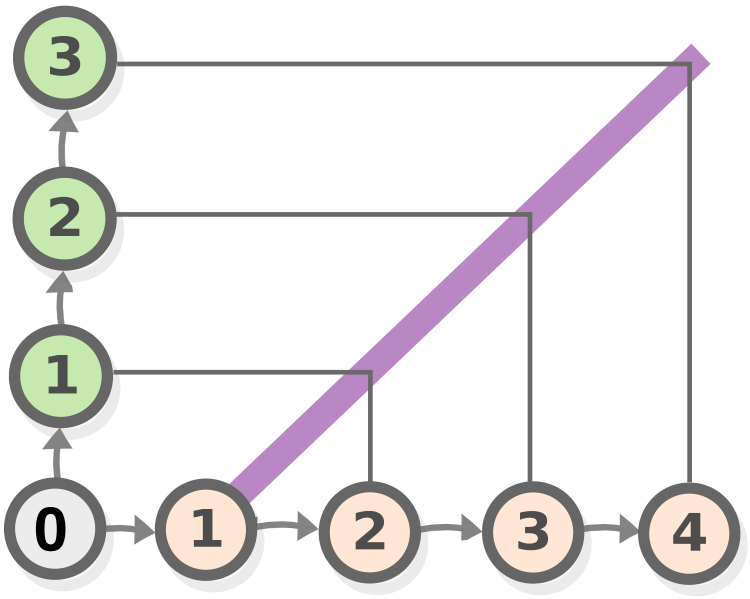
As we will see later, they are example of functors in the *category of vector spaces.



Task 6: Show that linear functions preserve addition.

And if we view that natural numbers as an order, linear functions are also functors as well, as all functions that are plotted with straight lines are obviously monotonic.

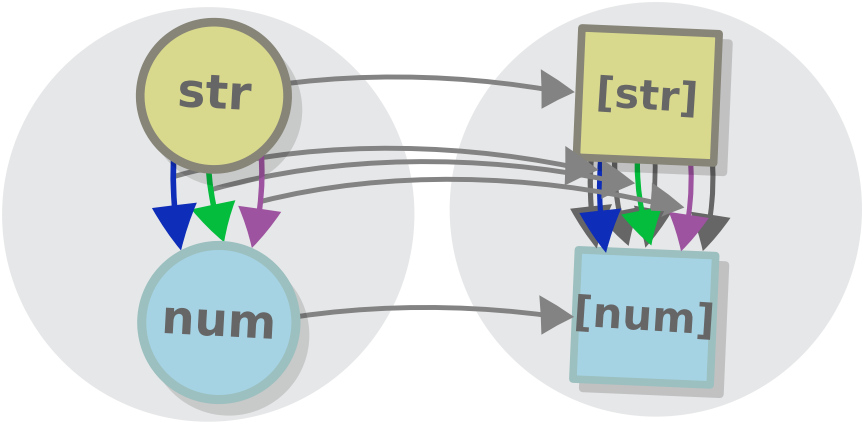
Note, however, that not all functions that are plotted straight lines preserve addition — functions of the form $f(x) = x * a + b$ in which b is non-zero, are also straight lines (and are also called linear) but they don't preserve addition.



For those, the above formula looks like this:
$f(x) + b + f(y) + b = f(x + y) + b$.

Functors in types. The list functor

A type theory/type system forms a category, and in that category there are some functors that programmers use every day, such as the list functor, that we will use as an example. The list functor is an example of a functor that maps the realm of simple (primitive) types and functions to the realm of more complex (generic) types and functions.



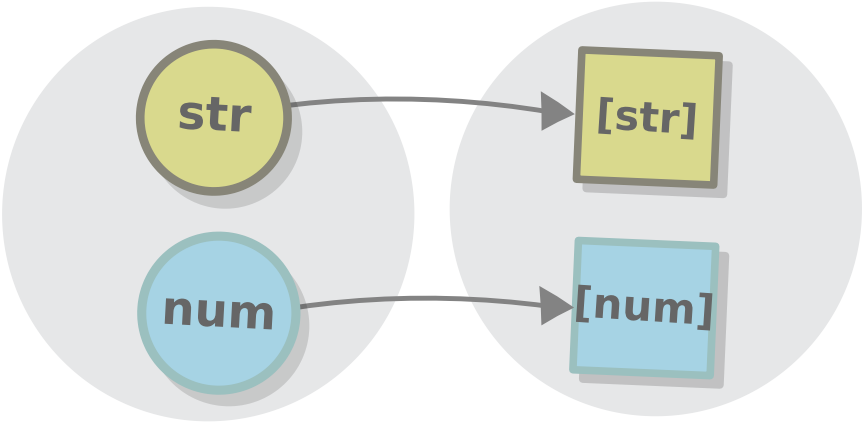
But let's start with the basics: defining the concept of a functor in type-theoretic context is as simple as changing some of the terms that are used, and optionally, changing the font we use in our formulas from "modern" to "monospaced".

> **DEFINITION:**
>
> A functor between two ~~categories~~ of types (let's call them A and B) consists of a mapping that maps each ~~object~~ *type* in A to a type in B and a mapping that maps each ~~morphism~~ *function* between types in A to a function between types in B, in a way that preserves the structure of the ~~category~~ type system.

Comparing these definitions makes us realize that mathematicians and programmers are two very different communities, that are united by the fact that they both use functors (and by their appreciation of peculiar typefaces).

Type mapping

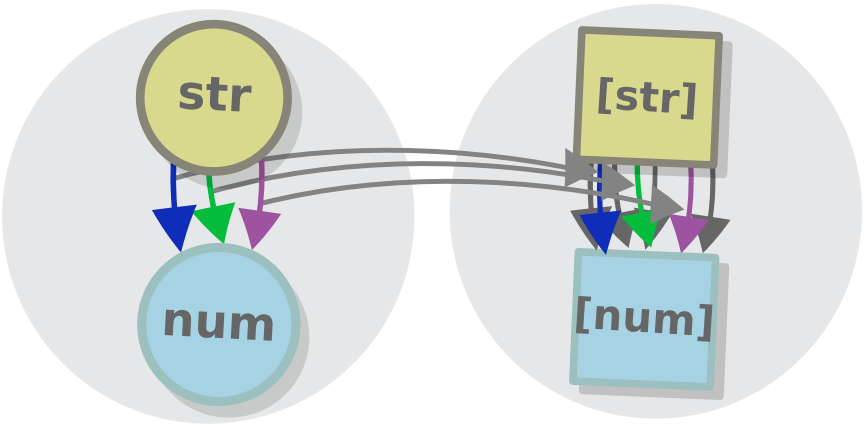
The first component of a functor is a mapping that converts each type to another type. so, it is *a type-level arrow*. But we already know about those: in type-theoretic terms, they are known as *polymorphic type* (as a polymorphic type is nothing but an arrow that maps all types to other types).



Note that although the diagrams they look similar, a polymorphic *type-level* arrow is completely different from a polymorphic *value-level* arrow i.e. type-level* arrow `List<A>` as that one converts each *type* a to a *type* $List\ a$ (e.g. the type `string` to the type $List\ string$, $number$ to $List\ number$ etc.) is not the same thing as a polymorphic value-level arrow from `a`, to `List<a>`, or in our mathy Haskell-inspired notation $\forall\ a.\ a \to List\ a$, which converts a value of type a to a value of type $Lista$ (we will learn about value-level polymorphic functions later in this chapter).

Function mapping

So the type mapping of a functor is simply a polymorphic type (we can also have functors between different two polymorphic types, but we will review those later). So, what is the *function mapping*? This is a mapping that convert any function operating on simple types, like $string \to number$ to a function between their more complex counterparts e.g. $List\ string \to List\ number$.



In type theory, this mapping is represented by a higher-order function called map with a signature $(a \to b) \to (Fa \to Fb)$, where F represents the generic type.

Note that, although any possible function that has this type signature (that that obeys the functor laws) gives rise to a functor, *not all such functors are useful*. Usually, there is only one of them that makes sense for a given generic type and that's why we talk about *the* list functor, and we define `map` directly in the in the generic datatype, as a method.

In the case of lists and similar structures, the *useful* implementation of `map` is the one that applies the original (simple) function to all elements of the list.

```
class Array<A> {
  map (f: A => B): Array<B> {
    let result = [];
    for (obj of this) {
      result.push(f(obj));
    }
    return result;
  }
}
```

Functor laws

Aside from facilitating code reuse by bringing in all standard functions of simple types in a more complex context, `map` allows us to work in a way that is predictable, courtesy of the functor laws, which in programming context look like this.

Identity law:

```
a.map(a => a) == a
```

Composition law:

```
a.map(f).map(g) == a.map((a) => g(f(a)))
```

Task 7: Use examples to convince yourself that the laws are followed.

What are functors for

Now, that we have seen so many examples of functors, we finally can attempt to answer the million-dollar question, namely what are functors for and why are they useful? (often formulated also as "Why are you wasting your/my time with this (abstract) nonsense?")

Well, we saw that *maps are functors* and we know that *maps are useful*, so let's start from there.

So, why is a map useful? Well, it obviously has to do with the fact that the points and arrows of the map corresponds to the cities and the roads in the place you are visiting in i.e. due to the very fact that it is a functor, but there is a second aspect as well - maps (or at least those of them that are useful) are *simpler to work with* than the actual things they represent. For example, road maps are useful, because they are *smaller* than the territory they represent, so it is much easier to go look up the routes between two given places by following a map, than to actually travel through all of them in real life.

And type-theoretic functors are used in programming for similar reason - functions that involve simple types like `string`, `number`, `boolean` etc. are ... simple, and least when compared with functions that work with lists and other generic types. Using the `map` function allows us to operate on such types without having to think about them and to derive functions that transform them, from functions that transform simple values. In other words, functors are means of *abstraction*.

Of course, not all routes on the map and no functions that between generic datatypes can be derived just by functions between the types they contain.

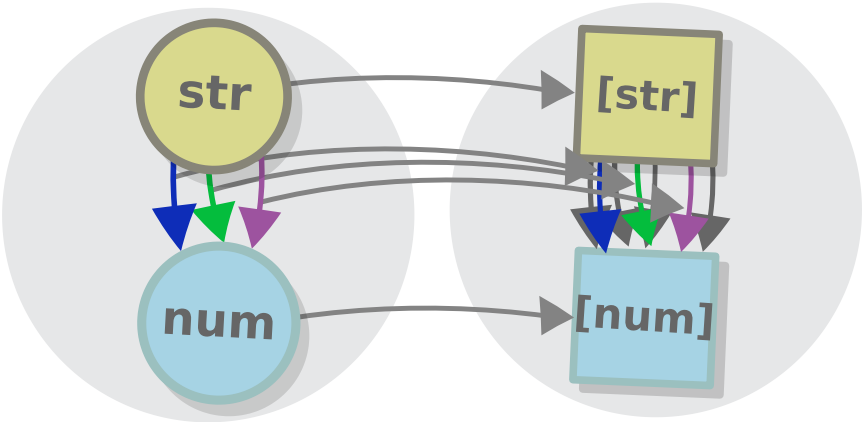
This is generally true for many “useful” functors: because their source categories are “simpler” than the target, some of the morphisms in the target have no equivalents in the source i.e. making the model simpler inevitably results in losing some of its capabilities. This is a consequence of “the map is not the territory” principle (“every abstraction is a leaky abstraction”, as Joel Spolsky puts it).

Pointed functors

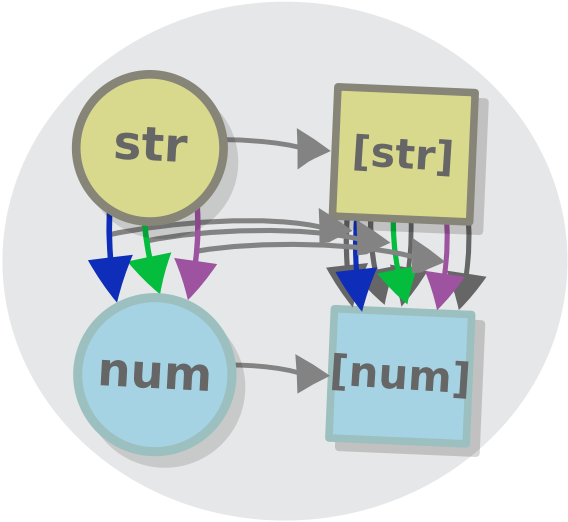
Now, before we close it off, we will review one more functor-related concept that is particularly useful in programming - *pointed endofunctors*.

Endofunctors

To understand what pointed endofunctors are, we have to first understand what are *endofunctors*, and we already saw some examples of those in the last section. Let me explain: from the way the diagrams there looked like, we might get the impression that the functors in programming connect different categories.



But that is not the case a type system is a category, and all functors go from this category to itself.

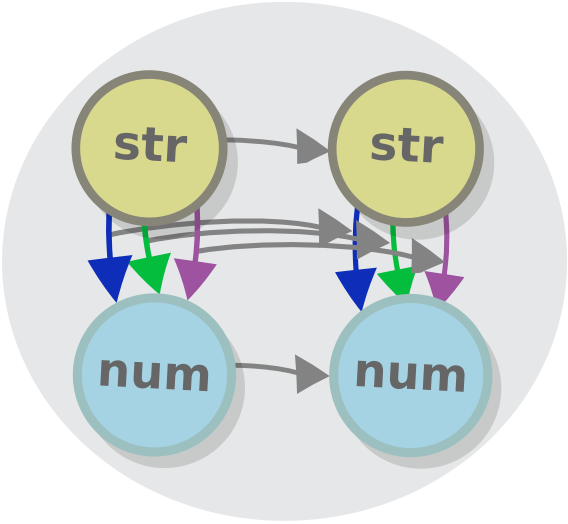


Yes, these are exactly what we call *endofunctors*.

> **DEFINITION:**
>
> Endofunctors are functors that have one and the same category as source and target.

The identity functor

So, what are some examples of endofunctors? I want to focus on one that will probably look familiar to you - it is the *identity functor* of each category, the one that maps each object and morphism to itself.



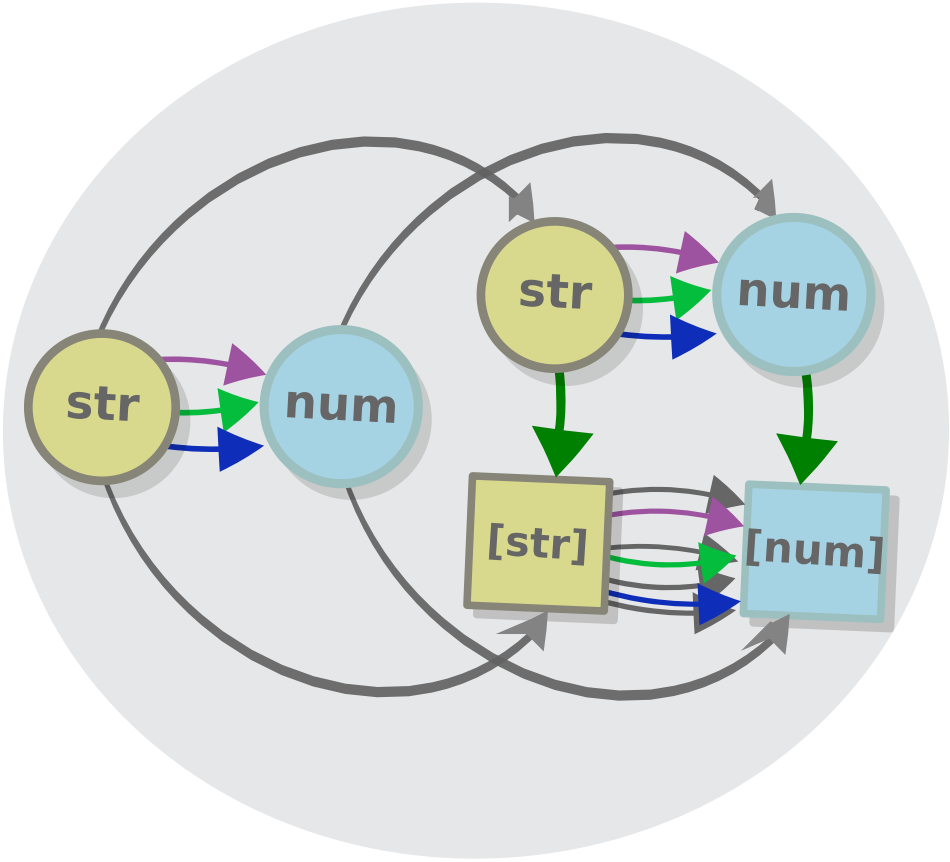
And it might be familiar, because an identity functor is similar to an identity morphism - it allow us to talk about value-related stuff without actually involving values.

Pointed functors

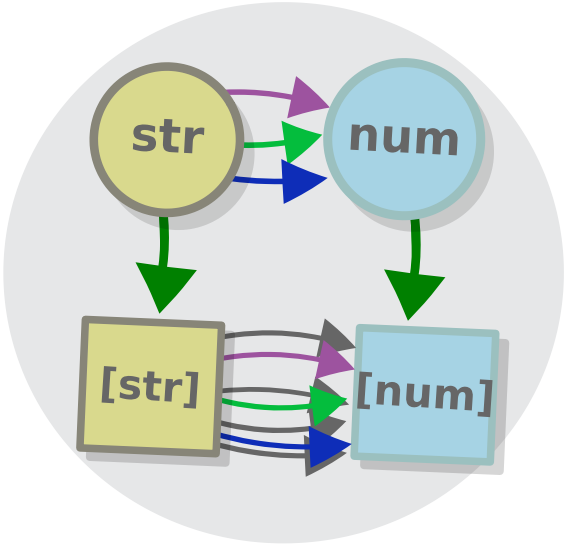
Finally, we can define pointed functors:

> **DEFINITION:**
>
> The identity functor, together with all other functors to which the identity functor can be *naturally transformed* are called *pointed functors* (i.e. a functor is pointed if there exist a natural transformation from the identity functor to it).

As we will see shortly, the list functor is a pointed functor.



We still haven't discussed what does it mean for one functor to be naturally transformed to another one (although the commuting diagram above can give you some idea), however, if we concentrate solely on the category of types in programming languages, then *a natural transformation is just a polymorphic function* e.g. this one is $a \to List\ a$, that preserves the structure of the types i.e. one for which this diagram commutes.



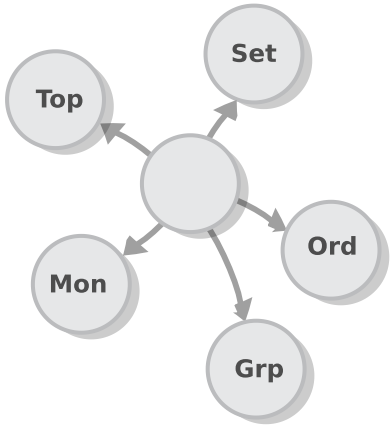
In the case of this functor, the function in question is $a \to [\, a \,]$ – the function that puts every value in a "singleton" list.

We will stop here, as natural transformations are a complex thing, and we want to examine them in a whole chapter (the next one).

The category of small categories

Ha, I got you this time (or at least I *hope* I did) - you probably thought that I won't introduce another category in this chapter, but this is exactly what I am going to do now. And (surprise again) the new category won't be the category of functors (don't worry, we will introduce that in the next chapter). Instead, we will examine the category of (small) categories:

> **DEFINITION:**
>
> The category of small categories is a category that has small categories (like Set - the category of sets, Mon, the category of monoids, Ord, the category of orders etc) for objects and functors for morphisms



We haven't yet mentioned the fact that functors compose (and in an associative way at that), but since a functor is just a bunch of functions, it is no wonder that it does.

Task 8: Go through the functor definition and see how do functors compose.

Task 9: What are the initial and terminal object of the category of small categories.

Categories all the way down

The recursive nature of category theory might sometimes leave us confused: we started by saying that categories are *composed of objects and morphisms*, but now we are saying that there are *morphisms between categories* (functors). And on top of that, there is a category where *the objects are categories themselves*. Does that mean that categories are an example of... categories? Sounds a bit weird on intuitive level (as for example biscuits don't contain other biscuits and houses don't use houses as building material), but it is actually the case. Like, for example, every monoid is a category with just one object, but at the same time, monoids can be seen as belonging to one category – the category of monoids, where they are connected by monoid homomorphisms. We also have the category of groups, for example, which contains the category of monoids as a subcategory, as all monoids are groups etc.

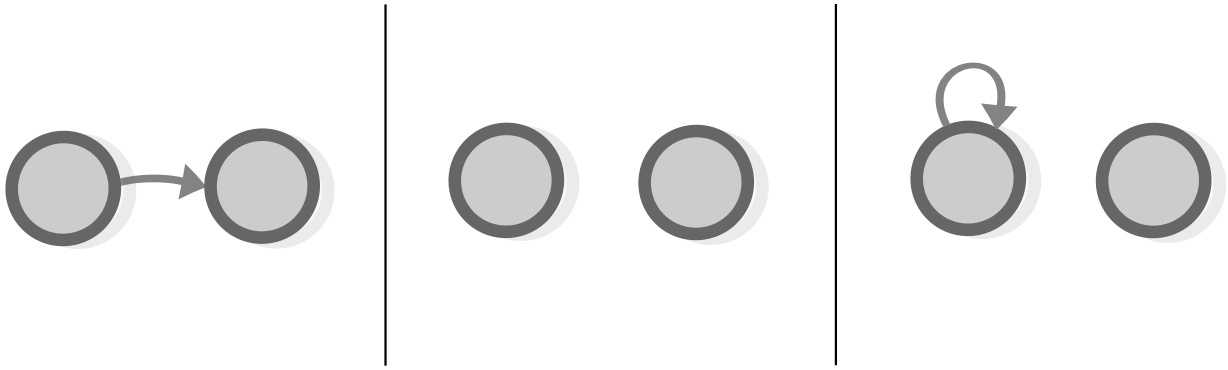
Category theory does *categorize* everything, so, from a category-theoretic standpoint, all of maths is *categories all the way down*. Whether you would treat a given category as a universe or as a point depends solely on the context. Category theory is an *abstract* theory. That is, it does not seek to

represent an actual state of affairs, but to provide a language that you can use to express many different ideas.

Answers

Task 1: There are just two more categories that have 2 objects and at most one morphism between two objects, draw them.

The answer is this:



Isomorphism means equality in category theory, so flipping the arrow from the left to the right doesn't count as a new category.

For the third category we have to specify what composing the morphism with itself would yield:

[f \circ f = id]

This law guarantees that there would be just 1 morphism besides identity: if this law doesn't exist, you would end up with the category with an infinite amount of morphisms (see the free monoid in the chapter on monoids), if it is changed to something else, they would be a finite number of morphisms, but still more than 1.

Task 2: What is the morphism mapping for orders?

The morphism mapping of orders consist just of mapping the only existing morphism between a given pair of objects in one order, to the only existing morphism of their counterparts in the other order. The "order-preserving"

condition of order isomorphisms guarantees that this morphism exists i.e. that if you have $A \to B$ in one order, you would have $F(A) \to F(B)$ in the other.

Task 3: Prove that the first functor law (preservation of identities) of groups can be proven from the second law i.e. that if C and D are groups and $F : C \to D$ – a group homomorphism, you have $ID_C = F(ID_D)$. Note that this is valid for groups, but not monoids.

We know that

$$ID_D = ID_D \circ ID_D$$

So

$F(ID_D) = F(ID_D) \circ F(ID_D)$ (second functor law)

But we can add the identity operation anywhere, without changing the equations, so we have

$F(ID_D) \circ ID_C = F(ID_D) \circ F(ID_D)$ (adding identity to the equation).

And cancelling out $F(ID_D)$, we have

$$ID_C = F(ID_D)$$

This last step is valid only for groups, as cancelling out a morphism is done by applying it's reverse, and monoid morphisms don't have reverses.

Task 4: Show why the law holds.

In categorical language, the problem states that if we have $f : a \to b$ and $g : b \to c$, then $F(g \circ f) = F(g) \circ F(f)$.

The proof uses the same approach as the one in Task 2: because in orders there can be just one morphism with a given type signature, two morphisms with equal type signatures must be equal to one another.

Then, if we compose those two morphisms in the target order ($F(g) \circ F(f)$), we get a morphism $F(a) \to F(c)$

[F(g)\circ F(f) : F(a) \to F(c)]

If we compose the two morphisms in the *source* order, and we use the functor law to obtain the corresponding morphism in the target order, we get another morphism from object $F(a) \to F(c)$

[F(g \circ f) : F(a) \to F(c)]

But because in orders there can be just one morphism with signature $F(a) \to F(c)$ so these two morphisms must be equal to one another:

[F(g)\circ F(f) = F(g \circ f)]

Task 5: Why are the graphs of linear functions comprised of straight lines?

A linear function grows with a constant rate, (i.e. their derivative is always constant).

The slope of a function reflects the change in its rate of growth, so in case of linear functions it is a straight line.

Task 6: Show that linear functions preserve addition.

Let's say we have a function f, such that it multiplies the number by a certain constant a, e.g. we have

$$f(x) = a \times x$$

and

$$f(y) = a \times y$$

If we sum the two we get

$$f(x) + f(y) = a \times x + a \times y$$

By the law of distributivity of addition we get

$$f(x) + f(y) = a \times (x + y)$$

but by the definition of f we also have

$$f(x + y) = a \times (x + y)$$

Uniting the two terms that are both equal to $a \times (x + y)$ we get:

$$f(x) + f(y) = f(x + y)$$

You can show that this works in the other direction as well, but it is a little more complicated.

Task 7: Use examples to convince yourself that the laws are followed.

The point here is playing a bit to familiarize yourself with the laws, you can use any set and functions, here is an example:

Identity law:

```
[1, 2, 3].map(a => a) == [1, 2, 3]
```

Composition law:

```
let f = (a) => a + 1
let g = (a) => a * 2
[1, 2, 3].map(f).map(g) == [1, 2, 3].map((a) => g(f(a)))
```

Task 8: Go through the functor definition and see how do functors compose.

Another trivial exercise, with, perhaps, non-trivial setup. It all comes down to go through the elements a functor consists of and to see how the elements of two functors can compose.

The two elements of a functor are the object mapping and the morphism mapping.

The object mapping is just a function, connecting two categories' underlying sets, so if we have two functors with the appropriate type signature e.g.

$$C \to D$$

and

$$D \to E$$

we also have two functions that connect these categories' underlying sets:

$$set(C) \to set(D)$$

and

$$set(D) \to set(E).$$

So the answer, for object mapping is to compose the two functions to get a function (which can be the basis for a $C \to D$ functor) $set(C) \to set(E)$.

For morphism mapping, do the same thing for the categories' hom-sets.

Task 9: What are the initial and terminal object of the category of small categories.

If you remember the concepts of initial and terminal objects from the category of sets Set, you will find out that they are basically the same for the category of categories Cat

The initial object in Set is the empty set, the initial category in Cat is the empty category. The unique functor from the empty category to any other category is the same – the functor which maps nothing to nothing.

The terminal object in Set is the one-element set, the terminal category in Cat is the category 1, which has one object and no morphisms other than identity. The unique functor from any object to the terminal object is the functor that maps all objects to the only object and all morphisms to the only morphism.

Natural transformations

> I didn't invent categories to study functors; I invented them to study natural transformations. — Saunders Mac Lane

In this chapter, we will introduce the concept of a morphism between functors, or *natural transformation*. Understanding natural transformations will enable us to define category equality and some other advanced concepts.

Natural transformations really are at the heart of category theory, however, their importance is not obvious at first. So, before introducing them, I like to talk, once more, about the body of knowledge that this heart maintains (I am good with metaphors... in principle).

Equivalent categories

Our first section aims to introduce natural transformation as a motivating example for creating a way to say that two categories are equal. But for that, we need to understand what equal categories are and should be.

So, are you ready to hear about equivalent categories and natural transformations? Actually it is my opinion that you are not (no offence, they are just very hard!). So, we will take a longer route. I can put this next section anywhere in this book, and it would always be neither here nor there. But anyway, if you are studying math, you are probably interested in the *nature of the universe*. "What is the quintessential characteristic of all things in this world?" I hear you ask...

Objects are overrated AKA Heraclitus was right!

> The world is the collection of facts, not of things. — Ludwig Wittgenstein

What is the quintessential characteristic of all things in this world? Some 2500 years ago, the philosopher Parmenides gave an answer to this question, postulating that the nature of the universe is permanence, stasis. According to his view, what we perceive as processes/transformations/change is merely illusory appearances ("Whatever is is, and what is not cannot be"). He said that that things never really change, they only *appear* to change, or (another way to put it), only appearances change, but the *essence* does not (I think this is pretty much how the word "essence" came to exist).

Although far from obviously true, his view is easy for people to relate to — objects are all around us, everything we "see", both literally (in real life), or metaphorically (in mathematics and other disciplines), can be viewed as *objects*, persisting through space and time. If we subscribe to this view, then we would think that the key to understanding the world is understanding *what objects are*. In my opinion, this is what set theory does, to some extent, as well as classical logic (Plato was influenced by Parmenides when he created his theory of forms).

However, there is another way to approach the question about the nature of the universe, which is equally compelling. Because, what is an object, when viewed by itself? Can we study an object in isolation? And will there anything left to study about it, once it is detached from its environment? If a given object undergoes a process to get all of it's part replaced, is it still the same object?

Asking such questions might lead us to suspect that, although what we *see* when we look at the universe are the objects, it is the processes/relations/transitions or *morphisms* between the objects that are the real key to understanding it. For example, when we think hard about everyday objects we realize that each of them has a specific *functions* (note the term) without which, a thing would not be itself e.g. is a lamp that doesn't glow, still a lamp? Is there food that is non-edible (or an edible item that isn't food)?

And this is even more valid for mathematical objects, which, without the functions that go between them, are not objects at all.

So, instead of thinking about objects that just happen to have some morphisms between them, we might take the opposite view and say *that objects are only interesting as sources and targets of morphisms*.

Although old, dating back to Parmenides' alleged rival Heraclitus, this view has been largely unexplored, until the 20th century, when a real mathematical revolution happened: Bertrand Russell created type theory, his student Ludwig Wittgenstein wrote a little book, from which the above quote comes, and this book inspired a group of mathematicians and logicians, known as the "Vienna circle". Part of this group was Rudolph Carnap who coined the word "functor"...

Isomorphism invariance

An embodiment of Heraclitus' view in the realm of category theory is the concept of *isomorphism invariance* that we implicitly touched several times.

All categorical constructions that we covered (products/coproducts, initial/terminal objects, functional objects in logic) are *isomorphism-invariant*. Or, equivalently, they define an objects *up to an isomorphism*.

> **DEFINITION:**
>
> A property is called isomorphism-invariant when if there are two or more objects that are isomorphic to one another, and one of them has a given property, then the rest of them would to also have this property as well.

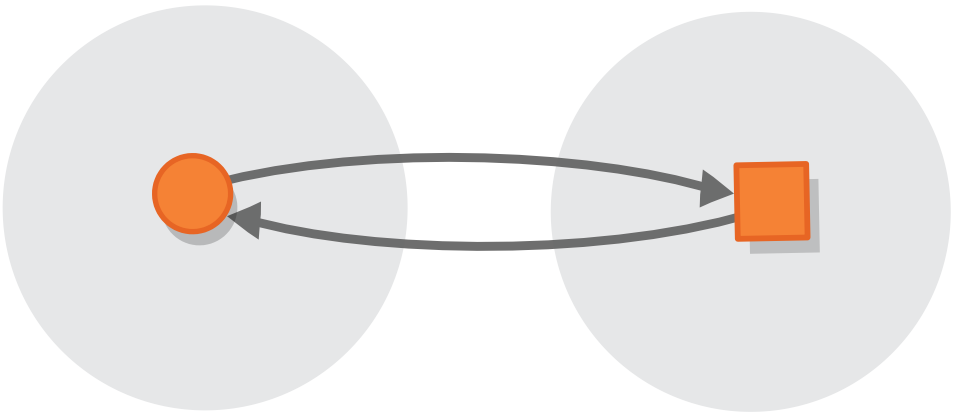
In short, in category theory **isomorphism = equality**.

The key to understanding category theory lies in understanding isomorphism invariance. And the key to understanding isomorphism invariance are natural transformations.

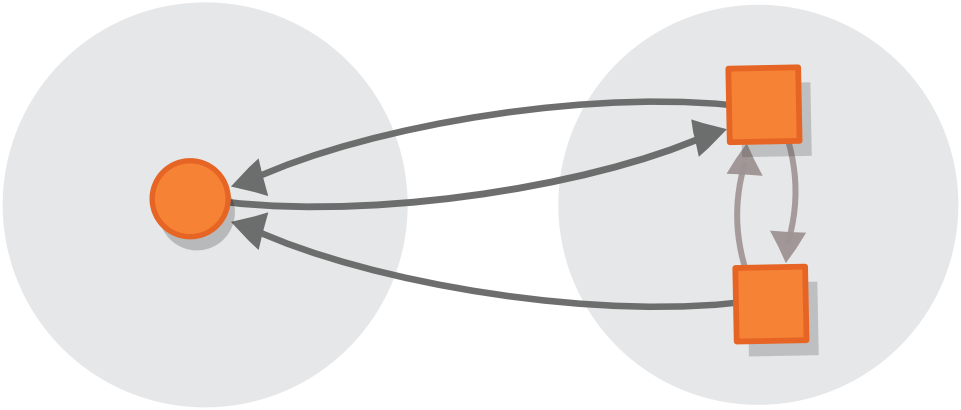
Categorical isomorphisms are not isomorphism-invariant

Let's return to the question that we were pondering at the beginning of the previous chapter — what does it mean for two categories to be equal?

In the prev chapter, we talked a lot about how great isomorphisms are and how important they are for defining the concept of equality in category theory, but at the same time we said that *categorical isomorphisms* do not capture the concept of equality of categories.



This is because (though it may seem contradictory at first) *categorical isomorphisms are not isomorphism invariant*, i.e. categories that only differ by having some additional isomorphic objects aren't isomorphic themselves.



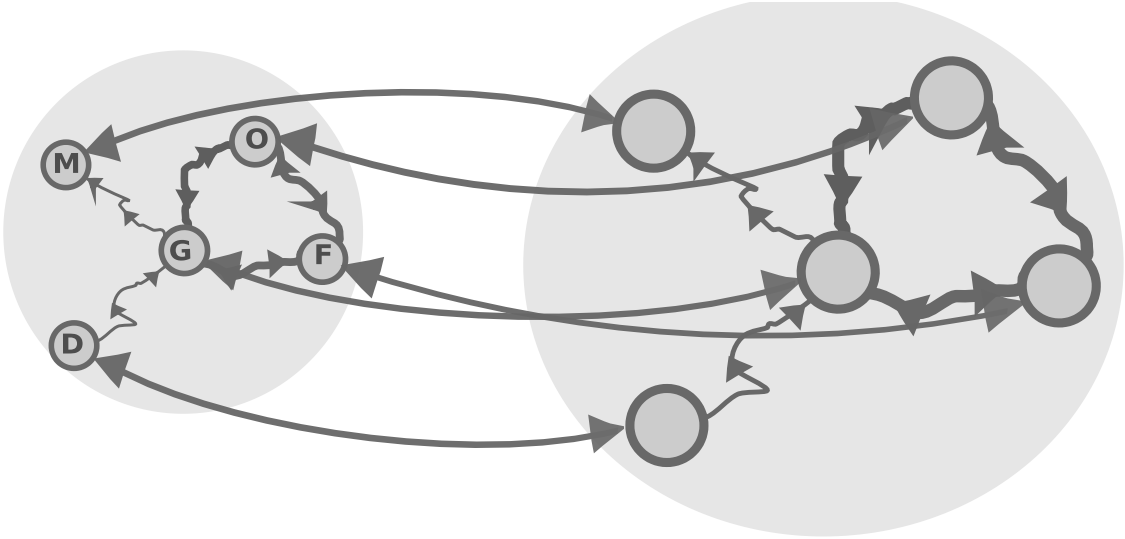
For this reason, we need a new concept of equality of categories. A concept that would elucidate the *differences* between categories with different structure, but also the *sameness* of categories that have the same categorical structures, disregarding the differences that are irrelevant for category-theoretic standpoint. That concept is *equivalence*.

Parmenides: This category surely cannot be equal to the other one — it has a different amount of objects!

Heraclitus: Who cares bro, they are isomorphic.

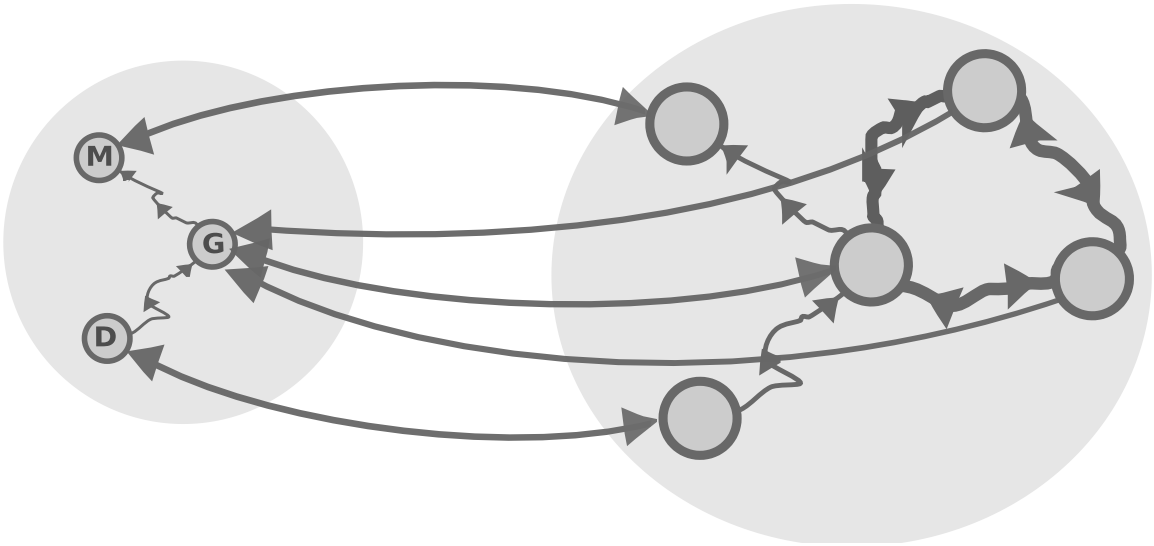
Equivalences are isomorphism invariant

To understand equivalent categories better, let's go back to the functor between a given map and the area it represents (we will only consider the thin categories (AKA preorders) for now). This functor would be invertible (and the categories — isomorphic) when the map should represent the area completely i.e. there should be arrow for each road and a point for each little place.



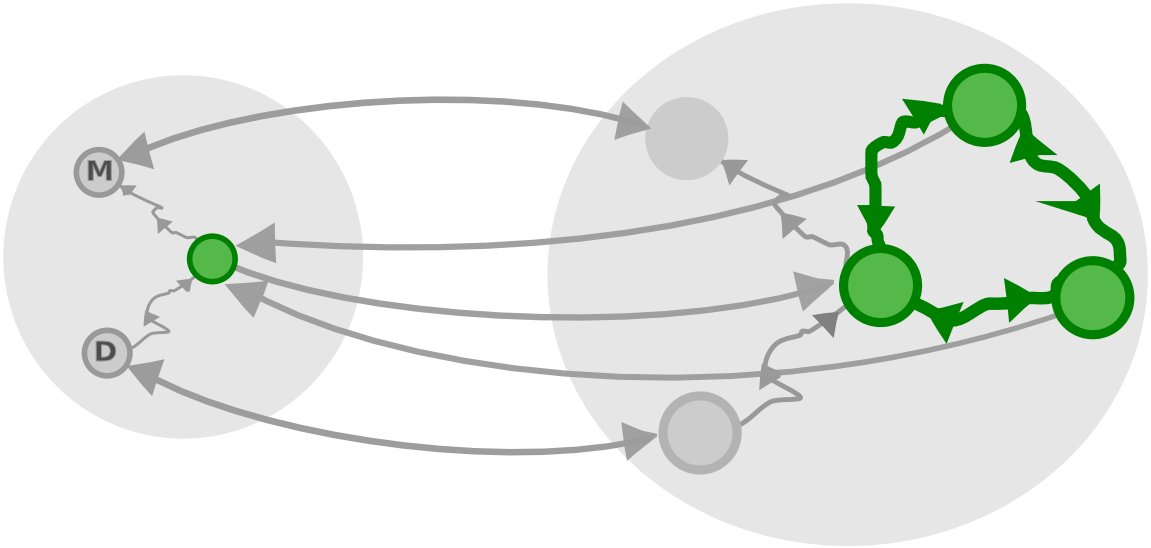
Such a map is necessary if your goal is to know about all *places*, however, like we said, when working with category theory, we are not so interested in *places*, but in the *routes* that connect them i.e. we focus not on *objects* but on *morphisms*.

For example, if there are intersections that are positioned in such a way that there are routes from one and to the other and vice-versa a map may collapse them into one intersection and still show all routes that exist (the tree routes would be represented by the "identity route").



These two categories are *not isomorphic* — going from one of them to the other and back again doesn't lead you to the same object.

However, going from one of them to the other would lead you at least to an *isomorphic object*.



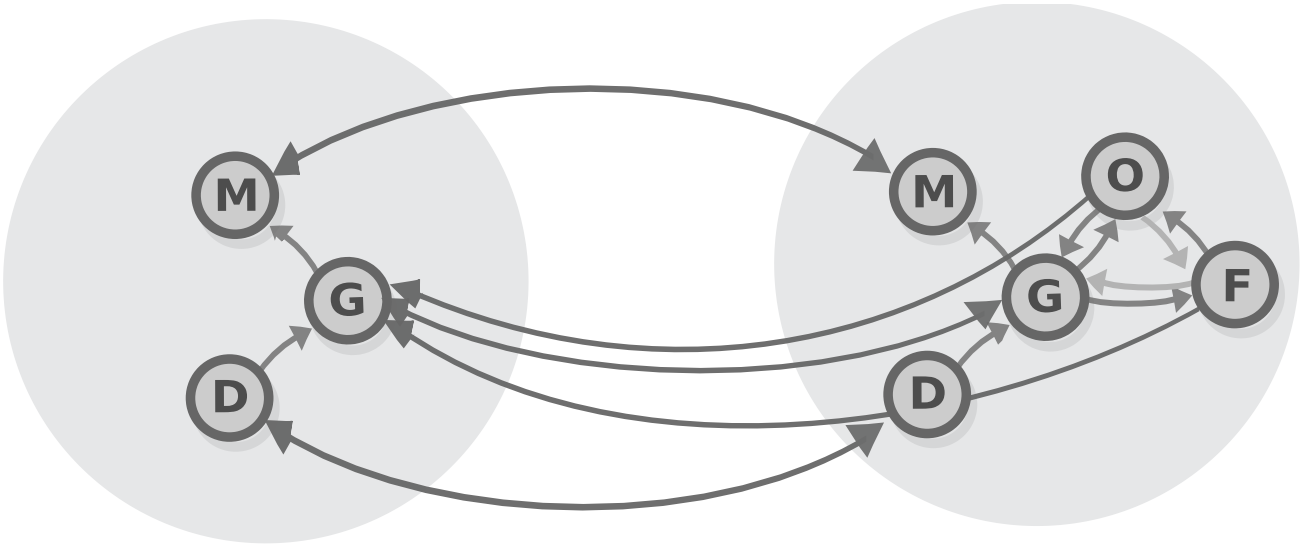
In this case we say that the preorders are *equivalent*.

Defining equivalence in terms of objects

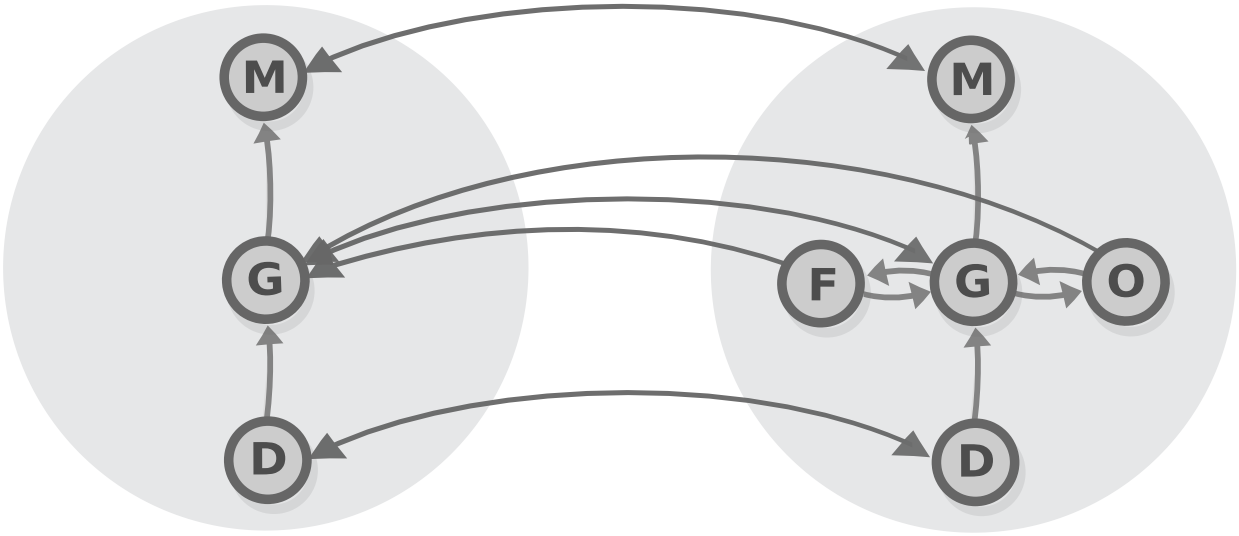
We know that two preorders are isomorphic if there are two functors, such that going from one to the other and back again leads you to the same object.

And two preorders are equivalent if going from one of them to the other and back again leads you to the same object, *or to an object that is isomorphic to*

the one you started with.

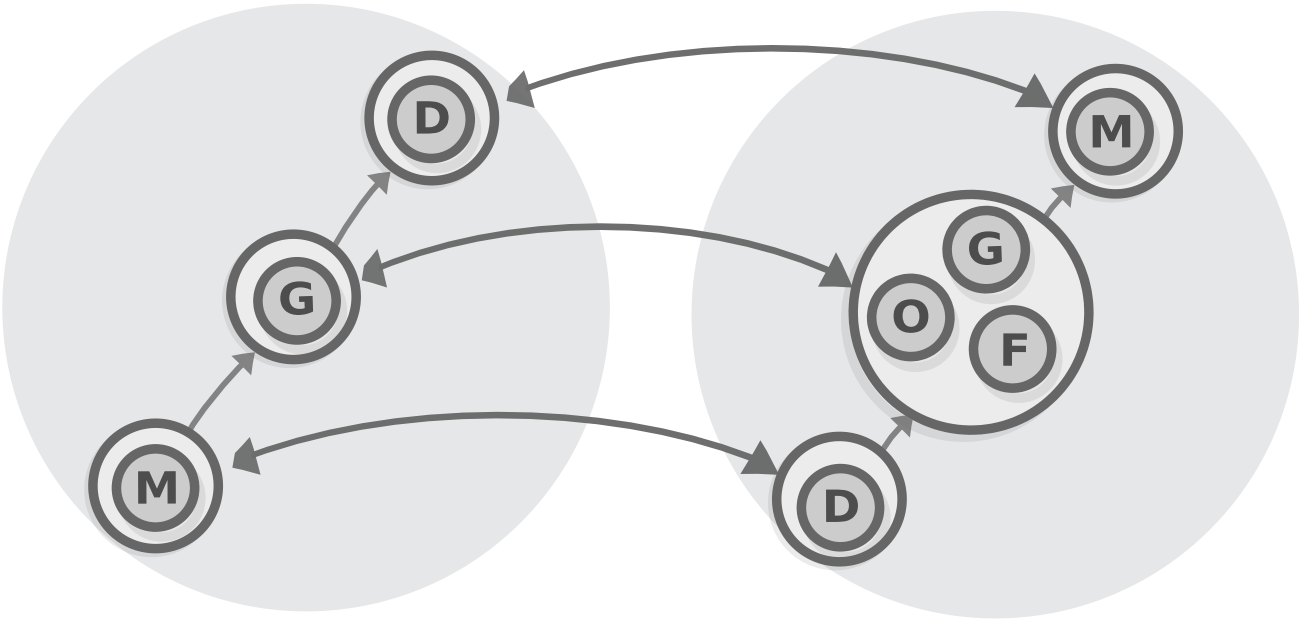


But when does this happen? To understand this, we plot the preorders as a Hasse diagram.



You can see that, although not all objects are connected one-to-one, *all objects at a given level are connected to objects of the corresponding level.*

To formalize that notion, we remember the concept of *equivalence classes* that we covered in the chapter about orders. Let's visualize the relationship of the equivalence classes of the two preorders that we saw above.

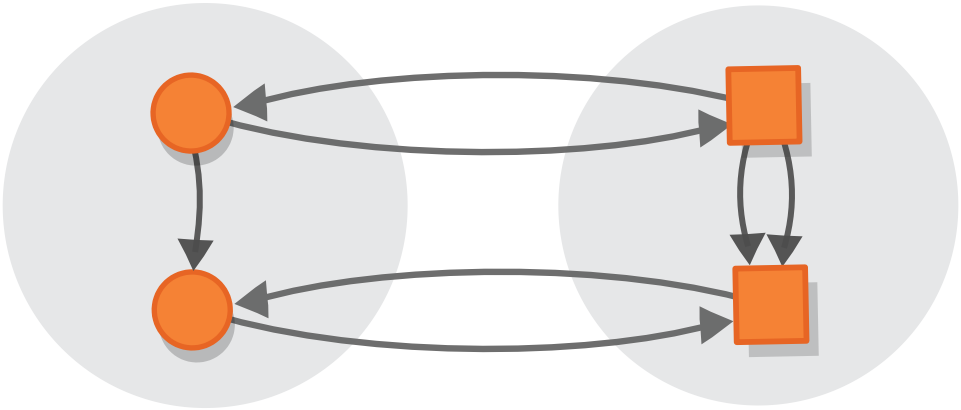


You can see that they are isomorphic. And that is no coincidence:

> **DEFINITION:**
>
> Two preorders are equivalent precisely when the preorders made of their equivalence classes are isomorphic.

This is a definition for equivalence of preorders, but unfortunately, it does not hold for all categories — when we are working with preorders, we can get away by just thinking about *objects*, but categories demands that we think about morphisms i.e. to prove two categories are equivalent, we should establish an isomorphism between their *morphisms*.

For example, the following two categories are *not* equivalent, although their equivalence classes are isomorphic — the category on the left has just one morphism, but the category on the right has two.



One way of defining equivalence of categories is by generalizing the notion of equivalence classes of preorders to what we call *skeletons* of categories, a skeleton of a category being a subcategory in which all objects that are isomorphic to one another are "merged" into one object (isomorphic objects are necessarily identical), i.ethe skeleton of a preorder is a partial order.

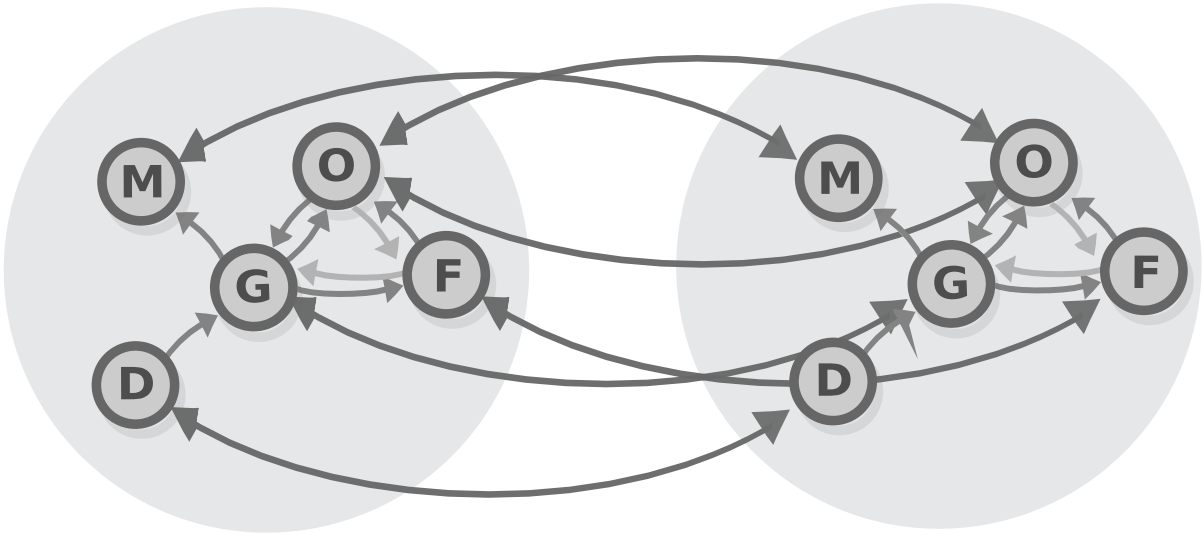
However, we will leave this (pardon my French) as an *exercise for the reader*. Why? We already did this when we generalized the notion of normal set-theoretic functions to *functors*, and so it makes more sense to build up on that notion. Also, we need a motivating example for introducing natural transformations, remember?

Defining equivalence in terms of morphisms

In the chapter about orders, we presented a definition of order *isomorphisms*, that is based on *objects*:

> **DEFINITION:**
>
> An order isomorphism is essentially an isomorphism between the orders' underlying sets (invertible function). However, besides their underlying sets, orders also have the arrows that connect them, so there is one more condition: in order for an invertible function to constitute an order isomorphism it has to *respect those arrows*, in other words it should be *order preserving*. More specifically, applying this function (let's call it F) to any two elements in one set (a and b) should result in two elements that have the same corresponding order in the other set (so $a \leq b$ if and only if $F(a) \leq F(b)$).

That a way to define them, but it is not the best way. Now that we know about functors (which, as we said, serve as functions between the orders and other categories), we can devise a new, simpler definition, which would also be valid for all categories, not just orders, and for all forms of equality (isomorphism and equivalence).



We begin with the definition of **set isomorphism**:

> **DEFINITION:**
>
> Two **sets** A and B are **isomorphic** (or $A \cong B$) if there exist functions $f : A \to B$ and its reverse $g : B \to A$, such that $f \circ g = ID_B$ and $g \circ f = ID_A$.

To amend it so it is valid for all categories by just replacing the word "function" with "functor" and "set" with "category":

> **DEFINITION:**
>
> Two **categories** A and B are **isomorphic** (or $A \cong B$) if there exist *functors* $f : A \to B$ and its reverse $g : B \to A$, such that $f \circ g = ID_B$ and $g \circ f = ID_A$.

Task 1: Check if that definition is valid.

Believe it or not, this definition, is just one find-and-replace operation away from the definition of *equivalence*. We get there only by replace equality with isomorphism (so, $=$ with $\cong$).

> **DEFINITION:**
>
> Two **categories** A and B are **equivalent** (or $A \simeq B$) if there exist *functors* $f : A \to B$ and its reverse $g : B \to A$, such that $f \circ g \cong ID_B$ and $g \circ f \cong ID_A$.

Like we said at the beginning, with isomorphisms, going back and forth brings us to the same object, while with equivalence the object is just *isomorphic* to the original one. This is truly all there is to it.

There is only one problem, though — *we never said what it means for functors to be isomorphic*.

Natural transformations, natural isomorphisms and categorical equivalence

So, how can we make the above definition "come to life"? The title of this chapter outlines the things we need to define:

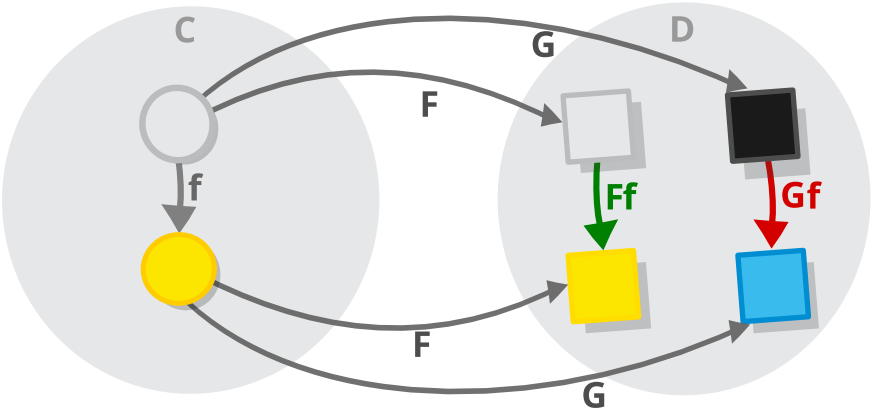
1. *Morphisms between functors* (called *natural transformations*).
2. *Functor isomorphisms* (called *natural isomorphisms*).
3. Finally *categorical equivalences*.

If this sounds complicated, remember that we are doing the same thing we always did — talking about isomorphisms.

In the very first chapter of this book, we introduced *set isomorphisms*, which are quite easy, and now we reached the point to examine *functor isomorphisms*. So, we are doing the same thing. Although actually...

But actually, natural transformations are quite different from morphisms and functors, (the definition is not "recursive", like the definitions of functor and morphism are). This is because functions and functors are both morphisms between objects (or *1-morphisms*), while natural transformations are *morphisms between morphisms* (known as *2-morphisms*).

But enough talking, let's draw some diagrams. We know that natural transformations are morphisms between functors, so let's draw two functors.



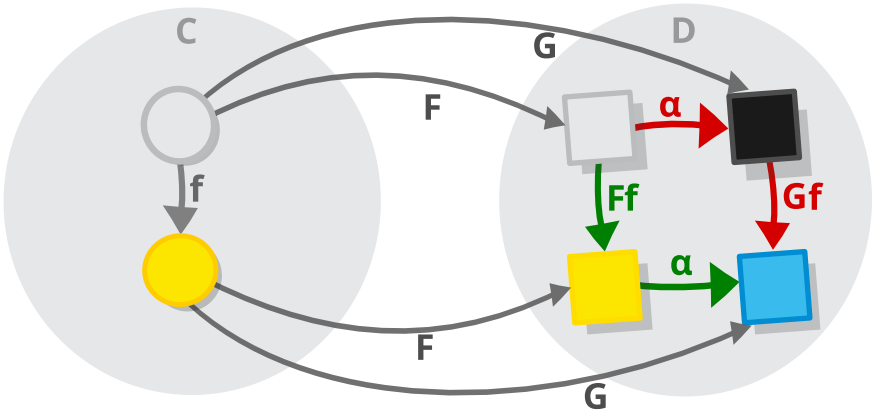
The functors have the same signature. Naturally. How else can there be morphisms between them?

Now, a functor is comprised of two mappings (object mapping and morphism mapping) so a mapping between functors, would consist of "object mapping mapping" and "morphism mapping mapping" (yes, I often do get in trouble with my choice of terminology, why do you ask?).

Object mapping mapping

Let's first connect the object mappings of the two functors, creating what we called "object mapping mapping".

It is simpler than it sounds when we realize that we only need to connect the object in functors' *target category* — the objects in the source category would just always be the same for both functors, as both functors would include *all* object from the source category (as that is what functors (and morphisms in general) do). In other words, mapping the two functors' object components involves nothing more than specifying a bunch of morphisms in the target category: one morphism for each object in the source category i.e. each object from the image of the first functor, should have one arrow coming from it (and to an object of the second functor, so, for example, our current source category has two objects and we specify two morphisms.

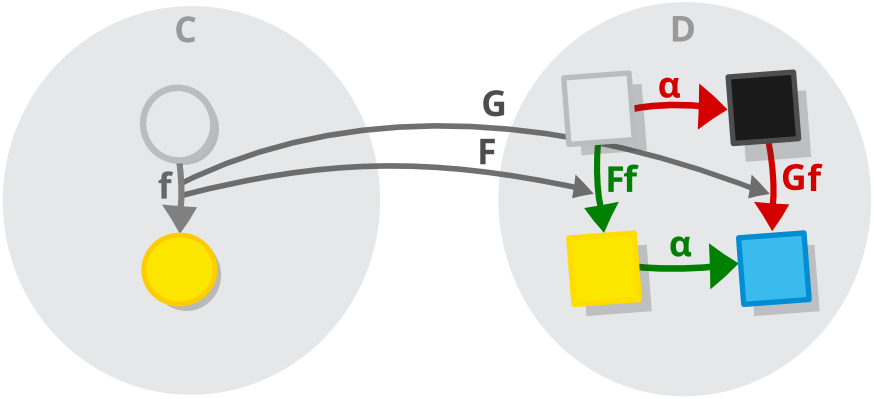


Note that this mapping does not map every object from the target category, i.e. not all objects have arrows coming from them (e.g. here the black and blue square do not have arrows), although, in some cases, it *might*.

Task 2: When exactly would the mapping encompass all objects?

Morphism mapping mapping

The morphism part might seem hard... until we realize that, once the connections between the object mappings are already established, there is only one way to connect the morphisms — we take each morphism of the source category and connect the two morphisms given by the two functors, in the target category. And that's all there is to it.



Oh, actually, there is also this condition that the above diagram should commute (the naturality condition), but that happens pretty much automatically.

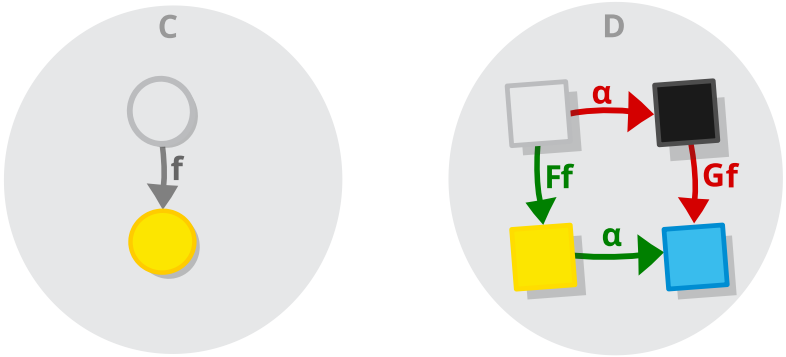
The naturality condition

Just like anything else in category theory, natural transformations have some laws that they are required to pass. In this case it's one law, typically called the naturality law, or the naturality condition.

Before we state this law, let's recap where are we now:

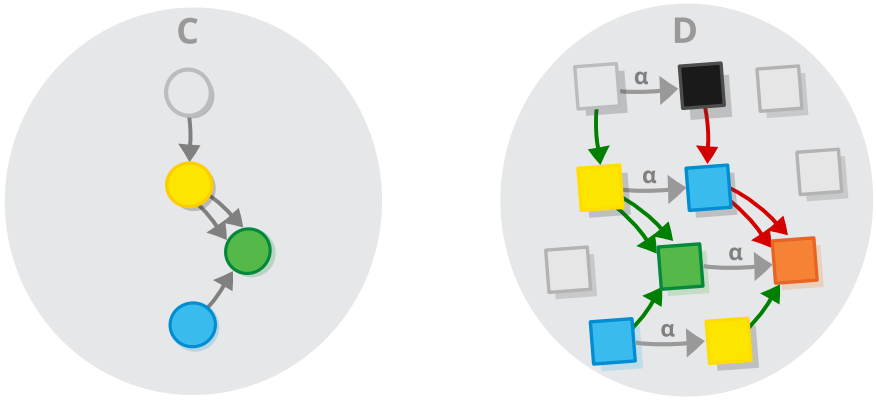
> **DEFINITION:**
>
> Let F and G be two functors that have the same type signature (so $F : C \to D$ and $G : C \to D$ for some categories C and D). A transformation between them is a family of morphisms in the target category D (denoted $\alpha : F \Rightarrow G$) one for each object in C, that map each object of the target of the functor F (or the image of F in D as it is also called) to some objects of the image of G.

This is a *transformation*, but not necessarily a *natural* one.

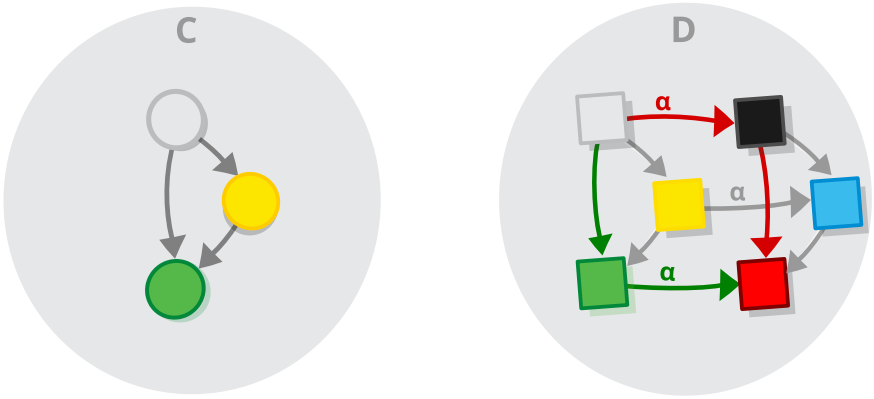
> **DEFINITION:**
>
> A transformation is natural when every morphism f in C is mapped to morphisms $F(f)$ by F and to $G(f)$ by G in such a way, that we have $\alpha \circ F(f) = G(f) \circ \alpha$ i.e. when this diagram commutes.
>
> 
>

i.e. when starting from the white square, when going right and then down (via the yellow square) is be equivalent to going down and then right (via the black one).

We may view a natural transformation is a mapping between morphisms and commutative squares: two functors and a natural transformation between two categories means that for each morphism in the source category of the functors, there exist one commutative square at the target category.



When we fully understand this, we realize that commutative squares are made of morphisms too, so, like morphisms, they compose — for any two morphisms with appropriate type signatures that have we can compose to get a third one, we have two naturality squares which compose the same way.



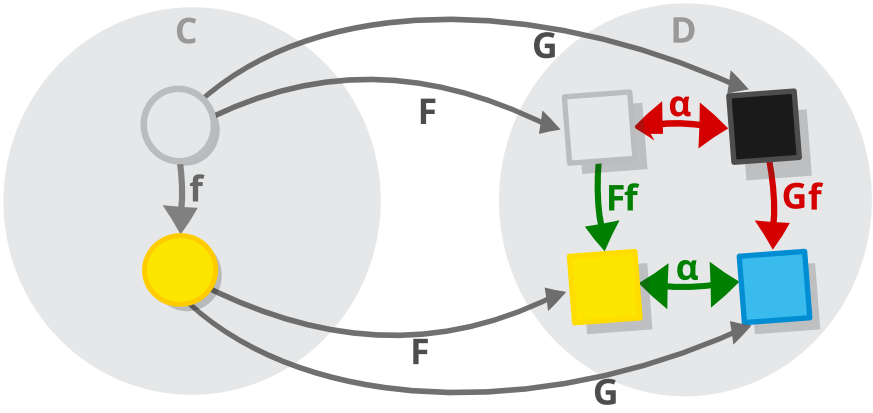
Which means natural transformation make up a...

(Oh wait, it's too early for that, is it?)

Natural isomorphisms

If a natural transformation is just a family of morphisms in a given category that satisfy certain criteria, then what would a natural *isomorphism* be? That's right — it is a family of *isomorphisms* that satisfy the same criteria.

The diagram is the same as the one for ordinary natural transformation, except that α are not just ordinary morphisms, but isomorphisms.



And the turning those morphisms into isomorphisms makes the diagram commute in more than one way i.e. if we have the naturality condition

$\alpha \circ F(f) = G(f) \circ \alpha$ i.e. the two paths going from **white** to **blue** are equivalent.

we also have:

$F(f) \circ \alpha = \alpha \circ G(f)$ i.e. the two paths going from **black** to **yellow** are also equivalent.

Constructing categorical equivalences

I am sorry, what were we talking about again? Oh yeah — categorical equivalence. Remember that categorical equivalence is the reason why we tackle natural transformations and isomorphisms? Or perhaps it was the other way around? Never mind, let's recap what we discussed so far:

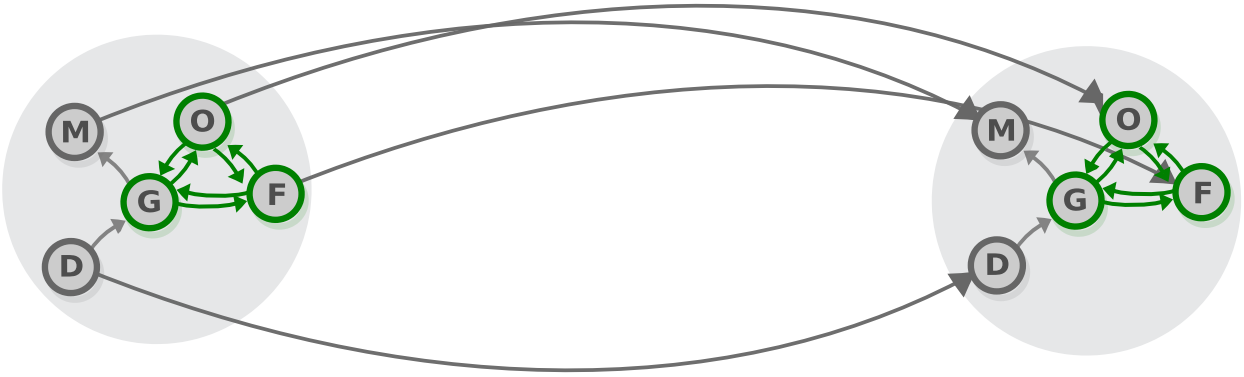
1. At the beginning of the section we introduced the notion of equivalence as two functors, such that going from one of them to the other and back again leads you to the same object, or to an *object that is isomorphic* to the one you started with.

2. And then, we discussed that for categories that are not thin (thick?) the situation is a bit more complex since they can have more than one morphism between two objects, and we should worry not only about isomorphic objects, but about *isomorphic morphisms*.

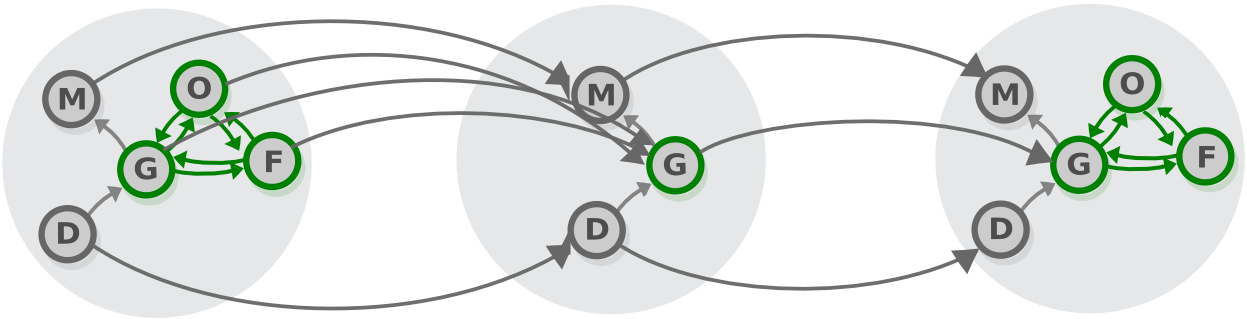
Now, we will show how these two notions are formalized by the definition that we presented.

> **DEFINITION:**
>
> Two **categories** A and B are **equivalent** (or $A \simeq B$) if there exist *functors* $f : A \to B$ and its reverse $g : B \to A$, such that $f \circ g \cong ID_A$ and $g \circ f \cong ID_B$.

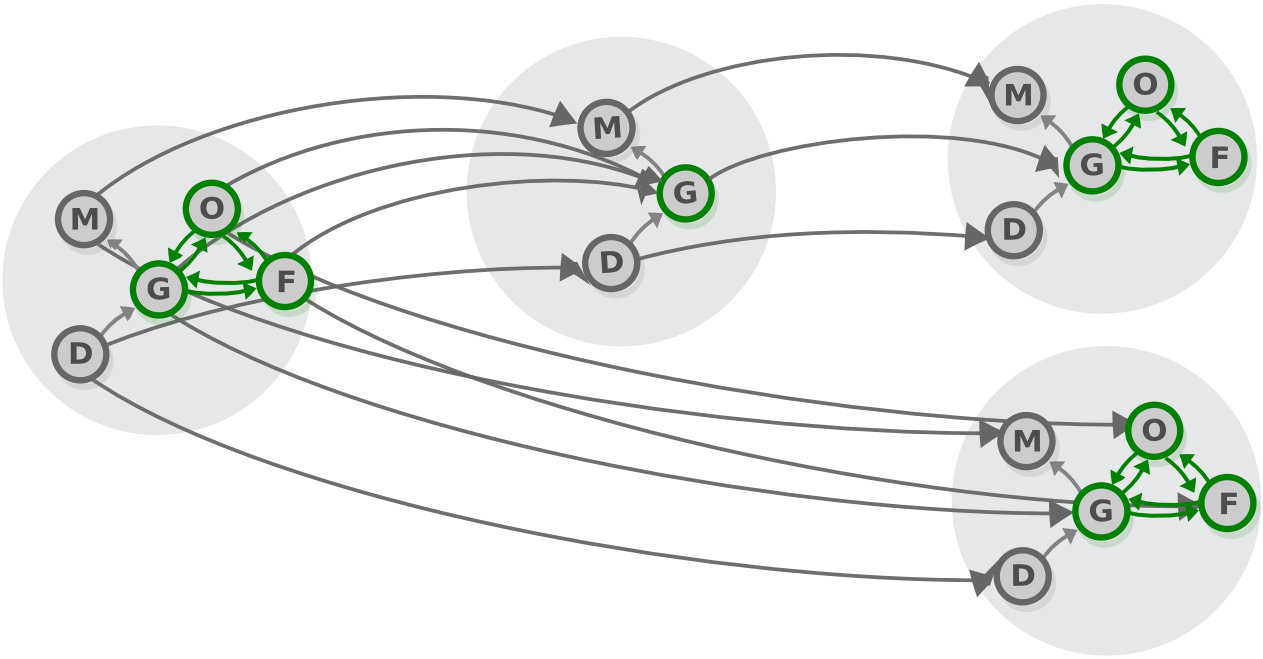
To understand, this how are the two related, let's construct the identity functor of the category that we have been using as an example all this time. Note that we are drawing the one and the same category two times (as opposed to just drawing an arrow coming from each object to itself), to make the diagrams more readable.



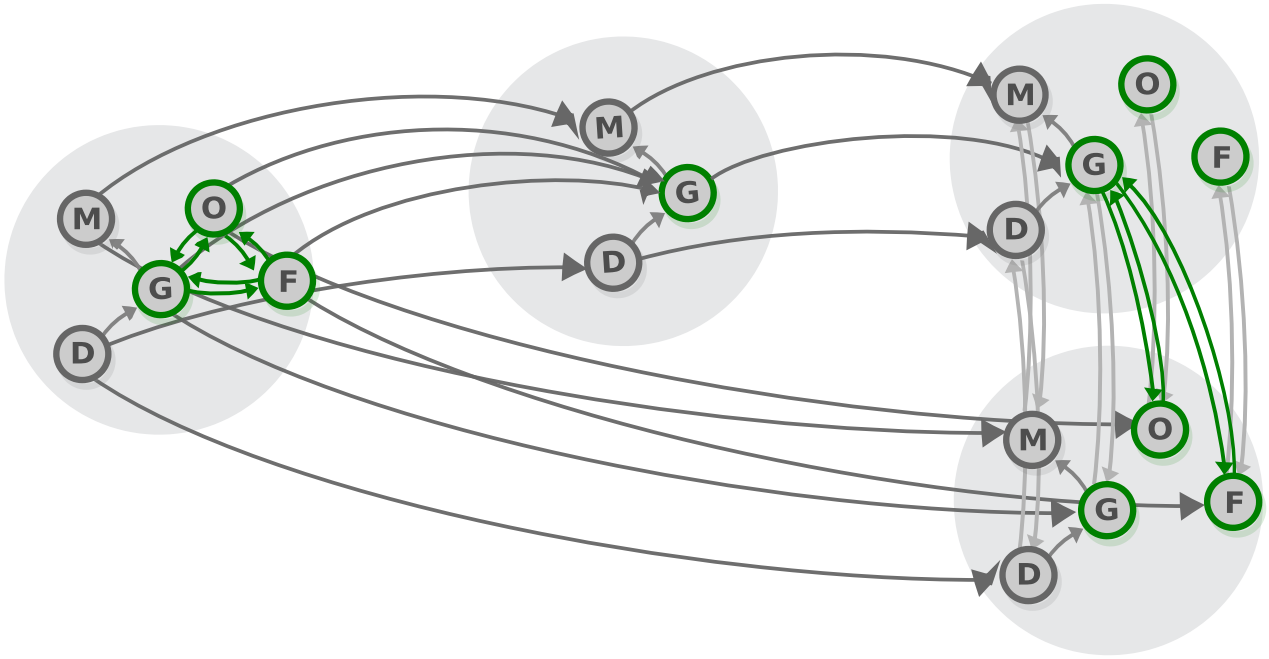
Then, we draw the composite of the two functors that establish an equivalence between the two categories, highlighting the 3 "interesting" objects, i.e. the ones due to which the categories aren't isomorphic.



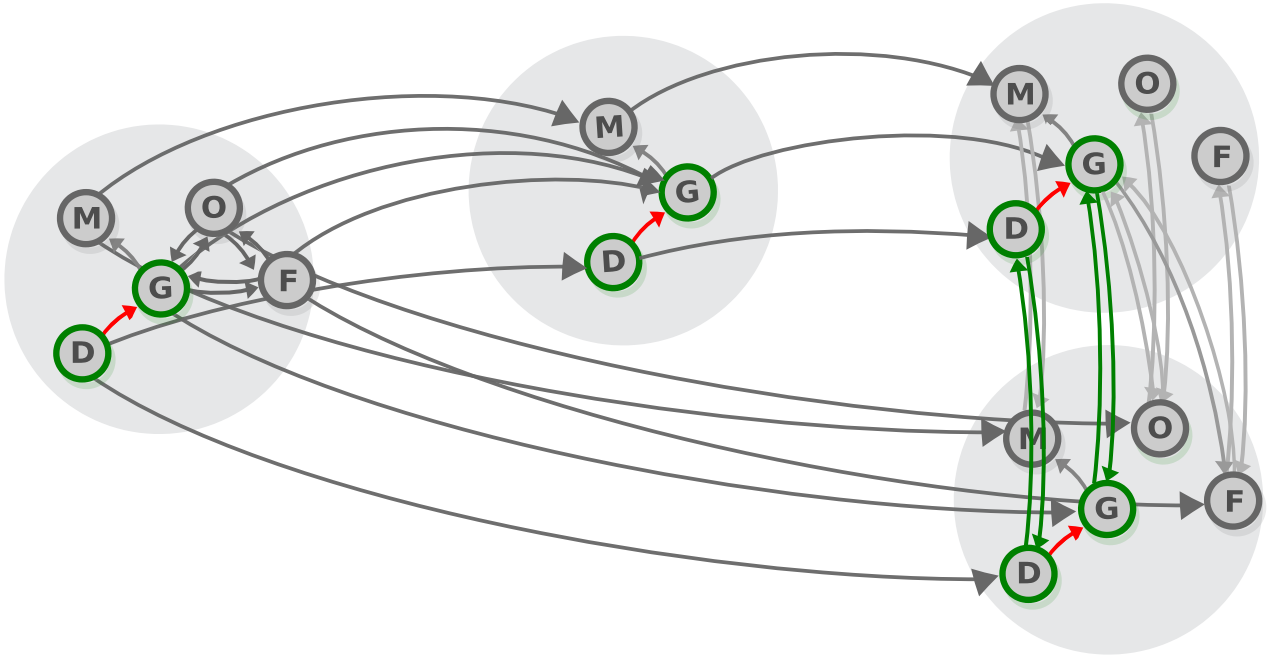
Now, we ask ourselves, in which cases does there exist an isomorphism between those two functors?



The answer becomes trivial if we draw the isomorphism arrows connecting the three “interesting” objects in a different way (remember, this is the same category on the top and the bottom) — we can see that these are exactly the arrows that enable us to construct an isomorphism between the two functors (the others are just identity arrows).



And when would this isomorphism be such that preserves the structure of the category (so that each morphism from the output of the composite functor has an equivalent one in the output of the identity)? Exactly when the isomorphism is *natural* i.e. when every morphism is mapped to a commuting square, e.g. here is the commuting square of the morphism that is marked in red.



i.e. naturality condition assures us that the morphisms in the target of the functor behave in the same way as their counterparts in the source.

With this, we are finished with categorical equivalence, but not with natural transformations — natural transformations are a very general concept, and categorical equivalences are only a very narrow case of them.

Natural transformations in programming. Natural transformations on the list functor

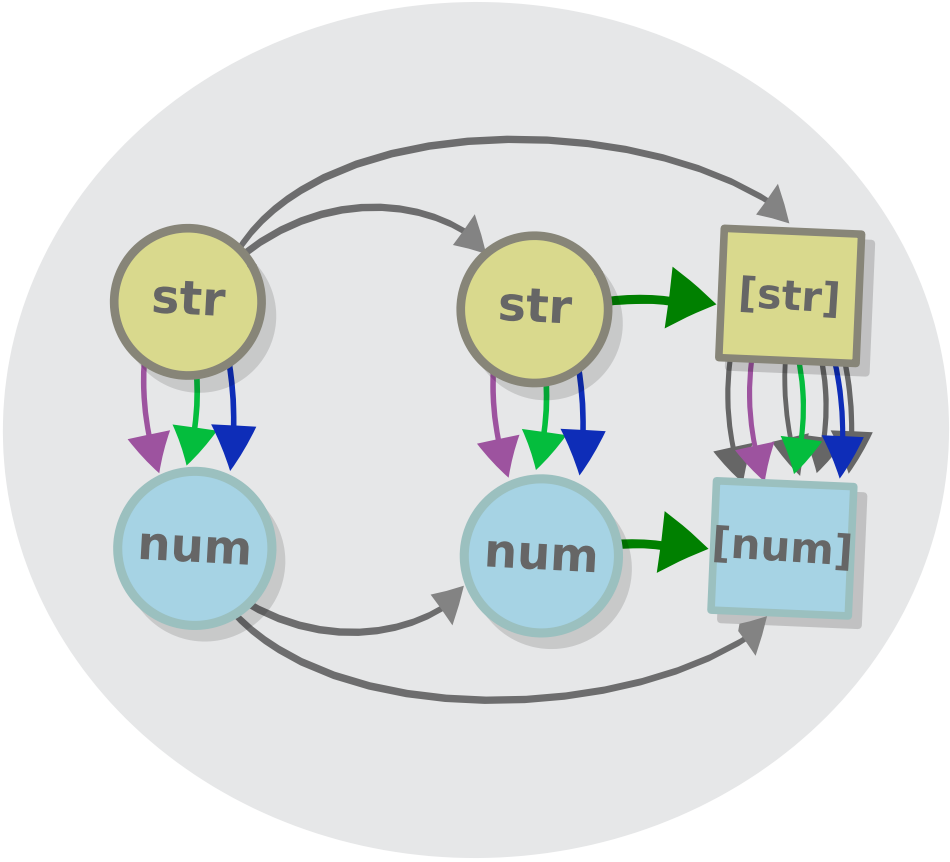
In the course of this book, we learned that programming/computer science is the study of the category of types in programming languages. However (in order to avoid this being too obvious) in the computer science context, we use different terms for the standard category-theoretic concepts.

We learned that objects are known as *types*, products and coproducts are, respectively, *objects/tuple* types and *sum* types. And, in the last chapter, we learned that functors are known as *generic types*. Now it's the time to learn what natural transformations are in this context. They are known as *(parametrically) polymorphic functions*.

Pointed functors again

Now, suppose this sounds a bit vague. If only we had some example of a natural transformation in programming, that we can use... But wait, we did show a natural transformation in the previous chapter, when we talked about pointed functors.

That's right, a functor is pointed when there is a natural transformation between it and the identity functor i.e. to have one green arrow for every object/type.



And this clearly is a natural transformation. As a matter of fact, if we get down to the nitty-gritty, we would see that it resembles a lot the equivalence diagram that we saw earlier — both transformations involve the identity functor, and both transformations have the same category as source and target, that's why we can put everything in one circle (we don't do that in the equivalence diagram, but that's just a matter of presentation).

Actually, the only difference between the two transformations is that an equivalence is defined by a natural *natural isomorphism* of a given functors to the identity functor ($ID \cong f \circ g$ and $ID \cong g \circ f$), while a pointed functor is defined by a one-way *natural transformation* from the identity functor ($ID \to f$) i.e. the equivalence functor is pointed, but not the other way around).

Polymorphic functions as natural transformations

We said that a natural transformation is equivalent to a (parametrically) polymorphic function in programming. But wait, wasn't natural transformation something else (and much more complicated):

> **DEFINITION:**
>
> Two functors F and G that have the same type signature (so $F : C \to D$ and $G : C \to D$ for some categories C and D), and a family of morphisms in the target category D (denoted $\alpha : F \Rightarrow G$) one for each object in C. Morphisms that map each object of the target of F (or the image of F in D as it is also called) to some object in the target of G.

Indeed it is (I wasn't lying to you, in case you are wondering), however, in the case of programming, the source and target categories of both functors are the same category (Set), so the whole condition regarding the functors' type signatures can be dropped.

> **DEFINITION:**
>
> Two ~~functors~~ generic types F and G ~~that have the same type signature~~ and a family of morphisms in Set (denoted $\alpha : Set \Rightarrow Set$) one for each object in Set, that map each target object of the functor F (or the image of F in D as it is also called) to some target objects of functor G.

As we know from the last chapter, a functor in programming is a generic type (which, has to have the `map` function with the appropriate signature).

And what is a "family of morphisms in Set one for each object in Set"? Well, the morphisms in the category Set are functions, so that's just a bunch of functions, one for each type. In Haskell/System F, if we denote a random type by the letter a), it is $alpha : \forall a.\, Fa \to Ga$. But that's exactly what polymorphic functions are.

Here is how would we write the above definition in a more traditional language (we use capital `<A>` instead of a, as customary.

```
function alpha<A>(a: F<A>) : G<A> {
}
```

Generic types work by replacing the `<A>` with some concrete type, like `string`, `int` etc. Specifically, the natural transformation from the identity functor to

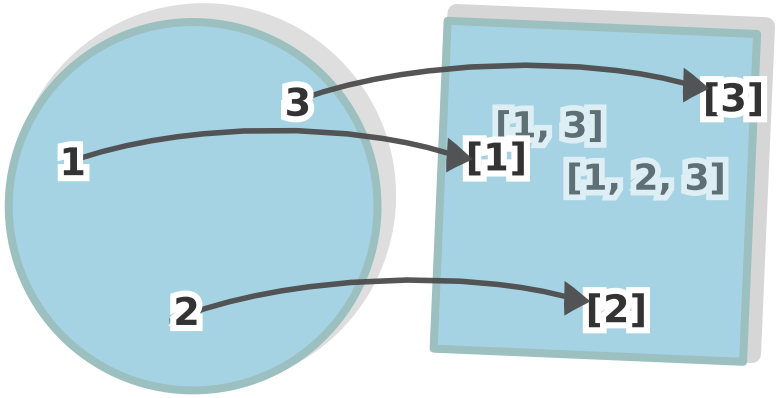
the list functor that puts each value in a singleton list looks like this $alpha :: \forall\ a.\ a \to List\ a$. Or in TypeScript:

```
function array<A>(a: A) : Array<A> {
    return [a]
}
```

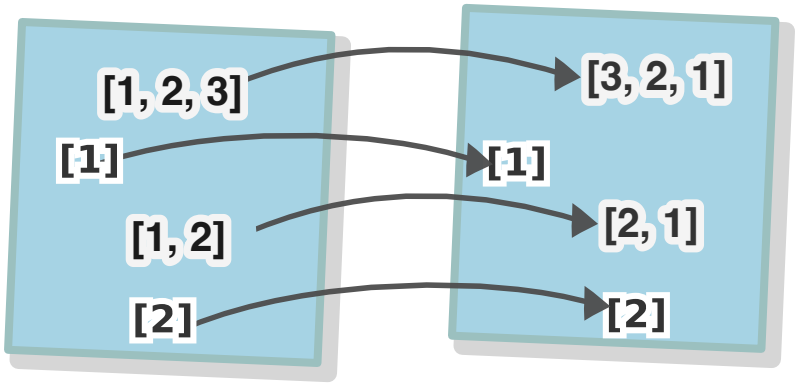
Some examples of natural transformations

Once we rid ourselves of the feeling of confusion, that such an excessive amount of new terminology and concepts impose upon us (which can take years, by the way), we realize that there are, of course, many polymorphic functions/natural transformations that programmers use.

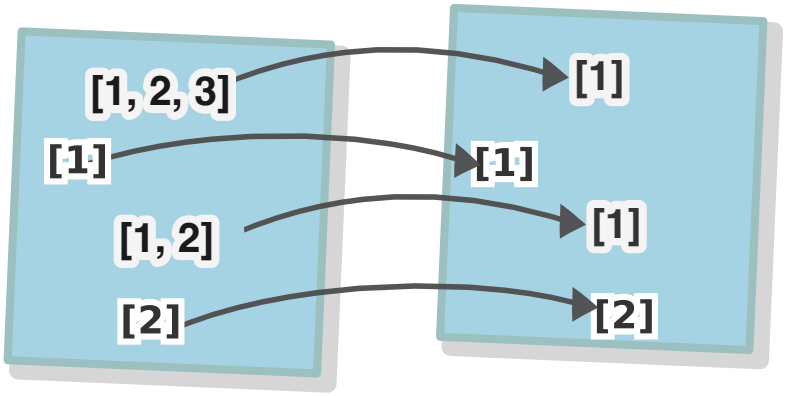
For example, in the previous chapter, we discussed one natural transformation/polymorphic function the function $\forall a.\ a \to [a]$ which puts every value in a singleton list. This function is a natural transformation between the identity functor and the list functor.



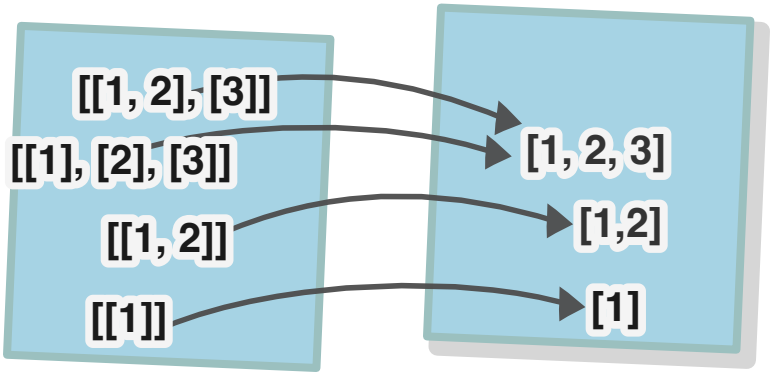
This is pretty much the only one that is useful with *this* signature (the others being $a \to [a, a]$, $a \to [a, a, a]$ etc.), but there are many examples with signature $list\ a \to list\ a$, such as the function to *reverse* a list.



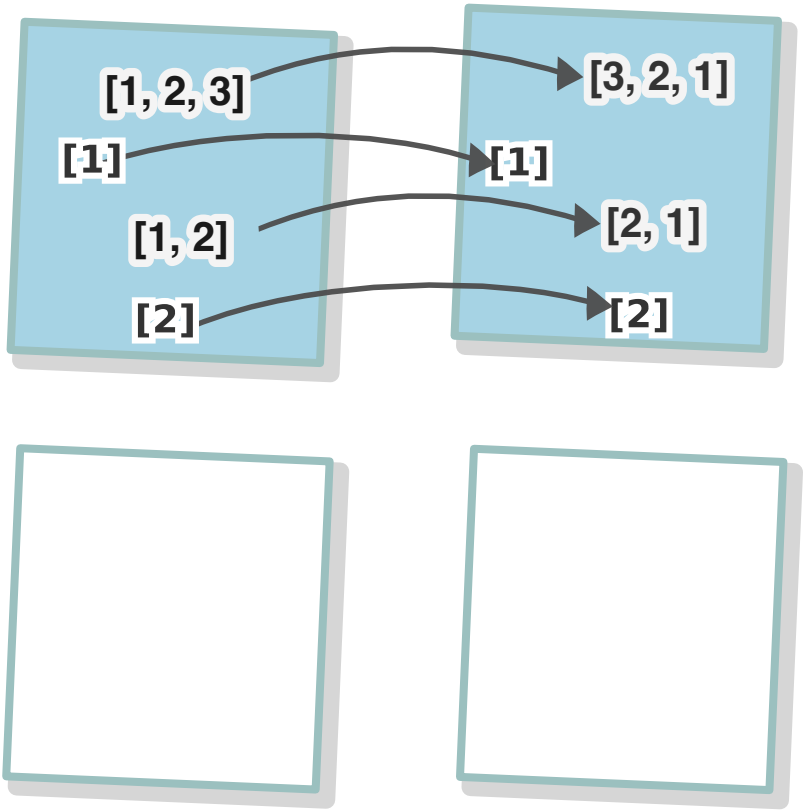
...or *take1* that retrieves the first element of a list



or *flatten* a list of lists of things to a regular list of things (the signature of this one is a little different, it's $list\ list\ a \to list\ a$).



Task 3: Draw example naturality squares of the $reverse$ natural transformation.



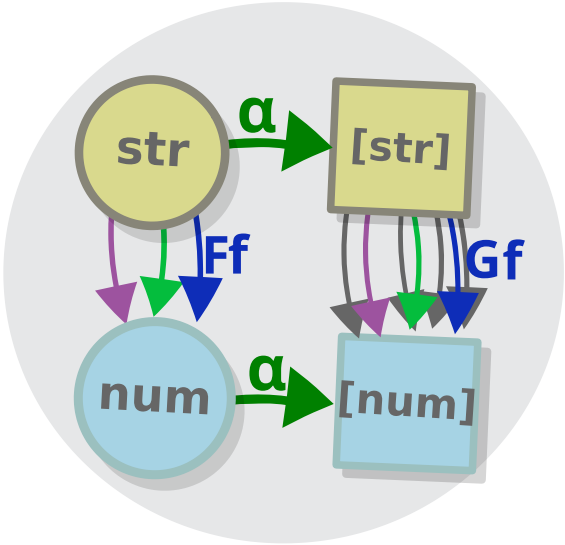
Do the same for the rest of the transformations.

The naturality condition

Before, we said that we shouldn't worry too much about naturality, as it is satisfied every time. Statistically, however, this is not true — as far as I am concerned, about 99.999 percent of transformations aren't really natural (I wonder if you can compute that percentage properly?). But at the same time, it just so happens (my favourite phrase when writing about maths) that all transformations that we care about *are* natural.

So, what does the naturality condition entail, in programming? To understand this, we construct some naturality squares of the transformations that we presented.

We choose two types that play the role of a, in our case $string$ and num and one natural transformation, like the transformation between the identity functor and the list functor.



The diagram commute when for all functions f, applying the Ff, the mapped/lifted version of f with one functor (in our case this is just $Ff : string \to num$ cause it is the identity functor), followed by ($alpha :: Fb \to G\ b$), is equivalent to applying ($alpha :: Fa \to G\ a$), and then the mapped version of f with the other functor (in our case $Gf :: List\ a \to List\ b$) i.e.

[\alpha \circ F\ f \cong G\ f \circ \alpha]

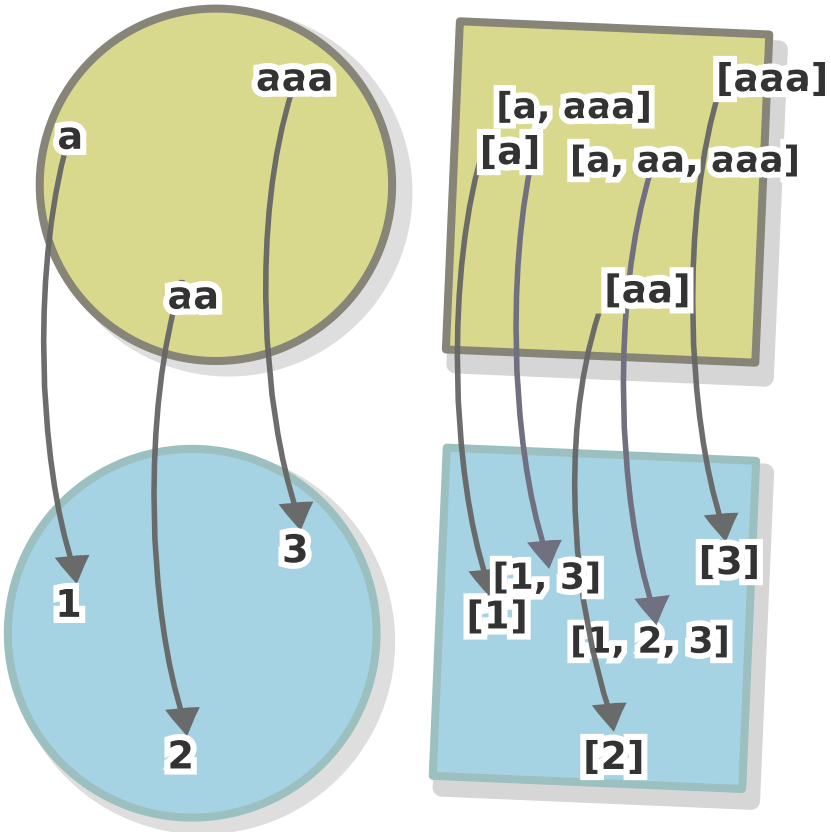
(in the programming world, you would also see it as something like $\alpha(map\ fx) = map\ f(\alpha x)$, but note that here map function means two different things on the two sides, Haskell is just smart enough to deduce which $fmap$ to use).

And in TypeScript, when we are talking specifically about the identity functor and the list functor, the equality is expressed as:

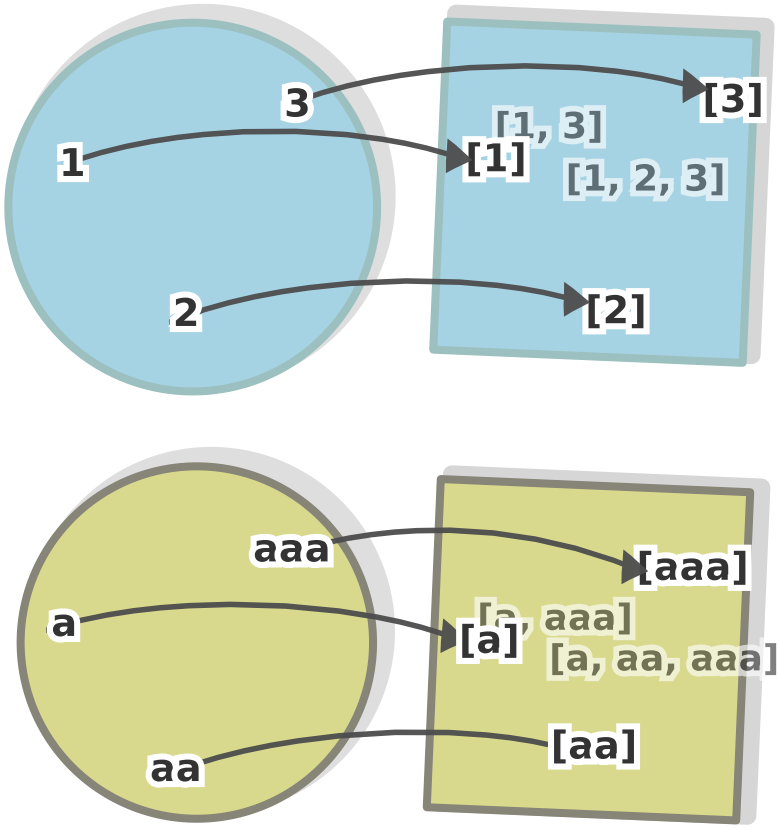
```
[x].map(f) == [f(x)]
```

So, is this equation true in our case? To verify it, we take one last peak at the world of values.

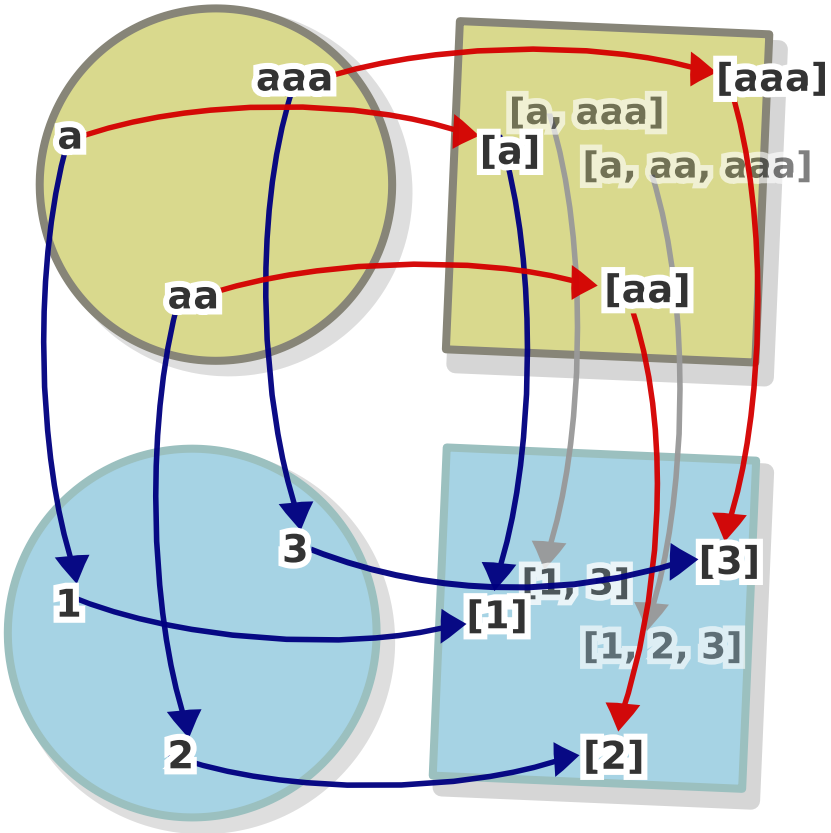
We acquire an f, that is, we a function that acts on simple values (not lists), such as the function $length : string \to num$, which returns the number of characters a string has and convert it, (or *lift* it, as the terminology goes) to a function that acts on more complex values, using the list functor, (and the higher-order function map).



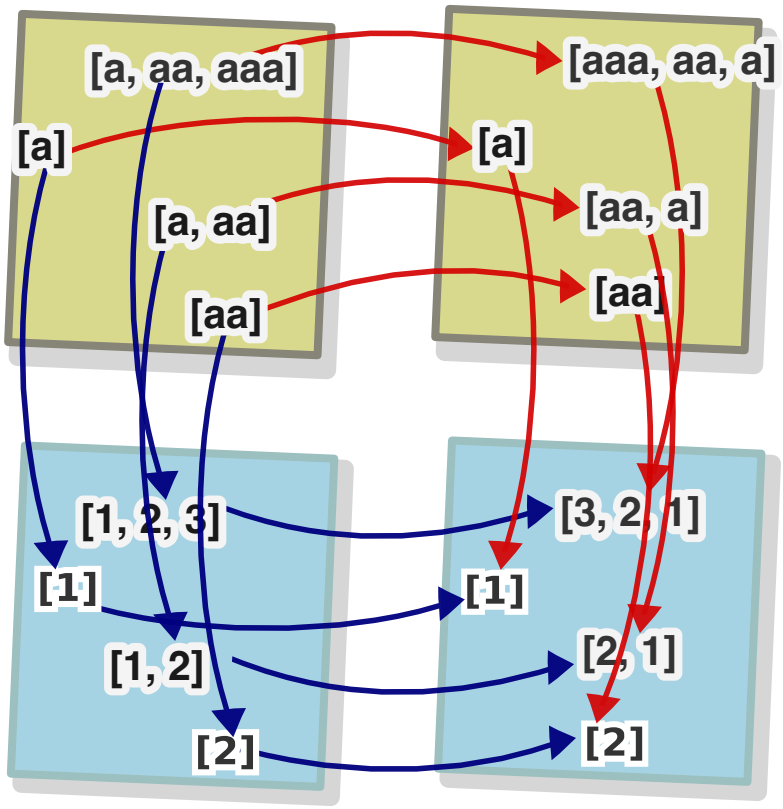
Then, we take the input and output types for this function (in this case $string$ and num), and the two morphisms of a natural transformation (e.g the abstract function $\forall a.\, a \to [a]$) that correspond to those two types.



When we compose these two pairs of morphisms we observe that they indeed commute — we get two morphisms that are actually one and the same function.



The above square shows the transformation $\forall a.\, a \to [a]$ (which is between the identity functor and the list functor, here is another one, this time between the list functor and itself ($\forall a.\, [a] \to [a]$) – $reverse$



(and you can see that this would work not just for $length$, but for any other function).

So, why does this happen? Why do these particular transformations make up a commuting square for each and every morphism?

The answer is simple, at least in our specific case: the original, unlifted function $f :: a \to b$ (like our $length :: string \to num$) can only work on the individual values (not with structure), while the natural transformation functions, i.e. ones with signature $list :: a \to list\ a$ only alter the structure, and not individual values. The naturality condition just says that these two types of functions can be applied in any order that we please, without changing the end result.

This means that if you have a sequence of natural transformations that you want to apply, (such as $reverse$, $take$, $flatten$ etc) and some lifted functions (Ff, Fg), you can mix and match between the two sequences in any way you like and you will get the same result e.g.

[take1 \circ reverse \circ F\ f \circ F\ g]

is the same as

[take1 \circ F\ f \circ reverse \circ F\ g]

...or...

[F\ f \circ F\ g \circ take1 \circ reverse]

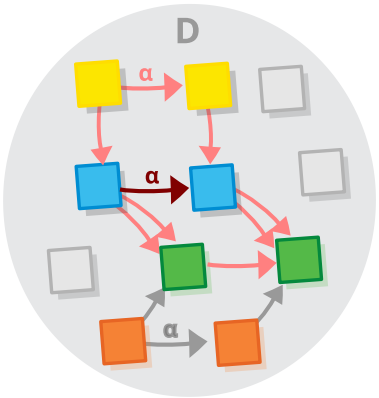
...or any other such sequence (the only thing that isn't permitted is to flip the members of the two sequences — $(take1 \circ reverse$ is of course different from $reverse \circ take1$and if you have $F\ f \circ F\ g$, then $F\ g \circ F\ f$ won't be permitted at all due to the different type signatures).

Task 4: Prove the above results, using the formula of the naturality condition.

Non-natural transformations

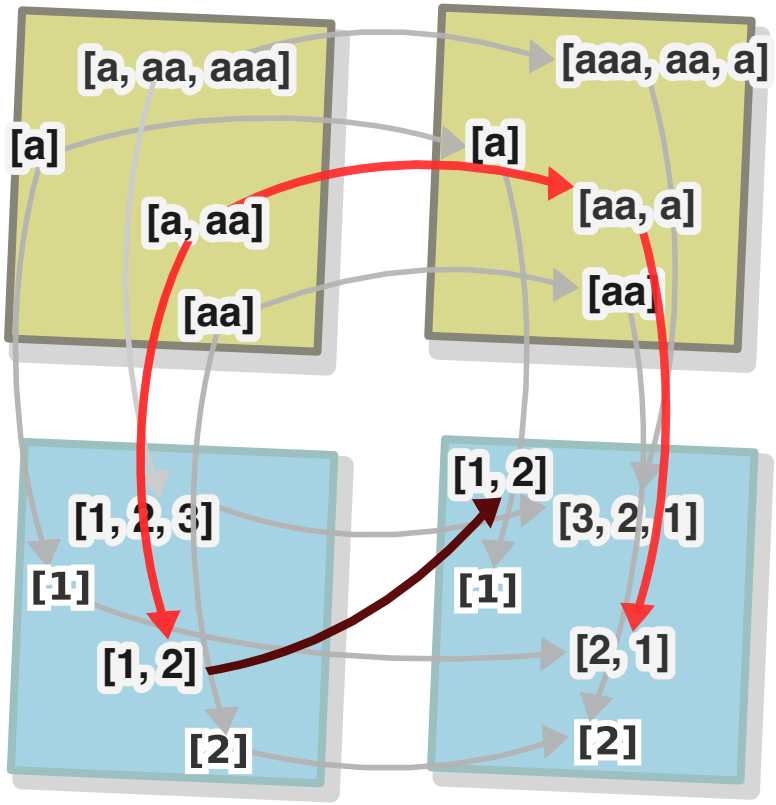
"Unnatural", or "non-natural" transformations (let's call them just *transformations*) are mentioned so rarely, that we might be inclined to ask if they exist. The answer is "yes and no". Why yes? On one hand, transformations, consist of an innumerable amount of morphisms, forming an ever more innumerable amount of squares and obviously nothing stops some of these squares to be non-commuting.

For example, if we substitute one morphism from the family of morphisms that make up the natural transformation with some other random morphism that has the same signature, all squares that have this morphism as a component would stop commuting.



This would result in something like an "almost-natural" transformation (e.g. an abstract function that reverses all lists, except lists of integers).

And in the category of sets, where morphisms are functions i.e. mappings between values, it is enough to move just one arrow of just one of those values in order to make the transformation "unnatural" (e.g. a function which reverses all lists, but one specific list).



Finally, if can just gather a bunch of random morphisms, one for each object, that fit the criteria, we get what I would call a "perfectly unnatural transformation" (but this is my terminology).

But, although they do exist, it is very hard to define non-natural transformations. For example, for categories that are *infinite*, there is no way to specify such "perfectly unnatural transformation" (ones where none of the squares commute) without resorting to randomness. And even transformations on finite categories, or the "semi-natural" transformations which we described above (the ones that include a single condition for a single value or type), are not possible to specify in some languages e.g. you can define such a transformation in Typescript, but not in Haskell.

To see why, let's see what the type of a natural transformation is.

[\forall\ a.\ F a \to G a]

The key is that the definition should be valid *for all* types a. For this reason, there is no way for us to specify a different arrows for different types, without resorting to type downcasting, which is not permitted in languages like Haskell (as it breaks the principle of parametricity).

Interlude: Skolem variables and parametrization

Let's try to define the "semi-natural" transformation that we described above (the ones that include a single condition for a single value or type) e.g. an abstract function that reverses all lists, except the list of booleans). In Typescript, it will look something like this.

```
function unnatural<A> (a: Array<A>): Array <A>{
        return a
    } else {
        return a.reverse()
    }
}
```

(Look at this piece of code! Doesn't this seem "unnatural"?)

This will work, but, if you try to do the same in Haskell, for example, it would immediately tell you that you cannot ("a is a "rigid type variable" (also known as "Skolem variable" in other context)). Why is it so? There are some *technical reasons*, as runtime type checks like this one, add performance overhead, because they require the runtime to preserve type information for each value, after compilation, but there is also a strong *philosophical reason*: a general function should work in a general way. And the generality of a function that checks the type of a value at runtime (and behaves differently for different types) is dubious at best.

Such function is like, (if we switch to the logic branch of the Curry-Howard isomorphism) proving a general statement, of the form "All a's have a given property" by merely pointing out that the as that are currently in existence happen to have it. Surely, even if valid in some contexts, such proofs are a very limited in terms of both scope and information they carry e.g. the

assertion that all people who are sited at the table next to you have brown hair doesn't tell you anything of substance, unless there is a deeper reason for it to be true.

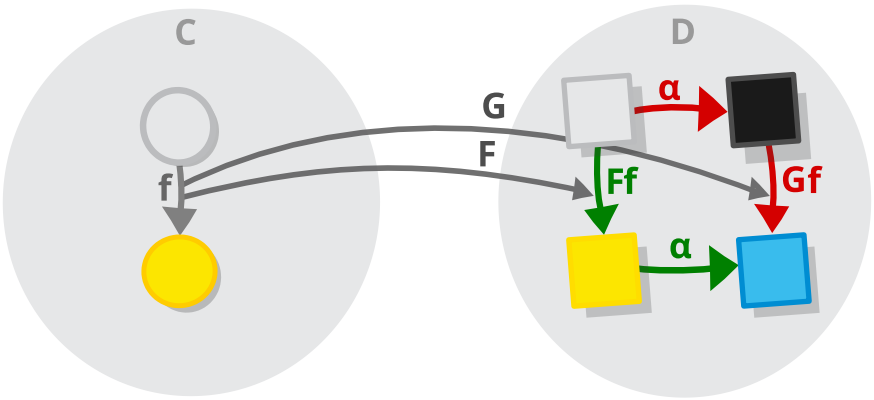
In other words, unnatural transformations wouldn't work in Haskell, simply because they are ... unnatural i.e. they do not follow the laws.

By the way, in programming, this principle is called "parametricity" and the natural abstract functions are called "parametrically polymorphic", whereas unnatural polymorphic functions are known as ad-hoc polymorphic.

Natural transformations again

Now, after we saw the definition of natural transformations, it is time to see the definition of natural transformations (and if you feel that the quality of the humour in this book is deteriorating, that's only because *things are getting serious*).

Let's review again the commuting diagram that represents a natural transformation.



This diagram might prompt us into viewing natural transformations as some kind of "two-arrow functors" that have not one but two arrows coming from each of their morphisms — this notion, can be formalized, by using *product categories*.

Oh wait, I just realized we never covered product categories... but don't worry, we will cover them now.

Product groups and product categories

We haven't covered product categories, however some pages ago, when we covered monoids and groups, we talked about the concept of a *product group*. The good news is that product *categories* are a generalization of product *groups*...

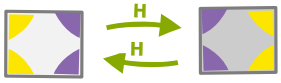
The bad news is that you probably don't remember much about product groups, as covered them briefly.

But don't worry, we will do a more in-depth treatment now:

Product groups

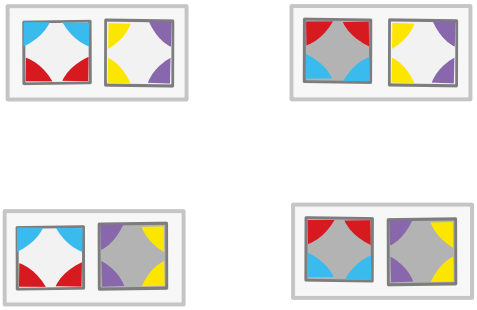
Given two groups G and H, whose sets of elements can also be denoted G and H...





(in this example we use two boolean groups, which we visualize as the groups of horizontal and vertical rotation of a square)

...the *product group* of these two groups is a group that has the cartesian product of these two sets $G \times H$ as its set of elements.



And what can the group operation of such a group be? Well, I would say that out of the few possible groups operations for this set that *exist*, this is the *only* operation that is *natural* (I didn't intend to involve natural transformation at this section, but they really do appear everywhere). So, let's try to derive the operation of this group.

We know what a group operation is, in principle: A group operation combines two elements from the group into a third element i.e. it is a function with the following type signature:

[\circ : (A, A) \to A]

or equivalently

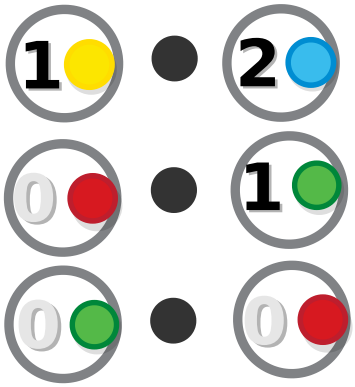
[\circ : A \to A \to A]

And for product groups, we said that the underlying set of the group (which we dubbed A above) is a cartesian product of some other two sets which we dubbed G and H. So, when we swap A for $G \times H$ the definition becomes:

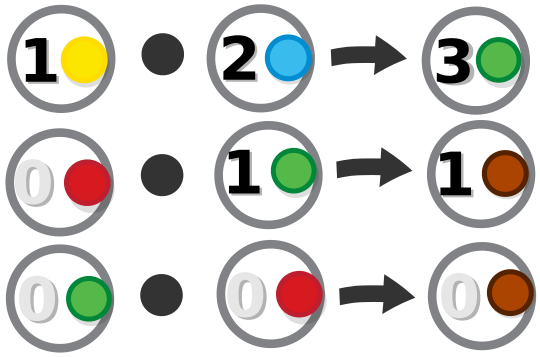
[\circ : G \times H \to G \times H \to G \times H]

i.e. the group operation takes one pair of elements from G and H and another pair of elements from G and H, only to return — guess what — a pair of elements G and H.

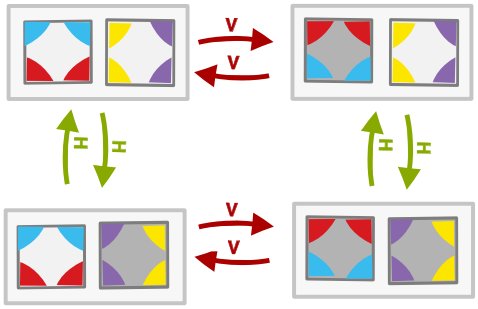
Let's take an example. To avoid confusion, we take two totally different groups — the color-mixing group and the group of integers under addition. That would mean that a value of $G \times H$ would be a pair, containing a random color and a random number, and the operation would combine two combine two such pairs and produce another one.



Now, the operation must produce a pair, containing a number and a color. Furthermore, it would be good if it produces a number *by using those two numbers*, not just picking one at random, and likewise for colors. And furthermore, we want it to work not just for monoids of numbers and colors, but all other monoids that can be given to us. It is obvious that there is only one solution, to get the elements of the new pair by combining the elements of the pairs given.



And the operation of the product group of the two boolean groups which we presented earlier is the combination of the two operations



So, the general definition of the operation is the following ($g1$, $g2$ are elements of G and $h1$ and $h2$ elements of H).

[(g1, h1) \circ (g2, h2) = ((g1 \circ g2), (h1 \circ h2))]

And that are product groups.

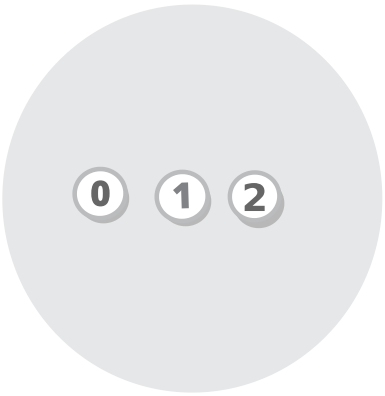
Product categories

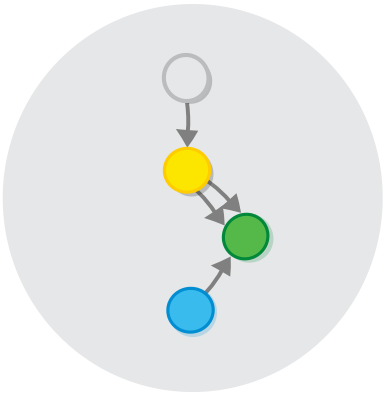
We are back at tackling product *categories*.

Since we know what product *groups* are, and we know that groups are nothing but categories with just one object (and the group objects are the category's morphisms, remember?), we are already almost there.

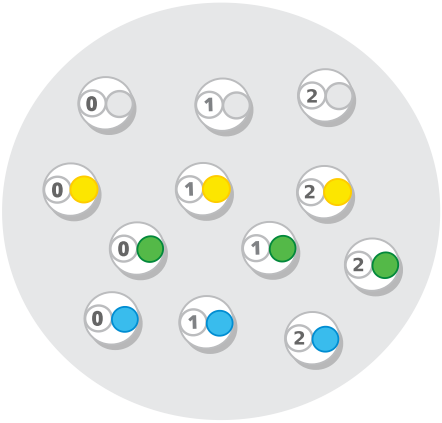
Here is a way to make a product category.

Take any two categories:

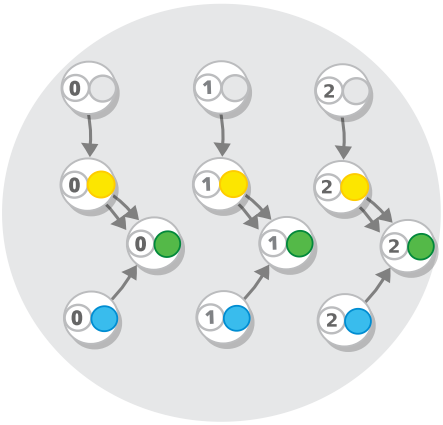




Then take the set of all possible pairs of the objects of these categories.



And, finally, we make a category out of that set by taking all morphisms coming from any of the two categories and replicate them to all pairs that feature some objects from their type signature, in the same way as we did for product groups (in this example, only one of the categories has morphisms).

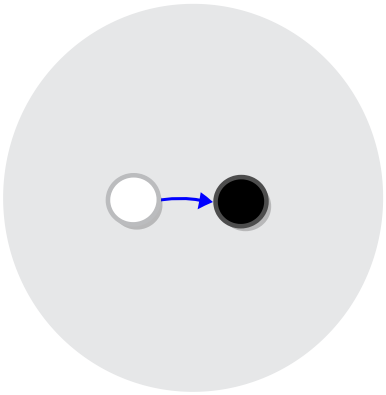


This is the *product category* of the two categories.

> **DEFINITION:**
>
> The product category of two categories C and D, denoted $C \times D$ is a category that has the product set of C and D's underlying sets as its underlying set. And it's morphisms are the combined morphisms of C, and D, transformed in such a way that they modify one element of the pair. That is for each morphism f in C we have a morphism $(f(c), d)$ in $C \times D$ and likewise for each morphism in D.

Natural transformations as functors of product categories

In this section we are interested with the products of one particular category, namely the category we called 2, containing two objects and one morphism (stylishly represented in black and white).



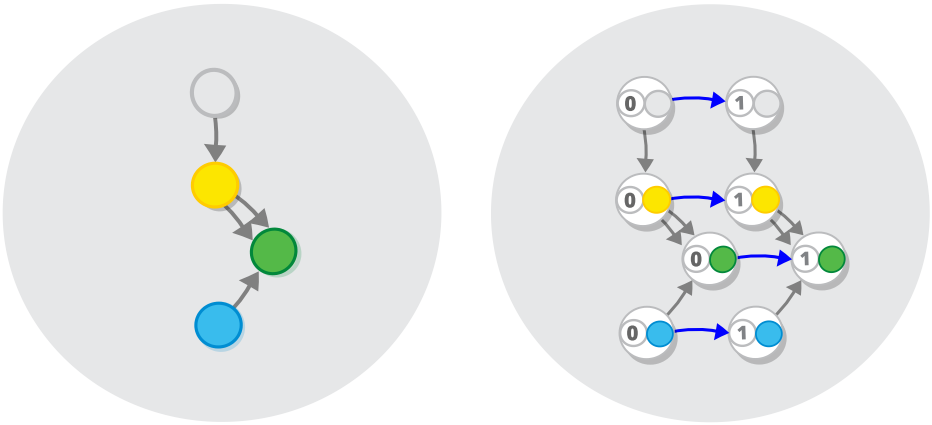
This category is the key to constructing a functor that is equivalent to a natural transformation:

- Because it has two objects, it produces two copies of the source category.
- because the two objects are connected, the two copies are connected in the same way as the two "images" in the target category are connected.

So, given a product category of 2 and some other category C...



...there exist a natural transformation between C and the product category $2 \times C$.



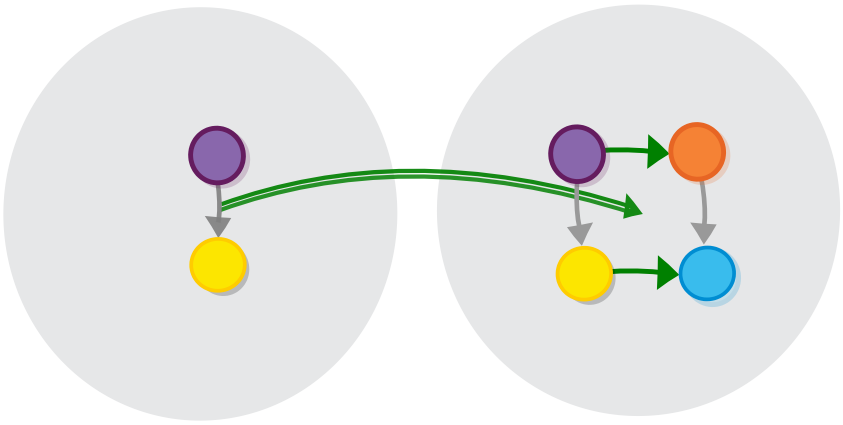
Furthermore, this connection is two-way:

> **THEOREM:**
>
> Any natural transformation from category C to any other category D, can be represented as a functor $2 \times C \to D$ and vice versa.

That is, if we have a natural transformations $\alpha : F \Rightarrow G$ (where $F : C \to D$ and $G : C \to D$), then, we also have a functor $2 \times C \to D$, such that if we take the subcategory of $2 \times C$ comprised of just those objects that have the 0 object as part of the pair, and the morphisms between them, we get a functor that is equivalent to F, and if we consider the subcategory that contains 1, then the functor is equivalent to G (we write $\alpha(-, 0) = F$ and $\alpha(-, 1) = G$). Et voilà!

Task 5: Show that the two definitions are equivalent.

This perspective helps us realize that a natural transformation can be viewed as a collection of commuting squares. The source functor defines the left-hand side of each square, the target functor – the right-hand side, and the transformation morphisms join these two sides.

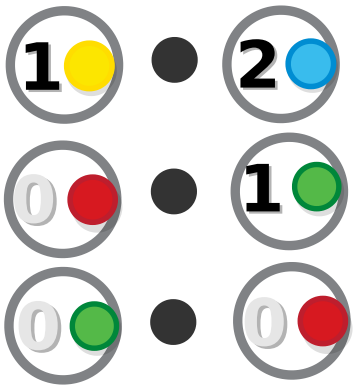


We can even retrieve the structure of the source category of these functors, which (as categories are by definition structure and nothing more) is equivalent to retrieving the category itself.

Interlude: Naturality in product group operations

Now, we will have one really peculiar interlude, in which we will show that the group operation of product groups is actually a natural transformation.

To understand why this is the case, let's look at the equations once more.



We said that in order for the solutions for this equations, to make sense, they would have to work for all monoids, not just numbers and colors. In other words, there should be a mechanism that, given two monoids, produces a third one. This mechanism can be nothing more (and nothing less) than a polymorphic function.

[\forall G H. G \to H \to G \times H]

We shown above how polymorphic functions are transformations (this one is harder to reason about, since it is parametrized over not one, but two types, but we learned how to represent functions that take two arguments as functions that take one argument in chapter 2).

And the naturality condition guarantees that our solution would produce a number/color etc. *by using the two numbers/colors provided*, not just picking one at random.

You can prove that the naturality condition indeed does hold (correct by construction) for all product groups.

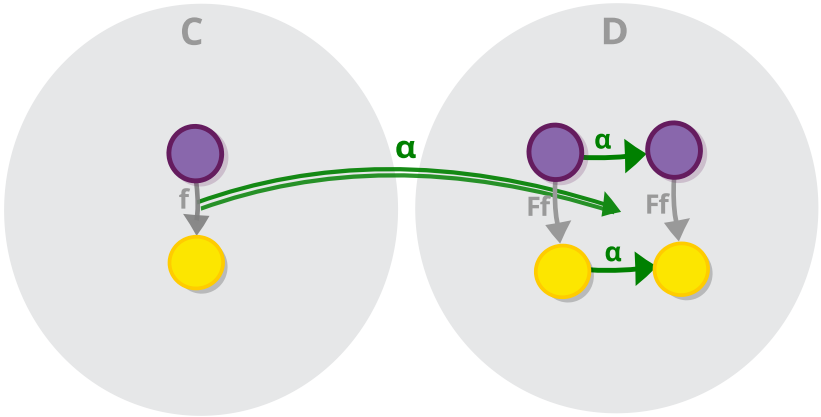
Composing natural transformations

Natural transformations are surely a different beast than normal morphisms and functors and so they don't compose in the same way. However, they do compose and here we will show how.

The identity natural transformation

Let's first get one trivial definition out of the way:

> **DEFINITION:**
>
> For each functor, we have the identity natural transformation (actually a natural isomorphism) between it and itself.

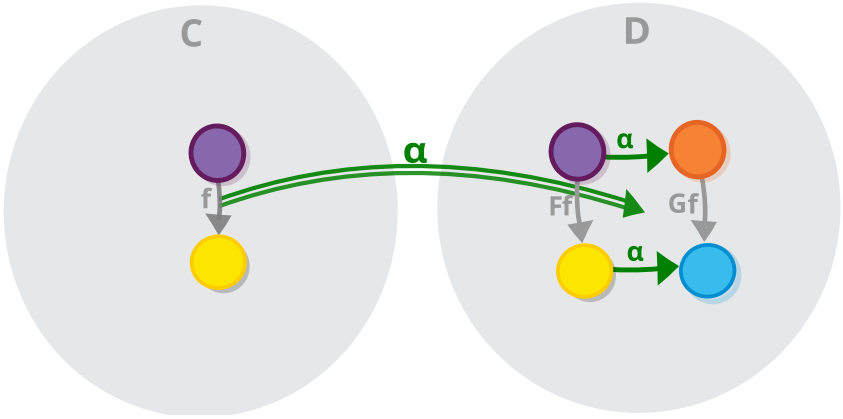


Horizontal composition

The setup for composing natural transformations may look complicated the first time you see it: we need three categories C, D and E (just as composition of morphisms requires three objects). We need a total of four functors, distributed on two pairs, one pair of functors that goes from C to D and one that goes from D to E (so we can compose these two pairs of functors together, to get a new pair of functors that go $C \to E$). However, we will try to keep it simple and we will treat the natural transformation as a map from a

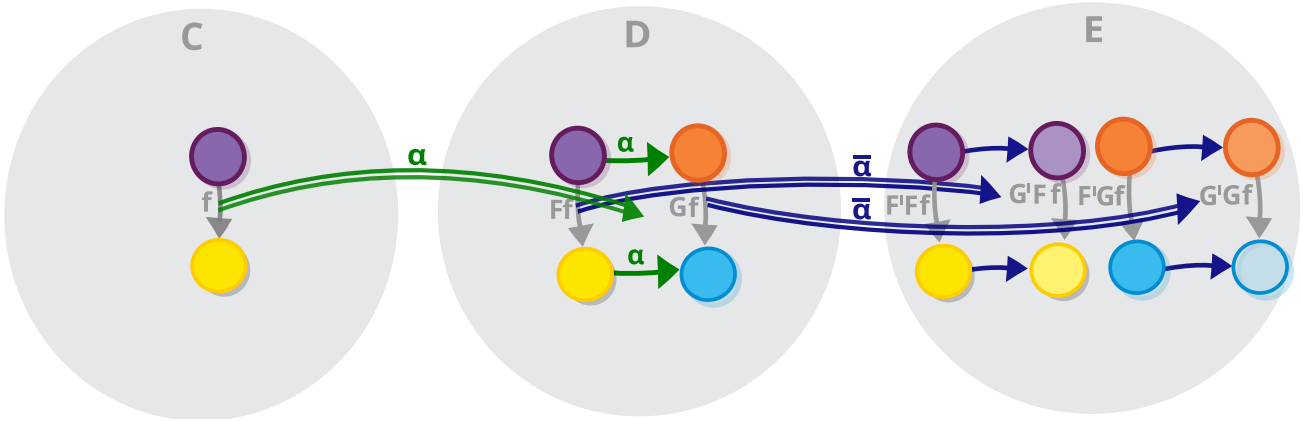
morphism to a commuting square. As we showed above, this mapping already contains the two functors in itself.

So, let's say that we have the natural transformation α involving the $C \to D$ functors (which we usually call F and G).

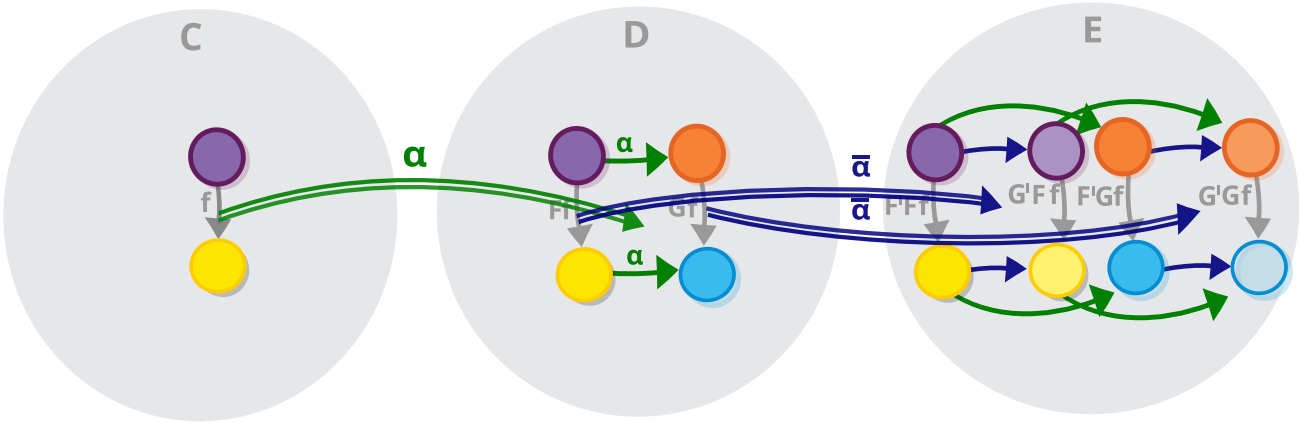


So, what will happen if we have one more transformation $\bar{\alpha}$ involving the functors that go $D \to E$ (which are labelled F' and G')? Well, since a natural transformation maps each morphism to a square, and a square contains four morphisms (two projections by the two functors and two components of the transformation), a square would be mapped to four squares.

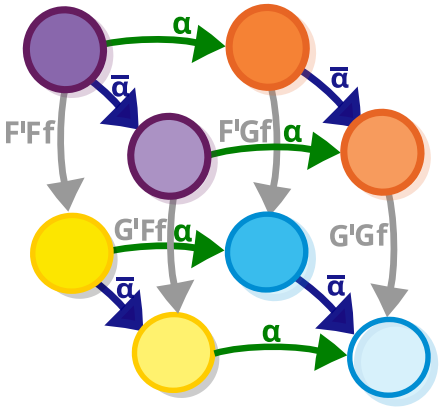
Let's start by drawing two of them for each projection of the morphism in C.



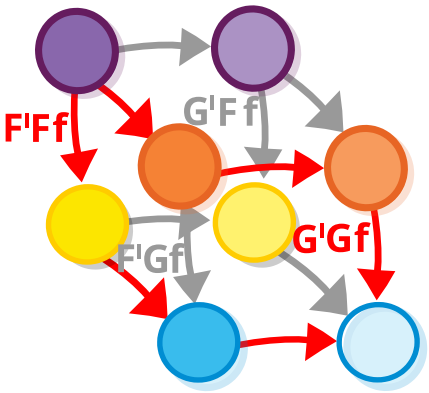
We have to have two more squares, corresponding to the two morphisms that are the components of the α natural transformation. However, these morphisms connect the objects that are the target of the two functors, objects that we already have on our diagram, so we just have to draw the connections between them.



The result is an interesting structure which is sometimes visualized as a cube.



More interestingly, when we compose the commuting squares from the sides of the cube horizontally, we see that it contains not one, but two bigger commuting squares (they look like *rectangles* in this diagram), visualized in grey and red. Both of them connect morphisms $F'Ff$ and $G'Gf$.

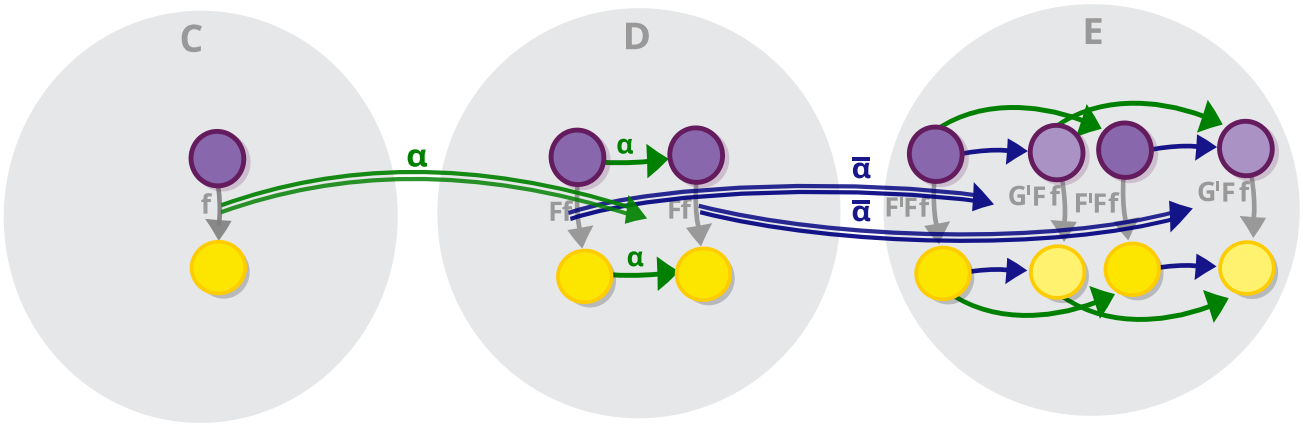


So, there is a natural transformation between the composite functor $F' \circ F : C \to E$ and $G' \circ G : C \to E$ – a natural transformation that is usually marked $\bar{\alpha} \bullet \alpha$ (with a black dot).

Task 6: Show that natural transformations indeed compose i.e. that if you have natural transformations $F'Ff \Rightarrow F'Gf$ and $F'Gf \Rightarrow G'Gf$ you have $F'Ff \Rightarrow G'Gf$.

Whiskering

And an interesting special case of horizontal composition is horizontal composition involving the identity natural transformation: given a natural transformation $\bar{\alpha}$ involving functors with signature $D \to E$ and some functor with signature $F : C \to D$, we can take α to be the identity natural transformation between functor F and itself and compose it with $\bar{\alpha}$.



We get a new natural transformation $\bar{\alpha} \bullet \alpha$, that is practically the same as the one we started with (i.e. the same as $\bar{\alpha}$) so what's the deal? We just found a way to *extend* natural transformations, using functors: i.e we can use a functor with signature $C \to D$ to extend a $D \to E$ natural transformation and make it $C \to E$.

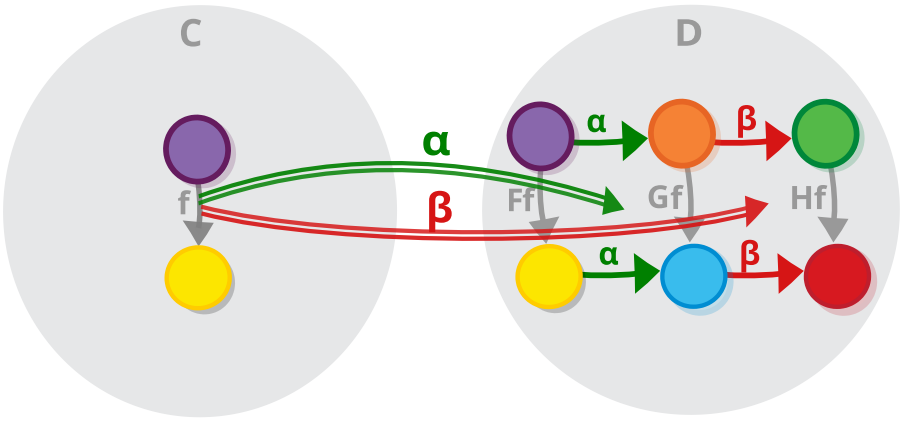
Task 7: Try to extend the natural transformation in the other direction (by taking $\bar{\alpha}$ to be identity).

So, this is how you compose natural transformations. It's too bad that this is form of composition is different from the standard categorical composition. So, I guess natural transformations do not form a category, like we hoped they would...

Well, OK, there is actually another way of composing categories, which might actually work.

Vertical composition

Recall that categorical composition involves three objects and two successive arrows between them. For vertical composition of natural transformations, we will need three (or more) *functors* with the same type signature, say $F, G, H : C \to D$ i.e. (same source and target category) and two successive *natural transformations* between those functors i.e. $\alpha : F \to G$ and $\beta : G \to H$.



We can combine each morphism of the natural transformation α (e.g. $a : F \to G$) and the corresponding morphism of the natural transformation β (say $b : G \to H$) to get a new morphism, which we call $b \circ a : F \to H$ (the composition operator is the usual white circle, as opposed to the black one, which denotes horizontal composition). And the set of all such morphisms are precisely the components of a new natural transformation: $\beta \circ \alpha : F \to H$.

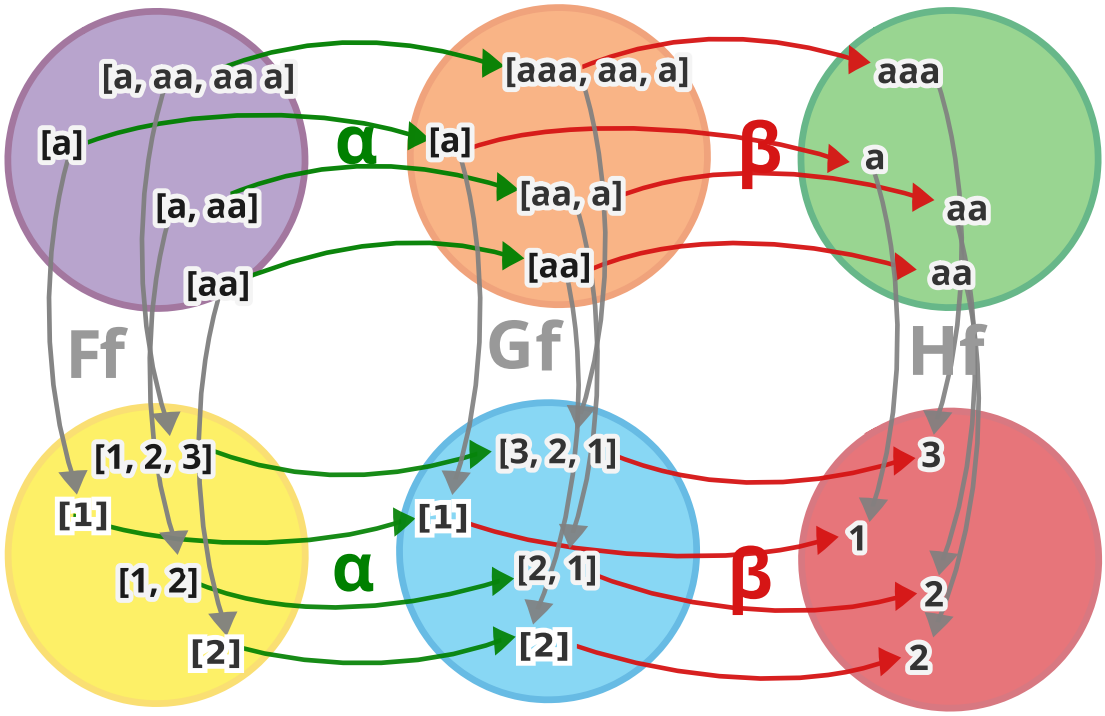
> Given three *functors* $F, G, H : C \to D$ and two successive *natural transformations* between those functors i.e. $\alpha : F \to G$ and $\beta : G \to H$, there is a natural transformation $\beta \circ \alpha : F \to H$, with the composition of each morphism of the natural transformation α (e.g. $a : F \to G$) and the corresponding morphism of the natural transformation β (say $b : G \to H$) as its components.

Categories of functors

Now, we are approaching the end of the chapter, we will introduce our category and call it quits. To do that, we first introduce a more compressed notation for vertical composition of natural transformations (where they do indeed look vertical).

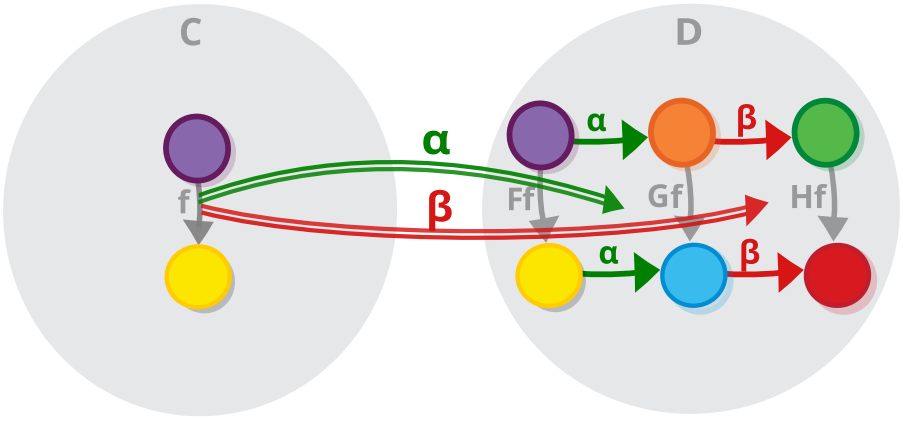
We started this chapter by looking at category of sets and using internal diagrams, displaying the set elements as points and the sets/objects as

collections.

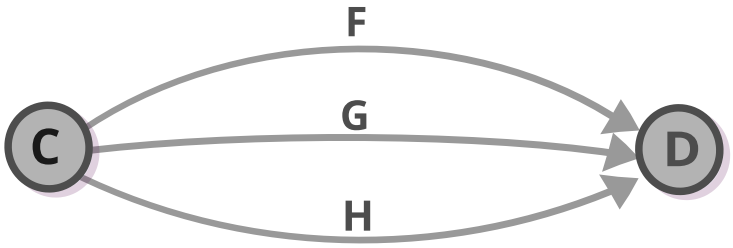


Task 8: identify the function, the three functors, and the two natural transformations used in this diagram.

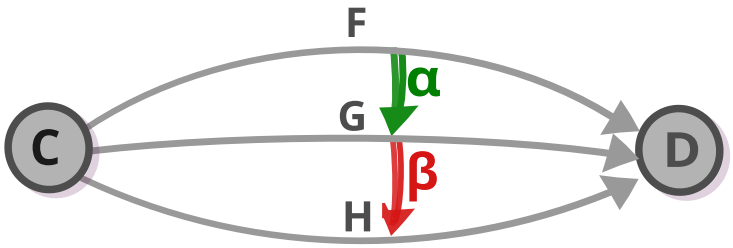
Then, we quickly passed to normal external diagrams, where objects are points and categories are collections.



And now we go one more level further, and show the category of categories, where categories are points and functors are morphisms.

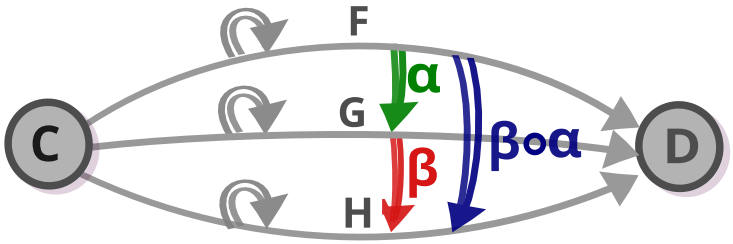


In this notation, we display natural transformations as (double) arrows between morphisms.



And you can already see the new category that is formed:

> **DEFINITION:**
>
> For each two categories C and D, there exists a category which has the $C \to D$ functors as objects and the natural transformations between those functors as morphisms.

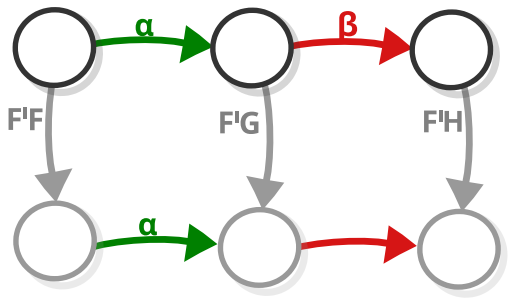


Natural transformations compose with vertical compositions, and, of course, the identity natural transformation is the identity morphism.

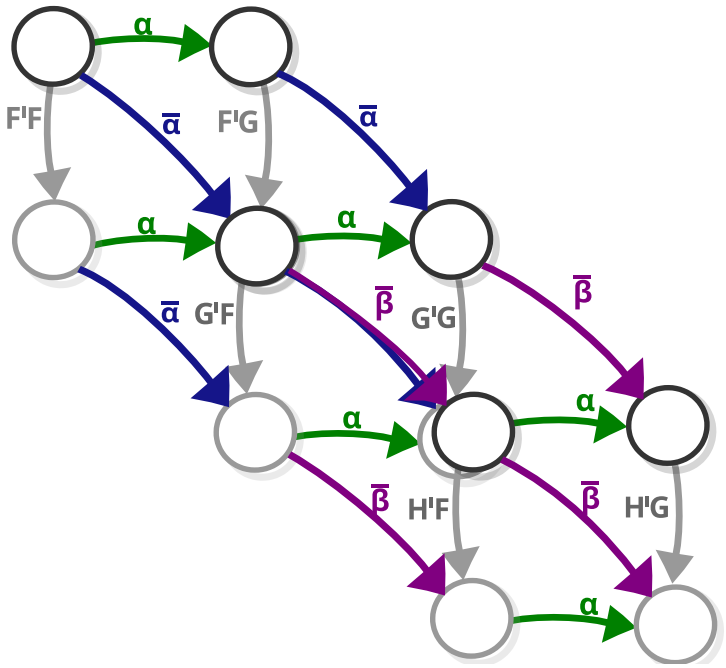
Interchange law

Vertical and horizontal composition of natural transformations are related to each other in the following way:

If we have (as we had) two successive natural transformations, in the vertical sense, like $\alpha : F \to G$ and $\beta : G \to H$.



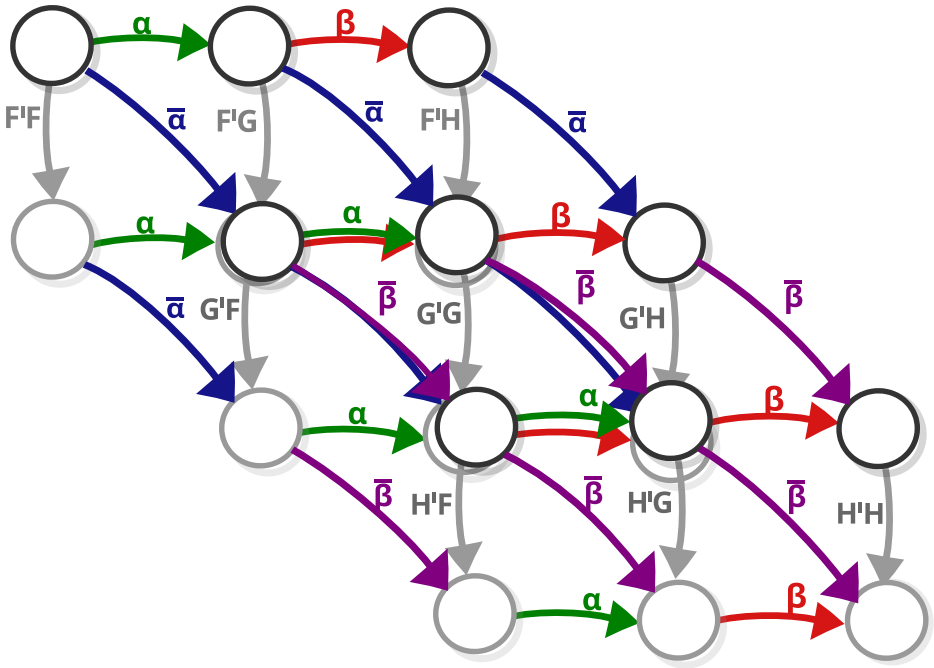
And two successive ones, this time in horizontal sense e.g. $\bar{\alpha} : F' \to G'$ and $\bar{\beta} : G' \to H'$. (note that α has nothing to do with $\bar{\alpha}$ as β has nothing to do with $\bar{\beta}$, we just call them that way to avoid using too many letters)



And if the two pairs of natural transformations both start from the same category and the same functor, then the compositions of the two pairs of natural transformations obey the following law

[(β \circ α) \bullet (\bar β \circ \bar α) = (β \bullet \bar β) \circ (α \bullet \bar α)]

Task 9: Draw the paths of the two compositions of the transformations (on the two sides of the equation) and ensure that they indeed lead to the same place.



2-Categories

At this point you might be wondering the following (although statistically you are more likely to wonder what the heck is all this about): We know that all categories are objects of Cat, the category of small categories, in which functors play the role of morphisms.

But, functors between given categories also form a category, under vertical composition. Which means that Cat not only has (as any other category) morphisms between objects, *but* also has *morphisms between morphisms*. And furthermore, those two types of morphisms compose in this very interesting way.

So, what does that make of Cat? I don't know, perhaps we can call natural transformations "2-morphisms" and Cat is some kind of "2-category"?

But wait, actually it's way too early for you to find out. We haven't even covered limits...

Answers

Task 1: Check if that definition is valid.

The definition in question is:

> Two **categories** A and B are **isomorphic** (or $A \cong B$) if there exist *functors* $f : A \to B$ and its reverse $g : B \to A$, such that $f \circ g = ID_B$ and $g \circ f = ID_A$.

We can check it by expanding the definition of a functor.

The functor consists of:

1. Object mapping
2. Morphism mapping
3. Laws

Expanding the object mapping gives us exactly the definition of set isomorphism:

> Two **sets** A and B are **isomorphic** (or $A \cong B$) if there exist functions $f : A \to B$ and its reverse $g : B \to A$, such that $f \circ g = ID_A$ and $g \circ f = ID_A$.

Expanding the morphism mapping and the law would give us the rest.

Task 2: When exactly would the mapping encompass all objects?

As we say above:

> mapping the two functors' object components involves nothing more than specifying a bunch of morphisms in the target category: one morphism for each object in the source category i.e. each object from the image of the first functor, should have one arrow coming from it.

So the mapping encompasses all object *from the image of the functor*. So when would that encompass all objects period. Simple — when the functor is one-to-one (or onto, if we are with there being more than one arrow per object). Most probably, that would happen when the two categories are actually one and the same category and the functor is the identity functor.

Task 3: Draw example naturality squares of the $reverse$ natural transformation.

To draw a square of the $reverse : List\ a \to List\ a$ natural transformation, we must first:

1. Pick a function, f say $+1 : number \to number$.
2. Lift it with the functor e.g $Ff : Listnumber \to Listnumber$ adds one to every number from the list.

Then we:

1. Draw the results of the application of the function on the sets that are at the top, at the corresponding boxes at the bottom e.g. $[1, 2]$ becomes $[2, 3]$.
2. Draw the connections of applying the $reverse$ natural transformation.

If we worked correctly, the square would commute.

Task 4: Prove the above results, using the formula of the naturality condition.

We want to prove that:

[take1 \circ reverse \circ F\ f \circ F\ g]

is the same as

[take1 \circ F\ f \circ reverse \circ F\ g]

We note that part of the equation contains a "lifted" function (one which is the result of the application of functor), followed by a natural transformation:

[take1 \circ (F\ f \circ reverse) \circ F\ g]

The naturality says that

[\alpha \circ F\ f \cong G\ f \circ \alpha]

We can replace α with $reverse$ (sinse $reverse$ is a natural transformation) and replace both the $F\ f$ and $G\ f$ with $F\ f$ (since F and G are the same functor (list))

[reverse \circ F\ f \cong F\ f \circ\ reverse]

Applying this we get:

[take1 \circ (reverse \circ F\ f) \circ F\ g]

Task 5: Show that the two definitions [of natural transformation] are equivalent.

This entails extracting a natural transformation from a functor $2 \times C \to D$, and the other way around.

Here,we will describe one direction of this process:

As we said, we can split the category $2 \times C$ into two subcategories (let's call those C^1 and C^2) which are both isomorphic to C, one containing the objects that are paired with the first object of 2 and one containing the objects paired with the second object.

Likewise, we can split the functor $2 \times C \to D$ into two functors $C^1 \to D$ and $C^2 \to D$.

Now, all we need to do is show that there is a natural transformation between those two functors i.e. we have to find a family of morphisms in category D that connects the images of those two functors.

This family is the image of the morphisms from the category 2 i.e. if we traverse $2 \times C \to D$ and collect all morphisms in D which come from the category 2, we already have our transformation.

This transformation is natural due to the way composition for product categories is defined (it is defined as the component-wise composition of the morphisms of the categories that are part of the product).

Task 6: Show that natural transformations indeed compose i.e. that if you have natural transformations $\alpha : F'F \Rightarrow F'G$ and $\bar{\alpha} : F'G \Rightarrow G'G$ you have $\bar{\alpha} \bullet \alpha : F'F \Rightarrow G'G$.

To show that two transformations compose, we need to find a way, given two natural transformations with the above signatures, to construct a new one.

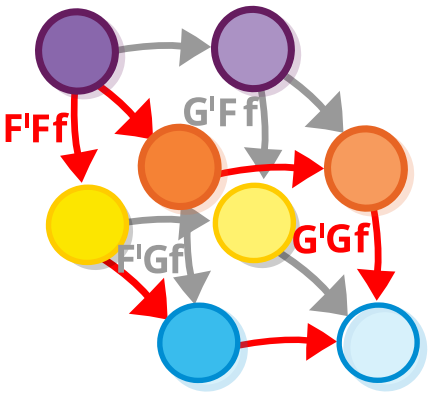
To do that, we remember that a natural transformation is just a family of morphisms, one for each object in the category C (and that we know how to compose morphisms).

For each object c in C, let $\bar{\alpha}_c$ be the morphism that corresponds to it in the natural transformation $\bar{\alpha}$ and let α_c be the morphism that corresponds to it in α.

Then, the morphism that corresponds to it in $\bar{\alpha} \bullet \alpha$ is either $\bar{\alpha}_c \circ F'(\alpha_c)$, if you follow the grey arrows of the cube, or alternatively $(\bar{\alpha} \bullet \alpha)_c = G'(\bar{\alpha}_c) \circ \alpha_c$, if you follow the grey arrows. Or, just $\bar{\alpha}_c \circ \alpha_c$ if you don't care so much about notation.

That such morphisms form a transformation follows from the signatures of the morphisms.

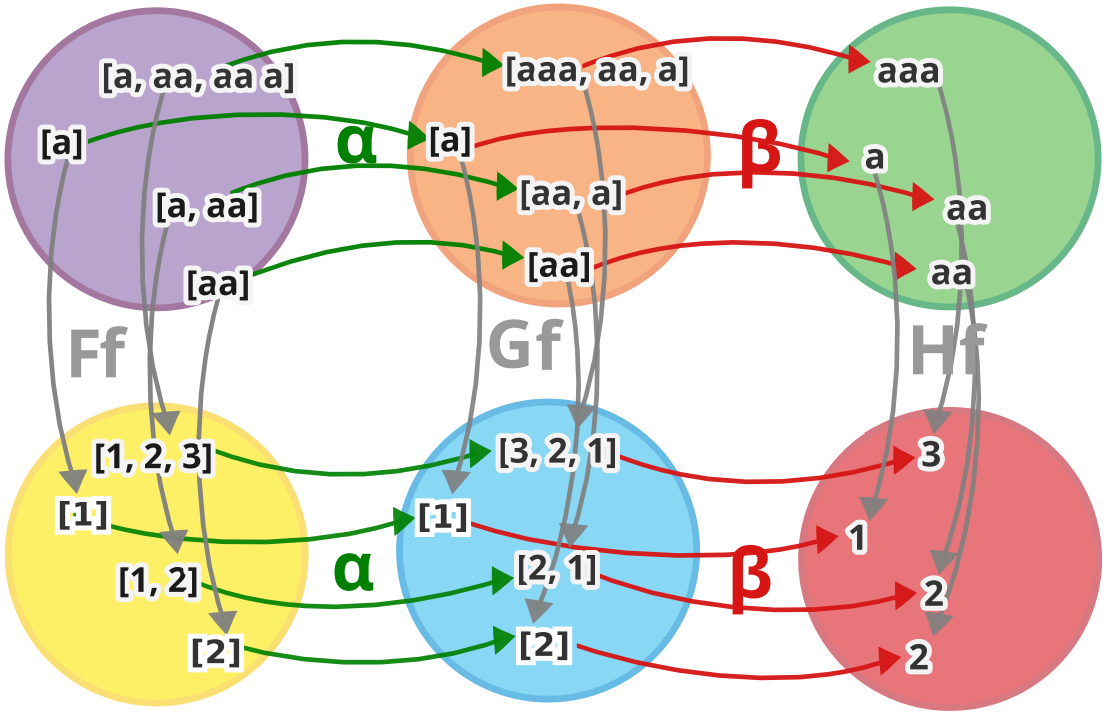
This transformation is natural, because for all those morphisms we have the commuting square.



Task 7: Try to extend the natural transformation in the other direction (by taking $\bar{\alpha}$ to be identity).

It works basically the same way and the result is also the same — you get a new natural transformation with a different signature.

Task 8: identify the function, the three functors, and the two natural transformations used in this diagram.



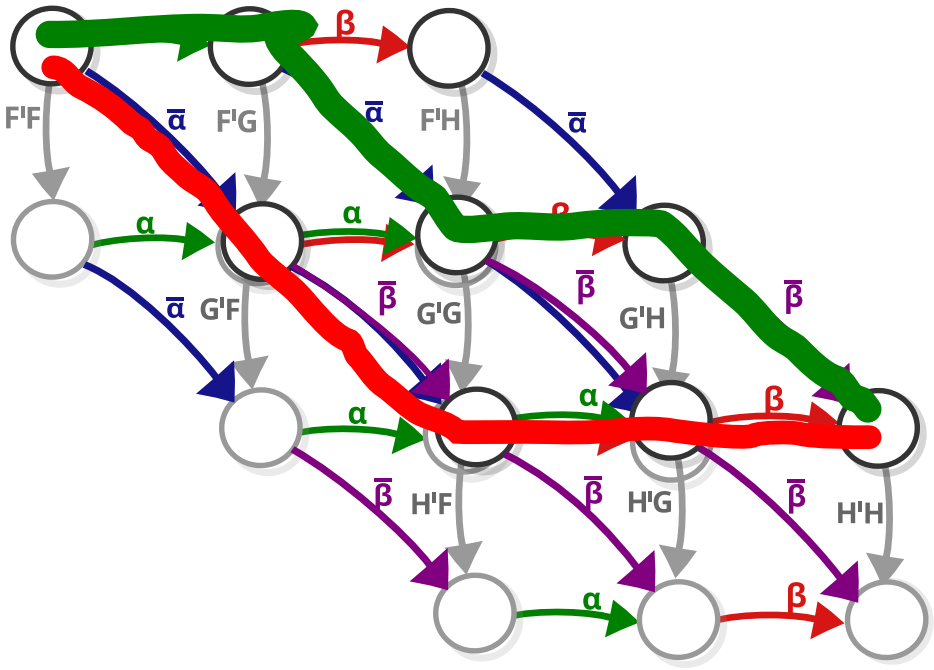
First the functors: F and G are the $List$ functor, H is the ID functor.

Natural transformations: α is $reverse : List \to List$ and β is $head : List \to ID$ (or, if we want to be punctual, the type should be its *Non-empty* list, as $head$ is only a partial natural transformation of $List$).

The function f is, as usual, $length : string \to int$.

Task 9: Draw the paths of the two compositions of the transformations (on the two sides of the equation) and ensure that they indeed lead to the same place.

One option is this:



Buy full version

Ko-fi

Patreon

Gumroad

Subscribe for updates

Enter your email Subscribe

Contents

About

Sets

Categories

Monoids

Orders

Logic

Types

Functors

Natural transformations

Created by Jencel Panic

personal blog

email

mastodon

twitter

Github

Made with Jekyll Book Boilerplate. Drawn in Inkscape. Set in Mell Sans.

This work is licensed under a Creative Commons Attribution-NonCommercial 4.0 International

License.

Presentation mode